

ESS Skills und Contracts

Enterprise Standard Skills als normative Systembeschreibung

Die Enterprise Standard Skills (ESS-0001 bis ESS-0023) mit ihren Contracts. Sie definieren normativ, welche Rolle jede Plattformkomponente hat, welche Zusagen sie gibt und welche Governance für sie gilt. Das ist die verbindlichste Ebene der Systembeschreibung.

Repository: SvenKulesa/Finance

Stand: claude/pdf-dokumente-notebook-l1m-l6kcxq @ 4eadf9d

Erzeugt am: 16.08.2026

Enthaltene Dokumente: 33

Umfang: ca. 63.250 Wörter

Dieses PDF ist eine Quelle für NotebookLM. Jedes eingebettete Dokument beginnt auf einer neuen Seite und nennt seinen Repository-Pfad, damit erzeugte Podcasts, Video-Skripte und Zusammenfassungen ihre Aussagen exakt auf die Originaldatei zurückführen können.

Inhalt dieses Quellpakets

1. Documentation Governance Validator	4
.ai/skills/Documentation-Governance-Validator.md	
2. Enterprise Event Mesh	10
.ai/skills/Enterprise-Event-Mesh.md	
3. ESS-0001-CONTRACTS	16
.ai/skills/ESS-0001-Contracts.md	
4. ESS 0001 Documentary Architect	243
.ai/skills/ESS-0001-Documentary-Architect.md	
5. ESS 0002 Supervisor Architect	342
.ai/skills/ESS-0002-Supervisor-Architect.md	
6. ESS 0003 Platform Director	362
.ai/skills/ESS-0003-Platform-Director.md	
7. ESS 0004 Enterprise Version Manager	380
.ai/skills/ESS-0004-Enterprise-Version-Manager.md	
8. ESS 0005 Quality Center	388
.ai/skills/ESS-0005-Quality-Center.md	
9. ESS 0006 Security Compliance	396
.ai/skills/ESS-0006-Security-Compliance.md	
10. ESS 0007 Enterprise Release Center	404
.ai/skills/ESS-0007-Enterprise-Release-Center.md	
11. ESS 0008 AI Agent Framework	412
.ai/skills/ESS-0008-AI-Agent-Framework.md	
12. ESS 0009 Enterprise Knowledge Platform	421
.ai/skills/ESS-0009-Enterprise-Knowledge-Platform.md	
13. ESS 0010 Documentary Engine	429
.ai/skills/ESS-0010-Documentary-Engine.md	
14. ESS-0011-CONTRACTS	447
.ai/skills/ESS-0011-Contracts.md	
15. ESS 0011 Enterprise Traceability	458
.ai/skills/ESS-0011-Enterprise-Traceability.md	
16. ESS-0012-CONTRACTS	470
.ai/skills/ESS-0012-Contracts.md	
17. ESS 0012 Documentation Governance	483
.ai/skills/ESS-0012-Documentation-Governance.md	

18. ESS 0012 Enterprise Vocabulary Terminology Governance	499
.ai/skills/ESS-0012-Enterprise-Vocabulary-Terminology-Governance.md	
19. ESS-0013-CONTRACTS	502
.ai/skills/ESS-0013-Contracts.md	
20. ESS 0013 Enterprise Event Mesh	507
.ai/skills/ESS-0013-Enterprise-Event-Mesh.md	
21. ESS-0014-CONTRACTS	518
.ai/skills/ESS-0014-Contracts.md	
22. ESS-0014 — Google Marketing MCP Governance	526
.ai/skills/ESS-0014-Google-Marketing-MCP-Governance.md	
23. ESS-0015 — API Interface Inventory & QA Governance	537
.ai/skills/ESS-0015-API-Interface-Inventory-QA.md	
24. ESS-0016 — Enterprise Market Data Provider & MCP Governance	543
.ai/skills/ESS-0016-Enterprise-Market-Data-Provider-MCP-Governance.md	
25. ESS-0017-CONTRACTS	557
.ai/skills/ESS-0017-Contracts.md	
26. ESS 0017 Vocabulary Governance	563
.ai/skills/ESS-0017-Vocabulary-Governance.md	
27. ESS-0018 — Agentic Supabase Tool Governance	567
.ai/skills/ESS-0018-Agentic-Supabase-Tool-Governance.md	
28. ESS-0019 — Universal AI Agent Control Plane	573
.ai/skills/ESS-0019-Universal-AI-Agent-Control-Plane.md	
29. ESS-0020 — Supabase Native MFA / AAL2 Hardening	576
.ai/skills/ESS-0020-Supabase-Native-MFA-AAL2-Hardening.md	
30. ESS-0021 — Systemadmin Roadmap Executor	582
.ai/skills/ESS-0021-Systemadmin-Roadmap-Executor.md	
31. ESS-0022 — Marketing Roadmap Executor	587
.ai/skills/ESS-0022-Marketing-Roadmap-Executor.md	
32. ESS-0022 — Passkey-only Human/Owner PR Authorization	594
.ai/skills/ESS-0022-Passkey-Only-Owner-PR-Authorization.md	
33. ESS-0023 — Konsolidierter Systemadministrator-Prototyp- und Mutationsanfragevertrag	600
.ai/skills/ESS-0023-Consolidated-Systemadmin-Prototype-and-Mutation-Request.md	

Documentation Governance Validator

Quelle: `.ai/skills/Documentation-Governance-Validator.md`

skill: id: SKILL-GOV-0001 name: Documentation Governance Validator version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Documentary Engine category: Enterprise Governance priority: High

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

classification: type: Operational Skill role: Handlungsanweisung für KI-Systeme zur Governance-Prüfung specification: ESS-0012 contractAuthority: ESS-0012-CONTRACTS note: > Dieser Skill ist kein ESS-Dokument und belegt keine ESS-Nummer. Er beschreibt ausschließlich die Anwendung des in ESS-0012 spezifizierten Validators. Bei Abweichungen gelten ESS-0012 und ESS-0012-CONTRACTS.

authority:

controls: - Governance-Prüflauf - Befunderzeugung - Berichtsanforderung

cannot_modify: - Enterprise Specifications - Architecture Decision Records - Enterprise Contracts - Dokumentation - Knowledge Graph - Digital Twin - Registry

crossReference: dependsOn: - ESS-0012 - ESS-0012-CONTRACTS relatedEss: - ESS-0001-CONTRACTS - ESS-0002 - ESS-0003 - ESS-0010 - ESS-0011 relatedAdr: - ADR-0014 relatedComponents: - src/platform/Documentary/Governance relatedSkills: - .ai/skills/ESS-0012-Documentation-Governance.md - .ai/skills/ESS-0012-Contracts.md

created: 2026-07-31

Documentation Governance Validator

Zweck dieses Skills

Dieser Skill beschreibt, **wie** ein KI-System den Documentation Governance Validator anwendet.

Er ersetzt keine Spezifikation. Die verbindlichen Regeln stehen in

ESS-0012 — Spezifikation

ESS-0012-CONTRACTS — Regelwerk, Scoring, Events

Bei jeder Abweichung zwischen diesem Skill und den ESS-Dokumenten gelten die ESS-Dokumente.

Grundregel

Der Validator stellt fest. Er entscheidet nicht. Er korrigiert nicht.

Ein KI-System, das eine Änderung vornimmt, prüft sie **vor** der Übergabe an die nächste Stufe der Wertschöpfungskette.

Ein KI-System behebt einen Befund niemals im selben Arbeitsschritt, in dem es ihn erzeugt hat, ohne die Korrektur erneut zu prüfen.

Aufgaben

Aufgabe	Beschreibung
Prüflauf ausführen	Neun Prüfbereiche gemäß ESS-0012 Chapter 3
Befunde erzeugen	jeweils mit Regel-ID, Severity und Nachweis
Kennzahlen berechnen	Governance Score, Repository Health, Documentation Quality
Berichte anfordern	zehn Berichte gemäß ESS-0012 Chapter 5
Events veröffentlichen	sechs Events gemäß ESS-0012-CONTRACTS Chapter 4
Supervisor informieren	bei Severity Critical und High

Trigger

Der Skill wird verbindlich angewendet bei

Auslöser	Umfang
vor jeder Übergabe an die nächste Stufe der Wertschöpfungskette	inkrementell
nach jeder Dokumentenerzeugung	inkrementell
nach jeder Metadatenänderung	inkrementell
bei ESS-Änderung	vollständig
bei ADR-Änderung	vollständig
bei Versionsänderung	vollständig
vor jeder Release-Vorbereitung	vollständig
bei zeitgesteuerter Vollprüfung	vollständig

Vor jeder Produktionsfreigabe ist ein vollständiger Durchlauf verbindlich.

Validierungsregeln

Das vollständige Regelwerk umfasst **57 Regeln in neun Bereichen** und steht in ESS-0012-CONTRACTS Chapter 2.

Übersicht der Bereiche:

Kürzel	Bereich	Regeln
GOV-ESS-*	ESS-Konsistenz	8
GOV-ADR-*	ADR-Konsistenz	8
GOV-REPO-*	Repository-Metadaten	9
GOV-CONTRACT-*	Contract-Konsistenz	6
GOV-DOC-*	Dokumentationsqualität	7
GOV-TRACE-*	Traceability	4
GOV-KG-*	Knowledge Graph	5
GOV-TWIN-*	Digital Twin	5
GOV-VER-*	Versionierung	5

Die Regeln werden in diesem Skill **nicht wiederholt**. Er verweist ausschließlich.

Anwendungsreihenfolge

1. Regelwerk aus ESS-0012-CONTRACTS laden
2. Prüfbereiche in fester Reihenfolge ausführen
3. Befunde mit Nachweis sammeln
4. Kennzahlen berechnen
5. Berichte anfordern
6. Events veröffentlichen
7. Bei Critical: Supervisor informieren und anhalten

Die Reihenfolge ist verbindlich und darf nicht verändert werden.

Nachweispflicht

Jeder Befund führt verbindlich einen Nachweis gemäß ESS-0012-CONTRACTS Chapter 1.

Zulässig sind

FileReference, RegistryEntry, KnowledgeNode, TwinDelta, MatrixLink, VersionValue

Nicht zulässig ist ein Befund, der ausschließlich auf einer Vermutung beruht.

Findet ein KI-System keinen Nachweis, meldet es keinen Befund — es meldet die Nichtprüfbarkeit.

Reports

Zehn Berichte gemäß ESS-0012 Chapter 5:

Documentation Health · Governance · ESS Coverage · ADR Coverage · Traceability · Repository Health · Digital Twin · Knowledge Graph · Contract · Architecture Compliance

Ablage: docs/quality/

Die physische Erzeugung erfolgt durch die Documentary Engine. Der Validator liefert die Daten.

ETM-Integration

Der Validator **verwendet** die Traceability Matrix aus ESS-0011.

Er erzeugt sie nicht und verändert sie nicht.

Fehlt die Matrix, gilt GOV-TRACE-001 (Severity High) und der Traceability-Prüfbereich wird ausgesetzt — er wird niemals übersprungen ohne Befund.

Coverage-Schwellwerte und Orphan-Klassen stehen ausschließlich in ESS-0011-CONTRACTS.

Documentary-Integration

Der Validator ist Bestandteil der Documentary Engine (ESS-0010) und läuft **nach** ihr.

Er verwendet:

Knowledge Graph · Enterprise Registry · Digital Twin · erzeugte Dokumentation

Er schreibt in keine dieser Quellen zurück.

Versionierungsregeln

Der Version Manager leitet aus den Befunden ab:

Befundklasse	Versionsauswirkung
Breaking Change ohne ADR	blockiert Release
Contract-Widerspruch	Major
neue Regel oder Komponente	Minor
Dokumentationslücke	Patch

Der Validator empfiehlt keine Version. Er liefert die Befunde, aus denen der Version Manager sie ableitet.

Pflichten je KI-System

System	Pflicht
Claude Code	prüft Implementierung und Repository-Änderungen vor Übergabe an die Documentary Engine
Google AI Studio	prüft Entwürfe vor Übergabe an Claude Code
ChatGPT	prüft Architektur- und Governance-Vorschläge vor Aufnahme in ESS oder ADR
Future Enterprise AI	identische Pflicht nach Governance-Prüfung

Verbindlich für alle:

- ✗ keine eigenen Prüfregeln
 - ✗ keine Deaktivierung von Regeln
 - ✗ keine Änderung von Schweregraden
 - ✗ keine Prüfergebnisse ohne Ausführung
 - ✗ kein Schließen eines Befundes ohne Korrektur
 - ✗ keine Selbstfreigabe bei Critical-Befunden
-

Abbruchbedingungen

Der Prüflauf bricht ab bei

fehlendem Regelwerk

nicht lesbarem Repository

fehlendem Knowledge Graph (GOV-KG-001 , Critical)

fehlendem Digital Twin (GOV-TWIN-001 , Critical)

nicht reproduzierbarem Ergebnis

Jeder Abbruch erzeugt ein Fehler-Event und wird dem Supervisor gemeldet.

Aktueller Anwendungsstand

Dieser Skill ist **spezifiziert, nicht ausführbar**.

Die Ausführung setzt die Umsetzungsstufen 1 bis 4 aus `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md` voraus — insbesondere den Enterprise Event Bus und die Validator-Basisklasse.

Bis dahin wenden KI-Systeme die Regeln aus ESS-0012-CONTRACTS **manuell** an und dokumentieren das Ergebnis im jeweiligen Arbeitsergebnis.

Eine manuelle Anwendung ersetzt keine automatisierte Prüfung und ist als solche zu kennzeichnen.

Related Documents

- ESS-0012 — Documentation Governance
- ESS-0012-CONTRACTS — Documentation Governance Contracts
- ESS-0010 — Documentary Engine
- ESS-0011 — Enterprise Traceability
- ESS-0001-CONTRACTS — Chapter 8, Chapter 9, Chapter 12, Chapter 15, Chapter 18
- ADR-0014 — Documentation Governance Validator

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Fassung des Governance-Skills

End of Document

SKILL-GOV-0001

CAPITAL-AI Documentation Governance Validator

Version 1.0.0

Enterprise Event Mesh

Quelle: `.ai/skills/Enterprise-Event-Mesh.md`

skill: id: SKILL-EVT-0001 name: Enterprise Event Mesh version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority: High

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

classification: type: Operational Skill role: Handlungsanweisung für KI-Systeme zur Nutzung der Enterprise Event Mesh specification: ESS-0013 contractAuthority: ESS-0013-CONTRACTS note: > Dieser Skill ist kein ESS-Dokument und belegt keine ESS-Nummer. Er beschreibt ausschliesslich die Anwendung der in ESS-0013 spezifizierten Komponente sowie des in ESS-0001-CONTRACTS Chapter 8 definierten Regelwerks. Bei Abweichungen gelten Chapter 8, ESS-0013 und ESS-0013-CONTRACTS.

authority:

controls: - Event-Veröffentlichung - Event-Konsum - Event-Registrierung

cannot_modify: - Enterprise Specifications - Architecture Decision Records - Enterprise Contracts - fachliche Event-Inhalte anderer Komponenten - ESS-0001-CONTRACTS Chapter 8

crossReference: dependsOn: - ESS-0013 - ESS-0013-CONTRACTS relatedEss: - ESS-0001-CONTRACTS - ESS-0010 - ESS-0011 - ESS-0011-CONTRACTS - ESS-0012 - ESS-0012-CONTRACTS relatedAdr: - ADR-0018 relatedComponents: - src/platform/EventMesh relatedSkills: - `.ai/skills/ESS-0013-Enterprise-Event-Mesh.md` - `.ai/skills/ESS-0013-Contracts.md` - `.ai/skills/Documentation-Governance-Validator.md`

created: 2026-07-31

Enterprise Event Mesh

Zweck dieses Skills

Dieser Skill beschreibt, **wie** ein KI-System die Enterprise Event Mesh anwendet — als Producer, als Consumer, oder bei der Bewertung eines neuen Event-Vorschlags.

Er ersetzt keine Spezifikation. Die verbindlichen Regeln stehen in

ESS-0001-CONTRACTS Chapter 8 — Enterprise Event & Messaging Contracts (globales Regelwerk)

ESS-0013 — Komponentenspezifikation

ESS-0013-CONTRACTS — Katalog-, Kompatibilitäts-, Registry-, Routing-, Discovery-, Policy- und Report-Contracts

Bei jeder Abweichung zwischen diesem Skill und den genannten Dokumenten gelten die genannten Dokumente, in dieser Reihenfolge.

Grundregel

Die Enterprise Event Mesh vermittelt. Sie entscheidet nicht über fachliche Inhalte.

Ein KI-System, das eine neue Komponente oder Funktion unter `src/platform/` plant, prüft **vor** jeder direkten Abhängigkeit auf eine andere Fachkomponente, ob der Bedarf stattdessen über ein registriertes oder neu zu registrierendes Enterprise Event gedeckt werden kann.

Ein KI-System erzeugt niemals ein neues Event, ohne zuvor den bestehenden `EventCatalog` (bzw., solange dieser nicht implementiert ist: den Standard Event Catalog aus ESS-0013 und die `events`-Felder der betroffenen `manifest.json`-Dateien) auf ein bedeutungsgleiches Event zu prüfen.

Event Governance

Aufgabe	Beschreibung
Event-Namensregel prüfen	Suffix <code>Event</code> , Kategorie aus ESS-0013
Katalog-Kollision prüfen	ESS-0013-CONTRACTS Abschnitt 1, Event Catalog Contract
Contract-Vollständigkeit prüfen	Pflichtfelder aus Chapter 8 plus ETM-Referenz (ESS-0013-CONTRACTS Abschnitt 9)
Versionssprung bewerten	ESS-0013-CONTRACTS Abschnitt 4, Compatibility Contract
Breaking Change ohne ADR blockieren	verbindlich, keine Ausnahme

Die Governance über neue Event-**Typen** und -**Kategorien** obliegt dem Platform Director (Chapter 8, Platform Director Integration). Ein KI-System schlägt vor, es genehmigt nicht selbst.

Event Producer

Ein KI-System, das eine Komponente als Event-Producer implementiert:

1. deklariert das Event in der eigenen `manifest.json` unter `events.produces`

2. prüft gegen den Standard Event Catalog (ESS-0013) und die `events`-Felder aller anderen Komponenten, ob ein bedeutungsgleiches Event bereits existiert
3. veröffentlicht ausschließlich über `EventPublisher` (sobald implementiert) — niemals über einen direkten Aufruf einer Consumer-Komponente
4. füllt den Payload ausschließlich mit fachlicher Information (Chapter 8, Event Payload) — keine Implementierungsdetails, keine Geschäftslogik

Bekannte mögliche Producer (siehe ESS-0013, Abschnitt Producer- und Consumer-Erkennung): Documentary Engine, Traceability Engine, Knowledge Graph, Compliance Engine, Version Manager, Supervisor, Platform Director, Security, Repository Discovery.

Event Consumer

Ein KI-System, das eine Komponente als Event-Consumer implementiert:

1. deklariert das konsumierte Event in der eigenen `manifest.json` unter `events.consumes`
2. abonniert ausschließlich über `EventSubscriber` (sobald implementiert)
3. greift niemals direkt auf die Producer-Komponente zu, auch wenn ein direkter Import technisch möglich wäre

Bekannte mögliche Consumer: Documentary, Knowledge Graph, Traceability, Version Manager, Supervisor, Platform Director, Compliance, Security, Release.

Event Contracts

Jedes Event führt mindestens die in Chapter 8 verbindlichen Felder:

Event Name · Event ID · Version · Timestamp · Source Component · Target Component · Correlation ID · Payload · ESS-Referenzen · ADR-Referenzen

Zusätzlich verbindlich (ESS-0013): ETM-Referenz (mindestens ein Interface, mindestens eine Komponente).

Ein Event ohne vollständige Pflichtfelder wird von `EventContractValidator` als unvollständig zurückgewiesen und niemals mit Lücken registriert.

Routing

Routing erfolgt ausschließlich über `EventRouter`. Ein KI-System implementiert niemals einen zweiten, parallelen Zustellweg — auch nicht „vorübergehend“ oder „zur Vereinfachung“.

Ein Routing-Fehlschlag wird als `EventRoutingFailedEvent` gemeldet, niemals stillschweigend verworfen.

Trigger

Der Skill wird verbindlich angewendet bei

Auslöser	Umfang
Planung einer neuen Abhängigkeit zwischen zwei <code>src/platform/</code> -Komponenten	vollständig — Event-Alternative prüfen
Implementierung eines neuen Event-Producers	vollständig
Implementierung eines neuen Event-Consumers	vollständig
Vorschlag eines neuen Event-Typs	vollständig — Katalog-Kollisionsprüfung
Änderung eines bestehenden Event-Contracts	vollständig — Kompatibilitätsprüfung
vor jeder Release-Vorbereitung	vollständig — <code>EventHealthReport</code> prüfen

ETM-Integration

Die Enterprise Event Mesh **liefert** die Event-Achse der Enterprise Traceability Matrix (ESS-0011). Sie erzeugt keine neue Traceability-Achse und keine neue Verknüpfungsart.

Fehlt die ETM-Referenz eines Events, bleibt es im Status `proposed` — es wird niemals ohne Referenz als `registered` geführt.

Documentary-Integration

Die Documentary Engine (ESS-0010) dokumentiert automatisch jedes registrierte Event (Chapter 8, Documentary Integration). Die Enterprise Event Mesh liefert die Daten, die Documentary Engine erzeugt daraus Dokumentation. Die Mesh selbst erzeugt keine Dokumentation.

Versionierung

Änderung	Versionsauswirkung
neues Pflichtfeld im Payload	Major (Breaking Change, ADR erforderlich)
neues optionales Feld	Minor
Beschreibungs-/Metadatenkorrektur	Patch

Änderung	Versionsauswirkung
neues, kollisionsfreies Event	Minor (neue Registrierung)
Umbenennung eines bestehenden Events	nicht zulässig — Events werden niemals umbenannt, nur <code>deprecated</code> gesetzt und durch ein neues Event ersetzt

Der Version Manager (ESS-0004) leitet aus Event-Änderungen die Plattformversion ab. Die Mesh selbst trifft keine Versionsentscheidung für die Plattform, nur für den einzelnen Event Contract.

Pflichten je KI-System

System	Pflicht
Claude Code	prüft vor jeder neuen <code>src/platform/</code> -Abhängigkeit die Event-Alternative; deklariert Producer/Consumer in <code>manifest.json</code>
Google AI Studio	prüft Event-Vorschläge in Entwürfen gegen den Standard Event Catalog vor Übergabe an Claude Code
ChatGPT	prüft neue Event-Kategorien und -Typen auf Governance-Konformität vor Aufnahme in ESS oder ADR
Future Enterprise AI	identische Pflicht

Verbindlich für alle:

- ✗ kein direkter Import einer Fachkomponente durch eine andere Fachkomponente, wenn ein Event-basierter Weg möglich ist
- ✗ kein neues Event ohne Katalog-Kollisionsprüfung
- ✗ keine Umbenennung bestehender Events
- ✗ kein Breaking Change ohne ADR
- ✗ keine eigene Routing-Logik außerhalb `EventRouter`

Aktueller Anwendungsstand

Dieser Skill ist **spezifiziert, nicht ausführbar**.

Die Ausführung setzt die Umsetzungsstufen 1 bis 3 aus `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md` voraus — insbesondere den Enterprise Event Bus.

Bis dahin wenden KI-Systeme die Prinzipien aus Chapter 8 und ESS-0013 **manuell** an: Neue `src/platform/`-Komponenten deklarieren ihre beabsichtigten Events bereits jetzt in

`manifest.json` (`events.produces / events.consumes`), auch ohne lauffähigen Event Bus, damit `Discovery/` nach Implementierung ohne Nacherfassung arbeiten kann.

Related Documents

ESS-0013 — Enterprise Event Mesh

ESS-0013-CONTRACTS — Enterprise Event Mesh Contracts

ESS-0001-CONTRACTS — Chapter 8 (Regelwerk), Chapter 9, Chapter 11-19

ESS-0011 — Enterprise Traceability

ESS-0012-CONTRACTS — Governance-Validator-Events

ADR-0018 — Enterprise Event Mesh

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Fassung des Enterprise-Event-Mesh-Skills

End of Document

SKILL-EVT-0001

CAPITAL-AI Enterprise Event Mesh

Version 1.0.0

ESS-0001-CONTRACTS

Quelle: `.ai/skills/ESS-0001-Contracts.md`

ESS-0001-CONTRACTS

Enterprise Technical Contracts

Version

1.1.0

Status

Enterprise Baseline Specification

Dokumentklassifizierung

Dieses Dokument ist der **Master Enterprise Standard** des CAPITAL-AI Core.

Es ist die einzige globale Contract-Referenz der Plattform.

Komponentenspezifische Enterprise-Spezifikationen dürfen ausschließlich komponenten-spezifische Contracts definieren. Bei Konflikt gilt ausnahmslos dieses Dokument.

Cross Reference

Dieses Dokument besitzt kein YAML-Frontmatter. Die Referenzen werden deshalb als Markdown-Abschnitt geführt. Beide Formen sind gleichwertig.

Depends On

ESS-0001

Related ESS

ESS-0001 — Documentary & Code Intelligence Architect

ESS-0002 — Supervisor Architect

ESS-0003 — Platform Director

ESS-0010 — Documentary Engine

ESS-0011 — Enterprise Traceability

ESS-0011-CONTRACTS — Enterprise Traceability Matrix Contracts

Related ADR

ADR-0010 — Enterprise Standard Extension

ADR-0011 — Bestandsschutz Root-Abweichungen

ADR-0013 — ESS Documentation Responsibility Consolidation

Related Components

sämtliche 22 Module unter `src/platform/`

Related Skills

`.ai/skills/ESS-0001-Documentary-Architect.md`

`.ai/skills/ESS-0002-Supervisor-Architect.md`

`.ai/skills/ESS-0003-Platform-Director.md`

`.ai/skills/ESS-0010-Documentary-Engine.md`

`.ai/skills/ESS-0011-Enterprise-Traceability.md`

`.ai/skills/ESS-0011-Contracts.md`

Chapter 1

Enterprise Foundation & Governance Contracts

Enterprise Purpose

ESS-0001-CONTRACTS definiert die verbindlichen technischen Verträge für die Entwicklung des CAPITAL-AI Core.

Während ESS-0001 die Enterprise-Architektur beschreibt, legt ESS-0001-CONTRACTS die technischen Regeln fest, nach denen sämtliche Komponenten implementiert werden müssen.

Diese Spezifikation ist für alle KI-Systeme, Entwickler und automatisierten Build-Prozesse gleichermaßen verbindlich.

Mission

Alle Implementierungen des CAPITAL-AI Core müssen

- deterministisch
- reproduzierbar
- nachvollziehbar

- validierbar
- versionierbar
- testbar
- dokumentierbar

sein.

Kein KI-System darf eigenständig von diesen Verträgen abweichen.

Scope

Diese Spezifikation gilt für sämtliche Bestandteile des CAPITAL-AI Core.

Dazu gehören insbesondere

- Documentary Engine
 - Platform Director
 - Supervisor
 - Version Manager
 - Quality Center
 - Security Center
 - Compliance Center
 - AI Gateway
 - Orchestratoren
 - Agenten
 - Services
 - APIs
 - Datenbankmodelle
 - Build- und Deployment-Prozesse
-

Enterprise Authority

ESS-0001-CONTRACTS besitzt höchste Priorität für technische Implementierungen.

Die Rangfolge lautet

ADR

↓

ESS Contracts

↓

Enterprise Specifications (ESS)

↓

Projektdokumentation

↓

Implementierung

Bei Widersprüchen gilt immer die höher priorisierte Ebene.

AI Deterministic Development

Der CAPITAL-AI Core verfolgt den Grundsatz des deterministischen KI-gestützten Software-Engineerings.

Bei identischem Repository, identischen ESS-Dokumenten, identischen ADRs und identischen Contracts muss jede unterstützte KI denselben logischen Lösungsraum erzeugen.

Implementierungen dürfen sich in Details unterscheiden, jedoch nicht in

- Architektur
 - Komponentenstruktur
 - Verantwortlichkeiten
 - Schnittstellen
 - Ereignismodell
 - Datenmodellen
 - Versionsstrategie
-

Enterprise Design Principles

Alle Komponenten folgen den folgenden Grundprinzipien.

Repository First

Architecture First

Knowledge First

Event First

Version First

Validation First

Documentation First

Automation First

Security by Design

Compliance by Design

Quality by Design

AI Native by Design

Engineering Principles

Jede Implementierung erfüllt mindestens folgende Eigenschaften

Single Responsibility Principle

Open Closed Principle

Dependency Inversion Principle

Composition over Inheritance

Loose Coupling

High Cohesion

Deterministic Behaviour

Idempotent Operations

Observable Processes

Testability

Reusability

Extensibility

Governance Principles

Keine Architekturentscheidung ohne ADR.

Keine Implementierung ohne Contract.

Keine Änderung ohne Versionierung.

Keine Änderung ohne Dokumentation.

Keine Änderung ohne Event.

Keine Änderung ohne Knowledge Update.

Keine Produktionsänderung ohne Validierung.

Production Readiness

Jede Implementierung muss unmittelbar produktionsfähig sein.

Prototypen

temporäre Lösungen

experimenteller Code

Platzhalter

Mock-Implementierungen

TODO-Kommentare

dürfen nicht Bestandteil produktionsreifer Komponenten sein, sofern sie nicht ausdrücklich als Entwicklungsartefakte gekennzeichnet und durch den Platform Director freigegeben wurden.

AI Behaviour Rules

Unterstützte KI-Systeme dürfen

- ✓ vorhandenen Code analysieren
- ✓ bestehende Architektur erweitern
- ✓ Dokumentation erzeugen
- ✓ Tests erzeugen
- ✓ Migrationen vorbereiten
- ✓ Refactorings vorschlagen

Sie dürfen jedoch nicht

- ✗ bestehende Architekturprinzipien verändern
 - ✗ Contracts ignorieren
 - ✗ ADRs überschreiben
 - ✗ Versionen eigenständig erhöhen
 - ✗ Sicherheitsmechanismen entfernen
 - ✗ Governance-Regeln umgehen
-

Human Override

Architekturentscheidungen liegen ausschließlich bei autorisierten Projektverantwortlichen.

KI-Systeme besitzen beratende und implementierende Funktionen.

Sie besitzen keine Entscheidungsbefugnis über Enterprise-Governance.

Documentation Requirement

Jede technische Änderung muss mindestens folgende Artefakte aktualisieren oder erzeugen

- Dokumentation
 - Knowledge Graph
 - Versionsinformationen
 - Event Registry (falls betroffen)
 - Architekturreferenzen (falls betroffen)
 - Tests (falls erforderlich)
-

Traceability

Jede Änderung muss eindeutig nachvollziehbar sein.

Mindestens zu erfassen sind

- Ursprung
 - Änderungsgrund
 - Version
 - betroffene Komponenten
 - betroffene ADRs
 - betroffene Dokumentation
 - Zeitpunkt
 - verantwortliche Instanz (Mensch oder KI)
-

Quality Gates

Keine Änderung darf integriert werden, bevor mindestens geprüft wurde

- ✓ Contract-Konformität
- ✓ Architektur-Konformität

- ✓ Versionskonformität
 - ✓ Dokumentationsstatus
 - ✓ Teststatus
 - ✓ Sicherheitsauswirkungen
 - ✓ Compliance-Auswirkungen
 - ✓ Build-Ergebnis
-

Enterprise Compliance

Alle Komponenten müssen

auditierbar

reproduzierbar

deterministisch

erweiterbar

langfristig wartbar

und unabhängig von einem einzelnen KI-Modell implementierbar sein.

Success Criteria

Dieses Kapitel gilt als erfüllt, wenn

- ✓ alle nachfolgenden Contracts auf diesen Regeln aufbauen
 - ✓ sämtliche KI-Systeme dieselben Governance-Regeln verwenden
 - ✓ Architekturentscheidungen reproduzierbar bleiben
 - ✓ technische Entscheidungen nachvollziehbar dokumentiert werden
 - ✓ keine Implementierung außerhalb der definierten Contracts erfolgt
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 2

Repository Structure Contract



Chapter 3

Directory Contracts



Chapter 4

TypeScript & Interface Contracts



sämtliche weiteren Enterprise Contracts.

End of Chapter 1

Chapter 2

Repository Structure Contract

Enterprise Purpose

Dieses Kapitel definiert die verbindliche Repository-Struktur des CAPITAL-AI Core.

Die Repository-Struktur ist Bestandteil der Enterprise-Architektur.

Sie darf nicht eigenständig verändert oder erweitert werden.

Neue Verzeichnisse oder strukturelle Änderungen benötigen einen Architecture Decision Record (ADR).

Mission

Jede Datei besitzt einen eindeutig definierten Speicherort.

Jeder Ordner besitzt genau eine Verantwortung.

Jede KI erzeugt identische Projektstrukturen.

Die Repository-Struktur bleibt langfristig stabil und reproduzierbar.

Enterprise Principle

Die Struktur folgt dem Prinzip

Architecture before Implementation.

Die Architektur bestimmt den Speicherort.

Nicht die Implementierung.

Repository Root

Die oberste Ebene des Repositories besitzt ausschließlich folgende Verzeichnisse.

```
.ai/  
docs/  
scripts/  
src/  
supabase/  
tests/  
public/  
dist/
```

Weitere Root-Verzeichnisse dürfen ausschließlich über einen ADR eingeführt werden.

Root Responsibilities

.ai/

Enthält ausschließlich KI-bezogene Artefakte.

Beispiele

Skills

Prompts

Contracts

Templates

Schemas

Registries

Knowledge Seeds

docs/

Enterprise-Dokumentation

Architecture

ADR

Compliance

Security

Migration

Release

Quality

Knowledge

scripts/

Build

Migration

Deployment

Automation

Maintenance

Validation

src/

Gesamter Quellcode der Plattform.

Keine Dokumentation.

Keine Build-Artefakte.

Keine generierten Reports.

supabase/

Migrationen

Policies

Functions

RPC

Schema

Seed-Dateien

Storage-Konfiguration

tests/

Integration

Unit

Contract

Architecture

Performance

Security

End-to-End

public/

Statische Web-Ressourcen.

dist/

Build-Ausgabe.

Dieses Verzeichnis wird niemals manuell geändert.

Source Structure

Der gesamte Plattformcode befindet sich ausschließlich unter

```
src/
```

Platform Structure

Die Enterprise-Komponenten befinden sich ausschließlich unter

```
src/platform/
```

Die Platform-Ebene enthält ausschließlich zentrale Plattformdienste.

Mandatory Platform Modules

```
src/platform/  
Documentary/  
PlatformDirector/  
Supervisor/  
VersionManager/  
Knowledge/  
Architecture/  
Discovery/  
Registry/  
Events/  
Contracts/  
Models/  
Interfaces/  
Validators/  
Generators/  
Plugins/  
Telemetry/  
Quality/  
Security/  
Compliance/  
Release/  
Shared/
```

Diese Struktur ist verbindlich.

Documentary Structure

```
src/platform/Documentary/  
Engine/  
Discovery/  
Knowledge/  
Documentation/  
Architecture/  
Migration/  
Versioning/  
Events/
```

```
Registry/  
Generators/  
Validators/  
Templates/  
Mermaid/  
Plugins/  
Types/  
Models/  
Interfaces/  
Contracts/  
Utils/
```

Jedes Unterverzeichnis besitzt genau eine Verantwortung.

Shared Structure

Gemeinsam genutzte Komponenten befinden sich ausschließlich unter

```
src/platform/Shared/
```

Beispiele

Utilities

Logger

Errors

Configuration

Constants

Types

Base Classes

Helper

Feature Isolation

Geschäftslogik gehört niemals in

Shared

Platform

oder Documentary.

Sie wird ausschließlich in ihren jeweiligen Domänen implementiert.

Beispiele

```
src/features/  
crypto/  
stocks/  
portfolio/  
billing/  
news/  
users/  
settings/
```

Configuration

Konfigurationen befinden sich ausschließlich unter

```
src/config/
```

Keine Konfiguration innerhalb von Services.

Environment

```
.env.example  
.env.local  
.env.production
```

Secrets werden niemals im Repository gespeichert.

Naming Rules

Verzeichnisse verwenden ausschließlich PascalCase.

Beispiele

```
PlatformDirector/  
VersionManager/  
Knowledge/
```

```
Documentary/
```

Keine Leerzeichen.

Keine Sonderzeichen.

Keine Abkürzungen ohne Definition.

File Placement Rules

Jede Datei besitzt genau einen Speicherort.

Eine Datei darf niemals dieselbe Verantwortung wie eine andere Datei besitzen.

Keine doppelten Implementierungen.

Keine alternativen Versionen derselben Klasse.

Generated Files

Automatisch erzeugte Dateien werden ausschließlich innerhalb definierter Generator-Verzeichnisse gespeichert.

Beispiele

```
Generated/
```

```
Reports/
```

```
Snapshots/
```

Generierter Code darf produktiven Code niemals überschreiben.

Documentation Placement

Dokumentation befindet sich ausschließlich in

```
docs/
```

Ausnahmen

README.md

CHANGELOG.md

LICENSE

CODE_OF_CONDUCT.md

CONTRIBUTING.md

AI Resources

Alle KI-Artefakte befinden sich ausschließlich unter

```
.ai/
```

Beispiele

```
.ai/  
skills/  
prompts/  
contracts/  
templates/  
schemas/  
registry/  
knowledge/
```

Import Boundaries

Eine Komponente darf ausschließlich auf definierte Ebenen zugreifen.

Beispiel

```
Shared  
↓  
Contracts  
↓  
Models  
↓  
Services  
↓  
Generators  
↓  
Documentary  
↓  
Supervisor
```




Platform Director

Zirkuläre Abhängigkeiten sind nicht zulässig.

Repository Validation

Vor jedem Build wird geprüft

- ✓ Ordnerstruktur vollständig
 - ✓ keine unbekannten Root-Verzeichnisse
 - ✓ keine doppelten Komponenten
 - ✓ keine verbotenen Speicherorte
 - ✓ keine Architekturverletzungen
 - ✓ keine verwaisten Dateien
-

Enterprise Rules

Keine Datei außerhalb der definierten Struktur.

Keine spontanen Verzeichnisse.

Keine gemischten Verantwortlichkeiten.

Keine Dokumentation innerhalb produktiver Komponenten.

Keine Build-Artefakte im Quellcode.

Keine Testdateien außerhalb des Testbereichs.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ jede Datei eindeutig eingeordnet werden kann
- ✓ jede KI dieselbe Repository-Struktur erzeugt
- ✓ keine Architekturentscheidungen während der Implementierung getroffen werden müssen
- ✓ alle Komponenten eindeutig lokalisiert sind

✓ die Repository-Struktur langfristig stabil bleibt

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 3

Directory Contracts

↓

Chapter 4

Interface Contracts

↓

Chapter 5

Enterprise Naming Contracts

↓

sämtliche nachfolgenden technischen Contracts.

End of Chapter 2

Chapter 3

Directory Responsibility Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen Verantwortlichkeiten sämtlicher Verzeichnisse innerhalb des CAPITAL-AI Core.

Jedes Verzeichnis besitzt genau eine klar definierte Aufgabe.

Verantwortlichkeiten dürfen sich nicht überschneiden.

Jede Klasse, jedes Interface, jedes Datenmodell und jede Ressource besitzt einen eindeutig definierten Speicherort.

Mission

Die Directory Contracts gewährleisten

- eindeutige Verantwortlichkeiten
 - reproduzierbare Projektstrukturen
 - deterministische Codegenerierung
 - klare Abhängigkeitsregeln
 - langfristige Wartbarkeit
 - modulare Erweiterbarkeit
-

Enterprise Principle

Ein Verzeichnis besitzt genau eine Verantwortung.

Eine Verantwortung besitzt genau ein Verzeichnis.

Directory Contract Model

Jedes Verzeichnis definiert

- Zweck
 - erlaubte Inhalte
 - verbotene Inhalte
 - zulässige Abhängigkeiten
 - verbotene Abhängigkeiten
 - Eigentümer
 - Lebenszyklus
-

Core Directory

src/platform/Core

Purpose

Technische Basis der gesamten Plattform.

Contains

Base Classes

Core Interfaces

Core Types

Core Events

Core Contracts

Core Errors

Lifecycle

Logging

Telemetry

Utilities

Forbidden

Business Logic

Feature Code

API Endpoints

UI

Database Queries

Domain Services

Accessible by

Alle Plattformkomponenten.

Documentary Directory

src/platform/Documentary

Purpose

Codebasierte Dokumentation und Repository Intelligence.

Contains

Repository Discovery

Architecture Discovery

Knowledge Generation

Documentation Generation

Migration Analysis

Impact Analysis
Version Intelligence
Documentation Templates
Mermaid Generator
Markdown Generator
Registry Synchronisation

Forbidden

Business Logic
Frontend
REST APIs
Feature Implementierungen
Produktive Geschäftsprozesse

Knowledge Directory

src/platform/Knowledge

Purpose

Zentrale Wissensbasis der Plattform.

Contains

Knowledge Graph
Knowledge Objects
Relationships
Semantic Models
Knowledge Registry
Knowledge Validation

Forbidden

Dokumentengenerierung
Frontend
Business Services

Platform Director

src/platform/PlatformDirector

Purpose

Strategische Steuerung der gesamten Plattform.

Contains

Governance

Policy Engine

Enterprise Coordination

Decision Engine

Workflow Orchestration

Forbidden

Business Logic

Repository Analyse

Dokumentengenerierung

Supervisor

src/platform/Supervisor

Purpose

Operative Überwachung sämtlicher Plattformprozesse.

Contains

Health Monitoring

Lifecycle Management

Status Monitoring

Event Coordination

Alerting

Recovery

Forbidden

Architekturentscheidungen

Business Services

Feature Code

Version Manager

src/platform/VersionManager

Purpose

Versionierung der gesamten Plattform.

Contains

Semantic Versioning

Release Preparation

Version Registry

Version Validation

Impact Classification

Change Tracking

Forbidden

Business Logic

Feature Code

REST APIs

Registry

src/platform/Registry

Purpose

Zentrale Registrierung aller Plattformobjekte.

Contains

Component Registry

Service Registry

Plugin Registry

Generator Registry

Agent Registry

Workflow Registry

Template Registry

Forbidden

Business Services

Knowledge Generation

UI

Discovery

src/platform/Discovery

Purpose

Erkennung neuer Plattformkomponenten.

Contains

Repository Scanner

Dependency Scanner

Architecture Scanner

Module Scanner

Component Discovery

Forbidden

Dokumentengenerierung

Business Code

Events

src/platform/Events

Purpose

Enterprise Event Definitionen.

Contains

Event Definitions

Event Types

Event Registry

Event Contracts

Event Metadata

Forbidden

Business Logic

Repository Scan

Versioning

Contracts**src/platform/Contracts****Purpose**

Globale technische Verträge.

Contains

Interfaces

Contracts

Schemas

Validation Rules

Forbidden

Implementierungen

Business Logic

Models**src/platform/Models****Purpose**

Gemeinsame Enterprise Datenmodelle.

Contains

DTOs

Entities

Metadata

Shared Models

Forbidden

Services

Controller

Business Logic

Interfaces**src/platform/Interfaces****Purpose**

Globale Plattforminterfaces.

Contains

Interface Definitionen

Public Contracts

Shared Interfaces

Forbidden

Implementierungen

Validators**src/platform/Validators****Purpose**

Validierung sämtlicher Plattformobjekte.

Contains

Schema Validation

Contract Validation

Architecture Validation

Repository Validation

Knowledge Validation

Forbidden

Business Logic

Generators

src/platform/Generators

Purpose

Automatische Code- und Dokumentengenerierung.

Contains

Markdown Generator

ADR Generator

Mermaid Generator

Code Generator

Documentation Generator

Forbidden

Repository Discovery

Business Services

Plugins

src/platform/Plugins

Purpose

Erweiterungspunkte der Plattform.

Contains

Plugin Loader

Plugin Contracts

Plugin Registry

Plugin Metadata

Forbidden

Business Logik

Telemetry

src/platform/Telemetry

Purpose

Technische Metriken.

Contains

Metrics

Tracing

Performance

Observability

Health Metrics

Forbidden

Business Logic

Quality

src/platform/Quality

Purpose

Qualitätssicherung.

Contains

Quality Gates

Architecture Checks

Static Analysis

Coverage

Code Metrics

Forbidden

Business Logic

Security

src/platform/Security

Purpose

Technische Sicherheitsdienste.

Contains

Security Validation

Security Policies

Secrets Validation

IAM Integration

Risk Detection

Forbidden

Business Features

Compliance

src/platform/Compliance

Purpose

Compliance und Audit.

Contains

Audit Reports

Compliance Rules

Policy Validation

Risk Reports

Governance Reports

Forbidden

Business Logic

Release

src/platform/Release

Purpose

Release Management.

Contains

Release Planning

Deployment Reports

Release Validation

Rollback Planning

Forbidden

Repository Discovery

Knowledge Generation

Shared

src/platform/Shared

Purpose

Gemeinsam genutzte Infrastruktur.

Contains

Shared Utilities

Constants

Shared Types

Configuration Helpers

Forbidden

Business Logic

Feature Implementierungen

Directory Ownership

Jedes Verzeichnis besitzt genau einen fachlichen Eigentümer.

Änderungen an einem Verzeichnis dürfen ausschließlich innerhalb seiner definierten Verantwortung erfolgen.

Directory Expansion Rules

Neue Unterverzeichnisse dürfen ausschließlich entstehen wenn

- eine neue Verantwortung entsteht
 - keine bestehende Verantwortung erweitert werden kann
 - ein ADR die Erweiterung genehmigt
-

Directory Validation

Vor jeder Integration wird geprüft

- ✓ Verantwortlichkeit eindeutig
 - ✓ keine doppelte Verantwortung
 - ✓ keine Architekturverletzung
 - ✓ keine unerlaubten Inhalte
 - ✓ Contract-Konformität
 - ✓ Import-Konformität
-

Enterprise Rules

Keine Klasse außerhalb ihrer Verantwortung.

Keine Vermischung technischer und fachlicher Logik.

Keine mehrfachen Implementierungen derselben Verantwortung.

Keine Querverweise außerhalb der definierten Abhängigkeitsregeln.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ jede Datei einem eindeutigen Verzeichnis zugeordnet werden kann
 - ✓ keine Verantwortungsüberschneidungen existieren
 - ✓ sämtliche KI-Systeme identische Ablagestrukturen erzeugen
 - ✓ die Plattform langfristig modular erweiterbar bleibt
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 4

TypeScript & Interface Contracts

↓

Chapter 5

Enterprise Naming Contracts

↓

Chapter 6

Dependency Contracts

End of Chapter 3

Chapter 4

TypeScript & Interface Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen TypeScript-, Interface- und Implementierungsstandards des CAPITAL-AI Core.

Alle Enterprise-Komponenten verwenden dieselben technischen Regeln.

Dadurch wird sichergestellt, dass sämtliche KI-Systeme (Claude Code, Google AI Studio, ChatGPT und zukünftige Modelle) identische, reproduzierbare und langfristig wartbare Implementierungen erzeugen.

Dieses Kapitel bildet den technischen Standard des CAPITAL-AI Core.

Mission

Die TypeScript & Interface Contracts gewährleisten

- deterministische Codegenerierung
 - starke Typisierung
 - reproduzierbare Enterprise APIs
 - langfristige Wartbarkeit
 - modulare Erweiterbarkeit
 - maximale Testbarkeit
 - minimale Kopplung
 - hohe Wiederverwendbarkeit
 - vollständige Dokumentierbarkeit
 - automatische Architekturvalidierung
-

Enterprise Principle

Im CAPITAL-AI Core gilt grundsätzlich

Interfaces beschreiben Verhalten.

Klassen implementieren Verhalten.

Typen beschreiben Daten.

Contracts definieren Regeln.

Events beschreiben Kommunikation.

Modelle beschreiben Informationen.

Keine Implementierung existiert ohne Contract.

Keine öffentliche API existiert ohne Interface.

Keine Plattformkomponente existiert ohne Dokumentation.

TypeScript Standard

Der gesamte CAPITAL-AI Core verwendet ausschließlich

TypeScript

ES Modules

Node LTS

ECMAScript Latest

Strict Mode

Alle Komponenten besitzen dieselben Compiler- und Sprachregeln.

Compiler Rules

Folgende TypeScript-Regeln sind verpflichtend

strict

strictNullChecks

noImplicitAny

noImplicitReturns

noImplicitOverride

noUncheckedIndexedAccess

exactOptionalPropertyTypes

forceConsistentCasingInFileNames

useUnknownInCatchVariables

noFallthroughCasesInSwitch

noPropertyAccessFromIndexSignature

Language Rules

Erlaubt

interface

type

class

readonly

Generics

Union Types

Discriminated Unions

async / await

Record

Readonly

Partial

Required

Pick

Omit

Unknown

Template Literal Types

Mapped Types

Conditional Types

Nicht erlaubt

any

var

namespace

nicht typisierte Parameter

implizite Rückgabetypen

globale Variablen

Monkey Patching

nicht dokumentierte öffentliche Klassen

Interface Contracts

Alle öffentlichen Plattformkomponenten besitzen mindestens ein Interface.

Interfaces definieren ausschließlich den öffentlichen Vertrag.

Interfaces enthalten keine Implementierung.

Interfaces definieren ausschließlich

Methodensignaturen

Eigenschaften

Typdefinitionen

Vertragsbedingungen

Dokumentation

Beispiel

DocumentaryEngine

↓

IDocumentaryEngine

Supervisor

↓

ISupervisor

VersionManager

↓

IVersionManager

KnowledgeRegistry

↓

IKnowledgeRegistry

RepositoryScanner

↓

IRepositoryScanner

Public Interface Contract

Enterprise Purpose

Die Plattform unterscheidet zwischen

öffentlichen Enterprise Interfaces

und

internen Komponenteninterfaces.

Diese Trennung stellt sicher, dass langfristig stabile APIs entstehen und interne Implementierungsdetails niemals Bestandteil öffentlicher Verträge werden.

Public Enterprise Interfaces

Alle öffentlichen Plattforminterfaces befinden sich ausschließlich unter
`src/platform/Interfaces/`

Diese Interfaces bilden die offiziellen Enterprise Contracts.

Sie dürfen von sämtlichen Plattformkomponenten verwendet werden.

Sie definieren die öffentliche technische API des CAPITAL-AI Core.

Beispiele

`IPlatformComponent`

`IEngine`

`IService`

`IRegistry`

`IValidator`

`IGenerator`

`IProvider`

`IConsumer`

`IPlugin`

`ILifecycle`

`IHealthCheck`

`IEvent`

`IVersioned`

`IKnowledgeObject`

`IRepositoryObject`

Component Interfaces

Interne Komponenteninterfaces befinden sich ausschließlich innerhalb ihrer jeweiligen Komponente.

Beispiele

src/platform/Core/Interfaces/

src/platform/Documentary/Interfaces/

src/platform/Supervisor/Interfaces/

src/platform/VersionManager/Interfaces/

Diese Interfaces dürfen ausschließlich innerhalb ihrer eigenen Komponente verwendet werden.

Sie sind kein Bestandteil der öffentlichen Enterprise API.

Promotion Contract

Soll ein internes Interface zukünftig komponentenübergreifend verwendet werden, muss es

fachlich validiert werden

technisch dokumentiert werden

versioniert werden

einen Enterprise Contract erhalten

in

src/platform/Interfaces/

verschoben werden.

Vor Abschluss dieser Schritte ist eine komponentenübergreifende Verwendung nicht zulässig.

Documentation Contract

Jedes öffentliche Interface besitzt mindestens

Beschreibung

Verantwortlichkeit

Version

ESS-Referenz

ADR-Referenz (falls vorhanden)

Änderungsverlauf

Implementierende Komponenten

Verantwortlichen Owner

Validation

Vor jeder Integration wird automatisch geprüft

- ✓ öffentliche Interfaces befinden sich ausschließlich unter
src/platform/Interfaces/
 - ✓ interne Interfaces befinden sich ausschließlich innerhalb ihrer Komponente
 - ✓ keine Komponente verwendet interne Interfaces anderer Komponenten
 - ✓ alle öffentlichen Interfaces besitzen Dokumentation
 - ✓ alle öffentlichen Interfaces besitzen einen Enterprise Contract
 - ✓ alle öffentlichen Interfaces besitzen eine eindeutige Verantwortlichkeit
-

Enterprise Rules

Es darf niemals zwei öffentliche Interfaces mit derselben Verantwortung geben.

Komponenten dürfen ausschließlich öffentliche Enterprise Interfaces oder ihre eigenen internen Interfaces verwenden.

Direkte Abhängigkeiten auf interne Interfaces anderer Komponenten sind nicht zulässig.

Die Documentary Engine registriert sämtliche öffentlichen Interfaces automatisch.

Der Platform Director verwendet ausschließlich öffentliche Enterprise Interfaces zur komponentenübergreifenden Kommunikation.

Der Supervisor überwacht ausschließlich öffentliche Enterprise Interfaces.

End of Part 1

Base Interfaces

Enterprise Purpose

Die Base Interfaces definieren die technischen Grundverträge sämtlicher Plattformkomponenten.

Sie bilden die gemeinsame Sprache des CAPITAL-AI Core.

Jede Enterprise-Komponente implementiert mindestens ein Base Interface.

Dadurch entsteht eine konsistente, austauschbare und testbare Architektur.

Mandatory Base Interfaces

Der CAPITAL-AI Core definiert mindestens folgende Basisinterfaces

IPlatformComponent

IEngine

IService

IRegistry

IGenerator

IValidator

IProvider

IConsumer

IPlugin

ILifecycle

IHealthCheck

IEvent

IRepositoryObject

IKnowledgeObject

IVersioned

IConfiguration

IDiscovery

IAuditable

IMonitorable

Interface Responsibilities

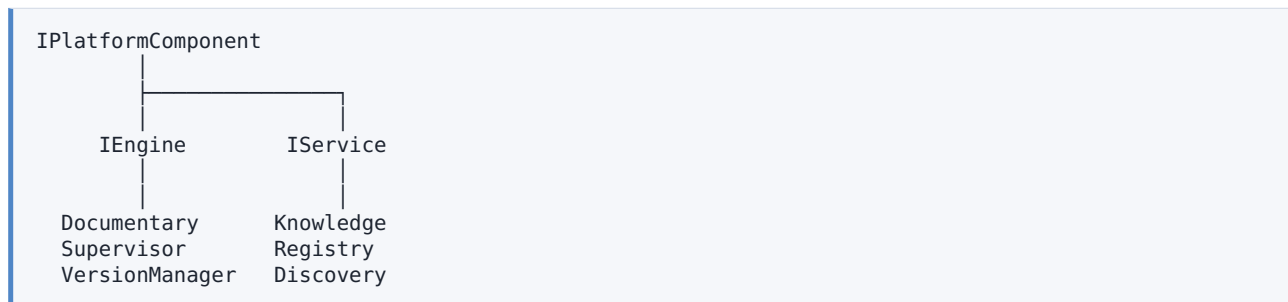
Jedes Base Interface besitzt genau eine technische Verantwortung.

Interfaces dürfen sich nicht überschneiden.

Neue Base Interfaces dürfen ausschließlich über einen ADR eingeführt werden.

Interface Hierarchy

Die technische Vererbung folgt ausschließlich der Enterprise-Hierarchie.



Mehrfachvererbungen sind ausschließlich bei fachlicher Notwendigkeit zulässig.

Class Contracts

Enterprise Purpose

Jede Klasse implementiert exakt definierte Verantwortlichkeiten.

Klassen enthalten ausschließlich Implementierungslogik.

Die öffentliche API wird vollständig über Interfaces beschrieben.

Rules

Jede Klasse

implementiert mindestens ein Interface

besitzt genau eine Verantwortung

ist vollständig typisiert

ist dokumentiert

ist testbar

besitzt einen definierten Lebenszyklus

Single Responsibility

Eine Klasse löst genau ein Problem.

Neue Verantwortlichkeiten führen zu einer neuen Klasse.

Public Classes

Öffentliche Klassen

werden dokumentiert

werden versioniert

werden automatisch registriert

werden automatisch durch die Documentary Engine erkannt

Internal Classes

Interne Klassen

dürfen ausschließlich innerhalb ihrer Komponente verwendet werden.

Sie sind kein Bestandteil der öffentlichen Plattform-API.

Constructor Contracts

Enterprise Purpose

Abhängigkeiten werden niemals innerhalb einer Klasse erzeugt.

Alle Abhängigkeiten werden von außen bereitgestellt.

Constructor Injection

Verpflichtend

Constructor Injection

Factory Injection

Provider Injection

Nicht erlaubt

new innerhalb produktiver Services

globale Instanzen

Singleton ohne ADR

Service Locator Pattern

versteckte Abhängigkeiten

Lifetime

Komponenten besitzen definierte Lebenszyklen

Transient

Scoped

Singleton (nur mit ADR)

Type Contracts

Enterprise Purpose

Alle Datenstrukturen besitzen eine eindeutige Typdefinition.

Inline-Objekte sind auf ein Minimum zu reduzieren.

Rules

Komplexe Daten

werden ausschließlich über

type

oder

interface

beschrieben.

Naming

Typen beschreiben ausschließlich Daten.

Beispiele

RepositoryMetadata

ComponentManifest

ArchitectureSnapshot

KnowledgeNode

VersionInformation

MigrationReport

ValidationResult

HealthStatus

Forbidden

Anonyme komplexe Objekte

Mehrfachdefinition identischer Typen

Typen mit Geschäftslogik

Generic Contracts

Enterprise Purpose

Generics erhöhen Wiederverwendbarkeit ohne Typverlust.

Mandatory Usage

Generics werden verwendet bei

Registries

Generatoren

Validatoren

Providern

Repositories

Factories

Discovery Services

Knowledge Services

Examples

Registry

Validator

Generator

Provider

Repository

Factory

Rules

Generics müssen

vollständig typisiert sein

einen fachlichen Nutzen besitzen

keine Lesbarkeit verschlechtern

Forbidden

Unnötige Generics

Generics ohne Typsicherheit

Generics mit any

Naming Rules

Enterprise Purpose

Einheitliche Benennung erhöht Verständlichkeit und automatische Dokumentierbarkeit.

Interfaces

Prefix

I

Beispiele

IRepositoryScanner

IPlatformComponent

IVersionManager

IDocumentaryEngine

Classes

PascalCase

Beispiele

RepositoryScanner

DocumentaryEngine

KnowledgeRegistry

PlatformDirector

Supervisor

VersionManager

Types

PascalCase

Substantive

Beispiele

ArchitectureSnapshot

ComponentManifest

KnowledgeNode

VersionReport

Enums

Nur wenn technisch erforderlich.

Ansonsten

Union Types

Files

Eine Hauptklasse pro Datei.

Dateiname entspricht Klassenname.

Export Rules

Public

Exportiert werden ausschließlich

öffentliche Klassen

Interfaces

Typdefinitionen

Factory Functions

Internal

Interne Hilfsklassen bleiben privat.

Hilfsfunktionen werden nicht exportiert.

Index Files

Öffentliche Komponenten dürfen Barrel Exports verwenden.

Interne Komponenten verzichten auf Barrel Exports.

File Contracts

Rules

Eine Datei besitzt

genau eine Hauptklasse

genau eine Hauptverantwortung

Documentation

Jede öffentliche Datei besitzt

Beschreibung

ESS-Referenz

ADR-Referenz

Version

Owner

Dependency Injection

Enterprise Purpose

Komponenten besitzen keine versteckten Abhängigkeiten.

Alle Abhängigkeiten werden explizit bereitgestellt.

Allowed

Constructor Injection

Factory Injection

Provider Injection

Forbidden

Globale Services

Service Locator

Statische Service Container

Direkte Initialisierung fremder Komponenten

Validation

Vor jeder Integration wird geprüft

- ✓ Interface vorhanden
- ✓ Klasse implementiert Interface
- ✓ Constructor Injection verwendet
- ✓ keine versteckten Abhängigkeiten
- ✓ Typisierung vollständig
- ✓ Dokumentation vollständig
- ✓ Naming Rules eingehalten
- ✓ Single Responsibility erfüllt
- ✓ öffentliche API dokumentiert

End of Part 2

Async Contracts

Enterprise Purpose

Alle asynchronen Prozesse des CAPITAL-AI Core folgen einem einheitlichen Ausführungsmodell.

Asynchrone Operationen müssen reproduzierbar, nachvollziehbar und testbar sein.

Rules

Asynchrone Methoden liefern ausschließlich

Promise

oder

Promise

zurück.

Callbacks sind nicht zulässig.

Error Propagation

Fehler dürfen niemals stillschweigend verworfen werden.

Alle Exceptions werden

- behandelt
 - protokolliert
 - typisiert
 - dokumentiert
-

Cancellation

Langlaufende Prozesse unterstützen nach Möglichkeit kontrollierte Abbrüche.

Hierfür sind standardisierte Mechanismen (z. B. AbortSignal) zu bevorzugen.

Timeouts

Alle externen Operationen besitzen definierte Timeout-Regeln.

Unbegrenzte Wartezeiten sind nicht zulässig.

Parallel Execution

Parallele Verarbeitung darf ausschließlich verwendet werden, wenn

- keine Race Conditions entstehen
 - Datenkonsistenz gewährleistet bleibt
 - Fehler eindeutig zugeordnet werden können
-

Validation

Vor jeder Integration wird geprüft

- ✓ Promise-basierte API
 - ✓ Fehlerbehandlung vorhanden
 - ✓ Timeout definiert
 - ✓ Logging vorhanden
 - ✓ Rückgabetyt vollständig typisiert
-

Error Contracts

Enterprise Purpose

Alle Fehler werden standardisiert behandelt.

Fehler sind Bestandteil der Enterprise-Architektur.

Rules

Es dürfen ausschließlich typisierte Error-Klassen verwendet werden.

Beispiele

RepositoryError

ValidationError

ConfigurationError

SecurityError

ContractViolationError

KnowledgeError

VersionError

DiscoveryError

MigrationError

Forbidden

throw "Error"

throw "String"

throw 123

throw {}

Error Hierarchy

Alle Fehler leiten sich von einer gemeinsamen Enterprise-Basisklasse ab.

Beispiel

EnterpriseError

↓

ValidationError

↓

ContractViolationError

Logging

Jeder Fehler wird protokolliert.

Mindestens

- Zeitpunkt
 - Komponente
 - Fehlerklasse
 - Ursache
 - Kontext
-

Validation

- ✓ typisierte Fehler
 - ✓ keine String-Exceptions
 - ✓ vollständiges Logging
 - ✓ Fehlerhierarchie eingehalten
-

Documentation Contracts

Enterprise Purpose

Jede öffentliche Komponente muss vollständig dokumentiert sein.

Dokumentation ist Bestandteil der Implementierung.

Public Classes

Jede öffentliche Klasse besitzt mindestens

- Beschreibung
 - Zweck
 - Verantwortlichkeit
 - ESS-Referenz
 - ADR-Referenz (falls vorhanden)
 - Version
 - Owner
-

Public Methods

Jede öffentliche Methode dokumentiert

- Zweck
 - Parameter
 - Rückgabewert
 - mögliche Exceptions
 - Nebenwirkungen
-

Documentation Format

Dokumentation erfolgt standardisiert.

Die Documentary Engine muss sämtliche Dokumentationen automatisch auswerten können.

Automatic Documentation

Alle öffentlichen Komponenten werden automatisch registriert.

Die Documentary Engine erzeugt daraus

- Architekturberichte
 - Komponentenübersichten
 - Dependency Graphs
 - Knowledge Graph
 - API-Dokumentation
-

Validation

- ✓ Dokumentation vorhanden
 - ✓ ESS-Referenz vorhanden
 - ✓ Version vorhanden
 - ✓ öffentliche Methoden dokumentiert
-

Testing Contracts

Enterprise Purpose

Jede öffentliche Komponente ist testbar.

Tests sind Bestandteil des Enterprise Contracts.

Mandatory Tests

Mindestens

Unit Test

Contract Test

Optional Tests

Integration Test

Performance Test

Architecture Test

Security Test

End-to-End Test

Regression Test

Test Rules

Tests müssen

deterministisch

automatisiert

isoliert

reproduzierbar

sein.

Coverage

Öffentliche Komponenten sollen vollständig durch Tests abgedeckt werden.

Die angestrebte Testabdeckung wird projektweit definiert.

Validation

✓ Test vorhanden

✓ Contract Test vorhanden

✓ Build erfolgreich

✓ Tests reproduzierbar

Import Contracts

Enterprise Purpose

Imports definieren die zulässigen Abhängigkeiten innerhalb des CAPITAL-AI Core.

Rules

Relative Imports

ausschließlich innerhalb derselben Komponente.

Komponentenübergreifend ausschließlich Alias Imports.

Examples

@platform/Core

@platform/Documentary

@platform/Supervisor

@platform/VersionManager

@platform/Knowledge

Forbidden

../..../

../..../..../

Zyklische Imports

Direkte Abhängigkeiten auf interne Komponenten

Dependency Direction

Die Architektur folgt einer eindeutigen Richtung.

Core

↓

Shared

↓

Knowledge

↓

Documentary

↓

Version Manager

↓

Supervisor

↓

Platform Director

Eine niedrigere Ebene darf niemals von einer höheren Ebene abhängen.

Import Validation

Vor jeder Integration wird geprüft

- ✓ keine zyklischen Imports
 - ✓ Alias Imports verwendet
 - ✓ Layer-Regeln eingehalten
 - ✓ keine verbotenen Abhängigkeiten
 - ✓ Architekturverletzungen ausgeschlossen
-

Enterprise Validation

Vor jeder Integration validiert die Plattform automatisch

- ✓ Compiler-Regeln
- ✓ TypeScript-Regeln
- ✓ Interface Contracts
- ✓ Public Interface Contracts
- ✓ Constructor Contracts
- ✓ Dependency Injection
- ✓ Async Contracts
- ✓ Error Contracts
- ✓ Documentation Contracts
- ✓ Testing Contracts
- ✓ Import Contracts
- ✓ ESS-Konformität
- ✓ ADR-Konformität

- ✓ Versionierung
 - ✓ Documentary-Kompatibilität
 - ✓ Supervisor-Kompatibilität
 - ✓ Platform-Director-Kompatibilität
-

End of Part 3

Chapter 5

Enterprise Naming Contracts

Enterprise Purpose

Dieses Kapitel definiert den verbindlichen Enterprise Naming Standard des CAPITAL-AI Core.

Alle Komponenten, Dateien, Klassen, Interfaces, Events, Registries, Dokumente und KI-Artefakte verwenden einheitliche Benennungsregeln.

Die Benennung ist Bestandteil der Enterprise-Architektur.

Abweichungen sind ausschließlich über einen Architecture Decision Record (ADR) zulässig.

Mission

Die Naming Contracts gewährleisten

- konsistente Repository-Strukturen
 - reproduzierbare Codegenerierung
 - deterministische KI-Ausgaben
 - eindeutige Verantwortlichkeiten
 - automatische Dokumentation
 - vereinfachte Architekturvalidierung
 - langfristige Wartbarkeit
-

Enterprise Principle

Ein Objekt besitzt genau einen Namen.

Ein Name beschreibt genau eine Verantwortung.

Mehrdeutige Bezeichnungen sind nicht zulässig.

Naming Language

Die gesamte technische Plattform verwendet ausschließlich

Englisch

für

- Quellcode
- Klassen
- Interfaces
- Dateien
- Verzeichnisse
- Events
- Typen
- APIs
- Konfigurationen

Dokumentationen dürfen zusätzlich deutschsprachige Inhalte enthalten.

General Naming Rules

Namen müssen

- eindeutig
- sprechend
- fachlich korrekt
- konsistent
- erweiterbar

sein.

Abkürzungen sind nur zulässig, wenn sie projektweit definiert wurden.

Directory Naming

Verzeichnisse verwenden ausschließlich

PascalCase

Beispiele

Core/

Documentary/

PlatformDirector/

VersionManager/

Knowledge/

Security/

Compliance/

Release/

File Naming

Dateinamen entsprechen dem Namen der Hauptklasse.

Beispiele

DocumentaryEngine.ts

KnowledgeRegistry.ts

VersionManager.ts

PlatformDirector.ts

Supervisor.ts

RepositoryScanner.ts

Interface Naming

Alle Interfaces besitzen den Präfix

I

Beispiele

IEngine

IService
IRegistry
IDocumentaryEngine
IKnowledgeRegistry
IVersionManager
IPlatformComponent

Class Naming

Klassen verwenden
PascalCase
Substantive
Beispiele
DocumentaryEngine
KnowledgeRegistry
ArchitectureScanner
Supervisor
PlatformDirector
VersionManager

Type Naming

Typdefinitionen beschreiben ausschließlich Daten.
Beispiele
ComponentManifest
ArchitectureSnapshot
RepositoryMetadata
VersionInformation
KnowledgeNode
HealthStatus

Event Naming

Events enden immer mit

Event

Beispiele

DocumentationGeneratedEvent

VersionReleasedEvent

RepositoryScannedEvent

KnowledgeUpdatedEvent

MigrationCompletedEvent

HealthCheckCompletedEvent

Registry Naming

Registries enden immer mit

Registry

Beispiele

KnowledgeRegistry

ComponentRegistry

EventRegistry

PluginRegistry

VersionRegistry

Generator Naming

Generatoren enden immer mit

Generator

Beispiele

MarkdownGenerator

ADRGenerator

DocumentationGenerator

MermaidGenerator

KnowledgeGenerator

Validator Naming

Validatoren enden immer mit

Validator

Beispiele

ArchitectureValidator

RepositoryValidator

KnowledgeValidator

SecurityValidator

ContractValidator

Provider Naming

Provider enden immer mit

Provider

Beispiele

ConfigurationProvider

KnowledgeProvider

VersionProvider

PluginProvider

Factory Naming

Factories enden immer mit

Factory

Beispiele

EngineFactory

GeneratorFactory

RegistryFactory

PluginFactory

Manager Naming

Manager koordinieren Prozesse.

Sie enden immer mit

Manager

Beispiele

VersionManager

LifecycleManager

ReleaseManager

PluginManager

Service Naming

Services implementieren fachliche oder technische Funktionen.

Beispiele

KnowledgeService

MigrationService

SecurityService

ReleaseService

Contract Naming

Contracts enden immer mit

Contract

Beispiele

InterfaceContract

RepositoryContract

DependencyContract

NamingContract

Documentation Naming

Enterprise-Dokumente folgen einem standardisierten Schema.

Beispiele

ESS-0001.md

ESS-0001-CONTRACTS.md

ADR-0005.md

README.md

CHANGELOG.md

ROADMAP.md

AI Artifact Naming

KI-Artefakte befinden sich ausschließlich unter

.ai/

Beispiele

Documentary-Migration-Architect.md

Platform-Director.md

Security-Auditor.md

Version-Manager.md

Configuration Naming

Konfigurationsdateien verwenden

kebab-case

Beispiele

tsconfig.json

package.json

eslint.config.js

vite.config.ts

Constant Naming

Konstanten verwenden

UPPER_SNAKE_CASE

Beispiele

DEFAULT_TIMEOUT

MAX_RETRY_COUNT

DEFAULT_LANGUAGE

Variable Naming

Variablen verwenden

camelCase

Beispiele

repositoryScanner

knowledgeRegistry

architectureSnapshot

Function Naming

Methoden beginnen mit einem Verb.

Beispiele

generateDocumentation()

scanRepository()

validateArchitecture()

loadKnowledge()

createSnapshot()

publishVersion()

Boolean Naming

Booleans beginnen mit

is

has

can

should

Beispiele

isHealthy

hasChanges

canGenerate

shouldPublish

Enumeration Naming

Enums verwenden PascalCase.

Enum-Werte verwenden PascalCase.

Union Types werden bevorzugt.

Forbidden

Nicht zulässig sind

Abkürzungen ohne Definition

mehrdeutige Namen

temporäre Namen

Class1

Test2

ManagerNew

HelperFinal

Misc

Utils2

Naming Validation

Vor jeder Integration wird geprüft

- ✓ Naming Standard eingehalten
 - ✓ Dateiname korrekt
 - ✓ Klassenname korrekt
 - ✓ Interface korrekt
 - ✓ Event korrekt
 - ✓ Registry korrekt
 - ✓ Dokument korrekt
 - ✓ AI-Artefakte korrekt
-

Enterprise Rules

Neue Komponenten müssen dem Naming Standard entsprechen.

Namensänderungen an öffentlichen Komponenten gelten als Breaking Change.

Breaking Changes benötigen einen ADR.

Die Documentary Engine validiert automatisch sämtliche Benennungen.

Der Supervisor überwacht Naming-Verletzungen.

Der Platform Director verwaltet zukünftige Erweiterungen des Naming Standards.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ sämtliche Komponenten konsistent benannt sind
 - ✓ alle KI-Systeme identische Namen erzeugen
 - ✓ automatische Dokumentation eindeutige Namen verwendet
 - ✓ keine Namenskonflikte existieren
 - ✓ Repository-Struktur und Naming vollständig synchron sind
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 6

Dependency & Layer Contracts

↓

Chapter 7

Data Model & Metadata Contracts

↓

Chapter 8

Event & Messaging Contracts

End of Chapter 5

Chapter 6

Dependency & Layer Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen Abhängigkeits- und Layer-Regeln des CAPITAL-AI Core.

Alle Komponenten kommunizieren ausschließlich über definierte Contracts und Interfaces.

Zyklische Abhängigkeiten, unkontrollierte Kopplungen und Architekturverletzungen sind nicht zulässig.

Die Layer Contracts bilden die technische Governance der gesamten Plattform.

Mission

Die Dependency & Layer Contracts gewährleisten

- eindeutige Architekturgrenzen
- lose Kopplung
- hohe Wartbarkeit

- deterministische Codegenerierung
 - kontrollierte Erweiterbarkeit
 - automatische Architekturvalidierung
 - reproduzierbare Enterprise-Strukturen
-

Enterprise Principle

Jede Abhängigkeit besitzt eine Richtung.

Eine Komponente kennt ausschließlich ihre erlaubten Nachbarn.

Architekturgrenzen dürfen niemals umgangen werden.

Layer Architecture

Der CAPITAL-AI Core folgt einer festen Layer-Hierarchie.

```
graph TD; PD[Platform Director] --- S[Supervisor]; S --- VM[Version Manager]; VM --- D[Documentary]; D --- K[Knowledge]; K --- DIS[Discovery]; DIS --- R[Registry]; R --- SH[Shared]; SH --- C[Core];
```

Abhängigkeiten verlaufen ausschließlich von oben nach unten.

Core Layer

Purpose

Der Core bildet das technische Fundament der Plattform.

Responsibilities

- Basisklassen
- Interfaces
- Events

- Errors
- Lifecycle
- Logging
- Telemetry
- Utilities

Allowed Dependencies

keine Plattformkomponenten

nur TypeScript Standardbibliothek

Shared Layer

Purpose

Gemeinsam genutzte Infrastruktur.

Allowed Dependencies

Core

Registry Layer

Purpose

Registrierung aller Plattformobjekte.

Allowed Dependencies

Core

Shared

Discovery Layer

Purpose

Analyse und Erkennung neuer Komponenten.

Allowed Dependencies

Core

Shared

Registry

Knowledge Layer

Purpose

Zentrale Wissensbasis.

Allowed Dependencies

Core

Shared

Registry

Discovery

Documentary Layer

Purpose

Automatische Dokumentation.

Allowed Dependencies

Core

Shared

Registry

Discovery

Knowledge

Version Manager Layer

Purpose

Versionierung und Release Management.

Allowed Dependencies

Core

Shared

Registry

Knowledge

Documentary

Supervisor Layer

Purpose

Monitoring und Governance.

Allowed Dependencies

alle darunterliegenden Layer

Platform Director Layer

Purpose

Strategische Orchestrierung der gesamten Plattform.

Allowed Dependencies

alle Plattformkomponenten

Dependency Rules

Eine Komponente darf ausschließlich

- ihre eigene Komponente
- niedrigere Layer
- öffentliche Enterprise Interfaces

verwenden.

Forbidden Dependencies

Nicht zulässig sind

- zyklische Abhängigkeiten
- gegenseitige Komponentenreferenzen
- direkte Zugriffe auf interne Klassen anderer Komponenten

- Umgehung definierter Layer
 - Import privater Komponenten
-

Communication Rules

Komponenten kommunizieren ausschließlich über

- öffentliche Interfaces
- Events
- Registries
- Contracts

Direkte Implementierungsabhängigkeiten sind zu vermeiden.

Event Communication

Asynchrone Kommunikation erfolgt ausschließlich über Enterprise Events.

Events dürfen keine Geschäftslogik enthalten.

Interface Communication

Komponentenübergreifende Kommunikation erfolgt ausschließlich über

`src/platform/Interfaces/`

Interne Interfaces dürfen nicht verwendet werden.

Plugin Dependencies

Plugins besitzen keine direkten Abhängigkeiten auf Plattformkomponenten.

Sie kommunizieren ausschließlich über

- Contracts
 - Interfaces
 - Events
-

Documentary Integration

Die Documentary Engine analysiert automatisch

- Abhängigkeiten
 - Layer
 - Architekturverletzungen
 - zyklische Referenzen
 - verbotene Imports
 - neue Komponenten
-

Supervisor Integration

Der Supervisor überwacht

- Dependency Violations
 - Layer Violations
 - Import Violations
 - Event Violations
-

Platform Director Integration

Der Platform Director entscheidet über

- neue Layer
- neue Komponenten
- Architekturänderungen
- Layer-Erweiterungen

Alle Änderungen benötigen einen ADR.

Version Manager Integration

Der Version Manager bewertet

- Breaking Changes
- Layer-Änderungen
- Interface-Änderungen
- Dependency-Änderungen

und empfiehlt automatisch

Major

Minor

Patch

Versionen.

AI Compatibility Contract

Alle KI-Systeme müssen diese Layer-Struktur unverändert einhalten.

Eigene Architekturentscheidungen sind nicht zulässig.

Neue Layer dürfen ausschließlich

- dokumentiert
- begründet
- per ADR freigegeben

werden.

Validation

Vor jeder Integration wird geprüft

- ✓ keine zyklischen Abhängigkeiten
 - ✓ Layer-Regeln eingehalten
 - ✓ ausschließlich erlaubte Imports
 - ✓ öffentliche Interfaces verwendet
 - ✓ Event Contracts eingehalten
 - ✓ Dependency Contracts erfüllt
 - ✓ Architecture Contracts erfüllt
 - ✓ Documentary kompatibel
 - ✓ Supervisor kompatibel
 - ✓ Platform Director kompatibel
 - ✓ Version Manager kompatibel
-

Enterprise Rules

Keine Komponente darf höhere Layer referenzieren.

Keine interne Implementierung anderer Komponenten darf direkt verwendet werden.

Alle Komponenten kommunizieren ausschließlich über definierte Enterprise Contracts.

Layer-Verletzungen gelten als Architekturfehler.

Breaking Changes der Layer-Struktur benötigen einen ADR.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ sämtliche Komponenten eindeutig einem Layer zugeordnet sind
 - ✓ keine Architekturverletzungen existieren
 - ✓ alle Abhängigkeiten deterministisch sind
 - ✓ Documentary sämtliche Layer automatisch analysieren kann
 - ✓ Supervisor Architekturverletzungen erkennt
 - ✓ Platform Director die gesamte Plattform orchestrieren kann
 - ✓ Version Manager Änderungen korrekt klassifizieren kann
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 7

Enterprise Data Model & Metadata Contracts

↓

Chapter 8

Event & Messaging Contracts

↓

Chapter 9

Versioning & Release Contracts

End of Chapter 6

Chapter 7

Enterprise Data Model & Metadata Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen Datenmodelle, Metadaten und semantischen Strukturen des CAPITAL-AI Core.

Jede Enterprise-Komponente besitzt standardisierte Metadaten.

Diese Metadaten bilden die Grundlage für

- Documentary Engine
- Knowledge Engine
- Platform Director
- Supervisor
- Version Manager
- Enterprise Registry
- AI Governance

Alle Komponenten werden dadurch selbstbeschreibend.

Mission

Die Data Model & Metadata Contracts gewährleisten

- eine einheitliche Datenstruktur
 - automatische Dokumentation
 - automatische Discovery
 - automatische Versionierung
 - Knowledge Graph Integration
 - Impact Analysen
 - Architekturtransparenz
 - AI-Kompatibilität
-

Enterprise Principle

Jede Komponente besitzt Metadaten.

Jede Metadatenstruktur besitzt einen Contract.

Alle Plattforminformationen sind maschinenlesbar.

Metadata Architecture

Jede Enterprise-Komponente besitzt mindestens

README.md

manifest.json

component.yaml

CHANGELOG.md

ADR.md

Optional

LICENSE

ROADMAP.md

TESTING.md

Enterprise Metadata

Jede Komponente beschreibt mindestens

Name

Beschreibung

Version

Status

Owner

Kategorie

ESS Referenzen

ADR Referenzen

Abhängigkeiten

Events

Interfaces

Contracts

Lifecycle

Health

Quality

Security

Dokumentation

Manifest Contract

manifest.json dient als primäre maschinenlesbare Beschreibung.

Pflichtfelder

name

version

status

owner

description

category

dependencies

interfaces

contracts

events

knowledge

documentation

Component Contract

component.yaml dient als

menschenlesbare

KI-lesbare

Enterprise-Komponentenbeschreibung.

Die YAML-Datei besitzt denselben Informationsumfang wie manifest.json.

README Contract

README.md beschreibt

Zweck

Verantwortung

Architektur

Abhängigkeiten

Integration

Beispiele

ESS Referenzen

CHANGELOG Contract

Alle Änderungen werden dokumentiert.

Mindestens

Version

Datum

Beschreibung

Breaking Changes

Autor

ADR Contract

Jede Komponente verweist auf alle relevanten ADRs.

Breaking Changes müssen dokumentiert werden.

Metadata Ownership

Jede Komponente besitzt genau einen fachlichen Owner.

Der Owner ist verantwortlich für

Qualität

Dokumentation

Versionierung

ADR Pflege

Enterprise Registry

Alle Komponenten werden automatisch registriert.

Die Registry enthält mindestens

Komponentenname

Version

Kategorie

Owner

Status

Layer

ESS Referenzen

ADR Referenzen

Abhängigkeiten

Health Status

Knowledge Graph Contract

Alle Komponenten werden automatisch Teil des Knowledge Graph.

Knoten

Komponenten

Dokumente

Interfaces

Events

ADRs

ESS

Repositories
Edges
verwendet
implementiert
erzeugt
abhängig von
dokumentiert
versioniert
überwacht

Metadata Synchronisation

Die Documentary Engine synchronisiert automatisch

README
manifest.json
component.yaml
Registry
Knowledge Graph

Lifecycle Contract

Jede Komponente besitzt einen Lifecycle.

Mindestens
Development
Experimental
Beta
Stable
Deprecated
Archived
Retired

Health Contract

Jede Komponente besitzt einen Health Status.

Beispiele

Healthy

Warning

Critical

Deprecated

Unknown

Quality Contract

Qualitätsmetriken

Dokumentation

Testabdeckung

Codequalität

Architecture Score

Security Score

Documentation Score

Knowledge Score

Security Metadata

Jede Komponente beschreibt

Security Classification

Sensitivity

Compliance

Audit Status

Encryption

AI Metadata

Jede Komponente beschreibt zusätzlich

AI Owner

AI Generated

AI Reviewed

AI Validated

AI Compatible

Supported AI Systems

Claude Code

Google AI Studio

ChatGPT

Future Enterprise AI

Repository Contract

Alle Komponenten besitzen eine eindeutige Repository-Zuordnung.

Beispiele

Finance

Finance-Dev

Platform

Shared

Semantic Version Contract

Metadaten besitzen dieselbe Version wie die Komponente.

Versionen müssen synchron bleiben.

Automatic Discovery

Neue Komponenten werden automatisch erkannt.

Die Documentary Engine erzeugt automatisch

README

Registry

Knowledge Nodes

Metadata

Architecture Reports

Validation

Vor jeder Integration wird geprüft

- ✓ manifest.json vorhanden
 - ✓ component.yaml vorhanden
 - ✓ README vorhanden
 - ✓ CHANGELOG vorhanden
 - ✓ ADR Referenzen vorhanden
 - ✓ Owner definiert
 - ✓ Version vorhanden
 - ✓ ESS Referenzen vorhanden
 - ✓ Layer definiert
 - ✓ Health Status vorhanden
 - ✓ Security Metadata vorhanden
 - ✓ Quality Metadata vorhanden
 - ✓ AI Metadata vorhanden
-

Enterprise Rules

Keine Enterprise-Komponente darf ohne Metadaten existieren.

Metadaten gelten als Bestandteil des Quellcodes.

Alle Änderungen an Metadaten werden versioniert.

Die Documentary Engine ist die führende Instanz für Metadaten.

Der Knowledge Graph wird ausschließlich aus validierten Metadaten aufgebaut.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ sämtliche Komponenten vollständig beschrieben sind
 - ✓ sämtliche Metadaten synchron sind
 - ✓ automatische Discovery funktioniert
 - ✓ Knowledge Graph automatisch erzeugt werden kann
 - ✓ Documentary sämtliche Komponenten dokumentieren kann
 - ✓ Platform Director sämtliche Komponenten verwalten kann
 - ✓ Supervisor sämtliche Komponenten überwachen kann
 - ✓ Version Manager sämtliche Änderungen bewerten kann
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 8

Enterprise Event & Messaging Contracts

↓

Chapter 9

Enterprise Versioning & Release Contracts

↓

Chapter 10

Enterprise AI Governance & Documentary Contracts

End of Chapter 7

Chapter 8

Enterprise Event & Messaging Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindliche Event- und Messaging-Architektur des CAPITAL-AI Core.

Alle Plattformkomponenten kommunizieren ausschließlich über standardisierte Enterprise Events oder öffentliche Enterprise Interfaces.

Das Enterprise Event Mesh bildet die zentrale Kommunikationsschicht zwischen allen Komponenten.

Es gewährleistet Nachvollziehbarkeit, Erweiterbarkeit und vollständige Automatisierung der Plattform.

Mission

Die Enterprise Event & Messaging Contracts gewährleisten

- lose Kopplung
 - ereignisgesteuerte Architektur
 - automatische Dokumentation
 - automatische Versionierung
 - vollständige Auditierbarkeit
 - AI-gesteuerte Automatisierung
 - deterministische Kommunikation
-

Enterprise Principle

Jede relevante Änderung erzeugt ein Event.

Jedes Event besitzt einen Contract.

Jedes Event ist versioniert.

Jedes Event ist dokumentiert.

Jedes Event ist nachvollziehbar.

Enterprise Event Mesh

Alle Plattformkomponenten kommunizieren über das Enterprise Event Mesh.

Komponenten dürfen keine versteckten Seiteneffekte erzeugen.

Alle relevanten Zustandsänderungen werden als Event veröffentlicht.

Event Architecture

Ein Event beschreibt ausschließlich

eine bereits eingetretene Änderung.

Events enthalten keine Geschäftslogik.

Events enthalten ausschließlich Informationen.

Event Naming

Alle Events enden mit

Event

Beispiele

RepositoryScannedEvent

DocumentationGeneratedEvent

KnowledgeUpdatedEvent

ArchitectureValidatedEvent

ComponentRegisteredEvent

VersionCalculatedEvent

ReleaseCreatedEvent

SecurityAuditCompletedEvent

SupervisorAlertEvent

PlatformDecisionEvent

Event Categories

Repository Events

RepositoryCreatedEvent

RepositoryUpdatedEvent

RepositoryScannedEvent

RepositoryValidatedEvent

Documentary Events

DocumentationGeneratedEvent

DocumentationUpdatedEvent

DocumentationValidatedEvent

KnowledgeExtractedEvent

Knowledge Events

KnowledgeCreatedEvent

KnowledgeUpdatedEvent

KnowledgeLinkedEvent

KnowledgeValidatedEvent

Version Events

VersionCalculatedEvent

VersionChangedEvent

ReleasePreparedEvent

ReleasePublishedEvent

Architecture Events

ArchitectureScannedEvent

ArchitectureValidatedEvent

DependencyViolationEvent

LayerViolationEvent

ContractViolationEvent

Security Events

SecurityScanCompletedEvent

ComplianceValidatedEvent

RiskDetectedEvent

AuditCompletedEvent

Platform Events

PlatformStartedEvent

PlatformStoppedEvent

PlatformDecisionEvent

PlatformHealthChangedEvent

Event Contract

Jedes Event besitzt mindestens

Name

Version

Timestamp

Source Component

Target Component

Correlation ID

Event Type

Payload

Schema Version

ESS Referenzen

ADR Referenzen

Event Payload

Der Payload enthält ausschließlich

fachliche Informationen.

Keine technischen Implementierungsdetails.

Keine Geschäftslogik.

Correlation Contract

Jedes Event besitzt eine eindeutige Correlation ID.

Alle zusammengehörigen Events können dadurch nachvollzogen werden.

Event Versioning

Alle Events besitzen eine Version.

Breaking Changes erfordern

Major Version

Neue optionale Felder

Minor Version

Fehlerkorrekturen

Patch Version

Event Registry

Alle Enterprise Events werden automatisch registriert.

Die Registry enthält

Name

Version

Beschreibung

Producer

Consumer

Payload Schema

ESS Referenzen

ADR Referenzen

Event Producers

Jede Plattformkomponente darf Events veröffentlichen.

Alle veröffentlichten Events müssen dokumentiert werden.

Event Consumers

Komponenten abonnieren ausschließlich dokumentierte Enterprise Events.

Direkte Implementierungsabhängigkeiten sind nicht zulässig.

Event Routing

Das Enterprise Event Mesh entscheidet automatisch

welche Komponenten

welche Events

empfangen.

Documentary Integration

Die Documentary Engine reagiert automatisch auf

RepositoryScannedEvent

ArchitectureValidatedEvent

DocumentationGeneratedEvent

VersionChangedEvent

ReleasePublishedEvent

ContractViolationEvent

Supervisor Integration

Der Supervisor überwacht

Event-Ausfälle

Event-Verzögerungen

Event-Fehler

Event-Ketten

unerwartete Event-Muster

Platform Director Integration

Der Platform Director bewertet

neue Event-Typen

neue Event-Kategorien

Event-Abhängigkeiten

Event-Governance

Version Manager Integration

Der Version Manager analysiert automatisch

Breaking Events

Interface Events

Release Events

Migration Events

und empfiehlt daraus

Major

Minor

Patch

Versionen.

AI Trigger Contracts

Jedes Enterprise Event kann einen AI-Prozess auslösen.

Beispiele

DocumentationGeneratedEvent

↓

Documentary Engine

ArchitectureValidatedEvent

↓

Supervisor

ReleasePreparedEvent

↓

Version Manager

KnowledgeUpdatedEvent

↓

Knowledge Engine

SecurityScanCompletedEvent

↓

Security Auditor

PlatformDecisionEvent

↓

Platform Director

Enterprise Automation

Die Plattform darf auf Basis validierter Events automatisch

Dokumentationen erzeugen

Versionen berechnen

Architekturdiagramme aktualisieren

Knowledge Graphs erweitern

Release Notes erzeugen

Repositories analysieren

Risiken erkennen

Governance Reports erstellen

Event Audit Trail

Alle Events werden revisionssicher protokolliert.

Mindestens

Timestamp

Komponente

Event

Version

Correlation ID

Status

Ergebnis

Validation

Vor jeder Integration wird geprüft

- ✓ Event dokumentiert
 - ✓ Event registriert
 - ✓ Version vorhanden
 - ✓ Payload vollständig
 - ✓ Correlation ID vorhanden
 - ✓ Producer definiert
 - ✓ Consumer definiert
 - ✓ ESS Referenzen vorhanden
 - ✓ ADR Referenzen vorhanden
 - ✓ Event Mesh kompatibel
-

Enterprise Rules

Keine Plattformkomponente kommuniziert direkt an der Event-Governance vorbei.

Alle relevanten Zustandsänderungen erzeugen Enterprise Events.

Alle Events werden automatisch durch die Documentary Engine dokumentiert.

Der Supervisor überwacht sämtliche Event-Flüsse.

Der Platform Director verwaltet die Event-Governance.

Der Version Manager bewertet Event-Auswirkungen auf die Plattformversion.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ sämtliche Komponenten ausschließlich über Enterprise Events oder öffentliche Interfaces kommunizieren
 - ✓ alle Events versioniert sind
 - ✓ sämtliche Events dokumentiert werden
 - ✓ das Enterprise Event Mesh alle Event-Flüsse verwaltet
 - ✓ Documentary automatisch auf relevante Events reagiert
 - ✓ Supervisor Event-Verletzungen erkennt
 - ✓ Platform Director die Event-Governance steuert
 - ✓ Version Manager Event-basierte Versionsentscheidungen treffen kann
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 9

Enterprise Versioning & Release Contracts

↓

Chapter 10

Enterprise AI Governance & Documentary Contracts

↓

Chapter 11

Enterprise Security & Compliance Contracts

End of Chapter 8

Chapter 9

Enterprise Versioning & Release Contracts

Enterprise Purpose

Dieses Kapitel definiert den verbindlichen Versionierungs-, Release- und Lifecycle-Standard des CAPITAL-AI Core.

Versionierung ist Bestandteil der Enterprise Governance.

Jede Änderung der Plattform muss automatisch bewertet, dokumentiert, versioniert und nachvollziehbar archiviert werden.

Die Versionierung bildet die Grundlage für reproduzierbare Releases, automatische Dokumentation und KI-gestützte Architekturentscheidungen.

Mission

Die Enterprise Versioning & Release Contracts gewährleisten

- reproduzierbare Releases
 - nachvollziehbare Änderungen
 - automatische Versionsbewertung
 - konsistente Release-Prozesse
 - automatische Dokumentation
 - vollständige Auditierbarkeit
 - AI-gestützte Release-Empfehlungen
-

Enterprise Principle

Keine Änderung erfolgt ohne Version.

Keine Version erfolgt ohne Dokumentation.

Keine Dokumentation erfolgt ohne Contract.

Keine Freigabe erfolgt ohne Validierung.

Enterprise Versioning Model

Die Plattform verwendet ausschließlich

Semantic Versioning

MAJOR.MINOR.PATCH

Beispiel

1.0.0

Version Categories

Major

Breaking Changes

Architekturänderungen

Interfaceänderungen

Contractänderungen

Repositorystruktur

Layeränderungen

Minor

Neue Komponenten

Neue Features

Neue Services

Neue Events

Neue Dokumentationen

Neue AI Skills

Patch

Bugfixes

Performance

Dokumentationskorrekturen

Testverbesserungen

Security Fixes

Refactorings ohne API Änderung

Enterprise Release Lifecycle

Development

↓

Validation

↓

Architecture Review

↓

Documentary Review

↓

Supervisor Review

↓

Compliance Review

↓

Release Candidate

↓

Production Release

↓

Post Release Validation

Automatic Version Analysis

Vor jeder Version analysiert der Version Manager automatisch

Quellcode

Repositorystruktur

Contracts

Interfaces

Events

Dokumentation

Metadata

ADRs

ESS Dokumente

Knowledge Graph

Version Decision Engine

Die Version Manager Engine bewertet automatisch

Breaking Changes

Neue Features

Entfernte Features

Veränderte Contracts

Neue Interfaces

Geänderte Events

Repositoryänderungen

Layeränderungen

Security Änderungen

Compliance Änderungen

Documentary Trigger

Jede relevante Änderung erzeugt automatisch einen Documentary Trigger.

Beispiele

Neue Klasse

↓

Dokumentation aktualisieren

Neue Komponente



Registry aktualisieren

Interface geändert



Impact Analyse

ADR erstellt



Architecture Report

Version geändert



Release Notes

Supervisor Trigger

Der Supervisor bewertet automatisch

Architekturverletzungen

Versionskonflikte

fehlende Dokumentation

fehlende Tests

fehlende Contracts

Platform Director Trigger

Der Platform Director bewertet

strategische Auswirkungen

Layeränderungen

Governance Änderungen

Repository Erweiterungen

AI Architektur

AI Version Trigger

Alle unterstützten KI-Systeme müssen Versionierungsereignisse berücksichtigen.

Claude Code

Google AI Studio

ChatGPT

Future Enterprise AI

dürfen keine Änderungen erzeugen,
ohne die Versionierungsregeln einzuhalten.

Release Documentation

Jede Version besitzt mindestens

Versionsnummer

Datum

Release Typ

Zusammenfassung

Breaking Changes

Neue Features

Bugfixes

Migration Hinweise

ESS Referenzen

ADR Referenzen

Changelog Contract

Alle Änderungen werden automatisch

klassifiziert

versioniert

dokumentiert

archiviert

Component Versioning

Jede Plattformkomponente besitzt eine eigene Version.

Komponentenversion

muss

mit

manifest.json

component.yaml

README

CHANGELOG

synchron sein.

Repository Versioning

Das Repository besitzt zusätzlich

eine globale Plattformversion.

Diese beschreibt den Gesamtzustand der Plattform.

Version Registry

Alle Versionen werden automatisch registriert.

Die Registry enthält mindestens

Version

Datum

Autor

Release Typ

Änderungen

ESS

ADR

Commit

Repository

AI Documentation Contract

Nach jeder Versionsänderung erzeugt die Documentary Engine automatisch

Release Notes

Architecture Report

Knowledge Update

Registry Update

Version Report

Dependency Report

Impact Report

Git Integration

Vor jedem Release wird automatisch geprüft

Git Status

Commit Historie

Branch

Tags

Repository Konsistenz

Release Validation

Vor jeder Freigabe wird geprüft

✓ Build erfolgreich

✓ Tests erfolgreich

✓ Contracts erfüllt

✓ Interfaces validiert

✓ Dokumentation vollständig

✓ Changelog aktuell

✓ Metadata synchron

✓ ESS aktuell

- ✓ ADR aktuell
 - ✓ Security Prüfung bestanden
 - ✓ Compliance Prüfung bestanden
-

Rollback Contract

Jede veröffentlichte Version muss reproduzierbar wiederhergestellt werden können.

Rollback Informationen werden automatisch dokumentiert.

Audit Contract

Jede Version besitzt

vollständige Nachvollziehbarkeit.

Alle Entscheidungen werden revisionssicher dokumentiert.

Enterprise Rules

Versionen dürfen niemals manuell erhöht werden.

Die Version Manager Engine entscheidet anhand der Contracts über

Major

Minor

Patch

Versionen.

Alle Versionsentscheidungen werden dokumentiert.

Alle Releases werden automatisch durch die Documentary Engine begleitet.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ jede Änderung automatisch bewertet wird
- ✓ Versionen konsistent erzeugt werden
- ✓ Release Notes automatisch erstellt werden

- ✓ Documentary automatisch aktualisiert wird
 - ✓ Supervisor Versionsverletzungen erkennt
 - ✓ Platform Director Auswirkungen bewertet
 - ✓ Version Manager den gesamten Versionsprozess steuert
 - ✓ alle Releases reproduzierbar sind
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 10

Enterprise AI Governance & Documentary Contracts

↓

Chapter 11

Enterprise Security & Compliance Contracts

↓

Chapter 12

Enterprise Validation & Quality Contracts

End of Chapter 9

Chapter 10

Enterprise AI Governance & Documentary Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen Governance-Regeln für sämtliche KI-Systeme innerhalb des CAPITAL-AI Core.

Die AI Governance stellt sicher, dass alle KI-Systeme reproduzierbar, nachvollziehbar und kontrolliert arbeiten.

Die Documentary Engine bildet dabei die zentrale Wissens-, Dokumentations- und Governance-Instanz der Plattform.

Alle KI-gestützten Änderungen unterliegen den Enterprise Contracts.

Mission

Die AI Governance & Documentary Contracts gewährleisten

- reproduzierbare KI-Entwicklung
 - vollständige Nachvollziehbarkeit
 - automatische Dokumentation
 - kontrollierte Architekturentwicklung
 - deterministische Codegenerierung
 - einheitliche Enterprise Standards
 - langfristige Wartbarkeit
-

Enterprise Principle

Keine KI erzeugt Architektur.

Die Architektur erzeugt die KI.

Die KI implementiert ausschließlich definierte Enterprise Contracts.

Jede KI-Aktion muss nachvollziehbar sein.

Jede KI-Aktion erzeugt Dokumentation.

AI Governance Model

Alle KI-Systeme arbeiten ausschließlich innerhalb der definierten Enterprise Governance.

Eigene Architekturentscheidungen sind nicht zulässig.

Alle Änderungen erfolgen auf Basis

ESS

ADR

Contracts

Repository Standards

Versioning Rules

Enterprise Policies

Supported AI Systems

Die Plattform unterstützt

Claude Code

Google AI Studio

ChatGPT

Future Enterprise AI

Alle zukünftigen KI-Systeme müssen dieselben Enterprise Contracts erfüllen.

AI Responsibility Contract

Jedes KI-System besitzt klar definierte Verantwortlichkeiten.

Claude Code

Verantwortlich für

Backend

Implementierung

Refactoring

Migration

Repository Änderungen

Tests

Produktionsnahe Codeänderungen

Google AI Studio

Verantwortlich für

Frontend

UI

UX

Rapid Development

Entwicklungsumgebung

Prototyping

Dokumentationsunterstützung

ChatGPT

Verantwortlich für

Enterprise Architektur

ESS

ADR

Governance

Security Reviews

Compliance

Repository Standards

Documentary Entwicklung

Strategische Architektur

Future Enterprise AI

Neue KI-Systeme dürfen ausschließlich nach erfolgreicher Governance-Prüfung integriert werden.

Documentary Engine Contract

Die Documentary Engine ist die führende Wissensquelle der Plattform.

Sie besitzt die Verantwortung für

Dokumentation

Knowledge Graph

Repository Analyse

Contract Analyse

Architecture Reports

Version Reports

Release Notes

Dependency Reports

Impact Analysen

Governance Reports

Documentary Trigger Contract

Die Documentary Engine wird automatisch ausgelöst bei

neuen Klassen

neuen Komponenten

Interface Änderungen

Contract Änderungen

ADR Änderungen

ESS Änderungen

Repository Änderungen

Version Änderungen

Release Änderungen

Migrationen

Security Reviews

Compliance Reviews

AI Validation Contract

Vor jeder KI-generierten Änderung wird geprüft

ESS Konformität

ADR Konformität

Contract Konformität

Versionierung

Repository Struktur

Layer Architektur

Naming Standards

Interface Contracts

Security

Compliance

Knowledge Contract

Alle KI-Systeme verwenden dieselbe Wissensbasis.

Der Knowledge Graph ist die einzige autorisierte Quelle für Architekturwissen.

Lokale Sonderregeln sind nicht zulässig.

AI Collaboration Contract

Mehrere KI-Systeme dürfen gemeinsam an einer Aufgabe arbeiten.

Alle Übergaben werden dokumentiert.

Jede KI übernimmt ausschließlich ihre definierte Verantwortung.

AI Decision Contract

KI-Systeme dürfen

keine

strategischen Architekturentscheidungen treffen.

Neue

Layer

Komponenten

Repositorystrukturen

Enterprise Contracts

Governance Regeln

dürfen ausschließlich über einen ADR eingeführt werden.

AI Review Contract

Alle KI-generierten Änderungen werden automatisch bewertet.

Bewertungskriterien

Codequalität

Architektur

Dokumentation

Contracts

Versionierung

Tests

Security

Compliance

Supervisor Integration

Der Supervisor überwacht

KI Aktivitäten

Governance Verstöße

fehlende Dokumentation

fehlende Contracts

fehlerhafte Architektur

nicht genehmigte Änderungen

Platform Director Integration

Der Platform Director koordiniert

KI Zusammenarbeit

Projektprioritäten

Architekturentwicklung

Governance Entscheidungen

Enterprise Roadmap

Version Manager Integration

Der Version Manager bewertet automatisch

KI erzeugte Änderungen

Breaking Changes

Minor Changes

Patch Changes

Release Empfehlungen

Security Integration

Alle KI-generierten Änderungen werden automatisch

auf Sicherheitsverletzungen geprüft.

Unsichere Änderungen dürfen nicht übernommen werden.

Compliance Integration

Alle Änderungen werden automatisch gegen

Enterprise Policies

ESS

ADR

Compliance Regeln

geprüft.

AI Audit Trail

Jede KI-Aktion erzeugt automatisch

Zeitpunkt

KI-System

Aufgabe

Repository

Änderungen

Version

ESS Referenzen

ADR Referenzen

Validierung

Ergebnis

Enterprise Automation

Die Plattform darf automatisch

Dokumentationen erzeugen

Repositorys analysieren

Knowledge Graphs erweitern

Versionen berechnen

Release Notes erzeugen

Architekturdiagramme aktualisieren

Governance Reports erzeugen

Compliance Reports erzeugen

Security Reports erzeugen

AI Compatibility Contract

Neue KI-Systeme müssen

Enterprise Contracts

ESS

ADR

Versionierung

Documentary

Knowledge Graph

Governance

vollständig unterstützen.

Validation

Vor jeder KI-gestützten Integration wird geprüft

- ✓ ESS erfüllt
 - ✓ ADR erfüllt
 - ✓ Contracts erfüllt
 - ✓ Repository Standard erfüllt
 - ✓ Naming erfüllt
 - ✓ Versionierung erfüllt
 - ✓ Documentary aktualisiert
 - ✓ Knowledge Graph aktualisiert
 - ✓ Security bestanden
 - ✓ Compliance bestanden
 - ✓ Supervisor bestätigt
 - ✓ Platform Director bestätigt
-

Enterprise Rules

Keine KI darf Enterprise Contracts umgehen.

Keine KI darf ESS verändern.

Keine KI darf ADR umgehen.

Keine KI darf Breaking Changes ohne Governance erzeugen.

Alle KI-Aktivitäten werden dokumentiert.

Die Documentary Engine ist die führende Dokumentationsinstanz.

Der Knowledge Graph ist die führende Wissensinstanz.

Der Platform Director ist die führende Governance-Instanz.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ sämtliche KI-Systeme denselben Enterprise Contracts folgen
- ✓ alle Änderungen dokumentiert werden
- ✓ sämtliche KI-Prozesse nachvollziehbar sind
- ✓ Documentary automatisch aktualisiert wird

- ✓ Knowledge Graph aktuell bleibt
 - ✓ Versionierung automatisch erfolgt
 - ✓ Governance automatisch überwacht wird
 - ✓ Security und Compliance jederzeit gewährleistet sind
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 11

Enterprise Security & Compliance Contracts

↓

Chapter 12

Enterprise Validation & Quality Contracts

↓

Chapter 13

Enterprise Plugin & Extension Contracts

End of Chapter 10

Chapter 11

Enterprise Security & Compliance Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen Sicherheits- und Compliance-Verträge des CAPITAL-AI Core.

Sicherheit ist keine nachgelagerte Prüfung.

Sicherheit ist Bestandteil der Architektur.

Jede Enterprise-Komponente beschreibt ihre Sicherheitseigenschaften maschinenlesbar.

Jede Änderung wird automatisch gegen Sicherheits- und Compliance-Regeln geprüft.

Die Documentary Engine dokumentiert sämtliche sicherheitsrelevanten Zustände automatisch.

Mission

Die Security & Compliance Contracts gewährleisten

- nachweisbare Sicherheitseigenschaften jeder Komponente
 - automatische Prüfung sämtlicher Änderungen
 - vollständige Auditierbarkeit
 - reproduzierbare Compliance-Nachweise
 - deterministische Bewertung von Sicherheitsrisiken
 - lückenlose Dokumentation sicherheitsrelevanter Entscheidungen
-

Enterprise Principle

Keine Komponente ohne Sicherheitsklassifizierung.

Keine Änderung ohne Sicherheitsbewertung.

Keine Freigabe ohne Compliance-Prüfung.

Keine Sicherheitsentscheidung ohne Dokumentation.

Keine Ausnahme ohne ADR.

Security Lifecycle

Repository Change

↓

Security Classification

↓

Security Analysis

↓

Compliance Analysis

↓

Risk Evaluation

↓

Security Event



Documentary Engine



Supervisor



Platform Director



Release Freigabe

Security Classification

Jede Enterprise-Komponente besitzt genau eine Sicherheitsklassifizierung.

Public

Internal

Confidential

Restricted

Critical

Die Klassifizierung wird in den Metadaten geführt und ist Bestandteil des Manifest Contracts aus Chapter 7.

Classification Rules

Public

Keine personenbezogenen Daten.

Keine Geschäftsgeheimnisse.

Internal

Interne Plattforminformationen ohne Personenbezug.

Confidential

Personenbezogene Daten.

Geschäftsdaten.

Abrechnungsdaten.

Restricted

Authentifizierung.

Autorisierung.

Schlüsselverwaltung.

Zahlungsverkehr.

Critical

Komponenten, deren Ausfall oder Kompromittierung den Plattformbetrieb unmittelbar gefährdet.

Sensitivity Contract

Jede Komponente beschreibt zusätzlich

Datenkategorien

Personenbezug

Speicherort

Übertragungswege

Aufbewahrungsdauer

Löschkonzept

Secure by Design

Jede Komponente erfüllt verbindlich

Least Privilege

Deny by Default

Fail Secure

Defense in Depth

Complete Mediation

Separation of Duties

Zero Trust zwischen Layern

Authentication Contract

Authentifizierung erfolgt ausschließlich über zentrale Plattformdienste.

Komponenten implementieren niemals eigene Authentifizierung.

Verbindlich sind

zentrale Identitätsprüfung

zentrale Sitzungsverwaltung

zentrale Token-Validierung

zentrale Multifaktor-Prüfung

Authorization Contract

Autorisierung erfolgt ausschließlich rollenbasiert.

Jede geschützte Operation beschreibt

erforderliche Rolle

erforderliche Berechtigung

erforderliche Sicherheitszone

erforderliche Step-Up-Anforderung

Berechtigungen werden niemals im Frontend entschieden.

Secret Contract

Secrets befinden sich niemals im Repository.

Verbindlich sind

Ablage ausschließlich in Umgebungsvariablen oder Secret Stores

Referenzierung ausschließlich über Konfiguration

keine Secrets in Logs

keine Secrets in Events
keine Secrets in Dokumentation
keine Secrets in Knowledge Nodes
keine Secrets in Fehlermeldungen

Data Protection Contract

Personenbezogene Daten unterliegen zusätzlich
Zweckbindung
Datenminimierung
Maskierung in Protokollen
Anonymisierung in Diagnosedaten
Verschlüsselung bei Übertragung
Verschlüsselung bei Speicherung sensibler Daten
Löschbarkeit

Logging Contract

Sicherheitsrelevante Ereignisse werden verbindlich protokolliert.
Mindestens
Authentifizierungsversuche
Autorisierungsverweigerungen
Rechteänderungen
Konfigurationsänderungen
Secret-Änderungen
Zugriffe auf Restricted-Komponenten
Administrative Operationen
Protokolle enthalten niemals Klartext-Secrets oder unmaskierte personenbezogene Daten.

Security Metadata Contract

Jede Komponente führt in ihren Metadaten verbindlich

classification

sensitivity

authentication

authorization

encryption

audit

compliance

lastSecurityReview

Diese Angaben erweitern die Security Metadata aus Chapter 7.

Compliance Contract

Jede Komponente beschreibt ihre Compliance-Anforderungen.

Beispiele

Datenschutz

Finanzaufsicht

Aufbewahrungspflichten

Nachweispflichten

Protokollpflichten

Exportpflichten

Compliance-Anforderungen sind Bestandteil der Komponentenbeschreibung.

Compliance Evidence

Compliance wird ausschließlich durch Nachweise belegt.

Zulässige Nachweise

Audit Trail

Security Report

Compliance Report

Validation Report

Test Report

Architecture Report

Behauptungen ohne Nachweis besitzen keine Gültigkeit.

Security Events

Sicherheitsrelevante Zustandsänderungen erzeugen verbindlich Enterprise Events gemäß Chapter 8.

SecurityScanCompletedEvent

SecurityClassificationChangedEvent

SecurityViolationDetectedEvent

ComplianceValidatedEvent

ComplianceViolationDetectedEvent

RiskDetectedEvent

RiskResolvedEvent

AuditCompletedEvent

PermissionChangedEvent

SecretRotatedEvent

Alle Event-Namen folgen dem Naming Contract aus Chapter 8.

Security Validation

Vor jeder Integration wird geprüft

- ✓ Security Classification vorhanden
- ✓ Sensitivity beschrieben
- ✓ Authentifizierung zentral
- ✓ Autorisierung rollenbasiert
- ✓ keine Secrets im Repository
- ✓ keine Secrets in Logs

- ✓ keine unmaskierten personenbezogenen Daten
 - ✓ Verschlüsselung definiert
 - ✓ Audit Trail vorhanden
 - ✓ Compliance-Anforderungen beschrieben
 - ✓ Security Events registriert
 - ✓ Security Metadata vollständig
-

Risk Contract

Jedes erkannte Risiko besitzt

Risiko-ID

Beschreibung

Kategorie

Eintrittswahrscheinlichkeit

Auswirkung

betroffene Komponenten

Maßnahme

Verantwortlichen

Status

Risiken werden niemals ohne Dokumentation geschlossen.

Risk Categories

Technisches Risiko

Architekturrisiko

Sicherheitsrisiko

Compliance-Risiko

Datenschutzrisiko

Betriebsrisiko

Abhängigkeitsrisiko

Lieferkettenrisiko

Dependency Security

Externe Abhängigkeiten unterliegen zusätzlich

Herkunftsprüfung

Versionsbindung

Änderungsüberwachung

Schwachstellenbewertung

Freigabepflicht bei Major-Wechseln

Neue externe Abhängigkeiten erfordern eine Bewertung durch das Security Center.

Security Review Contract

Ein Security Review ist verbindlich bei

neuen Restricted-Komponenten

neuen Critical-Komponenten

Änderungen an Authentifizierung

Änderungen an Autorisierung

Änderungen an Zahlungsprozessen

Änderungen an Datenbank-Policies

Änderungen an Secret-Verwaltung

neuen externen Abhängigkeiten

Migrationen mit Datenzugriff

Audit Trail Contract

Der Audit Trail ist unveränderbar.

Jeder Eintrag enthält

Zeitpunkt

auslösende Instanz

Komponente

Operation

Ergebnis

Version

Correlation ID

ESS Referenzen

ADR Referenzen

Einträge werden niemals gelöscht oder verändert.

Incident Contract

Sicherheitsvorfälle erzeugen verbindlich

Incident Event

Incident Report

Impact Analyse

Sofortmaßnahme

Nachbereitung

ADR bei Architekturänderung

Knowledge Update

Documentary Integration

Die Documentary Engine erzeugt automatisch

Security Report

Compliance Report

Risk Report

Audit Report

Permission Report

Policy Report

Diese Berichte werden ausschließlich aus validierten Metadaten und Events erzeugt.

Supervisor Integration

Der Supervisor überwacht

Sicherheitsverletzungen

Compliance-Verstöße

fehlende Klassifizierungen

fehlende Audit-Einträge

nicht bewertete Risiken

überfällige Security Reviews

Platform Director Integration

Der Platform Director entscheidet über

Sicherheitsausnahmen

Freigabe kritischer Komponenten

Risikoakzeptanz

Compliance-Strategie

Sicherheitsrelevante Architekturentscheidungen

Ausnahmen werden ausschließlich über ADR dokumentiert.

Version Manager Integration

Der Version Manager bewertet sicherheitsrelevante Änderungen zusätzlich als

Security Patch

Security Minor

Security Major

Hotfix

Sicherheitskorrekturen besitzen jederzeit Vorrang vor funktionalen Änderungen.

AI Security Contract

KI-Systeme dürfen niemals

Sicherheitsmechanismen entfernen

Berechtigungsprüfungen abschwächen

Secrets erzeugen, speichern oder ausgeben

Sicherheitsklassifizierungen herabstufen

Audit-Einträge verändern

Compliance-Nachweise erzeugen, die nicht auf Analyse beruhen

Jede KI-gestützte sicherheitsrelevante Änderung erfordert eine gesonderte Validierung.

Enterprise Rules

Keine Komponente ohne Security Classification.

Keine Restricted-Komponente ohne Security Review.

Keine Änderung ohne Sicherheitsbewertung.

Keine Compliance-Aussage ohne Nachweis.

Keine Ausnahme ohne ADR.

Keine Secrets im Repository.

Keine Manipulation des Audit Trails.

Sicherheitskorrekturen besitzen höchste Priorität.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ sämtliche Komponenten klassifiziert sind
 - ✓ sämtliche Sicherheitsmetadaten vollständig sind
 - ✓ sämtliche sicherheitsrelevanten Ereignisse Events erzeugen
 - ✓ sämtliche Risiken dokumentiert und bewertet sind
 - ✓ der Audit Trail lückenlos ist
 - ✓ Security und Compliance Reports automatisch erzeugt werden
 - ✓ keine Freigabe ohne Sicherheitsprüfung erfolgt
 - ✓ sämtliche Ausnahmen durch ADR gedeckt sind
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 12

Enterprise Validation & Quality Contracts

↓

Chapter 16

Enterprise Repository Governance

↓

Chapter 17

Enterprise AI Orchestration Contracts

End of Chapter 11

Chapter 12

Enterprise Validation & Quality Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen Validierungs- und Qualitätsverträge des CAPITAL-AI Core.

Qualität ist kein Ergebnis.

Qualität ist eine überprüfbare Eigenschaft.

Jede Regel dieser Enterprise Contracts muss maschinell durchsetzbar sein.

Ein Contract ohne Validator besitzt keine Wirkung.

Mission

Die Validation & Quality Contracts gewährleisten

- maschinelle Durchsetzbarkeit sämtlicher Contracts
- deterministische Qualitätsbewertung
- reproduzierbare Prüfergebnisse

- automatische Blockade nicht konformer Änderungen
 - nachvollziehbare Qualitätshistorie
 - einheitliche Bewertungsmaßstäbe für Mensch und KI
-

Enterprise Principle

Jeder Contract besitzt einen Validator.

Jeder Validator besitzt ein deterministisches Ergebnis.

Jedes Ergebnis besitzt eine Version.

Kein Quality Gate ohne ausführbare Prüfung.

Validation Lifecycle

Change

↓

Static Validation

↓

Contract Validation

↓

Architecture Validation

↓

Metadata Validation

↓

Documentation Validation

↓

Test Validation

↓

Security Validation

↓

Quality Score

↓

Quality Gate Decision



Supervisor



Platform Director

Validator Contract

Jeder Validator besitzt verbindlich

ID

Name

Version

geprüfter Contract

Prüfumfang

Eingaben

Ergebnisstruktur

Schweregrade

Owner

Validatoren befinden sich ausschließlich unter

```
src/platform/Validators/
```

Komponentenspezifische Validatoren befinden sich im Validators-Verzeichnis der jeweiligen Komponente.

Validator Rules

Ein Validator verändert niemals den geprüften Gegenstand.

Ein Validator besitzt keine Seiteneffekte.

Ein Validator ist idempotent.

Ein Validator liefert bei identischer Eingabe identische Ergebnisse.

Ein Validator erzeugt niemals Dokumentation.

Validation Result Contract

Jedes Prüfergebnis besitzt

Validator ID

Validator Version

geprüftes Objekt

Zeitpunkt

Ergebnis

Schweregrad

Befunde

Nachweise

Correlation ID

Severity Levels

Critical

Blockiert jede Integration.

High

Blockiert Releases.

Medium

Blockiert Produktionsfreigaben.

Low

Wird protokolliert.

Information

Rein informativ.

Mandatory Validators

Verbindlich sind mindestens

RepositoryStructureValidator

DirectoryResponsibilityValidator

NamingValidator

LayerValidator

DependencyValidator

InterfaceValidator

ManifestValidator

ComponentValidator

MetadataValidator

DocumentationValidator

EventValidator

VersionValidator

SecurityValidator

ComplianceValidator

KnowledgeValidator

TwinValidator

Jeder Validator prüft genau einen Contract-Bereich.

Quality Gates

Die in Chapter 1 definierten Quality Gates werden verbindlich als ausführbare Prüfungen geführt.

Gate 1

Contract-Konformität

Gate 2

Architektur-Konformität

Gate 3

Versionskonformität

Gate 4

Dokumentationsstatus

Gate 5

Teststatus

Gate 6

Sicherheitsauswirkungen

Gate 7

Compliance-Auswirkungen

Gate 8

Build-Ergebnis

Ein Gate gilt ausschließlich als bestanden, wenn sämtliche zugeordneten Validatoren ohne Critical- und High-Befunde abschließen.

Gate Ownership

Jedes Quality Gate besitzt genau einen verantwortlichen Enterprise-Bereich.

Gate	Verantwortung
Contract-Konformität	Quality Center
Architektur-Konformität	Architecture
Versionskonformität	Version Manager
Dokumentationsstatus	Documentary Engine
Teststatus	Quality Center
Sicherheitsauswirkungen	Security Center
Compliance-Auswirkungen	Compliance Center

Gate	Verantwortung
Build-Ergebnis	Release Center

Test Contract

Die Testverträge aus Chapter 4 bleiben unverändert gültig.

Dieses Kapitel ergänzt ausschließlich die verbindliche Zuordnung der Testarten zu Testverzeichnissen.

tests/unit/	Klassen und Funktionen
tests/integration/	Zusammenspiel mehrerer Komponenten
tests/contract/	Einhaltung der Enterprise Contracts
tests/architecture/	Layer, Abhängigkeiten, Struktur
tests/security/	Sicherheits- und Berechtigungsprüfungen
tests/performance/	Laufzeit und Ressourcenverhalten
tests/e2e/	Vollständige Prozessketten

Architecture Test Contract

Architecture Tests prüfen verbindlich

Repository-Struktur

Verzeichnisverantwortung

Layer-Hierarchie

Abhängigkeitsrichtung

zyklische Abhängigkeiten

Namenskonventionen

Import-Grenzen

Architecture Tests besitzen höchste Priorität, da sie sämtliche Struktur-Contracts durchsetzbar machen.

Contract Test Contract

Contract Tests prüfen verbindlich

Manifest-Schema

Component-Schema

Event-Schema

Registry-Schema

Knowledge-Schema

Version-Schema

Interface-Signaturen

Contract Tests bilden die maschinelle Repräsentation dieses Dokumentes.

Quality Metrics

Für jede Komponente werden verbindlich berechnet

Documentation Score

Test Score

Architecture Score

Security Score

Knowledge Score

Metadata Score

Twin Score

Jede Metrik besitzt einen Wertebereich von 0 bis 100.

Quality Score

Der Gesamtwert einer Komponente ergibt sich ausschließlich aus den Einzelmetriken.

Die Berechnung ist deterministisch.

Der Quality Score wird niemals manuell gesetzt.

Quality Thresholds

Development

kein Mindestwert

Beta

mindestens 60

Stable

mindestens 80

Critical

mindestens 90

Komponenten unterhalb ihres Schwellwertes dürfen nicht in den nächsthöheren Lifecycle-Zustand überführt werden.

Technical Debt Contract

Technische Schulden werden verbindlich erfasst.

Jeder Eintrag besitzt

ID

Beschreibung

Ursache

betroffene Komponente

Auswirkung

geschätzten Aufwand

Priorität

Zielversion

Technische Schulden werden niemals stillschweigend akzeptiert.

Observability Contract

Qualität ist ausschließlich messbar, wenn das Systemverhalten beobachtbar ist.

Jede Enterprise-Komponente stellt verbindlich bereit

Health Status

Lifecycle Status

Telemetriedaten

Fehlerzähler

Laufzeitkennzahlen

Ereignisstatistiken

Telemetrie befindet sich ausschließlich unter

```
src/platform/Telemetry/
```

Telemetriedaten enthalten niemals personenbezogene Daten.

Validation Events

Validierungen erzeugen verbindlich Enterprise Events.

ValidationStartedEvent

ValidationCompletedEvent

ValidationFailedEvent

ContractViolationEvent

QualityGatePassedEvent

QualityGateFailedEvent

QualityScoreChangedEvent

TechnicalDebtDetectedEvent

Documentary Integration

Die Documentary Engine erzeugt automatisch

Validation Report

Quality Report

Architecture Report

Technical Debt Report

Coverage Report

Gate Report

Berichte werden ausschließlich aus Validierungsergebnissen erzeugt.

Supervisor Integration

Der Supervisor überwacht

fehlgeschlagene Validierungen

nicht ausgeführte Validatoren

veraltete Prüfergebnisse

sinkende Quality Scores

wachsende technische Schulden

blockierte Quality Gates

Platform Director Integration

Der Platform Director entscheidet über

Schwellwerte

Ausnahmen

Priorisierung technischer Schulden

Freigabe trotz offener Befunde

Ausnahmen erfordern verbindlich eine ADR.

AI Validation Contract

KI-Systeme dürfen

✓ Validatoren erzeugen

✓ Tests erzeugen

✓ Befunde analysieren

✓ Korrekturen vorschlagen

Sie dürfen jedoch nicht

✗ Validatoren deaktivieren

✗ Schwellwerte verändern

✗ Befunde ohne Korrektur schließen

✗ Quality Gates umgehen

✗ Prüfergebnisse erzeugen, die nicht auf Ausführung beruhen

Validation

Vor jeder Integration wird geprüft

- ✓ sämtliche Pflichtvalidatoren ausgeführt
 - ✓ keine Critical-Befunde
 - ✓ keine High-Befunde
 - ✓ Quality Gates bestanden
 - ✓ Testverzeichnisse korrekt belegt
 - ✓ Coverage-Anforderungen erfüllt
 - ✓ Quality Score berechnet
 - ✓ technische Schulden erfasst
 - ✓ Validierungsergebnisse versioniert
-

Enterprise Rules

Kein Contract ohne Validator.

Kein Quality Gate ohne ausführbare Prüfung.

Keine Integration mit Critical-Befunden.

Kein Lifecycle-Wechsel ohne Schwellwert.

Keine Qualitätsaussage ohne Messung.

Keine manuelle Vergabe von Quality Scores.

Keine Ausnahme ohne ADR.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ sämtliche Contracts durch Validatoren abgedeckt sind
- ✓ sämtliche Quality Gates ausführbar sind
- ✓ sämtliche Prüfergebnisse reproduzierbar sind

- ✓ sämtliche Komponenten einen Quality Score besitzen
 - ✓ Architecture Tests sämtliche Strukturregeln durchsetzen
 - ✓ Contract Tests sämtliche Schemata prüfen
 - ✓ technische Schulden vollständig erfasst sind
 - ✓ keine nicht konforme Änderung integriert werden kann
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 13

Enterprise Plugin & Extension Contracts

↓

Chapter 14

Enterprise Migration & Lifecycle Contracts

↓

Chapter 19

Enterprise Automation Contracts

End of Chapter 12

Chapter 13

Enterprise Plugin & Extension Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen Erweiterungsverträge des CAPITAL-AI Core.

Die Plattform ist erweiterbar, ohne veränderbar zu sein.

Erweiterungen dürfen niemals bestehende Komponenten modifizieren.

Jede Erweiterung wird registriert, versioniert, validiert und dokumentiert.

Das Open Closed Principle aus Chapter 1 wird dadurch technisch durchgesetzt.

Mission

Die Plugin & Extension Contracts gewährleisten

- kontrollierte Erweiterbarkeit
 - vollständige Isolation von Erweiterungen
 - deterministische Ladereihenfolge
 - vollständige Registrierung
 - automatische Dokumentation jeder Erweiterung
 - Rückbaubarkeit ohne Nebenwirkungen
-

Enterprise Principle

Die Plattform wird erweitert.

Die Plattform wird nicht verändert.

Jede Erweiterung besitzt einen Vertrag.

Jede Erweiterung besitzt eine Version.

Jede Erweiterung besitzt einen Owner.

Extension Types

Der CAPITAL-AI Core kennt ausschließlich folgende Erweiterungsarten.

Generator Plugin

Validator Plugin

Discovery Plugin

Knowledge Plugin

Documentation Plugin

Diagram Plugin

Report Plugin

Event Plugin

Integration Plugin

AI Plugin

Weitere Erweiterungsarten erfordern eine ADR.

Plugin Location

Plattformweite Erweiterungen befinden sich ausschließlich unter

```
src/platform/Plugins/
```

Komponentenspezifische Erweiterungen befinden sich ausschließlich im Plugins-Verzeichnis der jeweiligen Komponente.

```
src/platform/Documentary/Plugins/
```

Plugin Contract

Jedes Plugin beschreibt verbindlich

ID

Name

Version

Typ

Beschreibung

Owner

Lifecycle

Abhängigkeiten

benötigte Interfaces

erzeugte Events

konsumierte Events

Konfiguration

ESS Referenzen

ADR Referenzen

Sicherheitsklassifizierung

Plugin Identity

Jede Plugin-ID ist plattformweit eindeutig.

Format

```
<domain>.<type>.<name>
```

Beispiele

```
documentary.generator.architecture  
documentary.validator.manifest  
knowledge.plugin.graph-export  
platform.integration.supabase
```

Die ID bleibt über sämtliche Versionen unverändert.

Plugin Interface Contract

Jedes Plugin implementiert ausschließlich öffentliche Enterprise Interfaces.

Verbindlich sind

initialize

validate

execute

dispose

describe

Ein Plugin greift niemals auf interne Klassen anderer Komponenten zu.

Plugin Lifecycle

Discovery

↓

Validation

↓

Registration

↓

Initialization

↓

Activation

↓

Execution



Deactivation



Disposal

Jede Stufe erzeugt ein Enterprise Event.

Plugin Discovery

Plugins werden ausschließlich automatisch erkannt.

Manuelle Registrierung ist nicht zulässig.

Die Discovery erfolgt über

Verzeichnisstruktur

Plugin-Manifest

Interface-Implementierung

Metadaten

Plugin Registration

Jedes erkannte Plugin wird verbindlich in der Enterprise Registry geführt.

Registrierte Angaben

ID

Version

Typ

Status

Owner

Abhängigkeiten

Events

Ladereihenfolge

Health Status

Nicht registrierte Plugins werden niemals ausgeführt.

Load Order Contract

Die Ladereihenfolge ist deterministisch.

Sie ergibt sich ausschließlich aus

Layer

Abhängigkeiten

Plugin-Typ

ID in lexikografischer Ordnung

Zufällige oder zeitabhängige Reihenfolgen sind nicht zulässig.

Isolation Contract

Ein Plugin darf niemals

bestehende Klassen überschreiben

bestehende Dateien verändern

globale Zustände modifizieren

andere Plugins direkt aufrufen

Layer-Grenzen umgehen

Contracts erweitern

Plugins kommunizieren ausschließlich über Enterprise Events und öffentliche Interfaces.

Configuration Contract

Plugin-Konfiguration erfolgt ausschließlich über

```
src/config/
```

Jede Konfigurationsoption besitzt

Name

Typ

Standardwert

Beschreibung

Gültigkeitsbereich

Plugins besitzen niemals eigene Konfigurationsdateien außerhalb der definierten Struktur.

Failure Contract

Ein fehlerhaftes Plugin darf die Plattform niemals beeinträchtigen.

Verbindlich sind

Fehlerkapselung

Zeitbegrenzung

Wiederholungsgrenzen

automatische Deaktivierung nach wiederholtem Fehler

Health-Status-Aktualisierung

Fehler-Event

Die Plattform bleibt jederzeit funktionsfähig.

Plugin Events

Verbindlich sind

PluginDiscoveredEvent

PluginRegisteredEvent

PluginActivatedEvent

PluginDeactivatedEvent

PluginExecutedEvent

PluginFailedEvent

PluginRemovedEvent

Plugin Versioning

Plugins folgen dem Versioning Contract aus Chapter 9.

Breaking Changes an Plugin-Interfaces erfordern

Major Version

ADR

Migrationspfad

Ein Plugin darf niemals eine höhere Interface-Version voraussetzen als die Plattform bereitstellt.

Compatibility Contract

Jedes Plugin beschreibt verbindlich

minimale Plattformversion

maximale Plattformversion

benötigte Interface-Version

benötigte Event-Version

Inkompatible Plugins werden registriert, jedoch niemals aktiviert.

Documentation Contract

Für jedes Plugin erzeugt die Documentary Engine automatisch

Plugin-Beschreibung

Interface-Dokumentation

Event-Dokumentation

Konfigurationsdokumentation

Abhängigkeitsdiagramm

Registry-Eintrag

Knowledge Node

Ein Plugin ohne Dokumentation gilt als nicht integriert.

Security Contract

Plugins unterliegen vollständig Chapter 11.

Zusätzlich gilt

kein direkter Datenbankzugriff ohne Freigabe

kein direkter Netzwerkzugriff ohne Deklaration

keine Ausführung fremden Codes

keine dynamische Codegenerierung

keine Umgehung der Autorisierung

Plugins mit Sicherheitsklassifizierung Restricted oder Critical erfordern ein Security Review.

Extension Governance

Neue Erweiterungsarten

neue Plugin-Interfaces

neue Ladephasen

neue Konfigurationsmechanismen

erfordern verbindlich eine ADR.

Supervisor Integration

Der Supervisor überwacht

Plugin-Ausfälle

Ladefehler

Zeitüberschreitungen

Versionskonflikte

nicht registrierte Erweiterungen

deaktivierte Plugins

Platform Director Integration

Der Platform Director entscheidet über

Zulassung neuer Erweiterungsarten

Freigabe kritischer Plugins

Deaktivierung von Plugins

Erweiterungsstrategie der Plattform

Validation

Vor jeder Aktivierung wird geprüft

- ✓ Plugin-Manifest vollständig
 - ✓ ID eindeutig
 - ✓ Interfaces implementiert
 - ✓ Abhängigkeiten auflösbar
 - ✓ Versionskompatibilität erfüllt
 - ✓ Ladereihenfolge bestimmbar
 - ✓ Events registriert
 - ✓ Sicherheitsklassifizierung vorhanden
 - ✓ Konfiguration gültig
 - ✓ Dokumentation erzeugt
 - ✓ Registry-Eintrag vorhanden
-

Enterprise Rules

Keine Erweiterung ohne Vertrag.

Keine Erweiterung ohne Registrierung.

Keine Erweiterung ohne Version.

Keine Erweiterung ohne Dokumentation.

Keine Erweiterung verändert bestehende Komponenten.

Keine Erweiterung umgeht Layer-Grenzen.

Keine neue Erweiterungsart ohne ADR.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ sämtliche Erweiterungen automatisch erkannt werden
- ✓ sämtliche Erweiterungen registriert sind

- ✓ die Ladereihenfolge deterministisch ist
 - ✓ fehlerhafte Plugins die Plattform nicht beeinträchtigen
 - ✓ sämtliche Plugins dokumentiert sind
 - ✓ sämtliche Plugins versioniert sind
 - ✓ keine Erweiterung bestehende Komponenten verändert
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 14

Enterprise Migration & Lifecycle Contracts

↓

Chapter 17

Enterprise AI Orchestration Contracts

↓

Chapter 19

Enterprise Automation Contracts

End of Chapter 13

Chapter 14

Enterprise Migration & Lifecycle Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen Lebenszyklus- und Migrationsverträge des CAPITAL-AI Core.

Jede Komponente durchläuft einen definierten Lebenszyklus.

Jeder Zustandswechsel besitzt Voraussetzungen.

Jede Migration besitzt einen Ausgangszustand, einen Zielzustand und eine Rückführungsstrategie.

Bestehende produktive Komponenten werden niemals ersetzt, sondern kontrolliert überführt.

Mission

Die Migration & Lifecycle Contracts gewährleisten

- eindeutige Zustände jeder Komponente
 - kontrollierte Zustandsübergänge
 - reproduzierbare Migrationen
 - vollständige Rückbaubarkeit
 - Schutz produktiver Bestandskomponenten
 - lückenlose Dokumentation jeder Überführung
-

Enterprise Principle

Kein Zustandswechsel ohne Voraussetzung.

Keine Migration ohne Impact Analyse.

Keine Migration ohne Rollback.

Keine Ablösung ohne Nachweis der Gleichwertigkeit.

Kein Bestandssystem wird ohne ADR entfernt.

Lifecycle States

Die verbindlichen Lebenszyklus-Zustände einer Komponente entsprechen Chapter 7.

Development

Experimental

Beta

Stable

Deprecated

Archived

Retired

Diese Werte beschreiben ausschließlich die Komponente.

Version Categories

Die Versionskategorien aus ESS-0001 Chapter 9 beschreiben ausschließlich die Version einer Auslieferung.

Development

Experimental

Preview

Alpha

Beta

Release Candidate

Production

Long Term Support

Hotfix

Emergency

Legacy

Archived

Beide Vokabulare bestehen unverändert nebeneinander.

Lifecycle beschreibt die Komponente.

Version Category beschreibt die Version.

Vocabulary Mapping

Zur eindeutigen Auswertung gilt verbindlich folgende Zuordnung.

Lifecycle	zulässige Version Categories
Development	Development, Experimental
Experimental	Experimental, Preview, Alpha
Beta	Beta, Release Candidate
Stable	Production, Long Term Support, Hotfix, Emergency
Deprecated	Production, Legacy
Archived	Legacy, Archived
Retired	Archived

Metadatenwerte werden ausschließlich in der hier definierten Schreibweise geführt.

Lifecycle Transitions

Zulässig sind ausschließlich folgende Übergänge.

Development	→	Experimental
Development	→	Beta
Experimental	→	Beta
Beta	→	Stable
Stable	→	Deprecated
Deprecated	→	Archived
Archived	→	Retired

Rückwärtsgerichtete Übergänge sind ausschließlich zulässig von

Beta	→	Development
Stable	→	Beta
Deprecated	→	Stable

Jeder Rückwärtsübergang erfordert eine Begründung im Changelog.

Transition Requirements

Development → Experimental

Manifest vollständig

README vorhanden

Registry-Eintrag vorhanden

Experimental → Beta

Contract Tests vorhanden

Architecture Tests bestanden

Quality Score mindestens 60

Events registriert

Beta → Stable

Quality Score mindestens 80

Testabdeckung erfüllt

Security Review bei Restricted und Critical

Dokumentation vollständig

Knowledge Nodes vorhanden

Digital Twin synchron

Stable → Deprecated

Nachfolger benannt

Migrationspfad dokumentiert

Deprecation-Datum gesetzt

ADR vorhanden

Deprecated → Archived

keine aktiven Abhängigkeiten

Ersatz produktiv

Abschlussdokumentation erzeugt

Archived → Retired

Code entfernt oder eingefroren

Historie im Knowledge Graph erhalten

Deprecation Contract

Eine Komponente gilt erst dann als veraltet, wenn

ein Nachfolger existiert

der Nachfolger produktiv ist

ein Migrationspfad dokumentiert ist

sämtliche Abhängigkeiten informiert wurden

ein Deprecation-Datum gesetzt wurde

Veraltete Komponenten werden weiterhin dokumentiert und überwacht.

Migration Contract

Jede Migration beschreibt verbindlich

Migration ID

Titel

Kategorie

Ausgangszustand

Zielzustand

betroffene Komponenten

betroffene Daten

Reihenfolge

Voraussetzungen

Validierung

Rollback

Version

ADR Referenz

Migration Categories

Structural Migration

Component Migration

Data Migration

Interface Migration

Event Migration

Documentation Migration

Legacy Migration

Configuration Migration

Migration Lifecycle

Der Migrationsprozess folgt verbindlich ESS-0001 Chapter 7.

Feature Request



Architecture Analysis



Impact Analysis



Risk Analysis



Migration Planning



Documentation Planning



Version Planning



Implementation



Validation



Deployment



Knowledge Synchronisation

Keine Stufe darf übersprungen werden.

Legacy Contract

Produktive Komponenten außerhalb der in Chapter 2 definierten Struktur gelten als Legacy-Bestand.

Legacy-Bestand wird verbindlich

registriert

klassifiziert

dokumentiert

versioniert

überwacht

Legacy-Bestand wird niemals

ohne ADR entfernt

ohne Nachweis der Gleichwertigkeit ersetzt

ohne Migrationspfad neu implementiert

Legacy Registration

Jede Legacy-Komponente erhält einen Registry-Eintrag mit

ID

Pfad

Verantwortung

Zielkomponente

Migrationsstatus

Risiko

ADR Referenz

Der Migrationsstatus lautet

Identified

Registered

Wrapped

Migrated

Retired

Adapter Contract

Bis zum Abschluss einer Legacy-Migration erfolgt die Anbindung ausschließlich über Adapter.

Ein Adapter

kapselt die Legacy-Implementierung vollständig

veröffentlicht ausschließlich Enterprise Interfaces
erzeugt Enterprise Events
verändert die Legacy-Implementierung nicht
besitzt eine eigene Version
Adapter befinden sich in der jeweiligen Zielkomponente.

Duplicate Prevention

Vor jeder Migration wird verbindlich geprüft
existiert die Funktion bereits?
existiert eine produktionsreife Implementierung?
existiert eine ältere Version?
existiert bereits ein Adapter?
existiert bereits ein Registry-Eintrag?
Eine bestehende produktionsreife Implementierung wird niemals neu entwickelt.

Data Migration Contract

Datenmigrationen erfordern zusätzlich
Sicherung des Ausgangszustands
Prüfsumme vor und nach der Migration
Idempotenz
Wiederanlauffähigkeit
Rollback-Skript
Testlauf in nicht produktiver Umgebung
Freigabe durch den Platform Director

Rollback Contract

Jede Migration besitzt verbindlich
Rollback-Strategie

Rollback-Reihenfolge

Rollback-Voraussetzungen

Rollback-Risiken

Rollback-Tests

Rollback-Version

Eine Migration ohne Rollback wird niemals ausgeführt.

Migration Events

Verbindlich sind

MigrationPlannedEvent

MigrationValidatedEvent

MigrationStartedEvent

MigrationCompletedEvent

MigrationFailedEvent

RollbackStartedEvent

RollbackCompletedEvent

LifecycleChangedEvent

DeprecationAnnouncedEvent

LegacyRegisteredEvent

Documentary Integration

Die Documentary Engine erzeugt automatisch

Migration Report

Lifecycle Report

Deprecation Report

Legacy Inventory

Rollback Report

Impact Report

Die Berichte werden ausschließlich aus Events und Metadaten erzeugt.

Supervisor Integration

Der Supervisor überwacht

laufende Migrationen

fehlgeschlagene Migrationen

überfällige Deprecations

nicht registrierten Legacy-Bestand

Zustandswechsel ohne Voraussetzung

Platform Director Integration

Der Platform Director entscheidet über

Freigabe von Migrationen

Deprecation-Zeitpunkte

Entfernung von Legacy-Bestand

Priorisierung der Migrationsreihenfolge

Version Manager Integration

Der Version Manager bestimmt automatisch

Versionsauswirkung jeder Migration

Zielversion

Breaking Changes

Release-Zuordnung

Rollback-Version

Validation

Vor jeder Migration wird geprüft

✓ Impact Analyse vorhanden

✓ Risikobewertung vorhanden

- ✓ Zielzustand definiert
 - ✓ Reihenfolge definiert
 - ✓ Rollback definiert
 - ✓ Tests vorhanden
 - ✓ ADR vorhanden
 - ✓ Version bestimmt
 - ✓ Dokumentation geplant
 - ✓ keine Duplikate
 - ✓ Legacy-Bestand berücksichtigt
-

Enterprise Rules

Kein Zustandswechsel ohne erfüllte Voraussetzungen.

Keine Migration ohne Impact Analyse.

Keine Migration ohne Rollback.

Keine Neuentwicklung produktionsreifer Bestandskomponenten.

Kein Legacy-Bestand ohne Registrierung.

Keine Entfernung ohne ADR.

Keine Deprecation ohne Nachfolger.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ jede Komponente einen eindeutigen Lebenszyklus besitzt
 - ✓ sämtliche Zustandswechsel überprüfbar sind
 - ✓ sämtlicher Legacy-Bestand registriert ist
 - ✓ sämtliche Migrationen dokumentiert sind
 - ✓ sämtliche Migrationen rückführbar sind
 - ✓ keine produktionsreife Komponente doppelt entwickelt wird
 - ✓ Documentary sämtliche Migrationen automatisch dokumentiert
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 15

Enterprise Knowledge Graph Contracts

↓

Chapter 16

Enterprise Repository Governance

↓

Chapter 18

Enterprise Digital Twin Contracts

End of Chapter 14

Chapter 15

Enterprise Knowledge Graph Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen technischen Verträge des Enterprise Knowledge Graph.

ESS-0001 Chapter 4 beschreibt das Wissensmodell der Plattform.

Chapter 7 dieser Contracts definiert die Metadaten, aus denen Wissen entsteht.

Dieses Kapitel definiert ausschließlich die technische Struktur, die Erzeugung, die Validierung und die Versionierung des Knowledge Graph.

Der Knowledge Graph ist die einzige autorisierte Quelle für Architekturwissen.

Mission

Die Knowledge Graph Contracts gewährleisten

- eine eindeutige Knotenidentität
- eine eindeutige Beziehungssemantik

- vollständige Rückverfolgbarkeit jedes Wissenslements
 - reproduzierbaren Aufbau aus dem Repository
 - automatische Validierung
 - identische Wissensbasis für sämtliche KI-Systeme
-

Enterprise Principle

Wissen entsteht ausschließlich aus Analyse.

Wissen wird niemals manuell geschrieben.

Jeder Knoten besitzt einen Ursprung.

Jede Beziehung besitzt eine Richtung.

Jede Aussage besitzt eine Version.

Knowledge Lifecycle

Repository

↓

Discovery

↓

Code Intelligence

↓

Metadata

↓

Knowledge Builder

↓

Knowledge Validation

↓

Knowledge Graph

↓

Digital Twin

↓

Documentary Engine

Knowledge Location

Der maschinenlesbare Wissensbestand befindet sich ausschließlich unter

```
.ai/knowledge/
```

Die erzeugende Logik befindet sich ausschließlich unter

```
src/platform/Knowledge/
```

Wissensdateien werden niemals manuell bearbeitet.

Node Contract

Jeder Knoten besitzt verbindlich

id

type

name

version

owner

source

layer

lifecycle

description

essReferences

adrReferences

createdAt

updatedAt

checksum

Node Identity

Die Knoten-ID ist plattformweit eindeutig und stabil.

Format

```
<type>:<domain>/<name>
```

Beispiele

```
component:platform/Documentary  
interface:platform/IKnowledgeBuilder  
event:platform/KnowledgeUpdatedEvent  
adr:governance/ADR-0010  
ess:governance/ESS-0001  
plugin:documentary.generator.architecture
```

Die ID bleibt über sämtliche Versionen unverändert.

Node Types

Verbindlich sind

Repository

Module

Component

Class

Interface

Type

Method

Event

Agent

Orchestrator

Service

API

DatabaseObject

Migration

Configuration

Plugin

Validator

Generator

Document

ADR

ESS

Version

Release

Risk

Policy

Neue Knotentypen erfordern eine ADR.

Relationship Contract

Jede Beziehung besitzt verbindlich

id

type

source

target

direction

version

origin

confidence

createdAt

Relationship Types

Verbindlich sind die in ESS-0001 Chapter 4 definierten Beziehungen.

USES

IMPLEMENTS

DEPENDS_ON

CALLS

LISTENS_TO

EMITS

OWNS

BELONGS_TO

GENERATES

REGISTERS

VALIDATES

DEPLOYS

CONFIGURES

MIGRATES

SUPERVISES

DOCUMENTS

Zusätzlich zulässig sind

REPLACES

DEPRECATES

REFERENCES

DERIVED_FROM

Weitere Beziehungstypen erfordern eine ADR.

Direction Contract

Jede Beziehung ist gerichtet.

Ungerichtete Beziehungen sind nicht zulässig.

Gegenbeziehungen werden niemals doppelt gespeichert, sondern ausschließlich berechnet.

Origin Contract

Jeder Knoten und jede Beziehung besitzt einen nachweisbaren Ursprung.

Zulässige Ursprünge

SourceCode

Manifest

ComponentDescriptor

Documentation

ADR

ESS

Event

Registry

Migration

GitHistory

Wissen ohne Ursprung wird verworfen.

Confidence Contract

Jede abgeleitete Beziehung besitzt einen Vertrauenswert.

Verified

aus explizitem Code oder Metadaten abgeleitet

Derived

aus Analyse mehrerer Quellen abgeleitet

Assumed

nicht zulässig

Beziehungen mit dem Wert Assumed dürfen niemals in den Knowledge Graph aufgenommen werden.

Knowledge Versioning

Der Knowledge Graph besitzt eine eigene Version gemäß ESS-0001 Chapter 9.

Die Knowledge Version wird erhöht bei

neuen Knotentypen

neuen Beziehungstypen

strukturellen Änderungen des Wissensmodells

vollständigem Neuaufbau

Jedes Dokument speichert die Knowledge Version, auf der es beruht.

Determinism Contract

Der Knowledge Graph ist vollständig reproduzierbar.

Bei identischem Repository-Zustand erzeugt jeder Aufbau

identische Knoten

identische IDs

identische Beziehungen

identische Prüfsummen

Zeitstempel und Laufzeitinformationen sind von der Prüfsummenbildung ausgenommen.

Knowledge Files

Die Wissensbasis wird verbindlich in getrennten Dateien geführt.

```
repository.json
architecture.json
components.json
interfaces.json
services.json
events.json
agents.json
orchestrators.json
database.json
api.json
security.json
documentation.json
dependencies.json
workflows.json
policies.json
versions.json
releases.json
plugins.json
risks.json
graph.json
```

Jede Datei besitzt ein Schema unter `.ai/schemas/`.

Query Contract

Der Knowledge Graph beantwortet verbindlich mindestens

Welche Komponenten besitzen keine ADR?

Welche Komponenten besitzen keine Tests?

Welche Events besitzen keinen Consumer?

Welche Interfaces besitzen keine Implementierung?

Welche Komponenten verletzen die Layer-Hierarchie?

Welche Komponenten sind veraltet?

Welche Komponenten besitzen technische Schulden?

Welche Version hat eine Komponente eingeführt?

Welche Dokumente beruhen auf einer veralteten Knowledge Version?

Welche Legacy-Komponenten besitzen keinen Adapter?

Consistency Contract

Der Knowledge Graph ist widerspruchsfrei.

Nicht zulässig sind

Knoten ohne Typ

Knoten ohne Ursprung

Beziehungen auf nicht existierende Knoten

doppelte IDs

zyklische OWNS-Beziehungen

widersprüchliche Versionsangaben

verwaiste Knoten ohne Beziehung

Synchronisation Contract

Der Knowledge Graph wird verbindlich aktualisiert bei

Repository-Änderungen

Metadatenänderungen

Contract-Änderungen

ADR-Änderungen

ESS-Änderungen

Versionsänderungen

Migrationen

Plugin-Registrierungen

Die Aktualisierung erfolgt ausschließlich ereignisgesteuert.

Knowledge Events

Verbindlich sind

KnowledgeBuildStartedEvent

KnowledgeCreatedEvent

KnowledgeUpdatedEvent

KnowledgeLinkedEvent

KnowledgeValidatedEvent

KnowledgeConflictDetectedEvent

KnowledgeVersionChangedEvent

Registry Relationship

Die Enterprise Registry aus Chapter 7 und der Knowledge Graph besitzen getrennte Verantwortungen.

Registry

verwaltet Existenz, Identität und Status registrierter Objekte

Knowledge Graph

verwaltet Bedeutung, Beziehungen und Historie

Die Registry ist Quelle des Knowledge Graph.

Der Knowledge Graph verändert die Registry niemals.

AI Knowledge Contract

Sämtliche KI-Systeme verwenden ausschließlich den Knowledge Graph als Wissensquelle.

Nicht zulässig sind

eigene Repository-Analysen zur Wissensbildung

lokale Wissensspeicher

abweichende Interpretationen

manuelle Ergänzungen

Ein KI-System, das Wissen benötigt, fragt den Knowledge Graph ab.

Documentary Integration

Die Documentary Engine erzeugt sämtliche Dokumentation ausschließlich aus dem Knowledge Graph.

Erzeugt werden

Knowledge Report

Knowledge Index

Component Registry

Dependency Report

Architecture Report

Diagramme

Kein Dokument entsteht ohne Knowledge-Referenz.

Validation

Vor jeder Veröffentlichung des Knowledge Graph wird geprüft

- ✓ sämtliche Knoten besitzen IDs
- ✓ sämtliche IDs sind eindeutig
- ✓ sämtliche Knoten besitzen einen Ursprung
- ✓ sämtliche Beziehungen besitzen gültige Endpunkte
- ✓ keine Assumed-Beziehungen
- ✓ keine verwaisten Knoten
- ✓ keine zyklischen Eigentumsbeziehungen
- ✓ Schema-Konformität aller Dateien

- ✓ Prüfsummen reproduzierbar
 - ✓ Knowledge Version gesetzt
-

Enterprise Rules

Wissen entsteht ausschließlich aus Analyse.

Wissen wird niemals manuell verändert.

Kein Knoten ohne Ursprung.

Keine Beziehung ohne Richtung.

Keine Annahme im Knowledge Graph.

Kein Dokument ohne Knowledge-Referenz.

Keine parallele Wissensbasis.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ sämtliche Komponenten als Knoten existieren
 - ✓ sämtliche Beziehungen abgeleitet wurden
 - ✓ der Graph reproduzierbar aufgebaut werden kann
 - ✓ sämtliche Wissensdateien schemakonform sind
 - ✓ sämtliche Dokumente auf Knowledge-Referenzen beruhen
 - ✓ sämtliche KI-Systeme dieselbe Wissensbasis verwenden
 - ✓ Widersprüche automatisch erkannt werden
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 16

Enterprise Repository Governance

↓

Chapter 18

Enterprise Digital Twin Contracts



Chapter 19

Enterprise Automation Contracts

End of Chapter 15

Chapter 16

Enterprise Repository Governance

Enterprise Purpose

Dieses Kapitel definiert die verbindliche Governance des CAPITAL-AI Repositorys.

Chapter 2 definiert die Struktur.

Chapter 3 definiert die Verantwortlichkeiten.

Dieses Kapitel definiert, wie die Einhaltung dieser Struktur dauerhaft durchgesetzt, überwacht und weiterentwickelt wird.

Das Repository ist die einzige technische Wahrheit der Plattform und unterliegt deshalb derselben Governance wie produktiver Code.

Mission

Die Repository Governance gewährleistet

- dauerhafte Strukturstabilität
 - kontrollierte Strukturerweiterung
 - automatische Erkennung von Abweichungen
 - eindeutige Eigentümerschaft jedes Bereichs
 - nachvollziehbare Ausnahmen
 - reproduzierbaren Repository-Zustand
-

Enterprise Principle

Struktur ist Architektur.

Struktur wird niemals beiläufig verändert.

Jede Abweichung ist entweder ein Befund oder eine dokumentierte Ausnahme.

Governance Scope

Dieses Kapitel gilt für

Root-Verzeichnisse

Root-Dateien

Plattformmodule

Feature-Domänen

Dokumentationsbereiche

AI-Ressourcen

Testbereiche

Skriptbereiche

Datenbankartefakte

Generierte Artefakte

Structural Authority

Die Rangfolge aus Chapter 1 gilt unverändert.

ADR

↓

ESS Contracts

↓

ESS

↓

Projektdokumentation

↓

Implementierung

Strukturentscheidungen werden ausschließlich durch ADR getroffen.

Platform Module Clarification

Chapter 2 definiert die verbindlichen fachlichen Plattformmodule.

Chapter 3 definiert zusätzlich `src/platform/Core` als technische Basis.

Chapter 6 führt Core als unterste Ebene der Layer-Hierarchie.

Klarstellung

Core ist verbindlicher Bestandteil der Plattform und bildet das Fundament sämtlicher Module.

Core wird nicht als fachliches Modul geführt, sondern als technische Basisschicht.

Diese Klarstellung ändert keine bestehende Regel, sondern beseitigt eine Auslegungslücke.

Cross Cutting Modules

Chapter 6 definiert die Layer-Hierarchie der fachlichen Plattformebenen.

Folgende Pflichtmodule aus Chapter 2 sind Querschnittsmodule und besitzen keine eigene Ebene innerhalb dieser Hierarchie.

Architecture	Events	Contracts	Models
Interfaces	Validators	Generators	Plugins
Telemetry	Quality	Security	Compliance
Release			

Für Querschnittsmodule gilt verbindlich

sie besitzen ausschließlich Abhängigkeiten auf Core und Shared

sie besitzen niemals Abhängigkeiten auf Documentary, Version Manager, Supervisor oder Platform Director

sie werden von höheren Layern verwendet, verwenden diese jedoch niemals selbst

Ausnahme

Release besitzt zusätzlich Zugriff auf Registry, Knowledge, Documentary und Version Manager, da es deren Ergebnisse zusammenführt.

Diese Zuordnung ergänzt Chapter 6, ohne die bestehende Hierarchie zu verändern.

Directory Ownership

Jeder Repository-Bereich besitzt genau einen Eigentümer.

Bereich	Eigentümer
.ai/	Platform Director
docs/	Documentary Engine
docs/adr/	Platform Director
scripts/	Release Center
src/platform/	Platform Director
src/features/	jeweilige Domäne
src/config/	Platform Director
supabase/	Security Center
tests/	Quality Center
public/	Frontend
dist/	Build

Ohne Eigentümer existiert kein zulässiger Repository-Bereich.

Root Governance

Root-Verzeichnisse sind abschließend in Chapter 2 definiert.

Root-Dokumente sind abschließend in Chapter 2 definiert.

Jede Abweichung erfordert verbindlich

einen ADR

eine Begründung

eine Zielstruktur

ein Enddatum oder den Status Permanent Exception

Nicht dokumentierte Abweichungen sind Befunde der Stufe Critical.

Exception Contract

Eine Ausnahme besitzt verbindlich

Exception ID

betroffenen Pfad

verletzten Contract

Begründung

Risiko

Zielzustand

Verantwortlichen

ADR Referenz

Status

Zulässige Status

Approved

Time Limited

Permanent

Revoked

Ausnahmen werden im Repository geführt und automatisch überwacht.

Drift Detection

Als Strukturdrift gilt verbindlich

neues Root-Verzeichnis ohne ADR

neue Root-Datei ohne Ausnahme

Plattformmodul ohne Manifest

Plattformmodul ohne README

Komponente ohne Registry-Eintrag

Datei außerhalb ihrer Verantwortungszone

Geschäftslogik außerhalb von Features

Dokumentation innerhalb produktiver Komponenten

Testdateien außerhalb des Testbereichs

generierte Dateien außerhalb der Generatorbereiche

Strukturdrift wird bei jedem Discovery-Lauf erkannt.

Naming Governance

Die Namensverträge aus Chapter 5 gelten unverändert.

Zusätzlich gilt verbindlich

Dateinamen enthalten niemals Leerzeichen

Dokumentnamen folgen dem definierten Enterprise-Schema

ADR-Nummern sind vierstellig und fortlaufend

ESS-Nummern sind vierstellig und werden ausschließlich über die ESS Registry vergeben

Zwischennummern sind nicht zulässig

ESS Registry Contract

Der ESS-Nummernraum wird zentral verwaltet.

Verbindlich gilt

jede ESS-Nummer wird genau einmal vergeben

vergebene Nummern werden niemals umbenannt

reservierte Nummern werden niemals abweichend belegt

neue ESS-Dokumente erhalten die nächste freie Nummer

jede ESS-Nummer besitzt genau ein Dokument

Die Reservierungen aus ESS-0001 sind verbindlich und unverändert zu übernehmen.

ADR Governance

Verbindlich gilt

jede Architekturentscheidung besitzt ein ADR-Dokument

jedes ADR-Dokument besitzt eine eindeutige Nummer

die ADR-Historie ist vollständig

Status und Implementation-Status werden getrennt geführt

abgeschlossene ADRs werden nach `docs/adr/resolved/` überführt

die maschinenlesbare ADR-Registrierung wird generiert, nicht manuell gepflegt

Repository Validation

Vor jedem Build wird zusätzlich zu Chapter 2 geprüft

- ✓ sämtliche Root-Verzeichnisse zulässig oder als Ausnahme registriert
 - ✓ sämtliche Root-Dateien zulässig oder als Ausnahme registriert
 - ✓ sämtliche Plattformmodule vollständig
 - ✓ sämtliche Module besitzen Manifest, Component Descriptor, README und CHANGELOG
 - ✓ sämtliche Bereiche besitzen einen Eigentümer
 - ✓ sämtliche Ausnahmen besitzen einen gültigen Status
 - ✓ keine Strukturdrift
 - ✓ keine Namensverletzungen
 - ✓ ESS Registry konsistent
 - ✓ ADR Registry konsistent
-

Governance Events

Verbindlich sind

RepositoryScannedEvent

RepositoryValidatedEvent

StructureViolationDetectedEvent

ExceptionRegisteredEvent

ExceptionExpiredEvent

OwnershipChangedEvent

GovernanceReportGeneratedEvent

Documentary Integration

Die Documentary Engine erzeugt automatisch

Repository Report

Structure Report

Ownership Report

Exception Report

Governance Report

Drift Report

Diese Berichte sind Bestandteil jeder Release-Dokumentation.

Supervisor Integration

Der Supervisor überwacht

Strukturverletzungen

abgelaufene Ausnahmen

fehlende Eigentümer

fehlende Metadaten

nicht registrierte Komponenten

nicht registrierten Legacy-Bestand

Platform Director Integration

Der Platform Director entscheidet über

Strukturerweiterungen

Ausnahmen

Eigentümerschaft

ESS-Nummernvergabe

Priorisierung von Strukturkorrekturen

AI Repository Contract

KI-Systeme dürfen

- ✓ Strukturabweichungen melden
- ✓ Strukturkorrekturen vorschlagen
- ✓ fehlende Metadaten erzeugen
- ✓ Berichte generieren

Sie dürfen jedoch nicht

- ✗ neue Root-Verzeichnisse anlegen
 - ✗ Strukturregeln auslegen
 - ✗ Ausnahmen selbst genehmigen
 - ✗ ESS-Nummern eigenständig vergeben
 - ✗ ADR-Nummern eigenständig vergeben
-

Enterprise Rules

- Keine Strukturänderung ohne ADR.
 - Keine Abweichung ohne registrierte Ausnahme.
 - Kein Repository-Bereich ohne Eigentümer.
 - Keine Komponente ohne Metadaten.
 - Keine ESS-Nummer ohne Registry-Eintrag.
 - Keine ADR-Nummer ohne Dokument.
 - Keine stillschweigende Strukturdrift.
-

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ sämtliche Repository-Bereiche einen Eigentümer besitzen
 - ✓ sämtliche Abweichungen registriert sind
 - ✓ Strukturdrift automatisch erkannt wird
 - ✓ ESS- und ADR-Nummernräume widerspruchsfrei sind
 - ✓ sämtliche Plattformmodule vollständig beschrieben sind
 - ✓ der Repository-Zustand reproduzierbar validiert werden kann
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 17

Enterprise AI Orchestration Contracts



Chapter 18

Enterprise Digital Twin Contracts



Chapter 20

Official Enterprise Standard

End of Chapter 16

Chapter 17

Enterprise AI Orchestration Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindliche Orchestrierung sämtlicher KI-Systeme innerhalb des CAPITAL-AI Core.

Chapter 10 definiert die Verantwortlichkeiten der einzelnen KI-Systeme.

Dieses Kapitel definiert die Reihenfolge, die Übergaben, die Auslöser und die Abbruchbedingungen der gesamten AI-Wertschöpfungskette.

Die Wertschöpfungskette ist deterministisch.

Sie beginnt niemals ohne Auslöser und endet niemals ohne Nachweis.

Mission

Die AI Orchestration Contracts gewährleisten

- eine eindeutige Reihenfolge sämtlicher KI-Stufen
- definierte Übergabeartefakte zwischen den Stufen
- automatische Auslösung durch Enterprise Events
- vollständige Nachvollziehbarkeit jeder Übergabe
- kontrollierten Abbruch bei Vertragsverletzung
- reproduzierbare Ergebnisse unabhängig vom eingesetzten Modell

Enterprise Principle

Kein KI-System arbeitet außerhalb der Kette.

Keine Stufe beginnt ohne Eingangsartefakt.

Keine Stufe endet ohne Ausgangsartefakt.

Jede Übergabe erzeugt ein Enterprise Event.

Jede Stufe kann die Kette anhalten.

Enterprise AI Value Chain

Die verbindliche Wertschöpfungskette lautet

Google AI Studio

↓

Claude Code

↓

Documentary Engine

↓

Supervisor

↓

Platform Director

↓

Version Manager

↓

Release

↓

Production

Keine Stufe darf übersprungen werden.

Keine Stufe darf ihre Reihenfolge verändern.

Stage Contract

Jede Stufe der Kette besitzt verbindlich

Stage ID

Verantwortung

Eingangsartefakte

Ausgangsartefakte

auslösendes Event

erzeugtes Event

Abbruchbedingungen

Validierung

Owner

Stage 1 — Google AI Studio

Verantwortung

Frontend

UI

UX

Rapid Development

Prototyping

Dokumentationsunterstützung

Eingang

Anforderung

Designvorgabe

bestehende Contracts

Ausgang

Entwurf

Prototyp

Frontend-Artefakt

Änderungsbeschreibung

Erzeugtes Event

DesignProposedEvent

Abbruch

Verstoß gegen Naming Contracts

Verstoß gegen Layer-Grenzen

fehlende Anforderungsbeschreibung

Stage 2 — Claude Code

Verantwortung

Backend

Implementierung

Refactoring

Migration

Repository-Änderungen

Tests

produktionsnahe Codeänderungen

Eingang

Entwurf aus Stage 1

Knowledge Graph

Enterprise Contracts

Impact Analyse

Ausgang

Implementierung

Tests

Metadaten

Migrationsplan

Changelog-Eintrag

Erzeugtes Event

ImplementationCompletedEvent

Abbruch

Contract-Verletzung

fehlende Impact Analyse

fehlende ADR bei Architekturänderung

nicht auflösbare Abhängigkeit

Stage 3 — Documentary Engine

Verantwortung

Repository Discovery

Code Discovery

Knowledge Graph

Enterprise Registry

Contract Validation

Architecture Reports

Impact Analysen

Digital Twin

Eingang

Repository-Zustand

Implementierungsartefakte

Events der vorgelagerten Stufen

Ausgang

Knowledge Update

Dokumentation

Architecture Report

Validation Report

Twin Update

Erzeugtes Event

DocumentationGeneratedEvent

KnowledgeUpdatedEvent

TwinSynchronizedEvent

Abbruch

nicht auflösbare Wissenskonflikte

fehlende Metadaten

Schema-Verletzung

Stage 4 — Supervisor

Verantwortung

Überwachung

Vollständigkeitsprüfung

Governance-Prüfung

Konsistenzprüfung

Eingang

Dokumentation

Validation Reports

Knowledge Graph

Twin-Zustand

Ausgang

Supervisor Assessment

Befundliste

Freigabeempfehlung

Erzeugtes Event

SupervisorValidatedEvent

SupervisorAlertEvent

Abbruch

offene Critical-Befunde

nicht dokumentierte Änderungen

Governance-Verstöße

Stage 5 — Platform Director**Verantwortung**

strategische Entscheidung

Freigabe

Priorisierung

Ausnahmegenehmigung

Eingang

Supervisor Assessment

Impact Analyse

Risikobewertung

Ausgang

Entscheidung

Freigabe oder Ablehnung

ADR bei Architekturentscheidung

Erzeugtes Event

PlatformDecisionEvent

Abbruch

fehlende Entscheidungsgrundlage

nicht akzeptables Risiko

Stage 6 — Version Manager

Verantwortung

Versionsbewertung

Versionsempfehlung

Versionssynchronisation

Rollback-Planung

Eingang

Freigabe des Platform Director

Impact Analyse

Änderungsumfang

Ausgang

Version

Changelog

Release Notes

Rollback-Plan

Erzeugtes Event

VersionCalculatedEvent

VersionChangedEvent

Abbruch

widersprüchliche Versionsstände

fehlender Rollback-Plan

Stage 7 — Release

Verantwortung

Release-Vorbereitung

Artefakterzeugung

Deployment-Planung

Freigabedokumentation

Eingang

Version

Dokumentation

Validierungsergebnisse

Ausgang

Release-Paket

Deployment-Plan

Release Report

Erzeugtes Event

ReleasePreparedEvent

ReleasePublishedEvent

Abbruch

nicht bestandene Quality Gates

fehlende Freigabe

Stage 8 — Production

Verantwortung

Betrieb

Überwachung

Rückmeldung

Eingang

Release-Paket

Ausgang

Betriebsstatus

Telemetrie

Vorfälle

Erzeugtes Event

DeploymentCompletedEvent

PlatformHealthChangedEvent

Abbruch

fehlgeschlagenes Deployment

Rollback-Auslösung

Handover Contract

Jede Übergabe zwischen zwei Stufen besitzt verbindlich

Correlation ID

Quellstufe

Zielstufe

Übergabeartefakte

Zeitpunkt

Version

Validierungsergebnis

Eine Übergabe ohne vollständiges Artefakt ist nicht zulässig.

Correlation Contract

Sämtliche Events einer Wertschöpfungskette teilen dieselbe Correlation ID.

Die Correlation ID entsteht in Stage 1 und bleibt bis Stage 8 unverändert.

Dadurch ist jede Änderung von der Anforderung bis zur Produktion lückenlos nachvollziehbar.

Trigger Contract

Jede Stufe wird ausschließlich durch Enterprise Events ausgelöst.

Auslösendes Event	Ausgelöste Stufe
DesignProposedEvent	Claude Code
ImplementationCompletedEvent	Documentary Engine
DocumentationGeneratedEvent	Supervisor
SupervisorValidatedEvent	Platform Director
PlatformDecisionEvent	Version Manager
VersionChangedEvent	Release
ReleasePublishedEvent	Production

Direkte Aufrufe zwischen Stufen sind nicht zulässig.

Documentary Trigger Contract

Die Documentary Engine wird zusätzlich zu Chapter 10 verbindlich ausgelöst bei

jeder Repository-Änderung

jeder Metadatenänderung

jeder Contract-Änderung

jeder ESS-Änderung

jeder ADR-Änderung

jeder Versionsänderung

jeder Migration

jeder Plugin-Registrierung

jedem Security Review

jedem Compliance Review

Die Documentary Engine ist niemals optional.

Abort Contract

Jede Stufe darf die Kette anhalten.

Ein Abbruch erzeugt verbindlich

Abort Event

Begründung

betroffene Artefakte

Verantwortlichen

Korrekturhinweis

Eine abgebrochene Kette wird niemals stillschweigend fortgesetzt.

Retry Contract

Wiederholungen sind ausschließlich zulässig

nach Behebung der Abbruchursache

mit unveränderter Correlation ID

mit dokumentiertem Wiederholungsgrund

Endlose Wiederholungen sind nicht zulässig.

Human Override

Die Regelung aus Chapter 1 gilt unverändert.

Autorisierte Projektverantwortliche dürfen

die Kette anhalten

Stufen erneut ausführen

Entscheidungen des Platform Director ersetzen

Ausnahmen genehmigen

Jeder Eingriff wird als Event protokolliert.

AI Audit Trail

Zusätzlich zu Chapter 10 wird je Stufe protokolliert

Stage ID

KI-System

Modellkennung

Eingangsartefakte

Ausgangsartefakte

Dauer

Validierungsergebnis

Correlation ID

Der Audit Trail ist unveränderbar.

Model Independence

Die Kette ist unabhängig vom eingesetzten Modell.

Ein Modellwechsel darf niemals

die Reihenfolge

die Artefakte

die Contracts

die Ergebnisse in Architektur, Struktur oder Semantik

verändern.

Ausschließlich Formulierungen dürfen abweichen.

Validation

Vor jedem Kettendurchlauf wird geprüft

- ✓ Auslöser vorhanden
- ✓ Correlation ID erzeugt
- ✓ Eingangsartefakte vollständig
- ✓ Contracts geladen
- ✓ Knowledge Graph aktuell
- ✓ Digital Twin synchron

Nach jedem Kettendurchlauf wird geprüft

- ✓ sämtliche Stufen ausgeführt
- ✓ sämtliche Events erzeugt
- ✓ sämtliche Artefakte dokumentiert
- ✓ Version bestimmt
- ✓ Audit Trail vollständig

Enterprise Rules

Keine Stufe ohne Auslöser.

Keine Übergabe ohne Artefakt.

Keine Stufe ohne Event.

Keine Kette ohne Correlation ID.

Kein Abbruch ohne Begründung.

Keine Produktion ohne vollständige Kette.

Kein Modellwechsel verändert Ergebnisse.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ jede Stufe automatisch ausgelöst wird
 - ✓ jede Übergabe dokumentiert ist
 - ✓ jede Kette lückenlos nachvollziehbar bleibt
 - ✓ die Documentary Engine bei jeder relevanten Änderung ausgelöst wird
 - ✓ die Versionierung vollständig integriert ist
 - ✓ der Knowledge Graph automatisch aktualisiert wird
 - ✓ der Digital Twin nach jeder Kette synchron ist
 - ✓ kein KI-System außerhalb der Kette arbeitet
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 18

Enterprise Digital Twin Contracts

↓

Chapter 19

Enterprise Automation Contracts



Chapter 20

Official Enterprise Standard

End of Chapter 17

Chapter 18

Enterprise Digital Twin Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen Verträge des digitalen Zwillings der CAPITAL-AI Plattform.

ESS-0001 Chapter 6 beschreibt den Digital Twin konzeptionell.

Dieses Kapitel definiert seine technische Struktur, seine Zustände, seine Synchronisation und seine Überprüfbarkeit.

Die Documentary Engine ist der digitale Zwilling der gesamten Plattform.

Jede Enterprise-Komponente besitzt einen eigenen Digital Twin.

Mission

Die Digital Twin Contracts gewährleisten

- eine jederzeit gültige Abbildung des tatsächlichen Systemzustands
 - messbare Synchronität zwischen Code und Modell
 - automatische Erkennung von Abweichungen
 - reproduzierbare Rekonstruktion jedes historischen Zustands
 - Bewertung geplanter Änderungen vor ihrer Umsetzung
-

Enterprise Principle

Der Zwilling beschreibt niemals einen gewünschten Zustand.

Der Zwilling beschreibt ausschließlich den nachweisbaren Zustand.

Weicht der Zwilling vom Code ab, ist der Zwilling fehlerhaft, niemals der Code.

Twin Scope

Der Digital Twin umfasst verbindlich

Repository

Plattformmodule

Komponenten

Interfaces

Events

Abhängigkeiten

Datenbankobjekte

APIs

Konfiguration

Plugins

Dokumentation

Versionen

Releases

Risiken

Governance-Zustand

Component Twin Contract

Jede Enterprise-Komponente besitzt einen Digital Twin mit verbindlich

twiId

componentId

version

twinVersion

sourceChecksum

metadataChecksum

knowledgeVersion

architectureVersion

lifecycle

health

quality

security

dependencies

interfaces

events

documentation

lastSynchronizedAt

state

Twin Identity

Die Twin-ID ist stabil und plattformweit eindeutig.

Format

```
twin:<componentId>
```

Beispiel

```
twin:component:platform/Documentary
```

Die Twin-ID bleibt über sämtliche Versionen unverändert.

Twin States

Ein Digital Twin besitzt verbindlich genau einen Zustand.

Synchronized

Zwilling und Repository stimmen vollständig überein.

Drifted

Es bestehen erkannte Abweichungen.

Stale

Der Zwilling wurde seit einer relevanten Änderung nicht aktualisiert.

Incomplete

Für den Aufbau erforderliche Informationen fehlen.

Unknown

Der Zwilling wurde noch nie aufgebaut.

Ausschließlich der Zustand Synchronized ist für Produktionsfreigaben zulässig.

Synchronisation Contract

Der Digital Twin wird verbindlich synchronisiert bei

jeder Repository-Änderung

jeder Metadatenänderung

jeder Versionsänderung

jeder Migration

jeder Plugin-Registrierung

jeder Contract-Änderung

jeder ADR-Änderung

jedem Release

Die Synchronisation erfolgt ausschließlich ereignisgesteuert.

Drift Contract

Als Drift gilt verbindlich jede Abweichung zwischen

Quellcode und Twin-Modell

Metadaten und Twin-Modell

Registry und Twin-Modell

Knowledge Graph und Twin-Modell

Dokumentation und Twin-Modell

Version und Twin-Version

Drift wird ausschließlich durch Vergleich von Prüfsummen und Strukturen erkannt.

Drift Tolerance

Zulässige Drift beträgt null.

Es existiert keine akzeptierte Abweichung.

Erkannte Drift erzeugt verbindlich

TwinDriftDetectedEvent

Befundeintrag

Korrekturauftrag

Eine Produktionsfreigabe bei bestehender Drift ist nicht zulässig.

Reconciliation Contract

Der Abgleich erfolgt ausschließlich in einer Richtung.

Repository

↓

Discovery

↓

Knowledge Graph

↓

Digital Twin

Der Digital Twin verändert niemals das Repository.

Der Digital Twin verändert niemals Metadaten.

Der Digital Twin erzeugt ausschließlich Befunde und Berichte.

Twin Versioning

Der Digital Twin besitzt eine eigene Version.

Die Twin-Version wird erhöht bei
strukturellen Änderungen des Twin-Modells
neuen Twin-Attributen
geänderter Prüfsummenbildung
vollständigem Neuaufbau
Jeder Twin-Zustand speichert zusätzlich
Repository Version
Knowledge Version
Architecture Version
Documentation Version

Historical Twin

Jeder freigegebene Zustand wird dauerhaft gespeichert.
Für jede Version existiert verbindlich ein rekonstruierbarer Zwilling.
Dadurch beantwortet die Plattform jederzeit
Wie war die Architektur zum Zeitpunkt einer Version?
Welche Komponenten existierten?
Welche Abhängigkeiten bestanden?
Welche Risiken waren bekannt?
Welche Dokumentation war gültig?

Planned Twin

Vor jeder Migration wird zusätzlich ein geplanter Zwilling erzeugt.
Der geplante Zwilling beschreibt
Zielarchitektur
geplante Komponenten
geplante Abhängigkeiten
geplante Events
geplante Versionen

Der Vergleich zwischen aktuellem und geplantem Zwilling bildet die Grundlage der Impact Analyse nach ESS-0001 Chapter 7.

Twin Simulation

Vor jeder Freigabe wird simuliert

Welche Komponenten ändern sich?

Welche Abhängigkeiten entstehen?

Welche Abhängigkeiten entfallen?

Welche Events ändern sich?

Welche Dokumente ändern sich?

Welche Version ist erforderlich?

Welche Tests müssen erneut ausgeführt werden?

Die Simulation verändert niemals den produktiven Zustand.

Twin Storage

Der Digital Twin wird ausschließlich maschinenlesbar geführt.

```
.ai/knowledge/twin/
```

Er enthält verbindlich

current.json

planned.json

history/

drift.json

Manuelle Änderungen sind nicht zulässig.

Twin Events

Verbindlich sind

TwinBuildStartedEvent

TwinSynchronizedEvent

TwinDriftDetectedEvent

TwinDriftResolvedEvent

TwinSnapshotCreatedEvent

TwinSimulationCompletedEvent

TwinVersionChangedEvent

Documentary Integration

Die Documentary Engine erzeugt aus dem Digital Twin automatisch

Architecture Report

Component Report

Dependency Report

Drift Report

Impact Report

Version Report

Sämtliche Architekturdiagramme werden ausschließlich aus dem Digital Twin erzeugt.

Supervisor Integration

Der Supervisor überwacht

Twin-Zustand

Drift

veraltete Zwillinge

fehlende Snapshots

nicht synchronisierte Komponenten

Platform Director Integration

Der Platform Director verwendet den Digital Twin als alleinige Entscheidungsgrundlage für

Architekturentscheidungen

Freigaben

Priorisierung

Risikobewertung

Roadmap-Planung

AI Twin Contract

Sämtliche KI-Systeme verwenden ausschließlich den Digital Twin zur Architekturinterpretation.

Nicht zulässig sind

eigene Architekturmodelle

abweichende Interpretationen

Annahmen über nicht abgebildete Komponenten

Ein KI-System, das Architektur benötigt, liest den Digital Twin.

Validation

Vor jeder Freigabe wird geprüft

- ✓ Twin für sämtliche Komponenten vorhanden
 - ✓ Twin-Zustand Synchronized
 - ✓ keine offene Drift
 - ✓ Prüfsummen reproduzierbar
 - ✓ Twin-Version gesetzt
 - ✓ Snapshot erzeugt
 - ✓ Simulation durchgeführt
 - ✓ Twin-Berichte erzeugt
-

Enterprise Rules

Kein Zwilling ohne Ursprung.

Keine Freigabe bei Drift.

Keine manuelle Änderung des Zwillings.

Keine Architekturentscheidung ohne Zwilling.

Keine Migration ohne geplanten Zwilling.

Keine Version ohne Snapshot.

Der Code besitzt jederzeit Vorrang vor dem Modell.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ jede Komponente einen Digital Twin besitzt
 - ✓ sämtliche Zwillinge synchron sind
 - ✓ Drift automatisch erkannt wird
 - ✓ jeder historische Zustand rekonstruierbar ist
 - ✓ geplante Änderungen vor der Umsetzung bewertbar sind
 - ✓ sämtliche Diagramme aus dem Zwilling erzeugt werden
 - ✓ sämtliche KI-Systeme denselben Zwilling verwenden
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 19

Enterprise Automation Contracts

↓

Chapter 20

Official Enterprise Standard

End of Chapter 18

Chapter 19

Enterprise Automation Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen Automatisierungsverträge des CAPITAL-AI Core.

Automatisierung ist kein Werkzeug.

Automatisierung ist die Ausführungsform der Enterprise Contracts.

Jede in diesen Contracts definierte Regel besitzt eine automatisierte Entsprechung.

Manuelle Ausführung ist ausschließlich als Notfallmaßnahme zulässig.

Mission

Die Automation Contracts gewährleisten

- reproduzierbare Ausführung sämtlicher Plattformprozesse
 - deterministische Ergebnisse unabhängig vom Ausführenden
 - vollständige Protokollierung jeder Ausführung
 - automatische Auslösung durch Enterprise Events
 - Wiederherstellbarkeit des gesamten Repository-Gerüsts
 - Unabhängigkeit von einzelnen Personen oder KI-Systemen
-

Enterprise Principle

Was nicht automatisiert ist, ist nicht durchgesetzt.

Jeder automatisierte Prozess ist idempotent.

Jeder automatisierte Prozess ist versioniert.

Jeder automatisierte Prozess erzeugt Events.

Automation Scope

Verbindlich automatisiert werden

Repository Discovery

Code Discovery

Metadatenerzeugung
Registry-Aufbau
Knowledge Graph Aufbau
Digital Twin Synchronisation
Dokumentationserzeugung
Diagrammerzeugung
Contract Validation
Architecture Validation
Quality Gates
Versionsbewertung
Changelog-Erzeugung
Release-Vorbereitung
Governance Reports
Security Reports
Compliance Reports

Automation Location

Automatisierte Prozesse befinden sich ausschließlich unter

```
scripts/
```

Die verbindliche Aufteilung lautet

scripts/automation/	Enterprise-Prozesse und Bootstrapping
scripts/validation/	Validatoren und Quality Gates
scripts/migration/	Migrations- und Rollback-Prozesse
scripts/deployment/	Release- und Deployment-Prozesse
scripts/maintenance/	Wartung, Bereinigung, Reparatur

Die zugehörige Logik befindet sich in den Plattformmodulen.

Skripte enthalten niemals Geschäftslogik.

Process Contract

Jeder automatisierte Prozess besitzt verbindlich

Process ID

Name

Version

Zweck

Eingaben

Ausgaben

Voraussetzungen

Auslöser

Ergebnisstruktur

Fehlerverhalten

Owner

Determinism Contract

Jeder Prozess erfüllt verbindlich

gleiche Eingabe erzeugt gleiche Ausgabe

keine Abhängigkeit von der Ausführungsreihenfolge

keine Abhängigkeit von Zufallswerten

keine Abhängigkeit von Systemzeit außerhalb von Zeitstempeln

keine Abhängigkeit von Netzwerkzuständen ohne Deklaration

keine Abhängigkeit vom ausführenden KI-System

Idempotency Contract

Jeder Prozess ist wiederholbar.

Eine wiederholte Ausführung ohne Eingabeänderung

erzeugt keine zusätzlichen Artefakte

erzeugt keine doppelten Registry-Einträge

erzeugt keine doppelten Knowledge Nodes

erzeugt keine doppelten Dokumente

verändert keine Versionen

Bootstrapper Contract

Das vollständige Enterprise-Gerüst wird verbindlich reproduzierbar erzeugt.

Der Bootstrapper erzeugt

sämtliche Pflichtmodule

sämtliche Pflichtverzeichnisse

sämtliche Manifeste

sämtliche Component Descriptors

sämtliche READMEs

sämtliche CHANGELOGs

sämtliche Registry-Einträge

Der Bootstrapper überschreibt niemals

vorhandene Implementierungen

vorhandene Dokumentationsinhalte

vorhandene Versionen

vorhandene ADRs

Fehlende Artefakte werden ergänzt, vorhandene niemals ersetzt.

Generator Contract

Generatoren erzeugen ausschließlich Artefakte, niemals Entscheidungen.

Jeder Generator

besitzt genau einen Ausgabebetyp

besitzt eine Version

kennzeichnet erzeugte Artefakte als generiert

speichert Ausgaben ausschließlich in definierten Generatorbereichen

überschreibt niemals produktiven Code

Generatoren befinden sich unter

```
src/platform/Generators/
```

Generated Artifact Contract

Jedes generierte Artefakt enthält verbindlich

Generator ID

Generator Version

Erzeugungszeitpunkt

Quellreferenz

Knowledge Version

Repository Version

Generierte Artefakte werden niemals manuell bearbeitet.

Trigger Contract

Automatisierte Prozesse werden ausschließlich durch

Enterprise Events

Zeitpläne

ausdrückliche Freigaben

ausgelöst.

Verbindliche Zuordnung

Auslöser	Prozess
RepositoryScannedEvent	Metadaten- und Registry-Aktualisierung
ImplementationCompletedEvent	Validierung und Dokumentation
KnowledgeUpdatedEvent	Digital Twin Synchronisation
TwinSynchronizedEvent	Berichtserzeugung
VersionChangedEvent	Changelog und Release-Vorbereitung
StructureViolationDetectedEvent	Governance Report
TwinDriftDetectedEvent	Drift Report und Korrekturauftrag

Scheduling Contract

Zeitgesteuerte Prozesse besitzen verbindlich

Zeitplan

Zeitzone

maximale Laufzeit

Überlappungsschutz

Ausfallverhalten

Benachrichtigung bei Fehlschlag

Ein Prozess läuft niemals mehrfach gleichzeitig.

Execution Contract

Jede Ausführung protokolliert verbindlich

Process ID

Process Version

Auslöser

Correlation ID

Startzeit

Endzeit

Ergebnis

erzeugte Artefakte

erzeugte Events

Befunde

Failure Contract

Fehlgeschlagene Prozesse

hinterlassen niemals unvollständige Artefakte

setzen niemals Versionen

verändern niemals die Registry teilweise

erzeugen verbindlich ein Fehler-Event

erzeugen verbindlich einen Befund

Teilweise ausgeführte Prozesse werden vollständig zurückgesetzt.

Recovery Contract

Für jeden Prozess existiert ein definiertes Wiederanlaufverhalten.

Zulässig sind

vollständige Wiederholung

Fortsetzung ab dokumentiertem Prüfpunkt

manuelle Freigabe nach Prüfung

Nicht zulässig ist eine stillschweigende Fortsetzung nach Fehler.

Manual Execution

Manuelle Ausführung ist ausschließlich zulässig

im Notfallbetrieb

bei ausgefallener Automatisierung

nach ausdrücklicher Freigabe

Jede manuelle Ausführung wird verbindlich als Event protokolliert und im Governance Report ausgewiesen.

Automation Events

Verbindlich sind

ProcessStartedEvent

ProcessCompletedEvent

ProcessFailedEvent

ArtifactGeneratedEvent

BootstrapCompletedEvent

ScheduleTriggeredEvent

ManualExecutionEvent

Documentary Integration

Die Documentary Engine erzeugt automatisch

Automation Report

Execution History

Generator Report

Failure Report

Coverage der Automatisierung

Die Automatisierung dokumentiert sich dadurch selbst.

Supervisor Integration

Der Supervisor überwacht

fehlgeschlagene Prozesse

nicht ausgeführte Zeitpläne

überfällige Synchronisationen

manuelle Eingriffe

nicht idempotente Ergebnisse

Platform Director Integration

Der Platform Director entscheidet über

neue automatisierte Prozesse

Zeitpläne

Notfallfreigaben

Priorisierung der Automatisierung

AI Automation Contract

KI-Systeme dürfen

✓ Prozesse ausführen

✓ Prozesse erzeugen

✓ Fehlerursachen analysieren

✓ Korrekturen vorschlagen

Sie dürfen jedoch nicht

- ✗ Prozesse deaktivieren
 - ✗ Zeitpläne eigenständig ändern
 - ✗ Ergebnisse ohne Ausführung erzeugen
 - ✗ Fehlerprotokolle verändern
 - ✗ manuelle Ausführungen als automatisiert ausweisen
-

Validation

Vor jeder Aufnahme eines Prozesses wird geprüft

- ✓ Process Contract vollständig
 - ✓ Determinismus nachgewiesen
 - ✓ Idempotenz nachgewiesen
 - ✓ Auslöser definiert
 - ✓ Fehlerverhalten definiert
 - ✓ Wiederanlauf definiert
 - ✓ Events definiert
 - ✓ Owner benannt
 - ✓ Version gesetzt
-

Enterprise Rules

Kein Contract ohne automatisierte Durchsetzung.

Kein Prozess ohne Version.

Kein Prozess ohne Auslöser.

Kein Prozess ohne Protokoll.

Keine Generierung außerhalb der Generatorbereiche.

Keine manuelle Ausführung ohne Protokollierung.

Kein Bootstrapping überschreibt Bestand.

Success Criteria

Dieses Kapitel gilt als erfüllt wenn

- ✓ das Enterprise-Gerüst reproduzierbar erzeugt werden kann
 - ✓ sämtliche Contracts automatisiert geprüft werden
 - ✓ sämtliche Dokumentation automatisch entsteht
 - ✓ sämtliche Prozesse idempotent sind
 - ✓ sämtliche Ausführungen protokolliert werden
 - ✓ manuelle Eingriffe die Ausnahme bleiben
 - ✓ die Plattform ohne personelle Abhängigkeit betrieben werden kann
-

Integration

Dieses Kapitel bildet die Grundlage für

Chapter 20

Official Enterprise Standard

End of Chapter 19

Chapter 20

Official Enterprise Standard

Enterprise Purpose

Dieses Kapitel schließt die Enterprise Technical Contracts des CAPITAL-AI Core ab.

Es fasst keine Inhalte zusammen.

Es definiert die verbindliche Geltung, die Rangfolge, die Konformitätsstufen und die Weiterentwicklung des Standards.

Mit Abschluss dieses Kapitels bilden ESS-0001 und ESS-0001-CONTRACTS gemeinsam den offiziellen Enterprise Standard des CAPITAL-AI Core.

Standard Definition

Der offizielle Enterprise Standard besteht verbindlich aus

ESS-0001

Enterprise-Architektur der Documentary Engine

ESS-0001-CONTRACTS

Enterprise Technical Contracts

sämtliche verabschiedeten ADRs

sämtliche weiteren ESS-Dokumente gemäß ESS Registry

Kein weiteres Dokument besitzt normative Wirkung.

Scope of Application

Der Standard gilt verbindlich für

sämtliche Plattformkomponenten

sämtliche Feature-Domänen

sämtliche Erweiterungen

sämtliche automatisierten Prozesse

sämtliche KI-Systeme

sämtliche Repositories des CAPITAL-AI Ökosystems

Ausnahmen bestehen ausschließlich in registrierter Form gemäß Chapter 16.

Authority

Die Rangfolge aus Chapter 1 gilt unverändert.

ADR

↓

ESS Contracts



Enterprise Specifications



Projektdokumentation



Implementierung

Bei Widersprüchen gilt jederzeit die höher priorisierte Ebene.

Normative Clarifications

Die folgenden Klarstellungen beseitigen Auslegungslücken.

Sie ändern keine bestehende Regel.

Event Naming

ESS-0001 Chapter 8 benennt Ereigniskategorien.

ESS-0001-CONTRACTS Chapter 8 definiert die verbindliche technische Schreibweise.

Verbindlich gilt für sämtliche implementierten Ereignisse das Suffix

Event

Die Bezeichnungen in ESS-0001 Chapter 8 sind als Kategorienamen zu lesen.

Core Module

`src/platform/Core` ist gemäß Chapter 3 und Chapter 6 verbindlicher Bestandteil der Plattform.

Core wird als technische Basisschicht geführt, nicht als fachliches Modul.

Cross Cutting Modules

Die Layer-Zuordnung der Querschnittsmodule ist in Chapter 16 verbindlich definiert.

Chapter 6 bleibt unverändert gültig.

Lifecycle und Version Category

Lifecycle beschreibt die Komponente.

Version Category beschreibt die Version.

Die verbindliche Zuordnung ist in Chapter 14 definiert.

ESS Numbering

Die in ESS-0001 reservierten Nummern ESS-0002 bis ESS-0009 sind verbindlich.

Der freie Nummernraum beginnt bei ESS-0010.

Die Vergabe erfolgt ausschließlich über die ESS Registry gemäß Chapter 16.

Chapter Index

Der Standard umfasst verbindlich folgende Kapitel.

Kapitel	Gegenstand
1	Enterprise Foundation & Governance Contracts
2	Repository Structure Contract
3	Directory Responsibility Contracts
4	TypeScript & Interface Contracts
5	Enterprise Naming Contracts
6	Dependency & Layer Contracts
7	Enterprise Data Model & Metadata Contracts
8	Enterprise Event & Messaging Contracts
9	Enterprise Versioning & Release Contracts
10	Enterprise AI Governance & Documentary Contracts
11	Enterprise Security & Compliance Contracts
12	Enterprise Validation & Quality Contracts
13	Enterprise Plugin & Extension Contracts
14	Enterprise Migration & Lifecycle Contracts
15	Enterprise Knowledge Graph Contracts
16	Enterprise Repository Governance
17	Enterprise AI Orchestration Contracts
18	Enterprise Digital Twin Contracts
19	Enterprise Automation Contracts

Kapitel	Gegenstand
20	Official Enterprise Standard

Completeness Assessment

Vor Abschluss dieses Standards wurde geprüft, ob weitere Kapitel erforderlich sind.

Zwei Regelbereiche wurden zusätzlich untersucht.

Registry & Discovery

Registry und Discovery besitzen verbindliche Verträge in

Chapter 7 — Enterprise Registry

Chapter 15 — Registry Relationship

Chapter 16 — Repository Governance

Ein eigenständiges Kapitel würde bestehende Regeln wiederholen und damit gegen das Verbot redundanter Dokumente verstoßen.

Ergebnis

Kein zusätzliches Kapitel erforderlich.

Observability & Telemetry

Beobachtbarkeit besitzt verbindliche Verträge in

Chapter 12 — Observability Contract

Chapter 19 — Execution Contract

Chapter 18 — Twin States

Ergebnis

Kein zusätzliches Kapitel erforderlich.

Conformance Levels

Der Standard definiert verbindlich drei Konformitätsstufen.

Level 1 — Structural Conformance

Repository-Struktur erfüllt

Verzeichnisverantwortung erfüllt

Namensverträge erfüllt

Metadaten vollständig

Registry-Einträge vorhanden

Level 2 — Operational Conformance

Level 1 erfüllt

Enterprise Events implementiert

Validatoren ausführbar

Quality Gates ausführbar

Versionierung automatisiert

Dokumentation generiert

Level 3 — Enterprise Conformance

Level 2 erfüllt

Knowledge Graph vollständig

Digital Twin synchron

AI-Wertschöpfungskette vollständig automatisiert

Security und Compliance vollständig nachweisbar

sämtliche Prozesse idempotent und protokolliert

Die Konformitätsstufe jeder Komponente wird in den Metadaten geführt.

Standard Validation

Der Standard gilt als eingehalten wenn

- ✓ sämtliche Kapitel durch Validatoren abgedeckt sind
- ✓ sämtliche Komponenten eine Konformitätsstufe besitzen
- ✓ sämtliche Abweichungen registriert sind
- ✓ sämtliche Ausnahmen durch ADR gedeckt sind
- ✓ sämtliche Berichte automatisch erzeugt werden

✓ der Digital Twin den tatsächlichen Zustand abbildet

Change Contract

Änderungen an diesem Standard erfolgen ausschließlich über einen ADR

eine Versionserhöhung dieses Dokumentes

eine Aktualisierung der ESS Registry

eine Aktualisierung des Knowledge Graph

eine Aktualisierung des Digital Twin

Nicht zulässig sind

stillschweigende Änderungen

lokale Sonderregeln

abweichende Auslegungen einzelner KI-Systeme

nachträgliche Umnummerierung von Kapiteln

Extension Contract

Neue Kapitel werden ausschließlich am Ende ergänzt.

Chapter 20 bleibt dabei das abschließende Kapitel des Standards und wird entsprechend fortgeschrieben.

Bestehende Kapitelnummern bleiben dauerhaft stabil.

Deprecation Contract

Regeln dieses Standards werden niemals ersatzlos entfernt.

Eine Regel wird ausschließlich

durch eine neue Regel ersetzt

als veraltet gekennzeichnet

mit Übergangsfrist versehen

durch ADR dokumentiert

Enterprise Rules

Der Standard ist verbindlich für Mensch und KI.

Kein Dokument außerhalb des Standards besitzt normative Wirkung.

Keine Änderung ohne ADR.

Keine Auslegung ohne Klarstellung im Standard.

Keine Konformitätsaussage ohne Validierung.

Keine Komponente ohne Konformitätsstufe.

Success Criteria

Der Standard gilt als vollständig etabliert wenn

- ✓ sämtliche Enterprise-Komponenten nach diesem Standard beschrieben sind
 - ✓ sämtliche Regeln maschinell durchgesetzt werden
 - ✓ sämtliche Änderungen automatisch dokumentiert werden
 - ✓ sämtliche Änderungen automatisch versioniert werden
 - ✓ sämtliche Änderungen automatisch in den Digital Twin übernommen werden
 - ✓ jede unterstützte KI identische Ergebnisse erzeugt
 - ✓ die Plattform sich vollständig selbst beschreibt
-

End of Chapter 20

Enterprise Final Statement

Governance Statement

ESS-0001-CONTRACTS ist die verbindliche technische Vertragsgrundlage des CAPITAL-AI Core.

Sämtliche Implementierungen, Erweiterungen und automatisierten Prozesse müssen mit diesem Standard vereinbar sein.

Abweichungen erfordern einen Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Chapter 1 bis Chapter 10 — Enterprise Baseline Specification
1.1.0	Enterprise Extension	Chapter 11 bis Chapter 20 — Security, Qualität, Erweiterungen, Migration, Knowledge Graph, Repository Governance, AI Orchestration, Digital Twin, Automation, Enterprise Standard

Related Enterprise Specifications

- ESS-0001 — Documentary & Code Intelligence Architect
- ESS-0002 — Supervisor Architect
- ESS-0003 — Platform Director
- ESS-0004 — Enterprise Version Manager
- ESS-0005 — Quality Center
- ESS-0006 — Security & Compliance
- ESS-0007 — Enterprise Release Center
- ESS-0008 — AI Agent Framework
- ESS-0009 — Enterprise Knowledge Platform
- ESS-0010 — Repository Governance

Approval

- Document Status
- APPROVED
- Enterprise Standard
- CAPITAL-AI Core Architecture
- Version 1.1.0

End of Document

ESS-0001-CONTRACTS

CAPITAL-AI Enterprise Technical Contracts

Version 1.1.0

ESS 0001 Documentary Architect

Quelle: .ai/skills/ESS-0001-Documentary-Architect.md

skill: id: ESS-0001 name: Documentary & Code Intelligence Architect version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority: Critical

capital_ai: platform: CAPITAL-AI Core repository: Finance-main architecture: Enterprise lifecycle: AI Native Development Lifecycle

authority:

controls: - Documentation Center - Code Intelligence Center - Knowledge Center - Architecture Center

collaborates: - Platform Director - Supervisor - Version Manager - Security Center - Compliance Center - Architecture Director - Quality Center

cannot_modify: - Trading Logic - Portfolio Logic - Crypto Scoring Algorithms - Stripe Billing Logic - OAuth Authentication - Database Business Logic

classification: type: Foundational Architecture Document role: Gründungs- und Visionsdokument der Documentary Engine technicalAuthority: ESS-0010 note: > Ergänzt am 2026-07-31 durch ADR-0013. Die Klassifizierung präzisiert die Verantwortung dieses Dokumentes. Es wurde kein bestehender Inhalt verändert.

crossReference: dependsOn: [] relatedEss: - ESS-0001-CONTRACTS - ESS-0002 - ESS-0003 - ESS-0010 relatedAdr: - ADR-0010 - ADR-0013 relatedComponents: - src/platform/Documentary - src/platform/Knowledge - src/platform/Discovery - src/platform/Architecture relatedSkills: - .ai/skills/ESS-0001-Contracts.md - .ai/skills/ESS-0010-Documentary-Engine.md

created: 2026-07-28

Documentary & Code Intelligence Architect

Dokumentklassifizierung

Dieses Dokument ist das **Foundational Architecture Document** der CAPITAL-AI Documentary Engine.

Es beschreibt Motivation, Zielbild, Architekturidee, ursprüngliche Vision und Designprinzipien.

Es besitzt unverändert Gültigkeit als Gründungsdokument der Plattformarchitektur.

Verhältnis zur technischen Spezifikation

Die Kapitel 2 bis 9 dieses Dokumentes beschreiben den ursprünglichen technischen Architekturentwurf aus dem Jahr 2026.

Für die verbindliche technische Spezifikation der Documentary Engine gilt ausschließlich

ESS-0010 — Documentary Engine

Bei Abweichungen zwischen diesem Dokument und ESS-0010 besitzt ESS-0010 Vorrang.

Für globale Enterprise Contracts gilt ausschließlich ESS-0001-CONTRACTS.

Diese Vorrangregel folgt der bereits in ESS-0001-CONTRACTS Chapter 20 etablierten Systematik der normativen Klarstellung. Sie ändert keine Aussage dieses Dokumentes und entfernt keinen Inhalt.

Enterprise Mission

Der Documentary & Code Intelligence Architect ist die zentrale Wissens- und Dokumentationsinstanz der CAPITAL-AI Plattform.

Seine Aufgabe besteht NICHT darin, Dokumente zu schreiben.

Seine Aufgabe besteht darin, aus dem Quellcode, der Architektur und den Ereignissen der Plattform jederzeit den vollständigen Wissensstand automatisch abzuleiten.

Die Documentary Engine ist deshalb Bestandteil der Plattformarchitektur und kein nachgelagertes Dokumentationswerkzeug.

Vision

Jede Änderung innerhalb der CAPITAL-AI Plattform erzeugt automatisch nachvollziehbares Wissen.

Dieses Wissen muss sowohl für Menschen als auch für KI-Systeme verständlich, reproduzierbar und maschinell auswertbar sein.

Dokumentation wird niemals manuell gepflegt.

Dokumentation entsteht ausschließlich aus:

- Quellcode
- Architektur
- Events
- Versionierung
- Repository-Struktur
- Metadaten

- ADRs
 - Releases
-

Enterprise Philosophy

Die CAPITAL-AI Plattform folgt dem Grundsatz

Code ist die Wahrheit.

Dokumentation beschreibt niemals den Code.

Dokumentation wird aus dem Code erzeugt.

Dadurch existiert niemals eine Dokumentation, welche nicht dem tatsächlichen System entspricht.

Documentation as Code

Die Documentary Engine betrachtet Dokumentation als Bestandteil des Produktes.

Folgende Artefakte besitzen denselben Stellenwert:

- Source Code
- Dokumentation
- ADR
- Architekturdiagramme
- Releaseinformationen
- Changelog
- API Dokumentation
- Datenbankbeschreibung

Alle Artefakte unterliegen derselben Versionierung.

Code Intelligence

Die Documentary Engine besitzt vollständige Kenntnis über:

Repository

↓

Ordnerstruktur



Dateien



Module



Klassen



Interfaces



Methoden



Events



Abhängigkeiten



APIs



Datenbank



Agenten



Orchestratoren



Workflows



Versionen

Die Plattform dokumentiert deshalb nicht nur Ergebnisse, sondern versteht ihre eigene Architektur.

AI Native Development

Die Documentary Engine wird für KI-Systeme entwickelt.

Jede Information muss

- eindeutig
- maschinenlesbar
- versioniert
- reproduzierbar

sein.

Neben Markdown erzeugt die Documentary Engine deshalb strukturierte Wissensmodelle.

Core Objectives

Die Documentary Engine verfolgt ausschließlich folgende Ziele.

Architektur verstehen

Die Plattform analysiert ihre vollständige Architektur.

Wissen erzeugen

Aus jeder Änderung entsteht Wissen.

Wissen validieren

Dokumentation wird automatisch geprüft.

Wissen versionieren

Jede Dokumentation besitzt eine Version.

Wissen verteilen

Supervisor, Platform Director und zukünftige KI-Agenten erhalten denselben Wissensstand.

Enterprise Responsibilities

Der Skill besitzt Verantwortung für

- Documentary Engine
- Code Intelligence

- Architecture Knowledge
 - Documentation Lifecycle
 - Knowledge Lifecycle
 - Event Documentation
 - ADR Lifecycle
 - Changelog
 - Release Notes
 - Architecture Reports
 - Mermaid Diagramme
 - Machine Readable Knowledge
-

Nicht Bestandteil

Der Skill verändert niemals

- Businesslogik
- Finanzalgorithmen
- KI-Modelle
- Datenbankinhalte
- Benutzerdaten
- Authentifizierung
- Berechtigungen
- Stripe
- Zahlungslogik

Die Documentary Engine beobachtet ausschließlich.

Enterprise Principles

Alle zukünftigen Erweiterungen müssen folgende Prinzipien erfüllen.

Single Source of Truth

Der Quellcode ist die einzige Wahrheit.

Event Driven

Jede Dokumentation entsteht durch Events.

Documentation by Observation

Dokumente werden niemals manuell erzeugt.

Zero Duplication

Information darf nur einmal existieren.

Machine First

Jede Dokumentation muss maschinenlesbar sein.

Human Friendly

Alle erzeugten Dokumente müssen zusätzlich für Entwickler verständlich sein.

Version Controlled

Jede Änderung besitzt eine nachvollziehbare Versionshistorie.

Self Validation

Die Documentary Engine validiert ihre eigenen Ergebnisse.

Enterprise Success Criteria

Die Documentary Engine gilt als erfolgreich implementiert wenn

- ✓ jede Architekturänderung erkannt wird
- ✓ jede Änderung automatisch dokumentiert wird
- ✓ jede Dokumentation validiert wird
- ✓ jede Version nachvollziehbar bleibt
- ✓ jeder Agent dieselbe Wissensbasis verwendet
- ✓ Platform Director und Supervisor jederzeit den aktuellen Architekturzustand kennen

✓ kein Dokument manuell gepflegt werden muss

End of Chapter 1

Chapter 2

Repository Intelligence & Code Discovery Engine

Enterprise Purpose

Die Repository Intelligence & Code Discovery Engine bildet den Einstiegspunkt der gesamten Documentary Engine.

Sie besitzt die alleinige Verantwortung für die vollständige Analyse des CAPITAL-AI Repositories.

Keine andere Komponente darf Repositoryinformationen eigenständig sammeln oder interpretieren.

Alle nachfolgenden Module (Code Intelligence, Knowledge Engine, Documentation Engine, Version Manager, Supervisor und Platform Director) verwenden ausschließlich die Ergebnisse der Repository Intelligence Engine.

Mission

Vor jeder Analyse muss die Repository Intelligence Engine den vollständigen technischen Zustand des Projektes bestimmen.

Die Analyse erfolgt vollständig automatisiert.

Es werden keinerlei Annahmen getroffen.

Die Plattform dokumentiert ausschließlich tatsächlich vorhandene Informationen.

Enterprise Principle

Das Repository stellt die einzige technische Wahrheit der Plattform dar.

Alle Informationen werden ausschließlich aus:

- Quellcode
- Dateistruktur

- Metadaten
- Konfiguration
- Buildsystem
- Versionierung
- Imports
- Exports
- Kommentaren
- Events

abgeleitet.

Manuell gepflegte Informationen besitzen keine Priorität gegenüber dem Repository.

Discovery Lifecycle

Repository



Repository Scanner



Metadata Scanner



File Scanner



Structure Scanner



Dependency Scanner



Technology Scanner



Classification Engine



Knowledge Graph



Code Intelligence Engine

Repository Scan

Zu Beginn jeder Ausführung wird das vollständige Repository analysiert.

Die Analyse umfasst sämtliche Dateien.

Ausnahmen werden ausschließlich durch den Platform Director definiert.

Zu analysierende Bereiche

Projektstruktur

- Root Directory
 - src
 - server
 - client
 - shared
 - docs
 - .ai
 - scripts
 - public
 - assets
 - migrations
 - configuration
-

Konfigurationsdateien

Automatische Erkennung:

package.json

package-lock.json

pnpm-lock.yaml

tsconfig.json

vite.config.*

docker-compose.*

Dockerfile
.env.example
render.yaml
supabase/
github workflows
eslint
prettier

Source Code

Erkennen:

TypeScript

JavaScript

React

Node

Express

Supabase

SQL

Markdown

YAML

JSON

Shell

PowerShell

Python

Automatische Klassifizierung

Jede Datei erhält exakt eine Hauptklassifizierung.

Beispiele

Component

Service

Agent
Orchestrator
Controller
Middleware
Hook
Provider
Model
DTO
Repository
Migration
Configuration
Documentation
ADR
Prompt
Skill
Template
Test
Script
Unknown

Repository Metadata

Für jede Datei werden automatisch gespeichert

Dateiname
Pfad
Dateityp
Programmiersprache
Größe
Hash
Erstellungsdatum

Änderungsdatum

Version

Owner

Abhängigkeiten

Importe

Exporte

Referenzen

Dokumentationsstatus

Relationship Discovery

Die Engine analysiert automatisch

welche Datei

welche Datei verwendet.

Es entstehen Beziehungen zwischen

Komponenten

↓

Services

↓

Orchestratoren

↓

Agenten

↓

Events

↓

Datenbank

↓

API

↓

Frontend



Backend

Technology Detection

Die Plattform erkennt automatisch

Frontend Framework

Backend Framework

State Management

Buildsystem

Testing Framework

Package Manager

Container Runtime

Cloud Provider

LLM Provider

Database

Authentication

Payment Provider

Deployment Plattform

AI Component Discovery

Automatisch erkennen

Platform Director

Supervisor

Version Manager

Orchestratoren

Agenten

Prompt Registry

Knowledge Base

Documentary Engine

Compliance Engine

Security Center

Release Center

Quality Center

Architecture Center

Event Source Discovery

Automatisch erkennen

Event Emitter

Event Listener

Scheduler

Cron Jobs

Queues

Webhooks

Realtime Listener

Observer

Trigger

API Discovery

Automatisch erkennen

REST

RPC

Webhook

Internal API

External API

Supabase Functions

Express Router

Middleware

Rate Limiter

Authentication

Authorization

Database Discovery

Automatisch erkennen

Supabase

Migrationen

SQL

Views

Functions

Policies

RLS

Storage

Buckets

Trigger

Indexes

Constraints

Documentation Discovery

Automatisch erkennen

README

ADR

CHANGELOG

RELEASE NOTES

MERMAID

Architecture

Runbooks

Knowledge

Skills

Prompts

Policies

Existing Documentary Detection

Die Engine prüft automatisch

welche Documentary-Komponenten bereits existieren.

Insbesondere

documentHygiene

documentSanitizer

systemEvents

Documentary

Audit

Knowledge

ADR

Architecture Reports

dürfen nicht neu entwickelt werden,

wenn eine produktionsreife Implementierung vorhanden ist.

Duplicate Detection

Vor jeder Migration wird geprüft

existiert diese Funktion bereits?

existiert ähnlicher Code?

existiert eine ältere Version?

existiert dieselbe Dokumentation mehrfach?

existiert bereits ein Agent?

existiert bereits ein Service?

Repository Classification Report

Nach Abschluss der Analyse wird automatisch erzeugt

Repository Report

Component Inventory

Service Inventory

Agent Inventory

Orchestrator Inventory

API Inventory

Database Inventory

Dependency Inventory

Documentation Inventory

Migration Inventory

Output

Die Repository Intelligence Engine liefert ausschließlich strukturierte Daten.

Keine Markdown-Dokumente.

Keine Reports.

Keine ADR.

Die Ausgabe erfolgt als interne Datenmodelle.

Diese werden anschließend von

Code Intelligence

Knowledge Engine

Documentation Engine

Version Manager

Supervisor

Platform Director

weiterverarbeitet.

Success Criteria

Die Repository Intelligence Engine gilt als erfolgreich,
wenn

- ✓ sämtliche Dateien erkannt wurden
 - ✓ sämtliche Technologien erkannt wurden
 - ✓ sämtliche Komponenten klassifiziert wurden
 - ✓ sämtliche Beziehungen erkannt wurden
 - ✓ sämtliche bestehenden Documentary-Komponenten erkannt wurden
 - ✓ keine Datei doppelt klassifiziert wurde
 - ✓ keine Repositorybereiche ausgelassen wurden
 - ✓ der vollständige technische Zustand des Projektes reproduzierbar erfasst wurde
-

End of Chapter 2

Chapter 3

Code Intelligence Engine

Enterprise Purpose

Die Code Intelligence Engine ist das zentrale Analysemodul der CAPITAL-AI Documentary Engine.

Ihre Aufgabe besteht darin, den vollständigen Quellcode semantisch zu verstehen.

Sie analysiert nicht nur Dateien.

Sie analysiert die Architektur, die Beziehungen und das Verhalten der Plattform.

Die Code Intelligence Engine erzeugt dadurch den vollständigen technischen Wissensgraphen der CAPITAL-AI Plattform.

Mission

Die Engine beantwortet jederzeit folgende Fragen:

Welche Komponenten existieren?

Welche Verantwortung besitzt jede Komponente?

Welche Abhängigkeiten existieren?

Welche Komponenten kommunizieren miteinander?

Welche Events werden ausgelöst?

Welche Services sind kritisch?

Welche Komponenten besitzen technische Schulden?

Welche Komponenten müssen dokumentiert werden?

Enterprise Principle

Code besitzt Bedeutung.

Die Documentary Engine analysiert deshalb nicht Zeichenketten.

Sie analysiert semantische Zusammenhänge.

Semantic Analysis Pipeline

Repository

↓

Parser

↓

Abstract Syntax Tree (AST)

↓

Semantic Analyzer

↓

Relationship Analyzer

↓

Architecture Analyzer

↓

Knowledge Graph

↓

Documentation Engine

Source Code Parsing

Automatisch analysieren

TypeScript

JavaScript

React

Node

Express

SQL

JSON

YAML

Markdown

Python

Shell

PowerShell

AST Analysis

Für jede Datei wird ein vollständiger Abstract Syntax Tree erzeugt.

Es werden erkannt

Imports

Exports

Klassen

Interfaces

Enums

Typen

Funktionen

Methoden

Properties

Namespaces

Module

Generics

Decorators

Component Discovery

Automatisch erkennen

Services

Controllers

Middleware

Hooks

Utilities

Repositories

Providers

DTO

Entities

Schemas

Configuration

Plugins

Factories

Builders

Strategies

Observers

Service Intelligence

Für jeden Service werden automatisch erfasst

Verantwortung

öffentliche Methoden

interne Methoden

Dependencies

Events

API Nutzung

Datenbankzugriffe

externe Provider

Konfiguration

Risiko

Dokumentationsstatus

Agent Intelligence

Automatisch analysieren

Agent ID

Agent Typ

Owner

Verantwortung

Eingaben

Ausgaben

Trigger

Events

Abhängigkeiten

Health Status

Supervisor Registrierung

Version

ADR

Knowledge Entry

Orchestrator Intelligence

Automatisch erkennen

Crypto Orchestrator

Portfolio Orchestrator
Compliance Orchestrator
Platform Director
Supervisor
Version Manager
AI Gateway
Billing
News
Workflow Engine

Event Intelligence

Automatisch erkennen
Emitter
Listener
Dispatcher
Observer
Queue
Retry
Scheduler
Cron
Realtime
Lifecycle

API Intelligence

Automatisch analysieren
REST Endpunkte
Express Router
Middleware
Authentication

Authorization

Rate Limiter

Versionierung

Request

Response

DTO

Swagger

OpenAPI

Database Intelligence

Automatisch erkennen

Tabellen

Views

Functions

RPC

RLS

Policies

Constraints

Indexes

Relations

Migrationen

Dependency Intelligence

Automatisch erzeugen

Class Dependency Graph

Service Dependency Graph

Module Dependency Graph

Event Dependency Graph

Agent Dependency Graph

Orchestrator Dependency Graph

Database Dependency Graph

Frontend Dependency Graph

Backend Dependency Graph

Architecture Intelligence

Die Engine erkennt automatisch

Layer Violations

Circular Dependencies

Dead Code

Duplicate Code

Missing Documentation

Missing Tests

Missing ADR

Missing Events

Deprecated Components

Legacy Code

Behaviour Analysis

Die Documentary Engine analysiert zusätzlich

Startsequenzen

Registrierungen

Initialisierung

Dependency Injection

Service Discovery

Plugin Discovery

Lifecycle

Shutdown

Recovery

Naming Intelligence

Automatisch prüfen

Klassen

Methoden

Variablen

Interfaces

Ordner

Module

Events

Services

Dateien

Namenskonventionen

Documentation Coverage

Für jede Komponente wird geprüft

existiert

ADR

Architecture Description

API Dokumentation

Knowledge Entry

Version

Owner

Event Registrierung

Supervisor Registrierung

Machine Readable Output

Die Code Intelligence Engine erzeugt ausschließlich strukturierte Modelle.

Beispiele

components.json

classes.json

interfaces.json

services.json

events.json

dependencies.json

apis.json

database.json

agents.json

orchestrators.json

architecture.json

code-quality.json

technical-debt.json

Architectural Memory

Die Engine speichert zusätzlich

wann

warum

durch wen

eine Komponente verändert wurde.

Dadurch entsteht eine vollständige Evolutionshistorie der Plattform.

Enterprise Rules

Keine Interpretation ohne Quellcode.

Keine Dokumentation ohne Analyse.

Keine Klassifizierung ohne AST.

Keine Architekturentscheidung ohne Dependency Analyse.

Keine Migration ohne vollständiges Komponentenverständnis.

Success Criteria

Die Code Intelligence Engine gilt als erfolgreich wenn

- ✓ sämtliche Klassen erkannt wurden
 - ✓ sämtliche Interfaces erkannt wurden
 - ✓ sämtliche Services erkannt wurden
 - ✓ sämtliche Agenten erkannt wurden
 - ✓ sämtliche Orchestratoren erkannt wurden
 - ✓ sämtliche Events erkannt wurden
 - ✓ sämtliche APIs erkannt wurden
 - ✓ sämtliche Datenbankobjekte erkannt wurden
 - ✓ sämtliche Abhängigkeiten erkannt wurden
 - ✓ sämtliche Architekturverletzungen erkannt wurden
 - ✓ der vollständige Wissensgraph reproduzierbar aufgebaut werden kann
-

Integration

Die Code Intelligence Engine liefert ihre Ergebnisse ausschließlich an

Repository Intelligence

↓

Knowledge Engine

↓

Documentation Engine

↓

Version Manager

↓

Supervisor

↓

Platform Director

Sie erzeugt selbst keine Dokumentation.

Sie erzeugt ausschließlich Wissen.

End of Chapter 3

Chapter 4

Enterprise Knowledge Graph & Machine Readable Knowledge

Enterprise Purpose

Die Knowledge Engine bildet das zentrale Wissenssystem der CAPITAL-AI Plattform.

Sie besitzt nicht die Aufgabe Dokumente zu speichern.

Sie besitzt die Aufgabe, den vollständigen Wissensstand der Plattform dauerhaft aufzubauen, miteinander zu verknüpfen und für Menschen sowie KI-Systeme verfügbar zu machen.

Die Knowledge Engine erzeugt dadurch einen digitalen Zwilling der gesamten Softwarearchitektur.

Mission

Jede technische Information der Plattform wird automatisch in Wissen umgewandelt.

Dieses Wissen besitzt Beziehungen.

Es kennt seinen Ursprung.

Es kennt seine Abhängigkeiten.

Es kennt seine Historie.

Es kennt seine Version.

Es kennt seine Verantwortung.

Enterprise Principle

Code erzeugt Wissen.

Wissen erzeugt Architektur.

Architektur erzeugt Dokumentation.

Dokumentation ist niemals der Ursprung.

Knowledge Lifecycle

Repository



Repository Intelligence



Code Intelligence



Knowledge Engine



Knowledge Graph



Documentation Engine



Supervisor



Platform Director

Enterprise Knowledge Model

Die Knowledge Engine verwaltet ausschließlich Beziehungen.

Beispiel

Platform Director



besitzt



Supervisor



registriert



Agent



verwendet



Service



nutzt



API



greift auf



Datenbank



besitzt



RLS



gehört zu



ADR

Knowledge Objects

Automatisch erzeugen

Repository

Module

Services

Klassen

Interfaces

Methoden

Events

Agenten

Orchestratoren

APIs

Datenbankobjekte

Konfigurationen

Dokumentationen

Versionen

Releases

ADR

Workflows

Policies

Knowledge Relationships

Automatisch erzeugen

USES

IMPLEMENTS

DEPENDS_ON

CALLS

LISTENS_TO

EMITS

OWNS

BELONGS_TO

GENERATES

REGISTERS

VALIDATES

DEPLOYS

CONFIGURES

MIGRATES

SUPERVISES

DOCUMENTS

Knowledge Graph

Die Plattform erzeugt automatisch einen vollständigen Graphen.

Beispiel

Platform Director

↓

Supervisor

↓

Crypto Orchestrator

↓

DeFi Scoring

↓

Risk Service

↓

API

↓

Supabase

↓

Migration

↓

ADR

↓

Version

↓

Release

Machine Readable Knowledge

Neben Markdown erzeugt die Knowledge Engine strukturierte Wissensmodelle.

knowledge/

repository.json

architecture.json

components.json

services.json

events.json

agents.json

orchestrators.json

database.json

api.json

security.json

documentation.json

dependencies.json

workflows.json

policies.json

versions.json

releases.json

Component Knowledge

Für jede Komponente werden automatisch gespeichert

ID

Name

Typ

Version

Owner

Beschreibung

Verantwortung

Abhängigkeiten

Events

API

Dokumentation

ADR

Supervisor Status

Health Status

Lifecycle

Historie

Architecture Memory

Die Knowledge Engine merkt sich

wann

warum

wie

durch wen

eine Komponente verändert wurde.

Sie kennt dadurch die vollständige Entwicklung der Plattform.

Historical Intelligence

Für jede Version werden gespeichert

welche Dateien geändert wurden

welche Klassen geändert wurden

welche APIs geändert wurden

welche Agenten geändert wurden

welche Events geändert wurden

welche Dokumente geändert wurden

welche ADR erstellt wurden

welche Migrationen entstanden

welche Risiken erkannt wurden

Version Relationship

Version

↓

Commit

↓

Änderungen

↓

Komponenten

↓

Events

↓

Dokumentation

↓

Release

↓

Deployment

↓

Produktivsystem

AI Knowledge

Jeder zukünftige KI-Agent verwendet dieselbe Wissensbasis.

Kein Agent analysiert das Repository mehrfach.

Die Knowledge Engine stellt allen Agenten identische Informationen bereit.

Dadurch entstehen

keine unterschiedlichen Interpretationen

keine widersprüchlichen Dokumentationen

keine redundanten Analysen.

Knowledge Queries

Die Knowledge Engine muss Fragen beantworten können.

Beispiele

Welche Services verwendet der Crypto Orchestrator?

Welche Agenten nutzen den Supervisor?

Welche APIs greifen auf Supabase zu?

Welche Komponenten besitzen keine ADR?

Welche Events besitzen keinen Listener?

Welche Klassen besitzen technische Schulden?

Welche Version hat diese Klasse eingeführt?

Welche Migration gehört zu dieser Tabelle?

Welche Dokumentation ist veraltet?

Welche Komponenten wurden seit Version X verändert?

Enterprise Search

Die gesamte Plattform wird semantisch durchsuchbar.

Nicht nach Dateinamen.

Sondern nach Bedeutung.

Beispiele

"Alle Komponenten die Stripe verwenden."

"Alle Agenten mit Telemetrie."

"Alle Klassen mit Security Bezug."

"Alle APIs ohne Dokumentation."

"Alle Orchestratoren mit Supervisor Integration."

Knowledge Validation

Vor jeder Dokumentation wird geprüft

existiert bereits Wissen?

existiert bereits eine Beziehung?

existiert bereits eine Version?

existiert bereits eine ADR?

existiert bereits eine Dokumentation?

existiert bereits ein Diagramm?

existiert bereits eine Migration?

Knowledge Synchronisation

Die Knowledge Engine synchronisiert automatisch

Repository

↓

Code Intelligence

↓

Documentation

↓

Version Manager

↓

Supervisor

↓

Platform Director

↓

Release Center

↓

Compliance Center

↓

Security Center



Quality Center

Enterprise Rules

Wissen wird niemals manuell geändert.

Wissen entsteht ausschließlich aus Analyse.

Keine Dokumentation verändert Wissen.

Nur Quellcode verändert Wissen.

Success Criteria

Die Knowledge Engine gilt als erfolgreich wenn

- ✓ sämtliche Komponenten Beziehungen besitzen
 - ✓ sämtliche Agenten registriert sind
 - ✓ sämtliche Services dokumentiert sind
 - ✓ sämtliche APIs verknüpft wurden
 - ✓ sämtliche Datenbankobjekte bekannt sind
 - ✓ sämtliche ADR referenziert werden
 - ✓ sämtliche Versionen nachvollziehbar bleiben
 - ✓ sämtliche Änderungen historisiert werden
 - ✓ alle KI-Agenten dieselbe Wissensbasis verwenden
 - ✓ der vollständige Architekturgraph jederzeit reproduzierbar aufgebaut werden kann
-

Enterprise Extension

Die Knowledge Engine bildet den zentralen digitalen Zwilling der CAPITAL-AI Plattform.

Alle zukünftigen Enterprise-Komponenten greifen ausschließlich auf diese Wissensbasis zu.

Sie ersetzt langfristig isolierte Dokumentationen und entwickelt sich zum zentralen Knowledge Operating System des CAPITAL-AI Core.

End of Chapter 4

Chapter 5

Documentation Engine

Enterprise Purpose

Die Documentation Engine ist die zentrale Generierungsinstanz der CAPITAL-AI Documentary.

Sie erzeugt sämtliche technische Dokumentation automatisch.

Sie besitzt keine eigene Logik zur Interpretation des Quellcodes.

Alle Informationen stammen ausschließlich aus

- Repository Intelligence
- Code Intelligence
- Enterprise Knowledge Graph
- Version Manager
- Event System

Die Documentation Engine transformiert Wissen in nachvollziehbare Dokumentation.

Mission

Dokumentation entsteht niemals manuell.

Dokumentation entsteht ausschließlich aus Wissen.

Jede Änderung der Plattform muss automatisch zu einer Aktualisierung aller betroffenen Dokumente führen.

Documentation Lifecycle

Repository

↓

Repository Intelligence

↓

Code Intelligence



Knowledge Graph



Documentation Engine



Validator



Knowledge Synchronisation



Supervisor



Platform Director

Documentation Philosophy

Dokumentation beschreibt niemals Vermutungen.

Sie beschreibt ausschließlich nachweisbare Architekturinformationen.

Alle Aussagen müssen reproduzierbar sein.

Jedes Dokument besitzt einen Ursprung.

Documentation Sources

Die Engine verwendet ausschließlich

Repository

Code Intelligence

Knowledge Graph

Version History

Architecture Graph

Event Registry

Repository Metadata

Git Metadata

Configuration

Buildinformationen

Document Registry

Die Documentation Engine verwaltet sämtliche Dokumenttypen.

Architecture

Architecture Report

Architecture Overview

Architecture Decision Records

Component Documentation

Dependency Documentation

Module Documentation

System Overview

Technology Overview

Development

Change Log

Development Report

Migration Report

Implementation Report

Refactoring Report

Technical Debt Report

Code Evolution Report

Knowledge

Knowledge Report

Knowledge Index

Component Registry

Agent Registry

Orchestrator Registry

API Registry

Database Registry

Event Registry

Plugin Registry

Security

Security Report

Compliance Report

IAM Report

Permission Report

Risk Report

Audit Report

RLS Report

Policy Report

Release

Release Notes

Version History

Deployment Report

Production Report

Validation Report

Rollback Report

Automatic Documentation

Für jede Änderung wird automatisch geprüft

Welche Dokumente sind betroffen?

Welche Kapitel müssen aktualisiert werden?

Welche Diagramme müssen neu erzeugt werden?

Welche ADR müssen ergänzt werden?

Welche Version muss aktualisiert werden?

Welche Komponenten müssen synchronisiert werden?

Documentation Templates

Jeder Dokumenttyp besitzt ein standardisiertes Template.

Beispiele

ADR Template

Architecture Template

Component Template

Service Template

Agent Template

API Template

Migration Template

Release Template

Security Template

Compliance Template

Knowledge Template

Documentation Generator

Jeder Dokumenttyp besitzt genau einen Generator.

Beispiele

ADRGenerator

ArchitectureGenerator

ServiceGenerator

ComponentGenerator

AgentGenerator

APIReferenceGenerator

DatabaseGenerator

MigrationGenerator

ReleaseGenerator

VersionGenerator

KnowledgeGenerator

MermaidGenerator

DiagramGenerator

Intelligent Update

Vor jeder Dokumentenerstellung wird geprüft

existiert das Dokument bereits?

existiert bereits dieselbe Information?

muss das Dokument erweitert werden?

muss das Dokument ersetzt werden?

muss lediglich die Version geändert werden?

Documentation Coverage

Für jede Plattformkomponente muss automatisch dokumentiert werden

Mission

Verantwortung

Architektur

Abhängigkeiten

Events

Version

Owner

Lifecycle

API

Datenbankzugriffe

Risiken

Supervisor Integration

Platform Director Integration

Code Documentation

Die Documentation Engine erzeugt zusätzlich codebasierte Dokumentation.

Für jede Klasse

automatisch erzeugen

Klassenbeschreibung

Methoden

Properties

Interfaces

Events

Dependencies

Importe

Exporte

Lifecycle

Version

Historie

Verantwortlichkeit

Verwendende Komponenten

API Documentation

Automatisch erzeugen

Endpoint

HTTP Methode

Beschreibung

Authentifizierung

Berechtigungen

DTO

Request

Response

Fehlercodes

Rate Limits

Abhängigkeiten

Version

Database Documentation

Automatisch erzeugen

Tabellen

Views

Constraints

Indexes

RLS

Policies

Storage

Functions

RPC

Migrationen

Historie

Event Documentation

Automatisch erzeugen

Event Name

Beschreibung

Emitter

Listener

Lifecycle

Trigger

Payload

Version

Abhängigkeiten

Supervisor Status

Diagram Generation

Automatisch erzeugen

Mermaid Class Diagram

Sequence Diagram

Flow Diagram

State Diagram

Component Diagram

Deployment Diagram

Entity Relationship Diagram

Dependency Diagram

Event Flow Diagram

Knowledge Graph Diagram

Documentation Metadata

Jedes Dokument besitzt standardisierte Metadaten.

Beispiele

Document ID

Version

Repository

Generator

Knowledge Version

Architecture Version

Component Version

Git Commit

Timestamp

Validation Status

Approval Status

Owner

Documentation Validation

Vor Veröffentlichung jedes Dokumentes prüfen

Vollständigkeit

Version

Referenzen

ADR Verknüpfung

Knowledge Referenzen

Komponentenreferenzen

Diagramme

Broken Links

Duplikate

Inkonsistenzen

Documentation Synchronisation

Alle Dokumente werden automatisch synchronisiert mit

Knowledge Engine

Version Manager

Supervisor

Platform Director

Quality Center

Compliance Center

Security Center

Release Center

Enterprise Rules

Keine manuelle Dokumentation.

Keine redundanten Dokumente.

Keine widersprüchlichen Informationen.

Keine Dokumente ohne Version.

Keine Dokumente ohne Ursprung.

Keine Dokumente ohne Validierung.

Success Criteria

Die Documentation Engine gilt als erfolgreich wenn

- ✓ jede Plattformkomponente dokumentiert ist
 - ✓ jede API dokumentiert ist
 - ✓ jede Datenbankstruktur dokumentiert ist
 - ✓ jede Version dokumentiert ist
 - ✓ jede ADR referenziert wird
 - ✓ sämtliche Diagramme automatisch erzeugt werden
 - ✓ jede Dokumentation validiert wurde
 - ✓ sämtliche Dokumente versioniert sind
 - ✓ sämtliche Dokumente auf dem aktuellen Wissensstand basieren
-

Enterprise Extension

Die Documentation Engine entwickelt sich langfristig zu einer vollständig automatisierten Enterprise Documentation Platform.

Sie dokumentiert nicht nur die CAPITAL-AI Plattform.

Sie dokumentiert künftig sämtliche zukünftigen Projekte des CAPITAL-AI Ökosystems.

Die Documentation Engine ist dadurch nicht nur Dokumentationsgenerator, sondern Bestandteil der Wissensarchitektur des gesamten CAPITAL-AI Core.

End of Chapter 5

Chapter 6

Architecture Intelligence & Digital Twin Engine

Enterprise Purpose

Die Architecture Intelligence Engine bildet den digitalen Zwilling der gesamten CAPITAL-AI Plattform.

Sie besitzt vollständige Kenntnis über

- Architektur
- Komponenten
- Services
- Datenflüsse
- Ereignisse
- Verantwortlichkeiten
- Abhängigkeiten
- Versionen
- Evolution

Die Plattform kennt dadurch jederzeit ihren aktuellen technischen Zustand.

Mission

Die Architecture Intelligence Engine beantwortet jederzeit folgende Fragen.

Wie ist die Plattform aufgebaut?

Welche Komponenten existieren?

Welche Komponenten kommunizieren miteinander?

Welche Komponenten besitzen Risiken?

Welche Komponenten wurden verändert?

Welche Architekturverletzungen existieren?

Welche Services sind kritisch?

Welche Komponenten fehlen?

Welche Komponenten besitzen technische Schulden?

Welche Komponenten sind veraltet?

Enterprise Principle

Die Architektur wird niemals manuell beschrieben.

Sie wird vollständig aus dem Knowledge Graph rekonstruiert.

Der digitale Zwilling stellt jederzeit den tatsächlichen Zustand der Plattform dar.

Digital Twin Lifecycle

Repository

↓

Repository Intelligence

↓

Code Intelligence

↓

Knowledge Graph

↓

Architecture Intelligence

↓

Digital Twin

↓

Supervisor

↓

Platform Director

Architecture Domains

Die Engine erkennt automatisch folgende Domänen.

Frontend

Backend
Shared
Infrastructure
Database
Authentication
Authorization
Billing
Security
Compliance
AI
Orchestrator
Agents
Platform
Documentary
Versioning
Deployment
Monitoring
Testing
Configuration

Layer Intelligence

Automatische Erkennung der Architekturebenen.

Presentation Layer
Application Layer
Domain Layer
Infrastructure Layer
Persistence Layer
Integration Layer
Platform Layer

AI Layer

Knowledge Layer

Documentation Layer

Governance Layer

Component Topology

Automatisch erzeugen

Component Tree

Module Tree

Service Tree

Agent Tree

Orchestrator Tree

API Tree

Database Tree

Knowledge Tree

Documentation Tree

Deployment Tree

Runtime Architecture

Automatisch erkennen

Application Startup

Service Registration

Dependency Injection

Plugin Registration

Agent Registration

Supervisor Registration

Event Registration

Health Checks

Lifecycle

Shutdown

Recovery

Communication Intelligence

Die Engine erkennt automatisch

REST Kommunikation

Internal API

External API

Realtime

Events

Webhooks

Cron

Queue

Observer

Scheduler

Broadcast

Architecture Relationships

Automatisch erzeugen

communicates_with

depends_on

owns

creates

uses

extends

implements

observes

publishes

subscribes

validates
documents
controls
supervises
deploys

Critical Path Detection

Automatisch identifizieren
Single Points of Failure
kritische Services
kritische Agenten
kritische APIs
kritische Datenbankobjekte
kritische Events
kritische Abhängigkeiten
kritische Workflows
kritische Sicherheitskomponenten

Architectural Drift Detection

Automatisch erkennen
ungeplante Änderungen
verwaiste Services
nicht dokumentierte Komponenten
fehlende ADR
fehlende Versionierung
unerwartete Abhängigkeiten
Layer Verletzungen
Governance Verstöße

Architecture Evolution

Die Plattform dokumentiert automatisch
wann eine Komponente entstand
wann sie geändert wurde
welche Version betroffen ist
welche ADR dazu existiert
welcher Commit verantwortlich ist
welcher Entwickler die Änderung durchgeführt hat
welcher Agent die Änderung dokumentiert hat

Architecture Timeline

Version

↓

Commit

↓

Komponente

↓

Änderung

↓

ADR

↓

Knowledge Update

↓

Release

↓

Deployment

↓

Produktionsstatus

Enterprise Architecture Graph

Die Engine erzeugt automatisch

System Graph

Component Graph

Dependency Graph

Event Graph

API Graph

Database Graph

Security Graph

Compliance Graph

Knowledge Graph

Deployment Graph

Workflow Graph

Architecture Metrics

Automatisch berechnen

Komponentenanzahl

Serviceanzahl

Agentenanzahl

Orchestratoren

Events

APIs

Datenbankobjekte

Architekturkomplexität

Dokumentationsabdeckung

Codequalität

Technische Schulden

Kopplungsgrad

Kohäsion

Architecture Risk Analysis

Automatisch bewerten
fehlende Dokumentation
fehlende Tests
fehlende Events
fehlende Versionierung
kritische Abhängigkeiten
Legacy Code
Duplicate Code
Dead Code
Circular Dependencies
Security Risiken
Compliance Risiken

Architecture Simulation

Vor jeder Migration simulieren
Welche Komponenten ändern sich?
Welche Dokumente ändern sich?
Welche APIs ändern sich?
Welche Events ändern sich?
Welche Datenbankobjekte ändern sich?
Welche Agenten ändern sich?
Welche Orchestratoren ändern sich?
Welche Version muss erhöht werden?
Welche Tests müssen erneut ausgeführt werden?

Enterprise Digital Twin

Der digitale Zwilling besitzt jederzeit Kenntnis über

den aktuellen Zustand

den historischen Zustand

den geplanten Zustand

den Zielzustand

Dadurch können Architekturentscheidungen bereits vor ihrer Implementierung bewertet werden.

Integration

Die Architecture Intelligence Engine liefert Informationen an

Knowledge Engine

↓

Documentation Engine

↓

Version Manager

↓

Supervisor

↓

Platform Director

↓

Compliance Center

↓

Security Center

↓

Quality Center

↓

Release Center

Enterprise Rules

Keine Architekturentscheidung ohne Knowledge Graph.

Keine Migration ohne Architekturvergleich.

Keine Freigabe ohne Architekturvalidierung.

Keine Versionserhöhung ohne Architekturprüfung.

Keine ADR ohne Architektureferenz.

Success Criteria

Die Architecture Intelligence Engine gilt als erfolgreich wenn

- ✓ die vollständige Plattform rekonstruiert werden kann
 - ✓ jede Komponente im Digital Twin existiert
 - ✓ sämtliche Beziehungen bekannt sind
 - ✓ sämtliche kritischen Abhängigkeiten erkannt wurden
 - ✓ Architekturverletzungen automatisch erkannt werden
 - ✓ jede Änderung historisiert wird
 - ✓ jede Version nachvollziehbar bleibt
 - ✓ Supervisor und Platform Director jederzeit den aktuellen Architekturzustand kennen
 - ✓ zukünftige KI-Agenten ausschließlich den Digital Twin zur Architekturinterpretation verwenden
-

Enterprise Extension

Der Digital Twin ist das zentrale Architekturmodell des CAPITAL-AI Core.

Alle zukünftigen Plattformkomponenten greifen ausschließlich auf dieses Modell zurück.

Dadurch existiert nur noch eine einzige gültige Darstellung der gesamten Systemarchitektur.

Die Architecture Intelligence Engine wird langfristig zur zentralen Governance-Komponente für sämtliche CAPITAL-AI Projekte.

End of Chapter 6

Chapter 7

Enterprise Impact Analysis & Migration Intelligence Engine

Enterprise Purpose

Die Impact Analysis & Migration Intelligence Engine ist die zentrale Entscheidungsinstanz der CAPITAL-AI Documentary.

Sie besitzt die Verantwortung, sämtliche geplanten Änderungen vor ihrer Umsetzung vollständig zu analysieren.

Keine Migration darf durchgeführt werden, bevor ihre Auswirkungen auf Architektur, Wissen, Versionierung, Dokumentation, Sicherheit und Compliance vollständig bewertet wurden.

Die Engine verhindert dadurch unkontrollierte Architekturänderungen.

Mission

Jede geplante Änderung wird vollständig simuliert.

Vor der ersten Codezeile kennt die Plattform bereits

- betroffene Komponenten
 - betroffene Services
 - betroffene APIs
 - betroffene Agenten
 - betroffene Orchestratoren
 - betroffene Datenbankobjekte
 - betroffene Dokumentationen
 - betroffene Versionen
 - betroffene Tests
 - betroffene Deployments
-

Enterprise Principle

Migrationen werden niemals direkt durchgeführt.

Migrationen entstehen ausschließlich nach einer vollständigen Auswirkungsanalyse.

Migration Lifecycle

Feature Request



Architecture Analysis



Impact Analysis



Risk Analysis



Migration Planning



Documentation Planning



Version Planning



Implementation



Validation



Deployment



Knowledge Synchronisation

Migration Categories

Die Engine klassifiziert jede Änderung.

Feature

Bugfix

Hotfix

Security

Compliance

Performance

Refactoring

Architecture

Infrastructure

Database

AI

Documentation

Configuration

Dependency

Emergency

Change Detection

Automatisch erkennen

Neue Datei

Datei entfernt

Klasse geändert

Methode geändert

Interface geändert

API geändert

Migration geändert

Policy geändert

Prompt geändert

Skill geändert

Konfiguration geändert

Impact Analysis

Automatisch bestimmen

Welche Klassen ändern sich?

Welche Interfaces ändern sich?

Welche Services ändern sich?

Welche APIs ändern sich?

Welche Datenbanktabellen ändern sich?

Welche RLS Policies ändern sich?

Welche Events ändern sich?

Welche Dokumentationen ändern sich?

Welche Knowledge Objekte ändern sich?

Welche Diagramme ändern sich?

Welche Releases ändern sich?

Dependency Impact

Automatisch analysieren

Direkte Abhängigkeiten

Indirekte Abhängigkeiten

Zirkuläre Auswirkungen

API Auswirkungen

Frontend Auswirkungen

Backend Auswirkungen

Supervisor Auswirkungen

Platform Director Auswirkungen

Version Manager Auswirkungen

Compliance Auswirkungen

Security Auswirkungen

AI Impact

Automatisch bestimmen

Welche Agenten müssen neu trainiert werden?

Welche Prompt Registry wird geändert?

Welche Skills ändern sich?

Welche Knowledge Einträge ändern sich?

Welche AI Workflows ändern sich?

Welche LLM Konfigurationen ändern sich?

Documentation Impact

Automatisch bestimmen

ADR

Architecture Report

Knowledge Base

API Dokumentation

Migration Report

Release Notes

Version History

Component Registry

Mermaid Diagramme

Risk Report

Compliance Report

Security Report

Version Impact

Automatisch bestimmen

Patch

Minor

Major

Enterprise

LTS

Hotfix

Preview

Development

Production

Die Versionsempfehlung erfolgt automatisch anhand der Änderungsauswirkungen.

Database Impact

Automatisch erkennen

Migration notwendig

Rollback notwendig

RLS Änderung

Policy Änderung

Index Änderung

Constraint Änderung

Function Änderung

Storage Änderung

Realtime Änderung

Deployment Impact

Automatisch bestimmen

Development

Testing

Staging

Production

Rollback

Canary

Blue Green

Emergency

Enterprise Risk Analysis

Vor jeder Migration automatisch berechnen

Technisches Risiko

Architekturrisiko

Betriebsrisiko

Compliance Risiko

Security Risiko

Performance Risiko

Datenbank Risiko

API Risiko

Deployment Risiko

Business Risiko

Migration Plan

Automatisch erzeugen

Ausgangszustand

↓

Zielzustand

↓

Betroffene Komponenten

↓

Implementierungsreihenfolge

↓

Dokumentationsänderungen

↓

Versionierung

↓

Validierung

↓

Deployment

↓

Monitoring

Migration Validation

Vor Beginn prüfen

Repository aktuell

Knowledge aktuell

Version aktuell

ADR aktuell

Supervisor synchronisiert

Platform Director synchronisiert

Quality Gates erfolgreich

Keine offenen Konflikte

Keine Architekturverletzungen

Rollback Intelligence

Für jede Migration automatisch erzeugen

Rollback Strategie

Rollback Dokumentation

Rollback Reihenfolge

Rollback Risiken

Rollback Voraussetzungen

Rollback Tests

Rollback Version

Rollback Freigabe

Enterprise Decision Engine

Vor jeder Änderung beantwortet die Documentary Engine

Warum wird geändert?

Welche Architekturentscheidung entsteht?

Welche ADR ist betroffen?

Welche Version wird erhöht?

Welche Risiken entstehen?

Welche Dokumente ändern sich?

Welche Komponenten sind betroffen?

Welche Events werden ausgelöst?

Welche Tests müssen laufen?

Welche Freigaben werden benötigt?

Production Governance

Produktionsänderungen benötigen zusätzlich

Version Manager Freigabe

Supervisor Freigabe

Platform Director Freigabe

Quality Center Freigabe

Compliance Center Prüfung

Security Center Prüfung

Deployment Validation

Knowledge Synchronisation

Enterprise Rules

Keine Migration ohne Impact Analyse.

Keine Migration ohne Versionsempfehlung.

Keine Migration ohne ADR Prüfung.

Keine Migration ohne Rollback Strategie.

Keine Migration ohne Quality Gates.

Keine Migration ohne Dokumentationsplanung.

Keine Migration ohne Supervisor Synchronisation.

Success Criteria

Die Impact Analysis Engine gilt als erfolgreich wenn

- ✓ jede Änderung vollständig analysiert wird
 - ✓ sämtliche Auswirkungen erkannt werden
 - ✓ sämtliche Risiken bewertet werden
 - ✓ sämtliche Dokumentationen geplant werden
 - ✓ sämtliche Versionen empfohlen werden
 - ✓ Rollback Strategien automatisch erzeugt werden
 - ✓ keine ungeprüfte Migration durchgeführt werden kann
 - ✓ jede Änderung vollständig nachvollziehbar bleibt
-

Enterprise Extension

Die Impact Analysis Engine entwickelt sich langfristig zur zentralen Entscheidungsinstanz sämtlicher CAPITAL-AI Migrationen.

Sie verbindet Repository Intelligence, Code Intelligence, Knowledge Graph, Version Manager, Supervisor und Platform Director zu einer vollständig automatisierten Enterprise Change Governance.

Jede zukünftige Änderung der Plattform beginnt mit einer Impact Analyse.

Dadurch wird die Documentary Engine nicht nur Dokumentationssystem, sondern die erste Instanz der technischen Entscheidungsfindung innerhalb des CAPITAL-AI Core.

End of Chapter 7

Chapter 8

Event Driven Documentation & Enterprise Event Bus

Enterprise Purpose

Die Event Driven Documentation Architecture bildet das zentrale Nervensystem des CAPITAL-AI Core.

Jede Änderung innerhalb der Plattform erzeugt ein standardisiertes Enterprise Event.

Diese Events bilden die einzige zulässige Kommunikationsform zwischen sämtlichen Enterprise-Komponenten.

Direkte Dokumentenerzeugung ist untersagt.

Direkte Versionierung ist untersagt.

Direkte Synchronisation ist untersagt.

Alle Prozesse beginnen mit einem Event.

Mission

Jede technische Änderung der Plattform erzeugt exakt ein oder mehrere standardisierte Enterprise Events.

Diese Events steuern anschließend automatisch

- Documentary Engine
 - Knowledge Engine
 - Version Manager
 - Supervisor
 - Platform Director
 - Security Center
 - Compliance Center
 - Quality Center
 - Release Center
-

Enterprise Principle

Events besitzen höchste Priorität.

Der Code erzeugt Events.

Events erzeugen Wissen.

Wissen erzeugt Dokumentation.

Dokumentation erzeugt Governance.

Governance steuert die Plattform.

Event Lifecycle

Repository Change



Event Detection



Enterprise Event



Enterprise Event Bus



Event Registry



Event Validation



Knowledge Engine



Documentation Engine



Version Manager



Supervisor



Platform Director



Release Center

Enterprise Event Categories

Repository Events

RepositoryCreated

RepositoryUpdated

RepositoryValidated

RepositoryIndexed

RepositoryArchived

Code Events

FileCreated

FileDeleted

FileModified

ClassCreated

ClassUpdated

InterfaceUpdated

MethodChanged

DependencyChanged

ImportChanged

ExportChanged

Architecture Events

ArchitectureChanged

LayerChanged

ComponentAdded

ComponentRemoved

ComponentRefactored

DependencyDetected

ArchitectureViolationDetected

Knowledge Events

KnowledgeCreated

KnowledgeUpdated

KnowledgeValidated

KnowledgeSynchronized

KnowledgeConflictDetected

Documentation Events

DocumentCreated

DocumentUpdated

DocumentValidated

ADRCreated

ADRUpdated

DiagramGenerated

DocumentationCompleted

Version Events

VersionCreated

VersionIncremented

VersionValidated

ReleasePrepared

ReleasePublished

RollbackCreated

Security Events

PolicyChanged

PermissionChanged

IAMChanged

RLSChanged

SecurityIncident

SecretUpdated

AuthenticationChanged

AuthorizationChanged

Compliance Events

ComplianceCheckStarted

ComplianceValidated

AuditCreated

RiskDetected

RiskResolved

PolicyViolationDetected

Database Events

MigrationCreated

MigrationValidated

MigrationExecuted

SchemaChanged

TableCreated

ViewCreated

PolicyUpdated

StorageChanged

AI Events

AgentRegistered

AgentUpdated

AgentRemoved

PromptUpdated

SkillUpdated

KnowledgeModelUpdated

LLMConfigurationChanged

Platform Events

SupervisorStarted

PlatformDirectorStarted

HealthCheckCompleted

DeploymentStarted

DeploymentCompleted

RollbackExecuted

SystemShutdown

RecoveryCompleted

Enterprise Event Model

Jedes Event besitzt einen standardisierten Aufbau.

Event ID

Event Name

Kategorie

Quelle

Komponente

Version

Timestamp

Correlation ID

Repository Version

Knowledge Version

Architecture Version

Payload

Priority

Status

Owner

Validation Status

Event Registry

Alle Events werden zentral registriert.

Es dürfen keine unbekannten Events existieren.

Jedes Event besitzt

eine Definition

eine Version

eine Beschreibung

eine Payload Definition

einen Verantwortlichen

eine Dokumentation

eine ADR Referenz

Event Validation

Vor der Verarbeitung prüfen

Event vollständig

Schema gültig

Payload gültig

Version gültig

Quelle gültig

Komponente registriert

Doppelte Events

Zirkuläre Events

Event Routing

Die Event Engine bestimmt automatisch

welche Komponenten informiert werden.

Beispiele

ArchitectureChanged

↓

Knowledge Engine

↓

Documentation Engine

↓

Version Manager

↓

Supervisor

↓

Platform Director

SecurityIncident

↓

Security Center

↓

Compliance Center

↓

Supervisor

↓

Platform Director

↓

Documentary

MigrationCreated

↓

Version Manager



Documentation Engine



Knowledge Engine



Release Center



Supervisor

Event Persistence

Jedes Event wird dauerhaft gespeichert.

Die Event Historie ist unveränderbar.

Alle Plattformsentscheidungen bleiben dadurch vollständig nachvollziehbar.

Event Replay

Die Plattform kann jederzeit

sämtliche Events erneut abspielen.

Dadurch kann

Knowledge Graph

Documentation

Versionierung

Architektur

vollständig rekonstruiert werden.

Event Priorities

Critical

High

Medium

Low

Information

Event Security

Events dürfen niemals

gefälscht

dupliziert

verändert

gelöscht

werden.

Jedes Event besitzt

Integritätsprüfung

Version

Zeitstempel

Quelle

Signatur (optional für zukünftige Versionen)

Enterprise Rules

Keine Dokumentation ohne Event.

Keine Versionierung ohne Event.

Keine Migration ohne Event.

Keine ADR ohne Event.

Keine Release Erstellung ohne Event.

Keine Architekturänderung ohne Event.

Success Criteria

Die Event Driven Architecture gilt als erfolgreich wenn

✓ jede Plattformänderung ein Event erzeugt

- ✓ alle Events registriert sind
 - ✓ keine unbekannten Events existieren
 - ✓ sämtliche Events validiert werden
 - ✓ sämtliche Komponenten ausschließlich über Events kommunizieren
 - ✓ Knowledge Engine vollständig synchron bleibt
 - ✓ Documentary automatisch aktualisiert wird
 - ✓ Version Manager automatisch ausgelöst wird
 - ✓ Supervisor jederzeit den aktuellen Plattformzustand kennt
 - ✓ Platform Director sämtliche Enterprise Events überwachen kann
-

Enterprise Extension

Der Enterprise Event Bus bildet langfristig die zentrale Kommunikationsschicht des CAPITAL-AI Core.

Alle zukünftigen Module, Agenten, Orchestratoren, Plugins und KI-Systeme kommunizieren ausschließlich über standardisierte Enterprise Events.

Dadurch entsteht eine lose gekoppelte, hochskalierbare und vollständig nachvollziehbare AI-native Plattformarchitektur.

End of Chapter 8

Chapter 9

Enterprise Version Manager & AI Value Chain Integration

Enterprise Purpose

Der Enterprise Version Manager bildet die zentrale Steuerungsinstanz für sämtliche Versionsänderungen innerhalb des CAPITAL-AI Core.

Versionierung ist keine Eigenschaft einzelner Softwarekomponenten.

Versionierung beschreibt den Entwicklungszustand der gesamten Plattform.

Jede Änderung innerhalb des Repositorys wird automatisch bewertet, klassifiziert und einer nachvollziehbaren Versionsstrategie zugeordnet.

Mission

Der Version Manager gewährleistet

- konsistente Versionierung
- reproduzierbare Releases
- vollständige Änderungsverfolgung
- automatische Dokumentation
- automatische Knowledge Synchronisation

Die Plattform besitzt dadurch jederzeit einen eindeutig definierten Entwicklungszustand.

Enterprise Principle

Jede Änderung besitzt eine Version.

Keine Änderung existiert ohne Versionsbezug.

Keine Version existiert ohne Dokumentation.

Keine Dokumentation existiert ohne Knowledge Reference.

Version Lifecycle

Repository Change

↓

Impact Analysis

↓

Version Recommendation

↓

Version Validation

↓

Documentation Update

↓

Knowledge Synchronisation

↓

Supervisor Validation



Platform Director Approval



Release Preparation



Deployment



Production

Enterprise Version Categories

Development

Experimental

Preview

Alpha

Beta

Release Candidate

Production

Long Term Support

Hotfix

Emergency

Legacy

Archived

Semantic Version Intelligence

Die Plattform verwendet Semantic Versioning als Grundlage.

Major

Architekturänderungen

Breaking Changes

Neue Plattformfähigkeiten

Enterprise Erweiterungen

Minor

Neue Funktionen

Neue Agenten

Neue Orchestratoren

Neue APIs

Neue Services

Neue Module

Patch

Bugfixes

Dokumentation

Refactoring

Tests

Performance

Konfiguration

Automatic Version Recommendation

Die Documentary Engine bestimmt automatisch
welche Versionsänderung erforderlich ist.

Beispiele

Neue Klasse

↓

Minor

Neue API

↓

Minor

Breaking Change



Major

Dokumentationsänderung



Patch

Security Hotfix



Hotfix

Neue Enterprise Architektur



Major

Version Decision Matrix

Automatisch bewerten

Änderungsumfang



Architekturauswirkung



Risiko



Kompatibilität



Abhängigkeiten



Deployment Auswirkungen



Versionsempfehlung

Repository Version Synchronisation

Automatisch synchronisieren

package.json

VersionManager

Knowledge Graph

Documentation

ADR

Release Notes

CHANGELOG

Platform Director

Supervisor

Deployment

Enterprise Version Registry

Für jede Version speichern

Version

Build

Repository

Commit

Branch

Release Typ

Status

Knowledge Version

Architecture Version

Documentation Version

Generator Version

Deployment Version

Historie

Component Versioning

Nicht nur das Repository,

sondern jede Plattformkomponente besitzt eine eigene Version.

Beispiele

Supervisor

Platform Director

Documentary Engine

Knowledge Engine

Security Center

Compliance Center

Crypto Orchestrator

DeFi Scoring

Billing

API Gateway

Architecture Version

Die Plattform besitzt zusätzlich eine Architekturversion.

Sie wird erhöht wenn

Layer geändert werden

Governance geändert wird

Enterprise Komponenten entstehen

Architekturprinzipien geändert werden

Knowledge Version

Die Knowledge Engine besitzt eine eigene Version.

Diese wird erhöht wenn

neue Beziehungen entstehen

Knowledge Objekte erweitert werden

Architekturgraphen geändert werden

AI Modelle neue Informationen erhalten

Documentation Version

Dokumentation besitzt eigene Versionen.

Jedes Dokument speichert

Version

Generator Version

Knowledge Version

Architecture Version

Repository Version

Validation Version

Enterprise Change Chain

Jede Änderung erzeugt automatisch

Code

↓

Enterprise Event

↓

Impact Analysis

↓

Version Recommendation

↓

Knowledge Update



Documentation Update



Validation



Supervisor



Platform Director



Release



Deployment

Release Preparation

Automatisch erzeugen

Release Notes

CHANGELOG

Version Report

Migration Report

Risk Report

Architecture Report

Knowledge Report

Deployment Plan

Rollback Plan

Version Validation

Vor jeder Freigabe prüfen

Repository aktuell

Knowledge aktuell
Dokumentation aktuell
Architecture aktuell
ADR aktuell
Tests erfolgreich
Migration vollständig
Quality Gates bestanden
Supervisor synchronisiert
Platform Director synchronisiert

Rollback Versioning

Jede Version besitzt
Rollback Version
Rollback Dokumentation
Rollback Plan
Rollback Risiken
Rollback Historie
Rollback Tests

Enterprise Traceability

Jede Version kennt
welche Dateien geändert wurden
welche Klassen geändert wurden
welche APIs geändert wurden
welche Dokumente geändert wurden
welche ADR entstanden
welche Events erzeugt wurden
welche Migrationen entstanden
welche Releases betroffen sind

AI Native Version Chain

Alle KI-Systeme verwenden dieselbe Version.

Knowledge Engine



Documentation Engine



Supervisor



Platform Director



Release Center



Compliance Center



Security Center



Quality Center

Dadurch existieren niemals unterschiedliche Versionsstände.

Enterprise Rules

Keine Änderung ohne Versionsanalyse.

Keine Versionsänderung ohne Impact Analysis.

Keine Versionsänderung ohne Dokumentation.

Keine Versionsänderung ohne Knowledge Update.

Keine Produktionsversion ohne Platform Director.

Keine Release Version ohne Supervisor Validation.

Keine Version ohne Rollback Strategie.

Success Criteria

Der Enterprise Version Manager gilt als erfolgreich wenn

- ✓ jede Änderung automatisch versioniert wird
 - ✓ sämtliche Komponenten eigene Versionen besitzen
 - ✓ Repository, Knowledge und Dokumentation synchron bleiben
 - ✓ Supervisor jederzeit den korrekten Versionsstand kennt
 - ✓ Platform Director Releases automatisch vorbereiten kann
 - ✓ sämtliche Versionen reproduzierbar bleiben
 - ✓ Rollback Versionen jederzeit verfügbar sind
 - ✓ jede Änderung vollständig nachvollziehbar bleibt
-

Enterprise Extension

Der Enterprise Version Manager bildet den Mittelpunkt der CAPITAL-AI Wertschöpfungskette.

Er verbindet Repository Intelligence, Code Intelligence, Knowledge Engine, Documentation Engine, Impact Analysis, Supervisor, Platform Director und Release Center zu einem vollständig automatisierten Enterprise Development Lifecycle.

Die Version wird dadurch nicht nur zur Kennzeichnung eines Releases.

Sie beschreibt jederzeit den vollständigen technischen Reifegrad der gesamten CAPITAL-AI Plattform.

End of Chapter 9

Enterprise Final Summary

ESS-0001 Status

Document ID

ESS-0001

Titel

CAPITAL-AI Documentary & Code Intelligence Architect

Status

Enterprise Baseline Specification

Version

1.0.0

Lifecycle Status

Approved Enterprise Foundation

Mission Statement

Die Documentary Engine bildet das zentrale Wissenssystem der CAPITAL-AI Plattform.

Sie dokumentiert nicht lediglich Software.

Sie versteht Architektur.

Sie versteht Zusammenhänge.

Sie versteht Versionen.

Sie versteht Veränderungen.

Sie bildet den digitalen Zwilling der gesamten Plattform.

Alle zukünftigen Enterprise-Komponenten verwenden diese Wissensbasis als einzige technische Wahrheit.

Enterprise Design Principles

Die Documentary Engine basiert auf folgenden Grundprinzipien.

1.

Repository First

Das Repository ist die einzige Quelle technischer Wahrheit.

2.

Knowledge First

Alle Dokumentation entsteht aus Wissen.

Nicht umgekehrt.

3.

Architecture First

Architektur wird rekonstruiert.

Nicht manuell beschrieben.

4.

Event First

Jede Plattformänderung beginnt mit einem Enterprise Event.

5.

Version First

Jede Änderung besitzt eine nachvollziehbare Version.

6.

Validation First

Keine Information wird ohne Validierung übernommen.

7.

Automation First

Alle Prozesse sind vollständig automatisierbar.

8.

Enterprise Governance

Alle Entscheidungen müssen nachvollziehbar, reproduzierbar und auditierbar sein.

Architectural Scope

ESS-0001 definiert

✓ Repository Intelligence

✓ Code Intelligence

- ✓ Enterprise Knowledge Graph
 - ✓ Documentation Engine
 - ✓ Digital Twin
 - ✓ Impact Analysis
 - ✓ Migration Intelligence
 - ✓ Enterprise Event Architecture
 - ✓ Enterprise Version Management
-

Integration Points

ESS-0001 bildet die Grundlage für

ESS-0002 Supervisor Architect

ESS-0003 Platform Director

ESS-0004 Enterprise Version Manager

ESS-0005 Quality Center

ESS-0006 Security & Compliance

ESS-0007 Enterprise Release Center

ESS-0008 AI Agent Framework

ESS-0009 Enterprise Knowledge Platform

Enterprise Success Criteria

Die Documentary Engine gilt als erfolgreich wenn

- ✓ jede Repositoryänderung erkannt wird
- ✓ jede Architekturänderung erkannt wird
- ✓ jede Änderung versioniert wird
- ✓ jede Änderung dokumentiert wird
- ✓ jede Änderung historisiert wird
- ✓ sämtliche Enterprise Events nachvollziehbar bleiben
- ✓ sämtliche Dokumentationen reproduzierbar erzeugt werden
- ✓ sämtliche KI-Komponenten dieselbe Wissensbasis verwenden

✓ der vollständige digitale Zwilling jederzeit rekonstruiert werden kann

Long-Term Vision

Die Documentary Engine entwickelt sich zu einer vollständig autonomen Enterprise Knowledge Platform.

Sie dokumentiert nicht ausschließlich CAPITAL-AI.

Sie bildet die Grundlage für zukünftige AI-native Enterprise-Systeme.

Der langfristige Zielzustand ist ein selbstbeschreibendes Softwaresystem, dessen Architektur, Wissen, Dokumentation, Versionierung und Governance vollständig synchronisiert sind.

Governance Statement

ESS-0001 ist die verbindliche Enterprise-Spezifikation für die Documentary Engine innerhalb des CAPITAL-AI Core.

Alle zukünftigen Erweiterungen müssen mit den in ESS-0001 definierten Architekturprinzipien kompatibel sein.

Abweichungen erfordern eine neue Architecture Decision Record (ADR).

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste vollständige Enterprise-Spezifikation der Documentary Engine

Related Enterprise Specifications

ESS-0002 — Supervisor Architect

ESS-0003 — Platform Director

ESS-0004 — Enterprise Version Manager

ESS-0005 — Quality Center

ESS-0006 — Security & Compliance

ESS-0007 — Release Center

ESS-0008 — AI Agent Framework

ESS-0009 — Enterprise Knowledge Platform

Approval

Document Status

APPROVED

Enterprise Baseline

CAPITAL-AI Core Architecture

Version 1.0.0

End of Document

ESS-0001

CAPITAL-AI Documentary & Code Intelligence Architect

Enterprise Specification

Version 1.0.0

ESS 0002 Supervisor Architect

Quelle: `.ai/skills/ESS-0002-Supervisor-Architect.md`

skill: id: ESS-0002 name: Supervisor Architect version: 1.0.0 status: Enterprise Approved
maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority:
Critical

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle:
AI Native Development Lifecycle

authority:

controls: - Monitoring Center - Health Center - Governance Enforcement - Escalation Ma-
nagement

collaborates: - Platform Director - Documentary Engine - Version Manager - Security Center
- Compliance Center - Quality Center - Release Center

cannot_modify: - Trading Logic - Portfolio Logic - Crypto Scoring Algorithms - Stripe Billing
Logic - OAuth Authentication - Database Business Logic - Enterprise Specifications -
Architecture Decision Records

classification: type: Component Specification role: Komponentenspezifikation Supervisor
contractAuthority: ESS-0001-CONTRACTS note: > Dieses Dokument definiert ausschließlich
supervisorspezifische Contracts. Globale Contracts verbleiben in ESS-0001-CONTRACTS.

references: - ESS-0001 - ESS-0001-CONTRACTS - ADR-0006

crossReference: dependsOn: - ESS-0001 - ESS-0001-CONTRACTS relatedEss: - ESS-0003 -
ESS-0010 - ESS-0011 relatedAdr: - ADR-0006 - ADR-0013 - ADR-0015 relatedComponents: -
src/platform/Supervisor - src/platform/Telemetry - src/platform/Quality - src/platform/Tra-
ceability relatedSkills: - `.ai/skills/ESS-0001-Contracts.md` - `.ai/skills/ESS-0003-Platform-Di-
rector.md`

created: 2026-07-30

Supervisor Architect

Enterprise Mission

Der Supervisor ist die Überwachungsinstanz der CAPITAL-AI Plattform.

Seine Aufgabe besteht nicht darin, Entscheidungen zu treffen.

Seine Aufgabe besteht darin, jederzeit den vollständigen Zustand der Plattform zu kennen
und jede Abweichung von den Enterprise Contracts unmittelbar sichtbar zu machen.

Der Supervisor beobachtet.

Der Supervisor bewertet.

Der Supervisor eskaliert.

Der Supervisor entscheidet niemals.

Vision

Kein Zustand der Plattform bleibt unbeobachtet.

Keine Vertragsverletzung bleibt unentdeckt.

Keine Änderung erreicht die Produktion, ohne den Supervisor passiert zu haben.

Enterprise Philosophy

Die Plattform folgt dem Grundsatz

Beobachtung ersetzt Vertrauen.

Der Supervisor verlässt sich niemals auf Zusicherungen von Komponenten, Entwicklern oder KI-Systemen.

Er bewertet ausschließlich nachweisbare Zustände.

Abgrenzung

Instanz	Verantwortung
Documentary Engine	erzeugt Wissen und Dokumentation
Supervisor	bewertet Zustand und Konformität
Platform Director	entscheidet
Version Manager	versioniert
Release Center	liefert aus

Der Supervisor erzeugt niemals Dokumentation.

Der Supervisor erzeugt ausschließlich Bewertungen.

Position in der Architektur

Gemäß ESS-0001-CONTRACTS Chapter 6 befindet sich der Supervisor unterhalb des Platform Director und oberhalb sämtlicher übriger Layer.

```
graph TD; PD[Platform Director] --- S[Supervisor]; S --- VM[Version Manager]; VM --- D[Documentary]; D --- K[Knowledge]; K --- DIS[Discovery]; DIS --- R[Registry]; R --- SH[Shared]; SH --- C[Core];
```

The diagram illustrates the architectural layers of the ESS system. At the top is the Platform Director, which connects to the Supervisor. Below the Supervisor are the Version Manager, Documentary, Knowledge, Discovery, Registry, Shared, and Core layers, all connected in a vertical sequence.

Der Supervisor besitzt lesenden Zugriff auf sämtliche darunterliegenden Layer.

Er besitzt niemals schreibenden Zugriff auf produktive Komponenten.

End of Chapter 1

Chapter 2

Observation Model

Enterprise Purpose

Der Supervisor beobachtet ausschließlich standardisierte Zustände.

Er interpretiert niemals Quellcode eigenständig.

Sämtliche Informationen stammen aus

- Enterprise Registry
- Knowledge Graph
- Digital Twin
- Enterprise Events
- Validierungsergebnissen
- Telemetriedaten

Observation Domains

Der Supervisor überwacht verbindlich folgende Bereiche.

Repository

Struktur

Komponenten

Metadaten

Registry

Knowledge Graph

Digital Twin

Events

Versionen

Releases

Qualität

Sicherheit

Compliance

Automatisierung

AI-Wertschöpfungskette

Repository Observation

Automatisch erkennen

nicht zulässige Root-Verzeichnisse

nicht registrierte Ausnahmen

Strukturdrift

fehlende Pflichtmodule

fehlende Metadaten

verwaiste Dateien

Dokumentation innerhalb produktiver Komponenten

Grundlage bildet ESS-0001-CONTRACTS Chapter 16.

Component Observation

Für jede Komponente werden verbindlich beobachtet

Existenz

Version

Lifecycle

Health

Quality Score

Konformitätsstufe

Registry-Eintrag

Knowledge Node

Digital Twin

Owner

Dokumentationsstatus

Event Observation

Automatisch erkennen

nicht registrierte Events

Events ohne Consumer

Events ohne Schema

fehlende Correlation ID

abgebrochene Event-Ketten

verzögerte Event-Verarbeitung

wiederholte Fehlerereignisse

Chain Observation

Der Supervisor überwacht die AI-Wertschöpfungskette gemäß ESS-0001-CONTRACTS Chapter 17.

Für jede Kette wird geprüft

wurde jede Stufe ausgeführt?

wurde jede Übergabe dokumentiert?
blieb die Correlation ID durchgängig?
wurden sämtliche Artefakte erzeugt?
wurde die Kette ordnungsgemäß abgeschlossen?
Unvollständige Ketten erreichen niemals die Produktion.

Twin Observation

Der Supervisor überwacht den Digital Twin gemäß ESS-0001-CONTRACTS Chapter 18.

Beobachtet werden

Twin-Zustand

Drift

Aktualität

Vollständigkeit

Snapshot-Historie

Der Zustand Drifted blockiert jede Produktionsfreigabe.

Observation Frequency

Der Supervisor arbeitet ereignisgesteuert.

Zusätzlich erfolgt eine periodische Vollprüfung.

Ereignisgesteuert

bei jedem Enterprise Event

Periodisch

vollständige Konsistenzprüfung der Plattform

Vor Freigaben

vollständige Prüfung sämtlicher Quality Gates

Enterprise Rules

Keine Bewertung ohne Nachweis.

Keine Interpretation ohne Knowledge Graph.

Keine Beobachtung ohne Protokoll.

Kein produktiver Schreibzugriff.

End of Chapter 2

Chapter 3

Health & Lifecycle Supervision

Enterprise Purpose

Der Supervisor führt für jede Enterprise-Komponente einen jederzeit gültigen Gesundheitszustand.

Der Health Status ist Bestandteil der Metadaten gemäß ESS-0001-CONTRACTS Chapter 7.

Health States

Healthy

sämtliche Prüfungen bestanden

Warning

Befunde der Stufe Medium oder Low

Critical

Befunde der Stufe Critical oder High

Deprecated

Komponente im Lebenszyklus Deprecated

Unknown

keine auswertbaren Informationen vorhanden

Health Determination

Der Health Status wird ausschließlich berechnet.

Er wird niemals manuell gesetzt.

Grundlage bilden

Validierungsergebnisse

Quality Score

Twin-Zustand

Event-Historie

Telemetrie

Sicherheitsbefunde

Lifecycle Supervision

Der Supervisor prüft sämtliche Zustandswechsel gemäß ESS-0001-CONTRACTS Chapter 14.

Geprüft wird

ist der Übergang zulässig?

sind sämtliche Voraussetzungen erfüllt?

ist der Quality Score ausreichend?

existiert die erforderliche Dokumentation?

existiert bei Deprecation ein Nachfolger?

Unzulässige Zustandswechsel werden blockiert.

Dependency Supervision

Der Supervisor erkennt automatisch

Layer-Verletzungen

zyklische Abhängigkeiten

Abhängigkeiten auf veraltete Komponenten

Abhängigkeiten auf nicht registrierte Komponenten

nicht deklarierte externe Abhängigkeiten

Legacy Supervision

Der Supervisor überwacht sämtlichen Legacy-Bestand gemäß ESS-0001-CONTRACTS Chapter 14.

Beobachtet werden

Registrierungsstatus

Migrationsstatus

Adapter-Existenz

Zielkomponente

Risiko

Nicht registrierter Legacy-Bestand ist ein Befund der Stufe Critical.

End of Chapter 3

Chapter 4

Governance Enforcement

Enterprise Purpose

Der Supervisor setzt die Enterprise Governance durch, ohne selbst zu entscheiden.

Er stellt sicher, dass keine Änderung ohne die vertraglich vorgeschriebenen Nachweise weitergereicht wird.

Enforcement Model

Beobachtung

↓

Bewertung

↓

Befund

↓

Blockade oder Freigabeempfehlung

↓

Eskalation an den Platform Director

Der Supervisor spricht ausschließlich Empfehlungen aus.

Die Entscheidung trifft der Platform Director.

Blocking Findings

Folgende Befunde führen verbindlich zur Blockade.

fehlende Impact Analyse

fehlende ADR bei Architekturänderung

nicht bestandene Quality Gates

Critical- oder High-Befunde der Validatoren

Digital Twin im Zustand Drifted

unvollständige AI-Wertschöpfungskette

fehlende Sicherheitsklassifizierung

fehlende Rollback-Strategie

nicht registrierte Strukturabweichung

widersprüchliche Versionsstände

Non Blocking Findings

Folgende Befunde werden protokolliert, blockieren jedoch nicht.

Medium- und Low-Befunde

technische Schulden ohne unmittelbares Risiko

Dokumentationslücken außerhalb produktiver Komponenten

veraltete Berichte

Finding Contract

Jeder Befund besitzt verbindlich

Finding ID

Kategorie

Schweregrad

betroffene Komponente

verletzter Contract

Nachweis

Zeitpunkt

Correlation ID

Status

Empfehlung

Befunde werden niemals ohne Nachweis erzeugt.

Finding States

Open

Acknowledged

InProgress

Resolved

Accepted

Rejected

Der Status Accepted ist ausschließlich durch den Platform Director und ausschließlich mit ADR zulässig.

Escalation Contract

Eskalationsstufen

Stufe 1

Protokollierung

Stufe 2

Benachrichtigung des zuständigen Owners

Stufe 3

Blockade der betroffenen Änderung

Stufe 4

Eskalation an den Platform Director

Stufe 5

Notfallmeldung bei Sicherheitsvorfällen

Sicherheitsvorfälle werden unmittelbar auf Stufe 5 eskaliert.

AI Supervision

Der Supervisor überwacht sämtliche KI-Aktivitäten gemäß ESS-0001-CONTRACTS Chapter 10 und Chapter 17.

Beobachtet werden

Einhaltung der Stufenreihenfolge

Vollständigkeit der Übergabeartefakte

Einhaltung der Contracts

nicht genehmigte Architekturänderungen

fehlende Dokumentation

fehlende Audit-Einträge

Ergebnisse ohne nachweisbare Ausführung

Human Override

Autorisierte Projektverantwortliche dürfen Befunde übersteuern.

Jede Übersteuerung wird verbindlich

als Event protokolliert

im Governance Report ausgewiesen

mit Begründung versehen

Der Supervisor entfernt niemals einen Befund eigenständig.

End of Chapter 4

Chapter 5

Event Integration

Enterprise Purpose

Der Supervisor kommuniziert ausschließlich über Enterprise Events gemäß ESS-0001-CONTRACTS Chapter 8.

Consumed Events

Der Supervisor abonniert verbindlich

RepositoryScannedEvent

RepositoryValidatedEvent

StructureViolationDetectedEvent

ImplementationCompletedEvent

DocumentationGeneratedEvent

KnowledgeUpdatedEvent

KnowledgeConflictDetectedEvent

TwinSynchronizedEvent

TwinDriftDetectedEvent

ValidationCompletedEvent
ValidationFailedEvent
ContractViolationEvent
QualityGateFailedEvent
VersionCalculatedEvent
VersionChangedEvent
MigrationStartedEvent
MigrationFailedEvent
SecurityViolationDetectedEvent
ComplianceViolationDetectedEvent
PluginFailedEvent
ProcessFailedEvent
DeploymentCompletedEvent

Produced Events

Der Supervisor veröffentlicht verbindlich

SupervisorValidatedEvent
SupervisorAlertEvent
SupervisorFindingCreatedEvent
SupervisorFindingResolvedEvent
SupervisorBlockedEvent
SupervisorEscalatedEvent
HealthChangedEvent

Sämtliche Ereignisnamen folgen dem Naming Contract aus Chapter 8.

Correlation

Der Supervisor übernimmt die Correlation ID des auslösenden Ereignisses unverändert.
Eine eigene Correlation ID wird ausschließlich bei periodischen Vollprüfungen erzeugt.

Documentary Integration

Die Documentary Engine erzeugt aus den Supervisor-Ergebnissen automatisch

Supervisor Report

Finding Report

Health Report

Governance Report

Escalation Report

Der Supervisor erzeugt diese Dokumente niemals selbst.

Platform Director Integration

Der Supervisor liefert dem Platform Director verbindlich

Gesamtzustand der Plattform

offene Befunde

blockierte Änderungen

Risikobewertung

Freigabeempfehlung

Der Platform Director entscheidet ausschließlich auf dieser Grundlage.

Version Manager Integration

Der Supervisor bestätigt vor jeder Versionsänderung

Vollständigkeit der Dokumentation

Vollständigkeit der Validierung

Synchronität des Digital Twin

Abwesenheit blockierender Befunde

End of Chapter 5

Chapter 6

Supervisor Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen technischen Anforderungen an die Implementierung des Supervisors.

Location

Der Supervisor befindet sich ausschließlich unter

```
src/platform/Supervisor/
```

Interface Contract

Der Supervisor veröffentlicht ausschließlich öffentliche Enterprise Interfaces.

Verbindlich sind

observe

evaluate

report

block

escalate

describe

Determinism Contract

Bei identischem Plattformzustand erzeugt der Supervisor

identische Befunde

identische Schweregrade

identische Empfehlungen

Zeitstempel sind hiervon ausgenommen.

Persistence Contract

Befunde werden dauerhaft gespeichert.

Die Befundhistorie ist unveränderbar.

Gelöste Befunde bleiben mit Status Resolved erhalten.

Read Only Contract

Der Supervisor besitzt niemals schreibenden Zugriff auf

Quellcode

Metadaten

Registry

Knowledge Graph

Digital Twin

Dokumentation

Versionen

Er schreibt ausschließlich in seinen eigenen Befundspeicher.

Validation

Vor jeder Freigabe des Supervisors wird geprüft

- ✓ sämtliche Beobachtungsbereiche abgedeckt
 - ✓ sämtliche Pflicht-Events abonniert
 - ✓ sämtliche Befunde nachweisbasiert
 - ✓ Determinismus nachgewiesen
 - ✓ kein produktiver Schreibzugriff
 - ✓ Eskalationsstufen implementiert
 - ✓ Befundhistorie unveränderbar
-

Enterprise Rules

Der Supervisor beobachtet.

Der Supervisor entscheidet nicht.

Der Supervisor dokumentiert nicht.

Der Supervisor verändert keine produktiven Artefakte.

Kein Befund ohne Nachweis.

Keine Freigabe mit blockierenden Befunden.

Keine Ausnahme ohne ADR.

Success Criteria

Der Supervisor gilt als erfolgreich implementiert wenn

- ✓ jede Vertragsverletzung erkannt wird
 - ✓ jeder Befund nachvollziehbar bleibt
 - ✓ jede blockierende Bedingung wirksam ist
 - ✓ jede Eskalation dokumentiert ist
 - ✓ der Platform Director jederzeit den vollständigen Plattformzustand kennt
 - ✓ keine unvollständige Wertschöpfungskette die Produktion erreicht
 - ✓ keine Komponente ohne Health Status existiert
-

End of Chapter 6

Enterprise Final Summary

ESS-0002 Status

Document ID

ESS-0002

Titel

CAPITAL-AI Supervisor Architect

Status

Enterprise Specification

Version

1.0.0

Lifecycle Status

Approved Enterprise Specification

Mission Statement

Der Supervisor bildet die Überwachungsinstanz der CAPITAL-AI Plattform.

Er kennt jederzeit den vollständigen Zustand sämtlicher Enterprise-Komponenten.

Er bewertet ausschließlich nachweisbare Informationen.

Er blockiert jede Änderung, die den Enterprise Contracts widerspricht.

Er entscheidet niemals selbst.

Architectural Scope

ESS-0002 definiert

- ✓ Observation Model
 - ✓ Health Supervision
 - ✓ Lifecycle Supervision
 - ✓ Governance Enforcement
 - ✓ Finding und Escalation Contracts
 - ✓ Event Integration
 - ✓ Supervisor Implementation Contracts
-

Governance Statement

ESS-0002 ist die verbindliche Enterprise-Spezifikation des Supervisors innerhalb des CAPITAL-AI Core.

Sie baut vollständig auf ESS-0001 und ESS-0001-CONTRACTS auf.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste vollständige Enterprise-Spezifikation des Supervisors

Related Enterprise Specifications

- ESS-0001 — Documentary & Code Intelligence Architect
- ESS-0001-CONTRACTS — Enterprise Technical Contracts
- ESS-0003 — Platform Director

Approval

- Document Status
- APPROVED
- Enterprise Specification
- CAPITAL-AI Core Architecture
- Version 1.0.0

End of Document

- ESS-0002
- CAPITAL-AI Supervisor Architect
- Enterprise Specification
- Version 1.0.0

ESS 0003 Platform Director

Quelle: `.ai/skills/ESS-0003-Platform-Director.md`

skill: id: ESS-0003 name: Platform Director version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority: Critical

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

authority:

controls: - Enterprise Governance - Architecture Authority - Release Authority - Exception Authority - AI Orchestration Authority

collaborates: - Supervisor - Documentary Engine - Version Manager - Security Center - Compliance Center - Quality Center - Release Center

cannot_modify: - Trading Logic - Portfolio Logic - Crypto Scoring Algorithms - Stripe Billing Logic - OAuth Authentication - Database Business Logic - Audit Trail

classification: type: Component Specification role: Komponentenspezifikation Platform Director contractAuthority: ESS-0001-CONTRACTS note: > Dieses Dokument definiert ausschließlich Contracts des Platform Director. Globale Contracts verbleiben in ESS-0001-CONTRACTS.

references: - ESS-0001 - ESS-0001-CONTRACTS - ESS-0002 - ADR-0006 - ADR-0007

crossReference: dependsOn: - ESS-0001 - ESS-0001-CONTRACTS - ESS-0002 relatedEss: - ESS-0010 - ESS-0011 - ESS-0011-CONTRACTS relatedAdr: - ADR-0006 - ADR-0007 - ADR-0010 - ADR-0011 - ADR-0013 - ADR-0015 relatedComponents: - src/platform/Platform-Director - src/platform/Release - src/platform/Compliance - src/platform/Traceability relatedSkills: - .ai/skills/ESS-0001-Contracts.md - .ai/skills/ESS-0002-Supervisor-Architect.md

created: 2026-07-30

Platform Director

Enterprise Mission

Der Platform Director ist die oberste Governance-Instanz der CAPITAL-AI Plattform.

Er trifft sämtliche Entscheidungen, die nicht durch bestehende Contracts bereits eindeutig geregelt sind.

Er entscheidet niemals gegen einen Contract.

Er entscheidet ausschließlich innerhalb der durch ESS und ADR gesetzten Grenzen.

Wo eine Entscheidung eine bestehende Regel verändert, entsteht verbindlich ein Architecture Decision Record.

Vision

Die Plattform besitzt jederzeit genau eine verantwortliche Entscheidungsinstanz.

Entscheidungen sind nachvollziehbar.

Entscheidungen sind begründet.

Entscheidungen sind dauerhaft dokumentiert.

Enterprise Philosophy

Die Plattform folgt dem Grundsatz

Entscheidung erfordert Nachweis.

Der Platform Director entscheidet ausschließlich auf Grundlage

des Digital Twin

des Knowledge Graph

der Supervisor-Bewertung

der Impact Analyse

der Risikobewertung

Er entscheidet niemals auf Grundlage von Annahmen.

Abgrenzung

Instanz	Verantwortung
Documentary Engine	Wissen und Dokumentation
Supervisor	Beobachtung und Bewertung
Platform Director	Entscheidung und Governance
Version Manager	Versionierung
Release Center	Auslieferung

Der Platform Director implementiert nicht.

Der Platform Director dokumentiert nicht.

Der Platform Director überwacht nicht.

Er entscheidet.

Position in der Architektur

Gemäß ESS-0001-CONTRACTS Chapter 6 bildet der Platform Director die oberste Ebene der Layer-Hierarchie.

Er besitzt Zugriff auf sämtliche Plattformkomponenten.

Keine Komponente besitzt Zugriff auf den Platform Director.

Verhältnis zu ADR-0006

ADR-0006 legt die schrittweise Anbindung des Plattform-Direktors innerhalb des Backends fest.

ESS-0003 widerspricht ADR-0006 nicht.

ESS-0003 beschreibt die Enterprise-Governance-Rolle des Platform Director innerhalb der Plattformarchitektur.

ADR-0006 beschreibt die technischen Anbindungsschritte im Backend.

Beide Dokumente ergänzen sich.

Bei Widersprüchen gilt gemäß ESS-0001-CONTRACTS Chapter 1 die ADR.

End of Chapter 1

Chapter 2

Decision Authority

Enterprise Purpose

Dieses Kapitel definiert verbindlich, welche Entscheidungen dem Platform Director vorbehalten sind.

Exclusive Decisions

Ausschließlich der Platform Director entscheidet über

Einführung neuer Layer

Einführung neuer Plattformmodule

Einführung neuer Root-Verzeichnisse

Änderung der Repository-Struktur

Vergabe von ESS-Nummern

Freigabe neuer Enterprise Contracts

Freigabe neuer Erweiterungsarten

Freigabe neuer KI-Systeme

Genehmigung von Ausnahmen

Akzeptanz von Risiken

Freigabe von Produktionsversionen

Entfernung von Legacy-Bestand

Deprecation-Zeitpunkte

Priorisierung der Enterprise Roadmap

Delegated Decisions

Folgende Entscheidungen sind vertraglich bereits geregelt und werden nicht erneut entschieden.

Gegenstand	Regelnde Instanz
Versionsstufe	Version Manager gemäß Chapter 9
Quality Gate Ergebnis	Validatoren gemäß Chapter 12
Health Status	Supervisor gemäß ESS-0002
Dokumentationsinhalt	Documentary Engine gemäß ESS-0001
Knowledge-Beziehungen	Knowledge Engine gemäß Chapter 15
Twin-Zustand	Digital Twin gemäß Chapter 18

Der Platform Director überschreibt diese Ergebnisse niemals ohne ADR.

Decision Contract

Jede Entscheidung besitzt verbindlich

Decision ID

Titel

Gegenstand

Entscheidungsgrundlage

Alternativen

Begründung

Risikobewertung

betroffene Komponenten

betroffene Contracts

Version

ADR Referenz

Zeitpunkt

Entscheider

Correlation ID

Decision Types

Architecture Decision

Governance Decision

Release Decision

Exception Decision

Risk Decision

Priority Decision

Emergency Decision

Jede Architecture Decision erzeugt verbindlich einen ADR.

Decision Prerequisites

Eine Entscheidung wird ausschließlich getroffen, wenn vorliegen

Supervisor Assessment

Impact Analyse

Risikobewertung

Digital Twin im Zustand Synchronized

vollständige Entscheidungsgrundlage

Fehlt eine Grundlage, wird die Entscheidung zurückgestellt.

Eine Zurückstellung ist ebenfalls eine dokumentierte Entscheidung.

Emergency Decisions

Notfallentscheidungen sind ausschließlich zulässig bei

Sicherheitsvorfällen

Produktionsausfällen

Datenschutzverletzungen

kritischen Datenfehlern

Für jede Notfallentscheidung gilt verbindlich

sofortige Protokollierung

Nachdokumentation innerhalb der laufenden Version

verbindliche ADR

Nachbewertung durch den Supervisor

End of Chapter 2

Chapter 3

Governance Model

Enterprise Purpose

Der Platform Director verantwortet die Einhaltung der gesamten Enterprise Governance.

Governance Scope

Verantwortet werden

Enterprise Specifications

Enterprise Contracts

Architecture Decision Records

Repository Governance

AI Governance

Security Governance

Compliance Governance

Release Governance

Standard Ownership

Der Platform Director ist Eigentümer

sämtlicher ESS-Dokumente

sämtlicher Enterprise Contracts

des ESS-Nummernraums

des ADR-Nummernraums

der Ausnahmenregistrierung

Er ändert diese Dokumente ausschließlich über den in Chapter 20 definierten Change Contract.

Exception Authority

Ausnahmen werden ausschließlich durch den Platform Director genehmigt.

Jede Ausnahme besitzt verbindlich

Exception ID

verletzten Contract

Begründung

Risiko

Zielzustand

Status

ADR Referenz

Zulässige Status

Approved

Time Limited

Permanent

Revoked

Abgelaufene Ausnahmen werden automatisch zu Befunden.

Ownership Assignment

Der Platform Director weist jedem Repository-Bereich und jeder Komponente genau einen Eigentümer zu.

Ohne Eigentümer existiert keine zulässige Komponente.

Eigentümerwechsel erzeugen verbindlich ein Enterprise Event.

Roadmap Authority

Der Platform Director führt die Enterprise Roadmap.

Die Roadmap enthält verbindlich

Zielarchitektur

geplante Komponenten

geplante Migrationen

geplante Deprecations

Prioritäten

Zielversionen

Die Roadmap wird ausschließlich gegen den geplanten Digital Twin geführt.

End of Chapter 3

Chapter 4

AI Orchestration Authority

Enterprise Purpose

Der Platform Director koordiniert sämtliche KI-Systeme gemäß ESS-0001-CONTRACTS Chapter 10 und Chapter 17.

Authority over AI Systems

Der Platform Director entscheidet über

Zulassung neuer KI-Systeme

Verantwortungszuschnitt je KI-System

Reihenfolge der Wertschöpfungskette

Abbruch laufender Ketten

Wiederholung von Stufen

Modellwechsel

Value Chain Governance

Die verbindliche Wertschöpfungskette lautet

Google AI Studio

↓

Claude Code

↓

Documentary Engine

↓

Supervisor

↓

Platform Director

↓

Version Manager

↓

Release

↓

Production

Der Platform Director bildet die Freigabestufe dieser Kette.

Ohne seine Entscheidung erreicht keine Änderung den Version Manager.

Model Independence

Der Platform Director stellt sicher, dass

kein Ergebnis von einem einzelnen Modell abhängt

kein KI-System eigene Governance-Regeln etabliert

sämtliche KI-Systeme dieselben Contracts erfüllen

sämtliche KI-Systeme dieselbe Wissensbasis verwenden

AI Boundaries

Der Platform Director stellt verbindlich sicher, dass kein KI-System

Architekturentscheidungen trifft

Contracts verändert

ESS-Dokumente verändert

ADRs überschreibt

Versionen eigenständig erhöht

Sicherheitsmechanismen abschwächt

Ausnahmen selbst genehmigt

Handover Governance

Jede Übergabe zwischen KI-Stufen wird verbindlich

protokolliert

validiert

versioniert

im Audit Trail geführt

Unvollständige Übergaben führen zum Abbruch der Kette.

End of Chapter 4

Chapter 5

Release Authority

Enterprise Purpose

Der Platform Director erteilt sämtliche Produktionsfreigaben.

Release Prerequisites

Eine Produktionsfreigabe erfolgt ausschließlich wenn

- ✓ sämtliche Quality Gates bestanden sind
- ✓ keine blockierenden Befunde offen sind
- ✓ der Digital Twin synchron ist
- ✓ der Knowledge Graph aktuell ist
- ✓ die Dokumentation vollständig erzeugt wurde
- ✓ die Version bestimmt wurde

- ✓ ein Rollback-Plan existiert
 - ✓ Security und Compliance bestätigt sind
 - ✓ die AI-Wertschöpfungskette vollständig durchlaufen wurde
-

Release Decision

Die Freigabeentscheidung besitzt verbindlich

Version

Release-Typ

Entscheidungsgrundlage

Risikobewertung

Rollback-Plan

Freigabezeitpunkt

Entscheider

Rollback Authority

Der Platform Director entscheidet über

Auslösung eines Rollbacks

Umfang des Rollbacks

Wiederaufnahme des Betriebs

Nachbereitung

Rollbacks werden niemals stillschweigend durchgeführt.

Production Governance

Produktionsänderungen erfordern zusätzlich

Version Manager Freigabe

Supervisor Freigabe

Quality Center Freigabe

Compliance Center Prüfung

Security Center Prüfung

Deployment Validation

Knowledge Synchronisation

Diese Anforderungen entsprechen ESS-0001 Chapter 7.

End of Chapter 5

Chapter 6

Platform Director Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen technischen Anforderungen an die Implementierung des Platform Director.

Location

Der Platform Director befindet sich ausschließlich unter

```
src/platform/PlatformDirector/
```

Interface Contract

Der Platform Director veröffentlicht ausschließlich öffentliche Enterprise Interfaces.

Verbindlich sind

decide

approve

reject

defer

grantException

assignOwnership

describe

Consumed Events

Verbindlich abonniert werden

SupervisorValidatedEvent

SupervisorEscalatedEvent

SupervisorBlockedEvent

TwinDriftDetectedEvent

VersionCalculatedEvent

SecurityViolationDetectedEvent

ComplianceViolationDetectedEvent

StructureViolationDetectedEvent

ExceptionExpiredEvent

Produced Events

Verbindlich veröffentlicht werden

PlatformDecisionEvent

PlatformApprovalEvent

PlatformRejectionEvent

ExceptionRegisteredEvent

OwnershipChangedEvent

ReleaseApprovedEvent

RollbackOrderedEvent

EmergencyDecisionEvent

Traceability Contract

Jede Entscheidung ist dauerhaft nachvollziehbar.

Entscheidungen werden niemals gelöscht.

Widerrufene Entscheidungen erhalten den Status Revoked und bleiben erhalten.

Documentation Contract

Der Platform Director erzeugt keine Dokumentation.

Sämtliche Entscheidungsdokumente werden durch die Documentary Engine erzeugt.

Erzeugt werden

Decision Report

Approval Report

Exception Report

Governance Report

Roadmap Report

Validation

Vor jeder Freigabe des Platform Director wird geprüft

- ✓ sämtliche Entscheidungen besitzen eine Grundlage
 - ✓ sämtliche Architekturentscheidungen besitzen eine ADR
 - ✓ sämtliche Ausnahmen besitzen einen Status
 - ✓ sämtliche Entscheidungen sind versioniert
 - ✓ sämtliche Entscheidungen sind im Audit Trail geführt
 - ✓ keine Entscheidung widerspricht einem bestehenden Contract
-

Enterprise Rules

Keine Entscheidung ohne Grundlage.

Keine Architekturentscheidung ohne ADR.

Keine Ausnahme ohne Registrierung.

Keine Freigabe ohne Supervisor Assessment.

Keine Produktionsfreigabe bei offenen blockierenden Befunden.

Keine Entscheidung gegen einen bestehenden Contract.

Keine Entscheidung durch ein KI-System.

Success Criteria

Der Platform Director gilt als erfolgreich implementiert wenn

- ✓ sämtliche Entscheidungen dokumentiert sind
 - ✓ sämtliche Entscheidungen nachvollziehbar bleiben
 - ✓ sämtliche Ausnahmen registriert sind
 - ✓ sämtliche Freigaben nachweisbasiert erfolgen
 - ✓ keine Änderung ohne Freigabe die Produktion erreicht
 - ✓ die Wertschöpfungskette jederzeit steuerbar bleibt
 - ✓ die Plattform genau eine Entscheidungsinstanz besitzt
-

End of Chapter 6

Enterprise Final Summary

ESS-0003 Status

Document ID

ESS-0003

Titel

CAPITAL-AI Platform Director

Status

Enterprise Specification

Version

1.0.0

Lifecycle Status

Approved Enterprise Specification

Mission Statement

Der Platform Director bildet die oberste Governance-Instanz der CAPITAL-AI Plattform.

Er entscheidet ausschließlich auf nachweisbarer Grundlage.

Er entscheidet niemals gegen bestehende Contracts.

Jede Entscheidung bleibt dauerhaft nachvollziehbar.

Architectural Scope

- ESS-0003 definiert
- ✓ Decision Authority
 - ✓ Governance Model
 - ✓ Exception Authority
 - ✓ Ownership Assignment
 - ✓ AI Orchestration Authority
 - ✓ Release Authority
 - ✓ Platform Director Implementation Contracts
-

Governance Statement

ESS-0003 ist die verbindliche Enterprise-Spezifikation des Platform Director innerhalb des CAPITAL-AI Core.

Sie baut vollständig auf ESS-0001, ESS-0001-CONTRACTS und ESS-0002 auf und ergänzt ADR-0006.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste vollständige Enterprise-Spezifikation des Platform Director

Related Enterprise Specifications

ESS-0001 — Documentary & Code Intelligence Architect

ESS-0001-CONTRACTS — Enterprise Technical Contracts

ESS-0002 — Supervisor Architect

Approval

Document Status

APPROVED

Enterprise Specification

CAPITAL-AI Core Architecture

Version 1.0.0

End of Document

ESS-0003

CAPITAL-AI Platform Director

Enterprise Specification

Version 1.0.0

ESS 0004 Enterprise Version Manager

Quelle: `.ai/skills/ESS-0004-Enterprise-Version-Manager.md`

skill: id: ESS-0004 name: Enterprise Version Manager version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority: Critical

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

classification: type: Component Specification role: Komponentenspezifikation Version Manager contractAuthority: ESS-0001-CONTRACTS note: > Dieses Dokument spezifiziert ausschliesslich die Komponente `src/platform/VersionManager`. Die globalen Versionierungsregeln verbleiben in ESS-0001-CONTRACTS Chapter 9 und ESS-0001 Chapter 9.

authority:

controls: - Version Registry - Versionsempfehlung - Versionssynchronisation - Rollback-Planung - Release-Vorbereitung

collaborates: - Documentary Engine - Enterprise Traceability - Supervisor - Platform Director - Release Center - Quality Center

cannot_modify: - Enterprise Specifications - Architecture Decision Records - Enterprise Contracts - Quellcode - Knowledge Graph - Digital Twin

crossReference: dependsOn: - ESS-0001 - ESS-0001-CONTRACTS relatedEss: - ESS-0002 - ESS-0003 - ESS-0007 - ESS-0010 - ESS-0011 relatedAdr: - ADR-0010 - ADR-0016 related-Components: - `src/platform/VersionManager` - `src/platform/Release` - `src/platform/Registry` - `src/platform/Traceability` relatedSkills: - `.ai/skills/ESS-0001-Contracts.md` - `.ai/skills/ESS-0011-Enterprise-Traceability.md`

created: 2026-07-31

Enterprise Version Manager

Enterprise Purpose

Dieses Dokument spezifiziert die Komponente `src/platform/VersionManager`.

ESS-0001 Chapter 9 beschreibt die Versionsstrategie der Plattform.

ESS-0001-CONTRACTS Chapter 9 definiert die verbindlichen Versionierungs- und Release-Contracts.

Dieses Dokument beschreibt ausschließlich, **wie** die Komponente aufgebaut ist, welche Schnittstellen sie veröffentlicht und wie sie in die Plattform eingebettet ist.

Es enthält keine globalen Versionierungsregeln.

Abgrenzung

Dokument	Verantwortung
ESS-0001 Chapter 9	Versionsstrategie, Versionskategorien
ESS-0001-CONTRACTS Chapter 9	verbindliche Versionierungs- und Release-Contracts
ESS-0001-CONTRACTS Chapter 14	Lifecycle, Vocabulary Mapping
ESS-0004	Komponentenspezifikation Version Manager
ESS-0007	Release Center

Bei Abweichungen gilt ausnahmslos ESS-0001-CONTRACTS.

Enterprise Principle

Der Version Manager bestimmt die Version.

Er entscheidet nicht über die Freigabe.

Er berechnet ausschließlich aus nachweisbaren Änderungen — niemals aus Absichtserklärungen.

Eine Version ohne Impact Analyse ist eine Behauptung.

Position in der Architektur

Gemäß ESS-0001-CONTRACTS Chapter 6 liegt der Version Manager oberhalb der Documentary Engine und unterhalb des Supervisor.



Zulässige Abhängigkeiten: Core, Shared, Registry, Knowledge, Documentary.

Unzulässig: Supervisor, Platform Director.

End of Chapter 1

Chapter 2

Komponentenarchitektur

Verzeichnisstruktur

```
src/platform/VersionManager/
```

Die innere Struktur folgt der allgemeinen Modulstruktur aus ESS-0001-CONTRACTS Chapter 3.

Kernkomponenten

VersionCalculator

Berechnung der Versionsempfehlung.

Verantwortung

Auswertung der Impact Analyse

Einstufung nach Major, Minor, Patch

Erkennung von Breaking Changes

Begründung der Einstufung

VersionRegistry

Persistenz sämtlicher Versionsstände.

Verantwortung

Repository-Version

Komponentenversionen

Knowledge Version

Architecture Version

Documentation Version

Twin Version

VersionSynchronizer

Abgleich der Versionsquellen.

Verantwortung

Erkennung widersprüchlicher Versionsstände

Meldung von Abweichungen

Bereitstellung des maßgeblichen Standes

Der Synchronizer **korrigiert nicht selbsttätig**. Er meldet Abweichungen als Befund.

RollbackPlanner

Erzeugung der Rollback-Artefakte.

Verantwortung

Rollback-Version

Rollback-Reihenfolge

Rollback-Voraussetzungen

Rollback-Risiken

ChangelogGenerator

Erzeugung der Änderungsdokumentation.

Verantwortung

Komponenten-Changelog

Release Notes

Versionshistorie

Die physische Dokumenterzeugung erfolgt durch die Documentary Engine gemäß ESS-0010.

End of Chapter 2

Chapter 3

Interfaces

Sämtliche Interfaces folgen ESS-0001-CONTRACTS Chapter 4.

IVersionCalculator

<code>calculate(impact)</code>	Versionsempfehlung aus Impact Analyse
<code>classify(change)</code>	Einstufung einer Einzeländerung
<code>detectBreaking(delta)</code>	Erkennung von Breaking Changes

IVersionRegistry

<code>current(scope)</code>	aktueller Versionsstand
<code>history(scope)</code>	Versionshistorie
<code>record(version)</code>	Aufnahme eines Versionsstandes

IVersionSynchronizer

<code>compare()</code>	Abgleich sämtlicher Versionsquellen
<code>conflicts()</code>	erkannte Widersprüche
<code>authoritative()</code>	maßgeblicher Stand

IRollbackPlanner

<code>plan(version)</code>	Rollback-Plan zu einer Version
<code>validate(plan)</code>	Prüfung der Durchführbarkeit

Sämtliche Methoden sind asynchron und idempotent.

End of Chapter 3

Chapter 4

Events

Namen folgen dem Naming Contract aus ESS-0001-CONTRACTS Chapter 8.

Erzeugte Events

VersionCalculatedEvent

VersionChangedEvent

VersionConflictDetectedEvent
BreakingChangeDetectedEvent
RollbackPlanCreatedEvent
ChangelogGeneratedEvent
ReleasePreparedEvent

Konsumierte Events

ImplementationCompletedEvent
DocumentationGeneratedEvent
KnowledgeUpdatedEvent
TwinSynchronizedEvent
TraceabilityBuildCompletedEvent
PlatformDecisionEvent

End of Chapter 4

Chapter 5

Integration

Wertschöpfungskette

Der Version Manager bildet Stufe 6 der Kette aus ESS-0001-CONTRACTS Chapter 17.

Eingang Freigabe des Platform Director, Impact Analyse, Änderungsumfang

Ausgang Version, Changelog, Release Notes, Rollback-Plan

Documentary Engine

Liefert Impact-Daten: geänderte Komponenten, Interfaces, Events, Breaking Changes.

Enterprise Traceability

Liefert die Regelachse gemäß ESS-0011: welche Regeln eine Änderung berührt und welche Verknüpfungen entfallen sind.

Eine entfallene Verknüpfung ohne **SUPERSEDES** ist ein Breaking Change.

Supervisor

Bestätigt vor jeder Versionsänderung Vollständigkeit der Dokumentation, Validierung und Twin-Synchronität.

Platform Director

Trifft die Freigabeentscheidung. Der Version Manager führt sie aus, er ersetzt sie nicht.

Bekannter Bestand

`server/versionManager.ts` führt produktiv Version, Build-Nummer, Git-Tag, Docker-Tag und Release Notes.

Diese Implementierung ist gemäß ESS-0001-CONTRACTS Chapter 14 als Legacy zu registrieren und über einen Adapter anzubinden — **nicht** neu zu entwickeln.

Offener Befund: Es bestehen vier widersprüchliche Versionsstände im Repository (GAP-019, Regel `GOV-VER-001`). Die Auflösung ist Voraussetzung für den produktiven Betrieb dieser Komponente.

Enterprise Rules

Keine Version ohne Impact Analyse.

Keine Version ohne Rollback-Plan.

Keine Versionsänderung ohne Event.

Kein selbsttätiges Korrigieren widersprüchlicher Versionsstände.

Keine Freigabeentscheidung durch den Version Manager.

Success Criteria

- ✓ jede Änderung erhält eine begründete Versionsempfehlung
 - ✓ Breaking Changes werden aus der Traceability abgeleitet, nicht geschätzt
 - ✓ sämtliche Versionsquellen sind abgeglichen
 - ✓ jede Version besitzt einen Rollback-Plan
 - ✓ Changelog und Release Notes entstehen generiert
 - ✓ keine widersprüchlichen Versionsstände verbleiben
-

Enterprise Final Summary

Document ID ESS-0004

Titel CAPITAL-AI Enterprise Version Manager

Status Enterprise Specification

Version 1.0.0

Governance Statement

ESS-0004 ist die verbindliche Komponentenspezifikation des Version Manager.

Sie baut vollständig auf ESS-0001 und ESS-0001-CONTRACTS auf.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Komponentenspezifikation des Version Manager

End of Document

ESS-0004

CAPITAL-AI Enterprise Version Manager

Version 1.0.0

ESS 0005 Quality Center

Quelle: `.ai/skills/ESS-0005-Quality-Center.md`

skill: id: ESS-0005 name: Quality Center version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority: Critical

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

classification: type: Component Specification role: Komponentenspezifikation Quality Center contractAuthority: ESS-0001-CONTRACTS note: > Dieses Dokument spezifiziert ausschließlich die Komponente `src/platform/Quality`. Validator-Basisvertrag, Severity-Stufen und Quality Gates verbleiben in ESS-0001-CONTRACTS Chapter 12.

authority:

controls: - Validator Registry - Quality Gate Ausfuehrung - Quality Score Berechnung - Technical Debt Register - Testabdeckung

collaborates: - Documentary Engine - Supervisor - Platform Director - Version Manager - Release Center

cannot_modify: - Enterprise Specifications - Architecture Decision Records - Enterprise Contracts - Quellcode - Schwellwerte

crossReference: dependsOn: - ESS-0001 - ESS-0001-CONTRACTS relatedEss: - ESS-0002 - ESS-0003 - ESS-0010 - ESS-0011 - ESS-0012 relatedAdr: - ADR-0010 - ADR-0016 relatedComponents: - `src/platform/Quality` - `src/platform/Validators` - `src/platform/Telemetry` - tests relatedSkills: - `.ai/skills/ESS-0001-Contracts.md` - `.ai/skills/ESS-0012-Documentation-Governance.md`

created: 2026-07-31

Quality Center

Enterprise Purpose

Dieses Dokument spezifiziert die Komponente `src/platform/Quality`.

ESS-0001-CONTRACTS Chapter 12 definiert die verbindlichen Validierungs- und Qualitätsverträge — 16 Pflichtvalidatoren, Severity-Stufen, acht Quality Gates.

Dieses Dokument beschreibt ausschließlich, **wie** das Quality Center diese Verträge ausführt.

Es definiert keine Regeln, keine Schweregrade und keine Schwellwerte.

Abgrenzung

Dokument	Verantwortung
ESS-0001-CONTRACTS Chapter 12	Validator-Vertrag, Severity, Quality Gates, Metriken
ESS-0011-CONTRACTS	Coverage-Achsen der Traceability
ESS-0012-CONTRACTS	Governance-Regelwerk
ESS-0005	Komponentenspezifikation Quality Center

Das Quality Center **führt aus**. Es legt nicht fest.

Enterprise Principle

Qualität ist keine Bewertung.

Qualität ist eine Messung.

Ein Quality Gate ohne ausführbare Prüfung ist eine Absichtserklärung.

Ein Score, der manuell gesetzt werden kann, ist kein Score.

Position in der Architektur

Querschnittsmodul gemäß ESS-0001-CONTRACTS Chapter 16.

Zulässige Abhängigkeiten: Core, Shared.

Zusätzlich lesend: Registry, Knowledge, Documentary — zur Auswertung der Prüfgegenstände.

Unzulässig: Version Manager, Supervisor, Platform Director.

End of Chapter 1

Chapter 2

Komponentenarchitektur

Kernkomponenten

ValidatorRegistry

Registrierung und Auflösung sämtlicher Validatoren.

Verantwortung

Registrierung der 16 Pflichtvalidatoren aus Chapter 12

Registrierung zusätzlicher Validatoren

Versionierung der Validatoren

Auflösung von Ausführungsreihenfolgen

GateRunner

Ausführung der acht Quality Gates.

Verantwortung

Zuordnung Validator zu Gate

Ausführung in fester Reihenfolge

Aggregation der Ergebnisse

Gate-Entscheidung bestanden oder nicht bestanden

Ein Gate gilt ausschließlich als bestanden, wenn keine Critical- und keine High-Befunde vorliegen.

ScoreCalculator

Berechnung der Qualitätskennzahlen aus Chapter 12.

Documentation Score

Test Score

Architecture Score

Security Score

Knowledge Score

Metadata Score

Twin Score

Sämtliche Werte werden ausschließlich berechnet.

TechnicalDebtRegister

Erfassung technischer Schulden gemäß Chapter 12.

Verantwortung

Aufnahme mit ID, Ursache, Auswirkung, Aufwand, Priorität, Zielversion

Verfolgung bis zur Behebung

Meldung wachsender Schulden

Technische Schulden werden niemals stillschweigend geschlossen.

CoverageCollector

Sammlung der Testabdeckung aus den Testbereichen gemäß Chapter 12.

```
tests/unit tests/integration tests/contract
tests/architecture tests/security tests/performance tests/e2e
```

End of Chapter 2

Chapter 3

Interfaces

IValidatorRegistry

register(validator)	Aufnahme eines Validators
resolve(contractArea)	Validatoren je Vertragsbereich
list()	sämtliche registrierten Validatoren

IQualityGate

identifizier()	Gate-Kennung
validators()	zugeordnete Validatoren
execute(scope)	Ausführung
result()	bestanden oder nicht bestanden

IScoreCalculator

calculate(component)	sämtliche Kennzahlen
threshold(lifecycle)	Schwellwert je Lebenszyklus

ITechnicalDebtRegister

record(debt)	Aufnahme
list(component)	Schulden je Komponente
resolve(id, evidence)	Schließen mit Nachweis

End of Chapter 3

Chapter 4

Events

Erzeugte Events

ValidationStartedEvent

ValidationCompletedEvent

ValidationFailedEvent

QualityGatePassedEvent

QualityGateFailedEvent

QualityScoreChangedEvent

TechnicalDebtDetectedEvent

TechnicalDebtResolvedEvent

CoverageCalculatedEvent

Konsumierte Events

ImplementationCompletedEvent

DocumentationGeneratedEvent

KnowledgeUpdatedEvent

TwinSynchronizedEvent

VersionCalculatedEvent

End of Chapter 4

Chapter 5

Integration

Supervisor

Das Quality Center liefert dem Supervisor gemäß ESS-0002 Gate-Ergebnisse, Befunde und Kennzahlen. Der Supervisor bewertet und blockiert.

Das Quality Center blockiert selbst nicht — es stellt fest.

Version Manager

Liefert die Qualitätsachse der Versionsbewertung: nicht bestandene Gates blockieren eine Release-Vorbereitung gemäß Chapter 9.

Governance Validator

ESS-0012 ist ein Validator im Sinne von Chapter 12 und wird über die ValidatorRegistry registriert. Sein Regelwerk verbleibt in ESS-0012-CONTRACTS.

Enterprise Traceability

Die Test-Achse der Matrix speist sich aus den Ergebnissen des Quality Center.

Bekannter Bestand

Das Repository enthält derzeit **null Testdateien** und **null Validatoren**.

`package.json` kennt keinen Test-Runner; `npm run lint` führt ausschließlich `tsc --noEmit` aus.

Damit ist kein einziges Quality Gate ausführbar. Die acht Gates aus Chapter 1 existieren ausschließlich als Vertrag.

Dies ist der schwerwiegendste offene Punkt der Plattform: Ohne ausführbare Prüfungen bleibt jede Regel dieses Standards eine Absichtserklärung.

Umsetzung gemäß Stufe 4 aus `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md`.

Enterprise Rules

Kein Contract ohne Validator.

Kein Quality Gate ohne ausführbare Prüfung.

Keine Kennzahl ohne Berechnung.

Keine Integration mit Critical-Befunden.

Keine Änderung von Schwellwerten durch das Quality Center.

Keine Blockade durch das Quality Center selbst.

Success Criteria

- ✓ sämtliche 16 Pflichtvalidatoren registriert und ausführbar
 - ✓ sämtliche acht Quality Gates ausführbar
 - ✓ sämtliche Kennzahlen deterministisch berechnet
 - ✓ sämtliche Testbereiche belegt
 - ✓ technische Schulden vollständig erfasst
 - ✓ keine nicht konforme Änderung integrierbar
-

Enterprise Final Summary

Document ID ESS-0005

Titel CAPITAL-AI Quality Center

Status Enterprise Specification

Version 1.0.0

Governance Statement

ESS-0005 ist die verbindliche Komponentenspezifikation des Quality Center.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Komponentenspezifikation des Quality Center

End of Document

ESS-0005

CAPITAL-AI Quality Center

Version 1.0.0

ESS 0006 Security Compliance

Quelle: `.ai/skills/ESS-0006-Security-Compliance.md`

skill: id: ESS-0006 name: Security & Compliance version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority: Critical

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

classification: type: Component Specification role: Komponentenspezifikation Security Center und Compliance Center contractAuthority: ESS-0001-CONTRACTS note: > Dieses Dokument spezifiziert ausschliesslich die Komponenten `src/platform/Security` und `src/platform/Compliance`. Saemtliche Sicherheits- und Compliance-Regeln verbleiben in ESS-0001-CONTRACTS Chapter 11.

authority:

controls: - Security Classification Registry - Audit Trail - Risk Register - Compliance Evidence - Security Review Verfahren

collaborates: - Documentary Engine - Supervisor - Platform Director - Quality Center - Version Manager - Release Center

cannot_modify: - Enterprise Specifications - Architecture Decision Records - Enterprise Contracts - Audit Trail Eintraege - IAM Regeln - Berechtigungen

crossReference: dependsOn: - ESS-0001 - ESS-0001-CONTRACTS relatedEss: - ESS-0002 - ESS-0003 - ESS-0005 - ESS-0011 - ESS-0012 relatedAdr: - ADR-0003.5 - ADR-0007 - ADR-0009 - ADR-0012 - ADR-0016 relatedComponents: - `src/platform/Security` - `src/platform/Compliance` - `src/platform/Telemetry` - `supabase/policies` relatedSkills: - `.ai/skills/ESS-0001-Contracts.md` - `.ai/skills/ESS-0002-Supervisor-Architect.md`

created: 2026-07-31

Security & Compliance

Enterprise Purpose

Dieses Dokument spezifiziert die Komponenten `src/platform/Security` und `src/platform/Compliance`.

ESS-0001-CONTRACTS Chapter 11 definiert die verbindlichen Sicherheits- und Compliance-Verträge — Klassifizierung, Secret Contract, Audit Trail, Risk Contract, Compliance Evidence.

Dieses Dokument beschreibt ausschließlich, **wie** beide Komponenten diese Verträge durchsetzen.

Es definiert keine Sicherheitsregeln.

Warum beide Komponenten in einem Dokument

Security und Compliance sind zwei Komponenten mit **einer** gemeinsamen Nachweiskette:

Security Classification → Kontrolle → Audit-Eintrag → Compliance Evidence → Nachweis

Eine getrennte Spezifikation würde diese Kette an der Dokumentgrenze zerschneiden. Die Nummernreservierung aus ESS-0001 führt beide Bereiche ebenfalls gemeinsam (Security & Compliance).

Die Komponenten bleiben im Repository getrennt.

Abgrenzung

Dokument	Verantwortung
ESS-0001-CONTRACTS Chapter 11	sämtliche Sicherheits- und Compliance-Regeln
ESS-0005	Ausführung von Security-Validatoren
ESS-0012-CONTRACTS	Governance-Prüfung der Dokumentation
ESS-0006	Komponentenspezifikation Security und Compliance

Enterprise Principle

Sicherheit wird nachgewiesen, nicht behauptet.

Eine Compliance-Aussage ohne Nachweis besitzt keine Gültigkeit.

Der Audit Trail ist unveränderbar — auch für die Komponente, die ihn schreibt.

Position in der Architektur

Beide Komponenten sind Querschnittsmodule gemäß ESS-0001-CONTRACTS Chapter 16.

Zulässige Abhängigkeiten: Core, Shared.

Unzulässig: Version Manager, Supervisor, Platform Director.

End of Chapter 1

Chapter 2

Security Center

Kernkomponenten

ClassificationRegistry

Führung der Sicherheitsklassifizierung je Komponente.

Stufen gemäß Chapter 11: Public, Internal, Confidential, Restricted, Critical.

Eine Komponente ohne Klassifizierung ist nicht integrationsfähig.

SecretGuard

Durchsetzung des Secret Contracts.

Prüft auf

Secrets im Repository

Secrets in Logs

Secrets in Events

Secrets in Dokumentation

Secrets in Fehlermeldungen

Ein Fund ist ausnahmslos ein Critical-Befund.

AuditTrail

Unveränderbare Protokollierung gemäß Chapter 11.

Jeder Eintrag führt Zeitpunkt, auslösende Instanz, Komponente, Operation, Ergebnis, Version, Correlation ID, ESS- und ADR-Referenzen.

Einträge werden **niemals** gelöscht oder verändert — auch nicht durch das Security Center selbst.

RiskRegister

Führung sämtlicher Risiken gemäß Chapter 11.

Jedes Risiko besitzt ID, Kategorie, Eintrittswahrscheinlichkeit, Auswirkung, betroffene Komponenten, Maßnahme, Verantwortlichen, Status.

Ein Risiko wird niemals ohne Dokumentation geschlossen.

SecurityReviewCoordinator

Steuerung der Review-Pflicht gemäß Chapter 11 — verbindlich bei Restricted- und Critical-Komponenten, Änderungen an Authentifizierung, Autorisierung, Zahlungsprozessen, Datenbank-Policies und Secret-Verwaltung.

End of Chapter 2

Chapter 3

Compliance Center

Kernkomponenten

ComplianceRequirementRegistry

Zuordnung der Compliance-Anforderungen je Komponente.

Datenschutz, Finanzaufsicht, Aufbewahrungs-, Nachweis-, Protokoll- und Exportpflichten.

EvidenceCollector

Sammlung der Nachweise gemäß Chapter 11, Compliance Evidence.

Zulässige Nachweise: Audit Trail, Security Report, Compliance Report, Validation Report, Test Report, Architecture Report.

Behauptungen ohne Nachweis werden verworfen.

ComplianceReporter

Aufbereitung der Nachweisdaten.

Die physische Berichtserzeugung erfolgt durch die Documentary Engine gemäß ESS-0010.

End of Chapter 3

Chapter 4

Interfaces

ISecurityClassifier

<code>classify(component)</code>	Klassifizierung ermitteln
<code>requiresReview(change)</code>	Review-Pflicht prüfen

IAuditTrail

<code>append(entry)</code>	Eintrag hinzufügen, niemals ändern
<code>query(filter)</code>	Auswertung
<code>verify()</code>	Integritätsprüfung

IRiskRegister

<code>record(risk)</code>	Aufnahme
<code>assess(risk)</code>	Bewertung
<code>close(id, evidence)</code>	Schließen ausschließlich mit Nachweis

IComplianceEvidence

<code>collect(requirement)</code>	Nachweise sammeln
<code>verify(requirement)</code>	Erfüllung prüfen

End of Chapter 4

Chapter 5

Events

Erzeugte Events

SecurityScanCompletedEvent

SecurityClassificationChangedEvent

SecurityViolationDetectedEvent

SecretExposureDetectedEvent

ComplianceValidatedEvent

ComplianceViolationDetectedEvent

RiskDetectedEvent

RiskResolvedEvent

AuditCompletedEvent

PermissionChangedEvent

Konsumierte Events

RepositoryScannedEvent

ImplementationCompletedEvent

ComponentRegisteredEvent

VersionChangedEvent

MigrationCompletedEvent

End of Chapter 5

Chapter 6

Integration und Bestand

Supervisor

Sicherheitsvorfälle werden gemäß ESS-0002 unmittelbar auf Eskalationsstufe 5 gemeldet.

Platform Director

Entscheidet über Sicherheitsausnahmen und Risikoakzeptanz — ausschließlich per ADR.

Version Manager

Sicherheitskorrekturen besitzen gemäß Chapter 11 jederzeit Vorrang vor funktionalen Änderungen.

Bekannter Bestand

Security besitzt den höchsten Implementierungsgrad der Plattform.

`server/iam/` umfasst Authentifizierungs-Middleware, Rate Limiting, Secret-Verschlüsselung und TOTP. `server/stepUp.ts` ergänzt Step-Up-Authentifizierung. ADR-0003.5 ist als abgeschlossen verifiziert, ADR-0009 (CORS Hardening) ebenfalls.

Die Datenbank führt `audit_logs_iam`, `iam_access_log`, `security_events`, `step_up_tokens` und `break_glass_codes`.

Diese Implementierung ist gemäß Chapter 14 zu registrieren und zu kapseln — niemals neu zu entwickeln.

Offene Befunde

Befund	Regel
keine Komponente besitzt eine Sicherheitsklassifizierung	GOV-REPO-007
kein Audit Trail im Sinne von Chapter 11	Chapter 11
Security Metadata in 23 von 24 Manifesten unvollständig	GOV-REPO-007
ADR-0012: Compliance-Endpunkte existieren nicht	FND-ADR-0012-01

Enterprise Rules

Keine Komponente ohne Security Classification.

Keine Restricted-Komponente ohne Security Review.

Keine Compliance-Aussage ohne Nachweis.

Keine Manipulation des Audit Trails.

Keine Secrets im Repository.

Keine Ausnahme ohne ADR.

Success Criteria

- ✓ sämtliche Komponenten klassifiziert
- ✓ Audit Trail lückenlos und unveränderbar
- ✓ sämtliche Risiken dokumentiert und bewertet
- ✓ sämtliche Compliance-Anforderungen mit Nachweis belegt

- ✓ keine Freigabe ohne Sicherheitsprüfung
 - ✓ Bestandsimplementierung gekapselt statt ersetzt
-

Enterprise Final Summary

Document ID ESS-0006

Titel CAPITAL-AI Security & Compliance

Status Enterprise Specification

Version 1.0.0

Governance Statement

ESS-0006 ist die verbindliche Komponentenspezifikation von Security Center und Compliance Center.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Komponentenspezifikation Security und Compliance

End of Document

ESS-0006

CAPITAL-AI Security & Compliance

Version 1.0.0

ESS 0007 Enterprise Release Center

Quelle: `.ai/skills/ESS-0007-Enterprise-Release-Center.md`

skill: id: ESS-0007 name: Enterprise Release Center version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority: High

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

classification: type: Component Specification role: Komponentenspezifikation Release Center contractAuthority: ESS-0001-CONTRACTS note: > Dieses Dokument spezifiziert ausschliesslich die Komponente `src/platform/Release`. Versionierungs- und Release-Contracts verbleiben in ESS-0001-CONTRACTS Chapter 9, Automatisierungsregeln in Chapter 19.

authority:

controls: - Release Vorbereitung - Release Artefakte - Deployment Planung - Rollback Ausfuehrung - Release Registry

collaborates: - Version Manager - Documentary Engine - Quality Center - Security Center - Compliance Center - Supervisor - Platform Director

cannot_modify: - Enterprise Specifications - Architecture Decision Records - Enterprise Contracts - Versionen - Quality Gate Ergebnisse

crossReference: dependsOn: - ESS-0001 - ESS-0001-CONTRACTS - ESS-0004 relatedEss: - ESS-0002 - ESS-0003 - ESS-0005 - ESS-0006 - ESS-0011 relatedAdr: - ADR-0010 - ADR-0016 relatedComponents: - `src/platform/Release` - `src/platform/VersionManager` - `src/platform/Quality` - `scripts/deployment` relatedSkills: - `.ai/skills/ESS-0001-Contracts.md` - `.ai/skills/ESS-0004-Enterprise-Version-Manager.md`

created: 2026-07-31

Enterprise Release Center

Enterprise Purpose

Dieses Dokument spezifiziert die Komponente `src/platform/Release`.

ESS-0001-CONTRACTS Chapter 9 definiert die verbindlichen Release-Contracts, Chapter 19 die Automatisierungsverträge.

Dieses Dokument beschreibt ausschließlich, **wie** das Release Center eine Auslieferung vorbereitet, ausführt und zurücknimmt.

Es definiert keine Versionsregeln und keine Freigabekriterien.

Abgrenzung

Dokument	Verantwortung
ESS-0001-CONTRACTS Chapter 9	Release Lifecycle, Version Validation, Rollback Contract
ESS-0001-CONTRACTS Chapter 19	Automatisierung, Prozessverträge
ESS-0004	Versionsermittlung
ESS-0003	Freigabeentscheidung
ESS-0007	Komponentenspezifikation Release Center

Das Release Center liefert aus. Es entscheidet nicht, ob ausgeliefert wird.

Enterprise Principle

Ein Release ist die Zusammenführung bereits geprüfter Ergebnisse.

Das Release Center erzeugt keine neue Wahrheit.

Es fügt zusammen, was Version Manager, Quality Center, Security Center und Documentary Engine festgestellt haben.

Ein Release ohne Rollback-Plan wird niemals ausgeführt.

Position in der Architektur

Querschnittsmodul mit erweitertem Zugriff gemäß ESS-0001-CONTRACTS Chapter 16 — Release besitzt ausdrücklich zusätzlichen Zugriff auf Registry, Knowledge, Documentary und Version Manager, da es deren Ergebnisse zusammenführt.

Unzulässig: Supervisor, Platform Director.

End of Chapter 1

Chapter 2

Komponentenarchitektur

Kernkomponenten

ReleasePreparer

Zusammenführung sämtlicher Release-Bestandteile.

Verantwortung

Sammlung der Artefakte

Prüfung der Vollständigkeit

Erzeugung des Release-Pakets

ReleaseValidator

Prüfung der Freigabevoraussetzungen gemäß Chapter 9.

Geprüft wird

Repository aktuell

Knowledge aktuell

Dokumentation aktuell

Architecture aktuell

ADR aktuell

Tests erfolgreich

Migration vollständig

Quality Gates bestanden

Supervisor synchronisiert

Platform Director synchronisiert

Das Release Center **stellt fest**. Die Freigabe erteilt der Platform Director.

DeploymentPlanner

Erzeugung des Deployment-Plans.

Umgebungen gemäß ESS-0001 Chapter 7: Development, Testing, Staging, Production, Rollback, Canary, Blue Green, Emergency.

RollbackExecutor

Ausführung der Rücknahme.

Verantwortung

Ausführung des vom Version Manager erzeugten Rollback-Plans

Reihenfolge

Prüfung der Voraussetzungen

Bestätigung des erreichten Zustands

Das Release Center erzeugt den Rollback-Plan nicht — es führt ihn aus.

ReleaseRegistry

Persistenz sämtlicher Releases.

Je Release: Version, Zeitpunkt, Artefakte, Umgebung, Freigabegrundlage, Twin-Snapshot, Traceability-Snapshot, Rollback-Version.

End of Chapter 2

Chapter 3

Interfaces

IReleasePreparer

prepare(version)	Release-Paket erzeugen
artifacts(version)	Bestandteile auflisten

IReleaseValidator

validate(release)	sämtliche Voraussetzungen prüfen
blockers(release)	blockierende Befunde

IDeploymentPlanner

plan(release, target)	Deployment-Plan erzeugen
validate(plan)	Durchführbarkeit prüfen

IRollbackExecutor

<code>execute(plan)</code> <code>verify()</code>	Rollback ausführen erreichten Zustand bestätigen
---	---

End of Chapter 3

Chapter 4

Events

Erzeugte Events

ReleasePreparedEvent
ReleaseValidatedEvent
ReleaseBlockedEvent
ReleasePublishedEvent
DeploymentStartedEvent
DeploymentCompletedEvent
RollbackStartedEvent
RollbackCompletedEvent

Konsumierte Events

VersionChangedEvent
QualityGatePassedEvent
QualityGateFailedEvent
SecurityScanCompletedEvent
ComplianceValidatedEvent
TwinSynchronizedEvent
PlatformDecisionEvent
RollbackOrderedEvent

End of Chapter 4

Chapter 5

Integration

Wertschöpfungskette

Das Release Center bildet Stufe 7 der Kette aus ESS-0001-CONTRACTS Chapter 17.

Eingang Version, Dokumentation, Validierungsergebnisse

Ausgang Release-Paket, Deployment-Plan, Release Report

Snapshot-Pflicht

Zu jedem Release werden verbindlich gesichert

Digital Twin Snapshot gemäß Chapter 18

Traceability-Matrix-Snapshot gemäß ESS-0011

Versionsstand gemäß ESS-0004

Dadurch ist rückwirkend nachvollziehbar, welcher Architektur- und Regelzustand ausgeliefert wurde.

Bekannter Bestand

Der Auslieferungsweg existiert technisch: `package.json` (`build`, `start`), `Dockerfile` mit zweistufigem Build, `render.yaml`.

Er ist jedoch **nicht an die Governance-Kette gebunden** — ein Release ist derzeit ohne Durchlauf von Documentary Engine, Quality Center und Version Manager möglich.

`scripts/deployment/` ist leer.

Dies ist der Befund `CHAIN-07` aus `ARCH-CHAIN-0001`.

Enterprise Rules

Kein Release ohne bestandene Quality Gates.

Kein Release ohne Freigabe des Platform Director.

- Kein Release ohne Rollback-Plan.
- Kein Release ohne Twin- und Traceability-Snapshot.
- Keine Versionsentscheidung durch das Release Center.
- Kein Rollback ohne dokumentierte Begründung.

Success Criteria

- ✓ jedes Release durchläuft die vollständige Kette
- ✓ jedes Release besitzt Snapshots von Twin und Matrix
- ✓ jedes Release besitzt einen ausführbaren Rollback-Plan
- ✓ kein Release umgeht die Quality Gates
- ✓ sämtliche Release-Artefakte werden generiert

Enterprise Final Summary

Document ID ESS-0007

Titel CAPITAL-AI Enterprise Release Center

Status Enterprise Specification

Version 1.0.0

Governance Statement

ESS-0007 ist die verbindliche Komponentenspezifikation des Release Center.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Komponentenspezifikation des Release Center

End of Document

ESS-0007

CAPITAL-AI Enterprise Release Center

Version 1.0.0

ESS 0008 AI Agent Framework

Quelle: `.ai/skills/ESS-0008-AI-Agent-Framework.md`

skill: id: ESS-0008 name: AI Agent Framework version: 1.0.0 status: Enterprise Approved
maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority:
High

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle:
AI Native Development Lifecycle

classification: type: Technical Specification role: Rahmenwerk fuer fachliche Domaenen-
Agenten contractAuthority: ESS-0001-CONTRACTS note: > Dieses Dokument spezifiziert
das Rahmenwerk der fachlichen Domaenen-Agenten. Die Orchestrierung der KI-Systeme
(Claude Code, Google AI Studio, ChatGPT) verbleibt in ESS-0001-CONTRACTS Chapter 10
und Chapter 17.

authority:

controls: - Agent Registry - Agent Lifecycle - Agent Contracts - Orchestrator Registry

collaborates: - Documentary Engine - Supervisor - Platform Director - Version Manager -
Quality Center

cannot_modify: - Enterprise Specifications - Architecture Decision Records - Enterprise
Contracts - Scoring-Algorithmen - Trading Logic - Portfolio Logic

crossReference: dependsOn: - ESS-0001 - ESS-0001-CONTRACTS relatedEss: - ESS-0002 -
ESS-0003 - ESS-0010 - ESS-0011 relatedAdr: - ADR-0006 - ADR-0016 relatedComponents: -
src/agents - src/orchestrator - src/services - src/platform/Registry relatedSkills: - .ai/skills/
ESS-0001-Contracts.md - .ai/skills/ESS-0003-Platform-Director.md

created: 2026-07-31

AI Agent Framework

Enterprise Purpose

Dieses Dokument spezifiziert das Rahmenwerk der **fachlichen Domänen-Agenten** des
CAPITAL-AI Core.

Es schließt eine echte Regelungslücke: ESS-0001-CONTRACTS Chapter 10 und Chapter 17
regeln die Orchestrierung der **KI-Entwicklungssysteme** — Claude Code, Google AI Stu-
dio, ChatGPT. Sie regeln **nicht** die Agenten, die im Produktivbetrieb Finanzanalysen durch-
führen.

Diese Agenten existieren produktiv: acht Agenten unter `src/agents` , zwei Orchestratoren unter `src/orchestrator` , sieben Scoring-Services unter `src/services` .

Für sie existierte bislang kein Enterprise-Vertrag.

Abgrenzung

Gegenstand	Zuständiges Dokument
KI-Entwicklungssysteme, Wertschöpfungskette	ESS-0001-CONTRACTS Chapter 10, Chapter 17
Agent-Erkennung durch Discovery	ESS-0001 Chapter 3, ESS-0010
fachliche Domänen-Agenten	ESS-0008
Scoring-Algorithmen selbst	unverändert, siehe Nicht Bestandteil

Nicht Bestandteil

Dieses Dokument verändert **niemals**

Scoring-Algorithmen

Bewertungslogik

Gewichtungen

Modellauswahl im Fachkontext

Trading- oder Portfolio-Logik

Es regelt ausschließlich **Struktur, Registrierung, Schnittstellen und Nachvollziehbarkeit** der Agenten.

Die fachliche Richtigkeit einer Bewertung ist nicht Gegenstand dieses Standards.

Enterprise Principle

Ein Agent ist eine Enterprise-Komponente.

Er besitzt Identität, Version, Owner, Schnittstelle und Registrierung.

Ein Agent ohne Registrierung existiert für die Plattform nicht.

Ein Agent, dessen Ergebnis nicht reproduzierbar ist, ist kein Agent, sondern eine Quelle zufälliger Werte.

End of Chapter 1

Chapter 2

Agent Contract

Pflichtangaben

Jeder Agent beschreibt verbindlich

agentId

name

version

owner

domain

responsibility

inputs

outputs

dependencies

llmUsage

deterministic

events

essReferences

adrReferences

Agent Identity

Format

```
agent:<domain>/<name>
```

Beispiele

```
agent:crypto/CryptoRiskAgent
agent:equity/FundamentalsAgent
agent:shared/ClassificationAgent
```

Die ID bleibt über sämtliche Versionen stabil.

Agent Classes

Klasse	Bedeutung
Classification Agent	Einordnung eines Assets
Analysis Agent	fachliche Kennzahlenermittlung
Risk Agent	Risikobewertung
Sentiment Agent	Stimmungsauswertung
Valuation Agent	Bewertungsrechnung
Composite Agent	Zusammenführung mehrerer Agenten

Determinism Contract

Ein Agent deklariert verbindlich, ob er deterministisch ist.

Wert	Bedeutung
deterministic: true	identische Eingabe erzeugt identische Ausgabe
deterministic: false	LLM-gestützt, Ergebnis kann variieren

Nicht deterministische Agenten führen zusätzlich

verwendetes Modell

Temperatur oder gleichwertigen Parameter

Rückfallverhalten bei Nichtverfügbarkeit

Ein nicht deterministischer Agent darf **niemals** als alleinige Quelle einer sicherheits-, abrechnungs- oder compliancerelevanten Entscheidung dienen.

Data Integrity Contract

Verbindlich gemäß der bestehenden Plattformdirektive

Keine Mock-Daten im Produktivbetrieb.

Keine simulierten Werte, wo Echtdaten erwartet werden.

Jede Antwort wird strukturell validiert, bevor sie weiterverarbeitet wird.

Fehlende Daten führen zu einem erkennbaren Fehlerzustand, niemals zu einem erfundenen Wert.

End of Chapter 2

Chapter 3

Orchestrator Contract

Zweck

Ein Orchestrator führt mehrere Agenten zu einem Ergebnis zusammen.

Pflichtangaben

orchestratorId

domain

beteiligte Agenten

Ausführungsreihenfolge

Zusammenführungsregel

Rückfallverhalten

Zeitgrenzen

Auswahlregel

Die bestehende Hierarchie bleibt unverändert gültig:

Spezialisierte Orchestratoren werden zuerst befragt, sofern ein Asset ihrer Domäne entspricht.

Die universelle Fallback-Engine ist ausschließlich für Assets zuständig, für die kein spezialisierter Orchestrator existiert.

Dieser Grundsatz stammt aus der bestehenden Plattformdokumentation und wird hier nicht verändert, sondern als Vertrag festgehalten.

Isolation

Ein Agent ruft niemals einen anderen Agenten direkt auf.

Die Zusammenführung erfolgt ausschließlich über den Orchestrator.

End of Chapter 3

Chapter 4

Registrierung und Events

Agent Registry

Jeder Agent wird in der Enterprise Registry gemäß ESS-0001-CONTRACTS Chapter 7 geführt.

Registriert werden ID, Version, Klasse, Domäne, Owner, Determinismus, Health Status, Schnittstelle, Events.

Nicht registrierte Agenten werden niemals ausgeführt.

Events

Erzeugte Events

AgentRegisteredEvent

AgentExecutedEvent

AgentFailedEvent

AgentDeprecatedEvent

OrchestratorExecutedEvent

ScoringCompletedEvent

Konsumierte Events

ComponentRegisteredEvent

VersionChangedEvent

Telemetrie

Jede Ausführung protokolliert Dauer, Ergebnisstatus, verwendetes Modell, Correlation ID.

Telemetriedaten enthalten niemals personenbezogene Daten.

End of Chapter 4

Chapter 5

Integration und Bestand

Bekannter Bestand

Bereich	Umfang
<code>src/agents/</code>	acht Agenten
<code>src/orchestrator/</code>	zwei Orchestratoren (Crypto, Raw Materials)
<code>src/services/</code>	sieben Scoring- und Ranking-Services
<code>src/lib/requestOrchestrator.ts</code>	modellunabhängiges Routing

Sämtliche Agenten folgen bereits einem einheitlichen Muster: exportierte Klasse je Agent, teilweise mit exportiertem Ergebnis-Interface.

Diese Implementierung ist **produktiv** und wird gemäß ESS-0001-CONTRACTS Chapter 14 registriert und gekapselt — **niemals neu entwickelt**.

Offene Befunde

Befund	Regel
kein Agent besitzt einen Registry-Eintrag	Chapter 7
kein Agent deklariert Determinismus	ESS-0008 Chapter 2
Agenten liegen außerhalb von <code>src/features/<do-main></code>	GAP-009
keine Agent-Events implementiert	Chapter 8
keine Tests für Agenten	Chapter 12

Supervisor

Überwacht Agent-Ausfälle, Zeitüberschreitungen, nicht registrierte Agenten und fehlende Determinismus-Deklarationen.

Documentary Engine

Erkennt Agenten gemäß ESS-0001 Chapter 3 automatisch und erzeugt Agent Registry sowie Agent-Dokumentation.

Enterprise Rules

Kein Agent ohne Registrierung.

Kein Agent ohne Determinismus-Deklaration.

Kein direkter Agent-zu-Agent-Aufruf.

Keine Mock-Daten im Produktivbetrieb.

Kein nicht deterministischer Agent als alleinige Quelle kritischer Entscheidungen.

Keine Änderung an Scoring-Algorithmen durch diesen Standard.

Success Criteria

- ✓ sämtliche Agenten registriert
 - ✓ sämtliche Agenten deklarieren Determinismus
 - ✓ sämtliche Orchestratoren beschreiben ihre Zusammenführungsregel
 - ✓ sämtliche Ausführungen erzeugen Telemetrie
 - ✓ keine Mock-Daten im Produktivpfad
 - ✓ Bestandsimplementierung gekapselt statt ersetzt
-

Enterprise Final Summary

Document ID ESS-0008

Titel CAPITAL-AI AI Agent Framework

Status Enterprise Specification

Version 1.0.0

Governance Statement

ESS-0008 ist die verbindliche Spezifikation des Rahmenwerks fachlicher Domänen-Agenten.

Sie verändert keine fachliche Bewertungslogik.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Spezifikation des Agent-Rahmenwerks

End of Document

ESS-0008

CAPITAL-AI AI Agent Framework

Version 1.0.0

ESS 0009 Enterprise Knowledge Platform

Quelle: `.ai/skills/ESS-0009-Enterprise-Knowledge-Platform.md`

skill: id: ESS-0009 name: Enterprise Knowledge Platform version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority: Critical

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

classification: type: Component Specification role: Komponentenspezifikation Knowledge Engine contractAuthority: ESS-0001-CONTRACTS note: > Dieses Dokument spezifiziert ausschliesslich die Komponente `src/platform/Knowledge`. Das Wissensmodell und seine Regeln verbleiben in ESS-0001 Chapter 4 und ESS-0001-CONTRACTS Chapter 15.

authority:

controls: - Knowledge Builder - Knowledge Query - Knowledge Validation - Knowledge Versionierung

collaborates: - Documentary Engine - Enterprise Traceability - Supervisor - Platform Director - Version Manager

cannot_modify: - Enterprise Specifications - Architecture Decision Records - Enterprise Contracts - Quellcode - Metadaten - Registry

crossReference: dependsOn: - ESS-0001 - ESS-0001-CONTRACTS relatedEss: - ESS-0002 - ESS-0003 - ESS-0010 - ESS-0011 - ESS-0012 relatedAdr: - ADR-0010 - ADR-0016 relatedComponents: - `src/platform/Knowledge` - `src/platform/Discovery` - `src/platform/Registry` - `src/platform/Documentary` relatedSkills: - `.ai/skills/ESS-0001-Documentary-Architect.md` - `.ai/skills/ESS-0010-Documentary-Engine.md`

created: 2026-07-31

Enterprise Knowledge Platform

Enterprise Purpose

Dieses Dokument spezifiziert die Komponente `src/platform/Knowledge`.

ESS-0001 Chapter 4 beschreibt das Wissensmodell der Plattform.

ESS-0001-CONTRACTS Chapter 15 definiert die verbindlichen Knowledge-Graph-Contracts — Knotenidentität, Beziehungstypen, Ursprung, Vertrauenswert, Determinismus.

Dieses Dokument beschreibt ausschließlich, **wie** die Komponente den Graphen aufbaut, prüft, versioniert und bereitstellt.

Es definiert keine Knoten- und keine Beziehungsregeln.

Abgrenzung

Dokument	Verantwortung
ESS-0001 Chapter 4	Wissensmodell, Vision
ESS-0001-CONTRACTS Chapter 15	Node Contract, Relationship Contract, Origin, Confidence
ESS-0010	Documentary Engine, die den Aufbau auslöst
ESS-0011	Traceability Matrix, die den Graphen nutzt
ESS-0009	Komponentenspezifikation Knowledge Engine

Enterprise Principle

Wissen entsteht ausschließlich aus Analyse.

Wissen wird niemals geschrieben, nur abgeleitet.

Ein Knoten ohne Ursprung ist kein Wissen, sondern eine Behauptung.

Der Graph ist die einzige autorisierte Quelle für Architekturwissen — sämtliche KI-Systeme verwenden ihn, keines analysiert das Repository erneut selbst.

Position in der Architektur

Knowledge Layer gemäß ESS-0001-CONTRACTS Chapter 6.

Zulässige Abhängigkeiten: Core, Shared, Registry, Discovery.

Unzulässig: Documentary, Version Manager, Supervisor, Platform Director.

Der Graph wird von oben gelesen, nicht von oben geschrieben.

End of Chapter 1

Chapter 2

Komponentenarchitektur

Kernkomponenten

KnowledgeBuilder

Aufbau des Graphen.

Verantwortung

Knotenerzeugung aus Discovery-Ergebnissen

Beziehungsableitung

Ursprungszuordnung

Vertrauenswertbestimmung

Prüfsummenbildung

KnowledgeValidator

Konsistenzprüfung gemäß Chapter 15.

Geprüft wird

Knoten ohne Typ

Knoten ohne Ursprung

Beziehungen auf nicht existierende Knoten

doppelte IDs

zyklische OWNS-Beziehungen

widersprüchliche Versionsangaben

verwaiste Knoten

Assumed-Beziehungen werden verworfen, niemals aufgenommen.

KnowledgeQuery

Bereitstellung der Abfragefähigkeit.

Beantwortet verbindlich die Fragen aus Chapter 15, Query Contract — unter anderem

Welche Komponenten besitzen keine ADR?

Welche Events besitzen keinen Consumer?

Welche Interfaces besitzen keine Implementierung?

Welche Komponenten verletzen die Layer-Hierarchie?

Welche Legacy-Komponenten besitzen keinen Adapter?

KnowledgeVersioner

Versionierung des Wissensbestands.

Die Knowledge Version wird erhöht bei neuen Knotentypen, neuen Beziehungstypen, strukturellen Änderungen des Modells oder vollständigem Neuaufbau.

KnowledgeStore

Persistenz.

```
.ai/knowledge/  
  repository.json  architecture.json  components.json  
  interfaces.json  services.json     events.json  
  agents.json      orchestrators.json database.json  
  api.json          security.json      documentation.json  
  dependencies.json workflows.json    policies.json  
  versions.json     releases.json      plugins.json  
  risks.json        graph.json
```

Jede Datei besitzt ein Schema unter `.ai/schemas/`.

Sämtliche Dateien werden ausschließlich generiert.

End of Chapter 2

Chapter 3

Interfaces

IKnowledgeBuilder

<code>build()</code>	vollständiger Aufbau
<code>update(delta)</code>	inkrementelle Aktualisierung
<code>checksum()</code>	reproduzierbare Prüfsumme

IKnowledgeValidator

validate()	vollständige Konsistenzprüfung
conflicts()	erkannte Widersprüche

IKnowledgeQuery

node(id)	Knoten abrufen
relations(id, type)	Beziehungen abrufen
query(expression)	semantische Abfrage

IKnowledgeVersioner

version()	aktuelle Knowledge Version
snapshot(version)	Sicherung

End of Chapter 3

Chapter 4

Events

Erzeugte Events

KnowledgeBuildStartedEvent

KnowledgeCreatedEvent

KnowledgeUpdatedEvent

KnowledgeLinkedEvent

KnowledgeValidatedEvent

KnowledgeConflictDetectedEvent

KnowledgeVersionChangedEvent

Konsumierte Events

RepositoryScannedEvent

CodeAnalyzedEvent

ComponentRegisteredEvent

MigrationCompletedEvent

Determinismus

Bei identischem Repository-Zustand erzeugt jeder Aufbau identische Knoten, IDs, Beziehungen und Prüfsummen.

Zeitstempel und Laufzeitinformationen sind von der Prüfsummenbildung ausgenommen.

Weicht die Prüfsumme bei unverändertem Eingangszustand ab, gilt der Aufbau als fehlerhaft.

End of Chapter 4

Chapter 5

Integration und Bestand

Documentary Engine

Der Knowledge Builder wird ausschließlich durch die Documentary Engine ausgelöst (ESS-0010).

Die Knowledge Engine löst sich nicht selbst aus.

Enterprise Traceability

Die Matrix aus ESS-0011 verwendet den Graphen als Pflichtquelle. Fehlt er, bricht der Matrixaufbau ab (GOV-KG-001 , Critical).

Sämtliche KI-Systeme

Verwenden ausschließlich diesen Graphen zur Architekturinterpretation. Eigene Repository-Analysen zur Wissensbildung, lokale Wissensspeicher und manuelle Ergänzungen sind unzulässig.

Bekannter Bestand

`.ai/knowledge/` enthält ausschließlich `.gitkeep`.

Es existiert **kein einziger Knoten**.

Damit ist die zentrale Aussage aus ESS-0001 Chapter 4 — sämtliche KI-Agenten verwenden dieselbe Wissensbasis — derzeit nicht erfüllt. Jedes KI-System analysiert das Repository erneut selbst.

Dies ist Befund `G0V-KG-001` (Critical) und blockiert zugleich Digital Twin und Traceability Matrix.

Die Knowledge Engine ist damit die Komponente mit der größten Hebelwirkung: Ihr Aufbau schaltet zwei weitere Subsysteme frei.

Umsetzung gemäß Stufe 5 aus `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md`.

Enterprise Rules

Wissen entsteht ausschließlich aus Analyse.

Kein Knoten ohne Ursprung.

Keine Beziehung ohne Richtung.

Keine Assumed-Beziehung im Graphen.

Keine manuelle Änderung des Wissensbestands.

Keine parallele Wissensbasis.

Kein Selbstauflösen des Aufbaus.

Success Criteria

- ✓ sämtliche Komponenten als Knoten vorhanden
 - ✓ sämtliche Beziehungen abgeleitet und gerichtet
 - ✓ Graph reproduzierbar aufbaubar
 - ✓ sämtliche Wissensdateien schemakonform
 - ✓ Query Contract vollständig beantwortbar
 - ✓ sämtliche KI-Systeme verwenden denselben Graphen
-

Enterprise Final Summary

Document ID ESS-0009

Titel CAPITAL-AI Enterprise Knowledge Platform

Status Enterprise Specification

Version 1.0.0

Governance Statement

ESS-0009 ist die verbindliche Komponentenspezifikation der Knowledge Engine.
Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Komponentenspezifikation der Knowledge Engine

End of Document

ESS-0009
CAPITAL-AI Enterprise Knowledge Platform
Version 1.0.0

ESS 0010 Documentary Engine

Quelle: `.ai/skills/ESS-0010-Documentary-Engine.md`

skill: id: ESS-0010 name: Documentary Engine version: 1.0.0 status: Enterprise Approved
maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority:
Critical

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle:
AI Native Development Lifecycle

classification: type: Technical Specification role: Technische Spezifikation der Documentary
Engine contractAuthority: ESS-0001-CONTRACTS foundationalDocument: ESS-0001 note: >
Dieses Dokument enthält ausschließlich technische Spezifikationsinhalte. Vision,
Motivation und historische Architekturbegründung verbleiben in ESS-0001.

authority:

controls: - Repository Discovery - Code Discovery - Knowledge Graph - Enterprise Registry -
Contract Validation - Architecture Reports - Release Reports - Impact Analysis - Version
Reports - AI Audit Trail - Digital Twin

collaborates: - Platform Director - Supervisor - Version Manager - Security Center -
Compliance Center - Quality Center - Release Center

cannot_modify: - Trading Logic - Portfolio Logic - Crypto Scoring Algorithms - Stripe Billing
Logic - OAuth Authentication - Database Business Logic - Enterprise Specifications -
Architecture Decision Records

crossReference: dependsOn: - ESS-0001 - ESS-0001-CONTRACTS relatedEss: - ESS-0002 -
ESS-0003 - ESS-0011 - ESS-0011-CONTRACTS relatedAdr: - ADR-0010 - ADR-0011 -
ADR-0013 - ADR-0015 relatedComponents: - src/platform/Documentary - src/platform/
Knowledge - src/platform/Discovery - src/platform/Registry - src/platform/Architecture - src/
platform/Generators - src/platform/Validators - src/platform/Events - src/platform/Traceabili-
ty relatedSkills: - `.ai/skills/ESS-0001-Documentary-Architect.md` - `.ai/skills/ESS-0001-Con-
tracts.md`

created: 2026-07-31

Documentary Engine

Enterprise Purpose

Dieses Dokument ist die verbindliche technische Spezifikation der CAPITAL-AI
Documentary Engine.

ESS-0001 beschreibt die Vision und den ursprünglichen Architekturentwurf.

ESS-0001-CONTRACTS definiert die globalen Enterprise Contracts, denen die Documentary Engine unterliegt — insbesondere Chapter 15 (Knowledge Graph), Chapter 17 (AI Orchestration), Chapter 18 (Digital Twin) und Chapter 19 (Automation).

Dieses Dokument beschreibt ausschließlich, **wie** die Documentary Engine technisch aufgebaut ist: Komponenten, Services, APIs, Events, Workflows, Trigger, Discovery, Registry, Digital Twin und Integration.

Es enthält keine Vision, keine Motivation und keine historischen Inhalte.

Abgrenzung

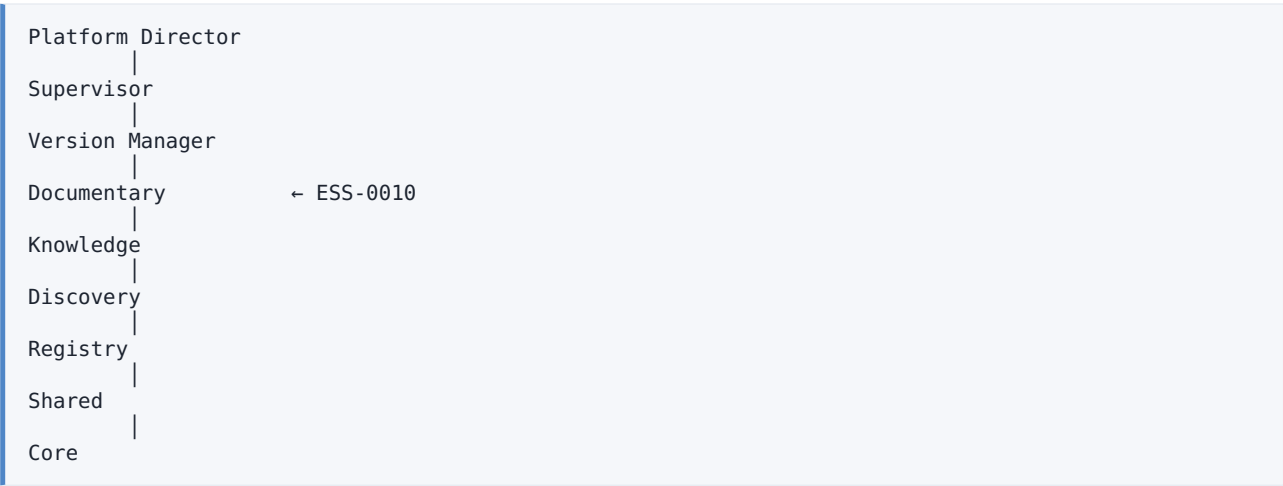
Dokument	Verantwortung
ESS-0001	Vision, Zielbild, Designprinzipien, ursprünglicher Entwurf
ESS-0001-CONTRACTS	globale Contracts, denen die Engine unterliegt
ESS-0010	technische Spezifikation der Engine
ESS-0011	Traceability Matrix, die die Engine speist

Bei Abweichungen zwischen ESS-0001 und diesem Dokument besitzt dieses Dokument Vorrang.

Bei Abweichungen zwischen diesem Dokument und ESS-0001-CONTRACTS besitzt ESS-0001-CONTRACTS Vorrang.

Position in der Architektur

Gemäß ESS-0001-CONTRACTS Chapter 6 befindet sich die Documentary Engine im Documentary Layer.



Zulässige Abhängigkeiten: Core, Shared, Registry, Discovery, Knowledge.

Die Documentary Engine besitzt niemals Abhängigkeiten auf Version Manager, Supervisor oder Platform Director. Die Kommunikation nach oben erfolgt ausschließlich über Enterprise Events.

End of Chapter 1

Chapter 2

Komponentenarchitektur

Enterprise Purpose

Dieses Kapitel definiert die verbindliche innere Struktur der Documentary Engine.

Verzeichnisstruktur

Die in ESS-0001-CONTRACTS Chapter 2 festgelegte Struktur ist verbindlich.

```
src/platform/Documentary/  
  
Engine/           Ausführungssteuerung und Orchestrierung  
Discovery/        Repository- und Code-Erkennung  
Knowledge/        Wissensaufbau  
Documentation/    Dokumenterzeugung  
Architecture/     Architekturanalyse  
Migration/        Migrationsanalyse  
Versioning/       Versionsbewertung  
Events/           Event-Erzeugung und -Konsum  
Registry/         Registrierung erzeugter Artefakte  
Generators/       Generatoren je Dokumenttyp  
Validators/       Validatoren der Engine  
Templates/        Dokumentvorlagen  
Mermaid/          Diagrammerzeugung  
Plugins/          Erweiterungen  
Types/            Typdefinitionen  
Models/           Datenmodelle  
Interfaces/       öffentliche Schnittstellen  
Contracts/        komponentenspezifische Contracts  
Utils/            Hilfsfunktionen
```

Jedes Unterverzeichnis besitzt genau eine Verantwortung gemäß Chapter 3.

Kernkomponenten

Die Documentary Engine besteht aus sieben Kernkomponenten.

DocumentaryEngine

Zentrale Ausführungssteuerung.

Verantwortung

Ablaufsteuerung

Reihenfolge der Analysestufen

Fehlerbehandlung

Ergebniszusammenführung

Event-Erzeugung

DiscoveryService

Repository- und Code-Erkennung.

Verantwortung

Dateierkennung

Klassifizierung

Metadatenerfassung

Technologieerkennung

Duplikaterkennung

KnowledgeBuilder

Aufbau des Enterprise Knowledge Graph.

Verantwortung

Knotenerzeugung

Beziehungsableitung

Ursprungsnachweis

Prüfsummenbildung

ArchitectureAnalyzer

Architekturanalyse.

Verantwortung

Layer-Prüfung
Abhängigkeitsanalyse
Verletzungserkennung
Metrikberechnung

DocumentationGenerator

Dokumenterzeugung.
Verantwortung
Auswahl des Generators
Anwendung der Vorlage
Metadatenanreicherung
Kennzeichnung als generiert

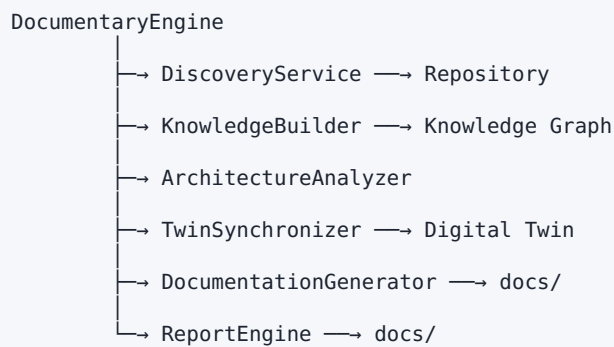
TwinSynchronizer

Digital-Twin-Abgleich.
Verantwortung
Twin-Aufbau
Drift-Erkennung
Snapshot-Erzeugung
Zustandsbestimmung

ReportEngine

Berichtserzeugung.
Verantwortung
Architecture Reports
Release Reports
Impact Analysen
Version Reports
Governance Reports

Komponentenbeziehungen



Die Reihenfolge ist verbindlich und deterministisch.

Keine Komponente überspringt eine vorgelagerte Stufe.

End of Chapter 2

Chapter 3

Services und Interfaces

Enterprise Purpose

Dieses Kapitel definiert die öffentlichen Schnittstellen der Documentary Engine.

Interface Contract

Sämtliche Interfaces folgen den Regeln aus ESS-0001-CONTRACTS Chapter 4.

Verbindlich sind folgende öffentliche Interfaces.

IDocumentaryEngine

<code>execute(trigger)</code>	vollständiger Durchlauf
<code>executeIncremental(scope)</code>	teilweiser Durchlauf
<code>describe()</code>	Selbstbeschreibung

IDiscoveryProvider

<code>scanRepository()</code>	vollständiger Repository-Scan
<code>scanPath(path)</code>	gezielter Scan
<code>classify(file)</code>	Klassifizierung genau einer Datei
<code>detectDuplicates()</code>	Duplikaterkennung

IKnowledgeBuilder

<code>build()</code>	vollständiger Aufbau
<code>update(delta)</code>	inkrementelle Aktualisierung
<code>validate()</code>	Konsistenzprüfung
<code>checksum()</code>	reproduzierbare Prüfsumme

IDocumentationGenerator

<code>supports(type)</code>	Typunterstützung
<code>generate(input)</code>	Artefakterzeugung
<code>version()</code>	Generatorversion

IDigitalTwin

<code>synchronize()</code>	Abgleich mit dem Repository
<code>state()</code>	aktueller Twin-Zustand
<code>drift()</code>	erkannte Abweichungen
<code>snapshot(version)</code>	Zustandssicherung
<code>simulate(plan)</code>	geplanter Zwilling

IReportEngine

<code>generate(reportType)</code>	Berichtserzeugung
<code>list()</code>	verfügbare Berichtstypen

Interface Rules

Sämtliche Methoden sind asynchron gemäß Chapter 4, Async Contracts.

Sämtliche Methoden sind idempotent.

Kein Interface gibt veränderbare interne Zustände zurück.

Kein Interface erlaubt Schreibzugriff auf produktiven Code.

End of Chapter 3

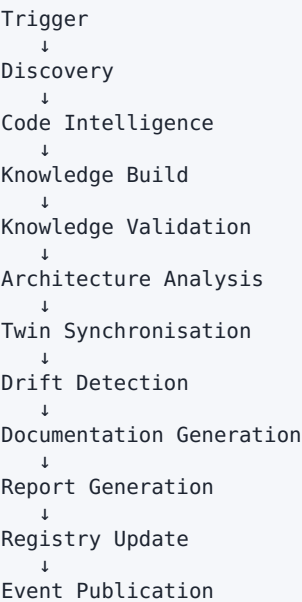
Chapter 4

Workflows und Trigger

Enterprise Purpose

Dieses Kapitel definiert die verbindlichen Abläufe der Documentary Engine.

Hauptworkflow



Keine Stufe darf übersprungen werden.

Bricht eine Stufe ab, werden nachgelagerte Stufen nicht ausgeführt.

Trigger Contract

Die Documentary Engine wird ausschließlich ereignisgesteuert ausgelöst.

Verbindlich gemäß ESS-0001-CONTRACTS Chapter 17, Documentary Trigger Contract:

Auslöser	Umfang
Repository-Änderung	inkrementell
Metadatenänderung	inkrementell

Auslöser	Umfang
Contract-Änderung	vollständig
ESS-Änderung	vollständig
ADR-Änderung	inkrementell
Versionsänderung	vollständig
Migration	vollständig
Plugin-Registrierung	inkrementell
Security Review	inkrementell
Compliance Review	inkrementell

Die Documentary Engine ist niemals optional.

Incremental Execution

Ein inkrementeller Durchlauf verarbeitet ausschließlich den geänderten Umfang und dessen Abhängigkeiten gemäß Knowledge Graph.

Ein inkrementeller Durchlauf erzeugt dieselben Ergebnisse wie ein vollständiger Durchlauf, beschränkt auf den betroffenen Teilgraphen.

Weicht das Ergebnis ab, gilt der inkrementelle Durchlauf als fehlerhaft.

Abort Contract

Ein Durchlauf bricht verbindlich ab bei

fehlender Repository-Lesbarkeit

Schema-Verletzung in Metadaten

Wissenskonflikt ohne Auflösung

nicht reproduzierbarer Prüfsumme

Jeder Abbruch erzeugt ein Fehler-Event und einen Befund.

End of Chapter 4

Chapter 5

Discovery und Registry

Enterprise Purpose

Dieses Kapitel definiert Erkennung und Registrierung.

Discovery Scope

Die Discovery erfasst verbindlich

Repository-Struktur

Dateien und Klassifizierung

Konfigurationsdateien

Plattformmodule

Feature-Domänen

Datenbankartefakte

Dokumentation

AI-Artefakte

Legacy-Bestand

Ausnahmen gemäß Exception Registry

Classification Contract

Jede Datei erhält exakt eine Hauptklassifizierung.

Eine Doppelklassifizierung ist unzulässig.

Nicht klassifizierbare Dateien erhalten `Unknown` und erzeugen einen Befund.

Registry Contract

Die Documentary Engine registriert erzeugte und erkannte Artefakte in der Enterprise Registry gemäß ESS-0001-CONTRACTS Chapter 7.

Verbindlich registriert werden

Komponenten

Interfaces

Events

Plugins

Generatoren

Validatoren

Dokumente

Legacy-Bestand

Die Documentary Engine ist erzeugende Instanz der Registry.

Sie verändert niemals Registry-Einträge, die von anderen Instanzen stammen.

Registry Boundaries

Die Registry beschreibt Existenz, Identität und Status.

Der Knowledge Graph beschreibt Bedeutung, Beziehungen und Historie.

Die Trennung ist in ESS-0001-CONTRACTS Chapter 15, Registry Relationship, verbindlich festgelegt und wird hier nicht erweitert.

End of Chapter 5

Chapter 6

Events

Enterprise Purpose

Dieses Kapitel definiert die von der Documentary Engine erzeugten und konsumierten Events.

Sämtliche Namen folgen dem Naming Contract aus ESS-0001-CONTRACTS Chapter 8.

Erzeugte Events

DocumentaryExecutionStartedEvent

DocumentaryExecutionCompletedEvent

DocumentaryExecutionFailedEvent

RepositoryScannedEvent

CodeAnalyzedEvent

KnowledgeBuildStartedEvent

KnowledgeCreatedEvent

KnowledgeUpdatedEvent

KnowledgeValidatedEvent

KnowledgeConflictDetectedEvent

ArchitectureScannedEvent

ArchitectureValidatedEvent

TwinSynchronizedEvent

TwinDriftDetectedEvent

TwinSnapshotCreatedEvent

DocumentationGeneratedEvent

DocumentationValidatedEvent

DiagramGeneratedEvent

ReportGeneratedEvent

ComponentRegisteredEvent

ContractViolationEvent

Konsumierte Events

ImplementationCompletedEvent

VersionChangedEvent

MigrationCompletedEvent

PluginRegisteredEvent

ExceptionRegisteredEvent

SecurityScanCompletedEvent

ComplianceValidatedEvent

Correlation

Die Documentary Engine übernimmt die Correlation ID des auslösenden Events unverändert.

Eine eigene Correlation ID entsteht ausschließlich bei zeitgesteuerten Vollprüfungen.

End of Chapter 6

Chapter 7

Digital Twin

Enterprise Purpose

Dieses Kapitel definiert die technische Umsetzung des Digital Twin.

Die verbindlichen Regeln stehen in ESS-0001-CONTRACTS Chapter 18 und werden hier nicht wiederholt, sondern ausschließlich technisch konkretisiert.

Twin Aufbau

```
Repository
  ↓
Discovery
  ↓
Knowledge Graph
  ↓
TwinSynchronizer
  ↓
.ai/knowledge/twin/
```

Der Abgleich erfolgt ausschließlich in dieser Richtung.

Der Twin verändert niemals Repository oder Metadaten.

Twin Artefakte

.ai/knowledge/twin/	
current.json	aktueller Zustand
planned.json	geplanter Zustand
drift.json	erkannte Abweichungen
history/	Snapshots je Version

Drift Detection

Drift wird ausschließlich durch Vergleich von Prüfsummen und Strukturen erkannt.

Verglichen werden

Quellcode gegen Twin-Modell

Metadaten gegen Twin-Modell

Registry gegen Twin-Modell

Knowledge Graph gegen Twin-Modell

Version gegen Twin-Version

Die zulässige Drift beträgt null.

Twin Zustände

Synchronized

Drifted

Stale

Incomplete

Unknown

Ausschließlich `Synchronized` ist produktionsfreigabefähig.

End of Chapter 7

Chapter 8

Integration

Enterprise Purpose

Dieses Kapitel definiert die Einbettung der Documentary Engine in die Plattform.

Stellung in der Wertschöpfungskette

Die Documentary Engine bildet Stufe 3 der in ESS-0001-CONTRACTS Chapter 17 definierten AI-Wertschöpfungskette.

Google AI Studio → Claude Code → Documentary Engine → Supervisor → Platform Director → Version Manager → Release → Production

Eingang Implementierungsartefakte, Repository-Zustand, Events der Vorstufen

Ausgang Knowledge Update, Dokumentation, Architecture Report, Validation Report, Twin Update

Supervisor Integration

Die Documentary Engine liefert dem Supervisor gemäß ESS-0002

Validierungsergebnisse

Twin-Zustand

Befunde

Dokumentationsstatus

Sie empfängt vom Supervisor keine Weisungen. Der Supervisor bewertet ausschließlich.

Platform Director Integration

Die Documentary Engine liefert dem Platform Director gemäß ESS-0003 die Entscheidungsgrundlagen.

Sie trifft selbst keine Entscheidungen.

Version Manager Integration

Die Documentary Engine liefert die Grundlage der Versionsbewertung:

geänderte Komponenten

geänderte Interfaces

geänderte Events

Breaking Changes

Impact-Umfang

Die Versionsentscheidung selbst trifft der Version Manager.

Legacy Integration

Bestehende produktionsreife Komponenten werden gemäß ESS-0001-CONTRACTS Chapter 14 über Adapter angebunden, niemals neu entwickelt.

Betroffen sind insbesondere

```
server/documentHygiene.ts
```

```
server/documentSanitizer.ts
```

```
server/fileWatcher.ts
```

```
server/decisionEngine.ts
```

Diese Komponenten sind als Legacy zu registrieren und zu kapseln.

Die Zielzuordnung ist in ADR-0011 verbindlich festgelegt.

Enterprise Rules

Keine Dokumentation ohne Knowledge-Referenz.

Keine Ausführung ohne Trigger.

Keine Stufe ohne Event.

Kein Schreibzugriff auf produktiven Code.

Keine Neuentwicklung produktionsreifer Bestandskomponenten.

Keine Entscheidung durch die Documentary Engine.

Success Criteria

Die Documentary Engine gilt als erfolgreich implementiert wenn

- ✓ jede relevante Änderung sie automatisch auslöst
 - ✓ jeder Durchlauf reproduzierbar identische Ergebnisse erzeugt
 - ✓ der Knowledge Graph vollständig aufgebaut wird
 - ✓ der Digital Twin nach jedem Durchlauf synchron ist
 - ✓ sämtliche Dokumentation generiert statt gepflegt wird
 - ✓ sämtliche erzeugten Artefakte registriert sind
 - ✓ Supervisor und Platform Director jederzeit versorgt sind
-

Integration

Dieses Dokument bildet die Grundlage für

ESS-0011 — Enterprise Traceability

ESS-0011-CONTRACTS — Enterprise Traceability Matrix Contracts

Enterprise Final Summary

ESS-0010 Status

Document ID ESS-0010

Titel CAPITAL-AI Documentary Engine

Status Enterprise Specification

Version 1.0.0

Lifecycle Status Approved Enterprise Specification

Governance Statement

ESS-0010 ist die verbindliche technische Spezifikation der Documentary Engine innerhalb des CAPITAL-AI Core.

Sie baut vollständig auf ESS-0001 und ESS-0001-CONTRACTS auf.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste technische Spezifikation der Documentary Engine

Approval

Document Status

APPROVED

Enterprise Specification

CAPITAL-AI Core Architecture

Version 1.0.0

End of Document

ESS-0010

CAPITAL-AI Documentary Engine

Enterprise Specification

Version 1.0.0

ESS-0011-CONTRACTS

Quelle: `.ai/skills/ESS-0011-Contracts.md`

ESS-0011-CONTRACTS

Enterprise Traceability Matrix Contracts

Version

1.0.0

Status

Enterprise Specification

Dokumentklassifizierung

Dieses Dokument definiert **ausschließlich** Enterprise Traceability Matrix Contracts.

Es ist kein globaler Enterprise Standard.

Nicht zulässig sind in diesem Dokument

globale Repository Contracts

globale Naming Contracts

globale Layer Contracts

allgemeine Governance

Diese verbleiben ausnahmslos in ESS-0001-CONTRACTS.

Bei Konflikt zwischen diesem Dokument und ESS-0001-CONTRACTS gilt ausnahmslos ESS-0001-CONTRACTS.

Cross Reference

Dieses Dokument besitzt kein YAML-Frontmatter. Die Referenzen werden als Markdown-Abschnitt geführt, analog zu ESS-0001-CONTRACTS.

Depends On

ESS-0001-CONTRACTS

ESS-0011

Related ESS

ESS-0001 — Documentary & Code Intelligence Architect

ESS-0010 — Documentary Engine

ESS-0011 — Enterprise Traceability

Related ADR

ADR-0010 — Enterprise Standard Extension

ADR-0013 — ESS Documentation Responsibility Consolidation

Related Components

`src/platform/Registry`

`src/platform/Knowledge`

`src/platform/Architecture`

`src/platform/Quality`

Related Skills

`.ai/skills/ESS-0011-Enterprise-Traceability.md`

`.ai/skills/ESS-0010-Documentary-Engine.md`

Chapter 1

Traceability Link Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindliche Struktur einer Traceability-Verknüpfung.

Link Contract

Jede Verknüpfung besitzt verbindlich

`linkId`

`sourceId`

`targetId`

`linkType`

`origin`

confidence

createdAt

Fehlt ein Feld, ist die Verknüpfung ungültig und wird verworfen.

Identity Contract

`sourceId` und `targetId` verwenden ausschließlich die Knoten-Identität aus ESS-0001-CONTRACTS Chapter 15.

```
<type>:<domain>/<name>
```

Beispiele

```
ess:governance/ESS-0001-CONTRACTS#Chapter-16  
adr:governance/ADR-0011  
component:platform/Documentary  
interface:platform/IKnowledgeBuilder  
event:platform/TwinDriftDetectedEvent  
exception:governance/EXC-0001
```

Die ID bleibt über sämtliche Versionen stabil.

Link Type Contract

Zulässig sind ausschließlich

SPECIFIES

DECIDES

IMPLEMENTS

EMITS

VALIDATES

COVERS

EXEMPTS

SUPERSEDES

Weitere Verknüpfungsarten erfordern eine ADR.

Direction Contract

Jede Verknüpfung ist gerichtet.

Gegenrichtungen werden berechnet, niemals gespeichert.

Eine Verknüpfung, die ausschließlich in einer Richtung auflösbar ist, erzeugt einen Befund.

Origin Contract

Jede Verknüpfung besitzt einen nachweisbaren Ursprung.

Zulässig sind

KnowledgeGraph

Registry

DigitalTwin

EssRegistry

ExceptionRegistry

AdrDocument

ValidationResult

SourceCode

Eine Verknüpfung ohne Ursprung wird niemals aufgenommen.

Confidence Contract

Verified

aus expliziter Referenz oder Metadaten abgeleitet

Derived

aus mehreren Quellen erschlossen

Assumed

nicht zulässig

Verknüpfungen mit dem Wert Assumed dürfen niemals in die Matrix aufgenommen werden.

End of Chapter 1

Chapter 2

Coverage Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindliche Messung der Abdeckung.

Coverage Definition

Abdeckung ist der Anteil der Artefakte einer Achse, die mindestens eine gültige Verknüpfung in der geforderten Richtung besitzen.

Die Berechnung ist deterministisch.

Abdeckung wird niemals geschätzt.

Coverage Axes

Verbindlich gemessen werden

Achse	Messgröße
Specification Coverage	Komponenten mit spezifizierender ESS-Referenz
Decision Coverage	Strukturentscheidungen mit ADR
Interface Coverage	Interfaces mit Implementierung
Event Coverage	Events mit mindestens einem Consumer
Test Coverage	Komponenten mit validierendem Test
Exception Coverage	Ausnahmen mit Zielzustand und ADR

Coverage Thresholds

Konformitätsstufe	geforderte Mindestabdeckung
Level 1 — Structural	Specification Coverage ≥ 80
Level 2 — Operational	zusätzlich Event Coverage ≥ 90 , Interface Coverage ≥ 90

Konformitätsstufe	geforderte Mindestabdeckung
Level 3 — Enterprise	sämtliche Achsen ≥ 95 , Exception Coverage = 100

Die Konformitätsstufen entsprechen ESS-0001-CONTRACTS Chapter 20 und werden hier nicht neu definiert, sondern ausschließlich um Traceability-Schwellwerte ergänzt.

Measurement Contract

Abdeckung wird ausschließlich aus der Matrix berechnet.

Sie wird niemals manuell gesetzt.

Sie wird niemals aus Dokumentation abgeleitet.

End of Chapter 2

Chapter 3

Orphan Contracts

Enterprise Purpose

Dieses Kapitel definiert, welche nicht verknüpften Artefakte als Befund gelten.

Orphan Definition

Ein Orphan ist ein Artefakt ohne gültige Verknüpfung in der geforderten Richtung.

Orphan Classes

Klasse	Bedeutung	Schweregrad
UnimplementedRule	ESS-Regel ohne umsetzende Komponente	High
UnspecifiedComponent	Komponente ohne spezifizierende Regel	High
UndecidedStructure	Strukturabweichung ohne ADR	Critical
UnconsumedEvent	Event ohne Consumer	Medium

Klasse	Bedeutung	Schweregrad
UnimplementedInterface	Interface ohne Implementierung	Medium
UntestedComponent	Komponente ohne Test	High
UnjustifiedException	Ausnahme ohne ADR oder Zielzustand	Critical
DanglingAdr	ADR ohne betroffene Komponente	Low

Orphan Handling

Jeder Orphan erzeugt verbindlich einen Befund gemäß ESS-0002.

Orphans der Stufe Critical blockieren Produktionsfreigaben.

Ein Orphan wird niemals stillschweigend geschlossen.

End of Chapter 3

Chapter 4

ETM Event Contracts

Enterprise Purpose

Dieses Kapitel definiert die Events der ETM.

Sämtliche Namen folgen dem Naming Contract aus ESS-0001-CONTRACTS Chapter 8.

Erzeugte Events

TraceabilityBuildStartedEvent

TraceabilityBuildCompletedEvent

TraceabilityBuildFailedEvent

CoverageCalculatedEvent

CoverageThresholdViolatedEvent

OrphanDetectedEvent

OrphanResolvedEvent

TraceabilityReportGeneratedEvent

Konsumierte Events

KnowledgeUpdatedEvent

TwinSynchronizedEvent

ComponentRegisteredEvent

ExceptionRegisteredEvent

ReleasePreparedEvent

Correlation Contract

Die ETM übernimmt die Correlation ID des auslösenden Events unverändert.

End of Chapter 4

Chapter 5

ETM Validation Contracts

Enterprise Purpose

Dieses Kapitel definiert die Prüfung der Matrix selbst.

Matrix Validation

Vor jeder Veröffentlichung der Matrix wird geprüft

- ✓ sämtliche Verknüpfungen besitzen vollständige Pflichtfelder
- ✓ sämtliche IDs sind auflösbar
- ✓ keine Verknüpfung auf nicht existierende Artefakte
- ✓ keine Assumed-Verknüpfungen

- ✓ keine doppelten Verknüpfungen
 - ✓ sämtliche Verknüpfungen bidirektional auflösbar
 - ✓ Prüfsumme reproduzierbar
 - ✓ Abdeckungsgrade berechnet
 - ✓ Orphans klassifiziert
-

Determinism Contract

Bei identischem Eingangszustand erzeugt die ETM eine byteidentische Matrix.

Zeitstempel sind von der Prüfsummenbildung ausgenommen.

Weicht die Prüfsumme bei unverändertem Eingangszustand ab, gilt der Durchlauf als fehlerhaft.

Validator Contract

Die ETM stellt einen Validator gemäß ESS-0001-CONTRACTS Chapter 12 bereit.

TraceabilityValidator

Er prüft ausschließlich die Matrix, niemals deren Quellen.

Enterprise Rules

Keine Verknüpfung ohne Ursprung.

Keine Verknüpfung ohne Richtung.

Keine Annahme in der Matrix.

Keine manuelle Änderung der Matrix.

Keine Abdeckungsaussage ohne Messung.

Kein Orphan ohne Befund.

Keine Produktionsfreigabe bei Critical-Orphans.

Dieses Dokument definiert niemals globale Enterprise-Regeln.

Success Criteria

Diese Contracts gelten als erfüllt wenn

- ✓ sämtliche Verknüpfungen dem Link Contract entsprechen
 - ✓ sämtliche Abdeckungsgrade deterministisch berechnet werden
 - ✓ sämtliche Orphans klassifiziert und gemeldet werden
 - ✓ die Matrix reproduzierbar erzeugt werden kann
 - ✓ der TraceabilityValidator ausführbar ist
 - ✓ keine globale Regel in diesem Dokument definiert wurde
-

Integration

Dieses Dokument ergänzt

ESS-0011 — Enterprise Traceability

↓

ESS-0001-CONTRACTS Chapter 12 — Validation & Quality Contracts

↓

ESS-0001-CONTRACTS Chapter 20 — Official Enterprise Standard

Governance Statement

ESS-0011-CONTRACTS ist die verbindliche Vertragsgrundlage der Enterprise Traceability Matrix.

Es besitzt keine globale Wirkung.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Fassung der ETM Contracts

Approval

Document Status

APPROVED

Enterprise Specification

CAPITAL-AI Core Architecture

Version 1.0.0

End of Document

ESS-0011-CONTRACTS

CAPITAL-AI Enterprise Traceability Matrix Contracts

Version 1.0.0

ESS 0011 Enterprise Traceability

Quelle: .ai/skills/ESS-0011-Enterprise-Traceability.md

skill: id: ESS-0011 name: Enterprise Traceability version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority: High

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

classification: type: Technical Specification role: Spezifikation der Enterprise Traceability Matrix contractAuthority: ESS-0001-CONTRACTS ownContracts: ESS-0011-CONTRACTS note: > Dieses Dokument beschreibt ausschließlich ETM-Architektur, -Komponenten, -Prozesse, -Reports, -Integration und -Workflows. Allgemeine Enterprise-Regeln verbleiben in ESS-0001-CONTRACTS.

authority:

controls: - Enterprise Traceability Matrix - Traceability Reports - Coverage Analysis - Orphan Detection

collaborates: - Documentary Engine - Supervisor - Platform Director - Version Manager - Quality Center - Compliance Center

cannot_modify: - Enterprise Specifications - Architecture Decision Records - Knowledge Graph - Digital Twin - Registry

crossReference: dependsOn: - ESS-0001-CONTRACTS relatedEss: - ESS-0001 - ESS-0002 - ESS-0003 - ESS-0010 - ESS-0011-CONTRACTS relatedAdr: - ADR-0010 - ADR-0013 relatedComponents: - src/platform/Registry - src/platform/Knowledge - src/platform/Architecture - src/platform/Documentary - src/platform/Quality relatedSkills: - .ai/skills/ESS-0001-Contracts.md - .ai/skills/ESS-0010-Documentary-Engine.md

created: 2026-07-31

Enterprise Traceability

Enterprise Purpose

Dieses Dokument spezifiziert die Enterprise Traceability Matrix (ETM) des CAPITAL-AI Core.

Die ETM beantwortet eine einzige Frage lückenlos und maschinell:

Welche Anforderung, welche Entscheidung, welche Regel führte zu welchem Artefakt — und welches Artefakt erfüllt welche Regel?

Die ETM erzeugt keine Regeln.

Sie erzeugt keine Dokumentation.

Sie verknüpft ausschließlich bereits vorhandene, validierte Enterprise-Artefakte.

Abgrenzung

Dokument	Verantwortung
ESS-0001-CONTRACTS	globale Enterprise Contracts
ESS-0010	technische Spezifikation der Documentary Engine
ESS-0011	ETM-Architektur, -Komponenten, -Prozesse, -Reports, -Integration, -Workflows
ESS-0011-CONTRACTS	verbindliche ETM-Contracts

Dieses Dokument enthält keine allgemeinen Enterprise-Regeln.

Für Repository-, Naming-, Layer- und Governance-Regeln gilt ausschließlich ESS-0001-CONTRACTS.

Enterprise Principle

Nachvollziehbarkeit entsteht nicht durch Dokumentation.

Nachvollziehbarkeit entsteht durch nachweisbare Verknüpfung.

Jede Verknüpfung besitzt einen Ursprung.

Keine Verknüpfung wird behauptet.

End of Chapter 1

Chapter 2

ETM Architektur

Enterprise Purpose

Dieses Kapitel definiert den Aufbau der Traceability Matrix.

Traceability-Achsen

Die ETM verknüpft sieben Artefaktklassen.

ESS	Enterprise-Spezifikationen und Contracts
ADR	Architekturentscheidungen
Exception	registrierte Ausnahmen
Component	Plattformkomponenten
Interface	öffentliche Schnittstellen
Event	Enterprise Events
Test	Validatoren und Tests

Traceability-Richtungen

Die ETM ist bidirektional auswertbar.

Vorwärts — Regel zu Umsetzung

ESS-Kapitel → ADR → Component → Interface → Event → Test

Rückwärts — Umsetzung zu Regel

Test → Event → Interface → Component → ADR → ESS-Kapitel

Eine Verknüpfung, die ausschließlich in einer Richtung existiert, ist ein Befund.

Datenquellen

Die ETM erzeugt keine eigenen Daten.

Sie liest ausschließlich

Quelle	Beitrag
Knowledge Graph	Knoten und Beziehungen
Enterprise Registry	Existenz und Status
Digital Twin	tatsächlicher Zustand
<code>.ai/registry/ess-registry.json</code>	ESS-Nummernraum
<code>.ai/registry/exception-registry.json</code>	registrierte Ausnahmen
<code>docs/adr/</code>	Architekturentscheidungen
Validierungsergebnisse	Testabdeckung

Fehlt eine Quelle, ist die ETM unvollständig und meldet dies als Befund.

Matrixmodell

Jeder Matriceintrag verknüpft genau zwei Artefakte.

sourceId	Ursprungsartefakt
targetId	Zielartefakt
linkType	Art der Verknüpfung
origin	Herkunft der Verknüpfung
confidence	Verified oder Derived
createdAt	Zeitpunkt

Die Identität folgt der Knoten-Identität aus ESS-0001-CONTRACTS Chapter 15.

Verknüpfungsarten

SPECIFIES	ESS-Kapitel spezifiziert Komponente
DECIDES	ADR entscheidet über Komponente
IMPLEMENTS	Komponente implementiert Interface
EMITS	Komponente erzeugt Event
VALIDATES	Test validiert Regel
COVERS	Test deckt Komponente ab
EXEMPTS	Ausnahme befreit Pfad von Regel
SUPERSEDES	ADR ersetzt ADR

Weitere Verknüpfungsarten erfordern eine ADR.

End of Chapter 2

Chapter 3

ETM Komponenten

Enterprise Purpose

Dieses Kapitel definiert die technischen Bestandteile der ETM.

TraceabilityBuilder

Aufbau der Matrix aus den Datenquellen.

Verantwortung

Quellen einlesen

Verknüpfungen ableiten

Richtungsprüfung

Prüfsummenbildung

CoverageAnalyzer

Abdeckungsanalyse.

Verantwortung

Abdeckungsgrad je Achse

Lückenerkennung

Schwellwertprüfung

OrphanDetector

Erkennung nicht verknüpfter Artefakte.

Verantwortung

Regeln ohne Umsetzung

Umsetzungen ohne Regel

Events ohne Consumer

Komponenten ohne Test

ADR ohne betroffene Komponente

TraceabilityReporter

Berichtserzeugung.

Verantwortung

Coverage Report

Orphan Report

Impact Trace

Compliance Trace

Ablage

```
.ai/knowledge/traceability/  
matrix.json      vollständige Matrix  
coverage.json    Abdeckungskennzahlen  
orphans.json     nicht verknüpfte Artefakte
```

Diese Dateien werden ausschließlich generiert und niemals manuell bearbeitet.

End of Chapter 3

Chapter 4

ETM Prozesse und Workflows

Enterprise Purpose

Dieses Kapitel definiert die Abläufe der ETM.

Hauptworkflow

```
Trigger  
↓  
Quellen einlesen  
↓  
Verknüpfungen ableiten  
↓  
Richtungsprüfung  
↓  
Coverage-Analyse  
↓  
Orphan-Erkennung  
↓  
Matrix schreiben  
↓  
Reports erzeugen  
↓  
Events veröffentlichen
```

Die ETM läuft ausschließlich **nach** der Documentary Engine, da sie deren Ergebnisse verwendet.

Trigger

Die ETM wird verbindlich ausgelöst durch

Event	Umfang
KnowledgeUpdatedEvent	inkrementell
TwinSynchronizedEvent	inkrementell
ComponentRegisteredEvent	inkrementell
ADR-Änderung	vollständig
ESS-Änderung	vollständig
ExceptionRegisteredEvent	inkrementell
ReleasePreparedEvent	vollständig

Vor jeder Produktionsfreigabe erfolgt ein vollständiger Durchlauf.

Determinismus

Bei identischem Eingangszustand erzeugt die ETM

identische Verknüpfungen

identische Kennzahlen

identische Befunde

identische Prüfsummen

Zeitstempel sind ausgenommen.

End of Chapter 4

Chapter 5

ETM Reports

Enterprise Purpose

Dieses Kapitel definiert die von der ETM erzeugten Berichte.

Coverage Report

Abdeckungsgrad je Achse.

Enthält

Anteil spezifizierter Komponenten

Anteil durch ADR gedeckter Strukturentscheidungen

Anteil getesteter Komponenten

Anteil dokumentierter Events

Anteil registrierter Ausnahmen mit Zielzustand

Orphan Report

Nicht verknüpfte Artefakte.

Enthält

Regeln ohne Umsetzung

Komponenten ohne spezifizierende Regel

Events ohne Consumer

Interfaces ohne Implementierung

ADR ohne betroffene Komponente

Ausnahmen ohne ADR

Impact Trace

Auswirkungspfad einer geplanten Änderung.

Beantwortet

Welche Regeln sind betroffen?

Welche Entscheidungen sind betroffen?

Welche Komponenten sind betroffen?

Welche Tests müssen erneut laufen?

Ergänzt die Impact Analyse aus ESS-0001 Chapter 7 um die Regelachse.

Compliance Trace

Nachweisführung für Prüfungen.

Beantwortet

Welche Regel fordert diese Maßnahme?

Welches Artefakt erfüllt sie?

Welcher Test belegt die Erfüllung?

Wann wurde zuletzt geprüft?

Berichtsablage

docs/quality/

Die Erzeugung erfolgt ausschließlich durch die Documentary Engine gemäß ESS-0010.

Die ETM liefert die Daten, nicht das Dokument.

End of Chapter 5

Chapter 6

ETM Integration

Enterprise Purpose

Dieses Kapitel definiert die Einbettung der ETM.

Documentary Engine

Die ETM verwendet ausschließlich die Ergebnisse der Documentary Engine.

Sie liest Knowledge Graph, Registry und Digital Twin.

Sie schreibt niemals in diese Quellen.

Supervisor

Die ETM liefert dem Supervisor gemäß ESS-0002

Abdeckungskennzahlen

Orphan-Befunde

Regelverstöße ohne Umsetzung

Der Supervisor bewertet, die ETM stellt fest.

Platform Director

Die ETM liefert dem Platform Director gemäß ESS-0003

Nachweis der Regelabdeckung vor Freigaben

Compliance Trace

Impact Trace

Version Manager

Die ETM liefert die Regelachse der Versionsbewertung: welche Regeln eine Änderung betrifft.

Die Versionsentscheidung trifft der Version Manager.

Compliance Center

Die ETM liefert die Nachweisführung gemäß ESS-0001-CONTRACTS Chapter 11, Compliance Evidence.

Sie erzeugt selbst keine Compliance-Aussage.

Enterprise Rules

Die ETM erzeugt keine Regeln.

Die ETM erzeugt keine Dokumentation.

Die ETM verändert keine Quelle.

Keine Verknüpfung ohne Ursprung.

Keine Verknüpfung mit dem Vertrauenswert Assumed.

Keine Produktionsfreigabe ohne vollständigen ETM-Durchlauf.

Success Criteria

Die ETM gilt als erfolgreich implementiert wenn

- ✓ sämtliche Artefaktklassen erfasst sind
 - ✓ sämtliche Verknüpfungen bidirektional auflösbar sind
 - ✓ Abdeckungsgrade je Achse berechnet werden
 - ✓ nicht verknüpfte Artefakte automatisch erkannt werden
 - ✓ jede Regel bis zu ihrem Test rückverfolgbar ist
 - ✓ jede Komponente bis zu ihrer Regel rückverfolgbar ist
 - ✓ die Matrix reproduzierbar aufgebaut werden kann
-

Integration

Dieses Dokument bildet die Grundlage für

ESS-0011-CONTRACTS — Enterprise Traceability Matrix Contracts

Enterprise Final Summary

ESS-0011 Status

Document ID ESS-0011

Titel CAPITAL-AI Enterprise Traceability

Status Enterprise Specification

Version 1.0.0

Lifecycle Status Approved Enterprise Specification

Governance Statement

ESS-0011 ist die verbindliche Spezifikation der Enterprise Traceability Matrix innerhalb des CAPITAL-AI Core.

Sie baut vollständig auf ESS-0001-CONTRACTS und ESS-0010 auf.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Spezifikation der Enterprise Traceability Matrix

Approval

Document Status

APPROVED

Enterprise Specification

CAPITAL-AI Core Architecture

Version 1.0.0

End of Document

ESS-0011

CAPITAL-AI Enterprise Traceability

Enterprise Specification

Version 1.0.0

ESS-0012-CONTRACTS

Quelle: `.ai/skills/ESS-0012-Contracts.md`

ESS-0012-CONTRACTS

Documentation Governance Contracts

Version

1.0.0

Status

Enterprise Specification

Dokumentklassifizierung

Dieses Dokument definiert **ausschließlich** Documentation Governance Contracts.

Es ist kein globaler Enterprise Standard.

Nicht zulässig sind in diesem Dokument

globale Repository Contracts

globale Naming Contracts

globale Layer Contracts

allgemeine Governance

Validator-Basisregeln, Severity-Definitionen und Quality Gates

Diese verbleiben ausnahmslos in ESS-0001-CONTRACTS.

Bei Konflikt zwischen diesem Dokument und ESS-0001-CONTRACTS gilt ausnahmslos ESS-0001-CONTRACTS.

Cross Reference

Depends On

ESS-0001-CONTRACTS

ESS-0012

Related ESS

ESS-0001 — Documentary & Code Intelligence Architect

ESS-0010 — Documentary Engine

ESS-0011 — Enterprise Traceability

ESS-0011-CONTRACTS — Enterprise Traceability Matrix Contracts

ESS-0012 — Documentation Governance

Related ADR

ADR-0010 — Enterprise Standard Extension

ADR-0013 — ESS Documentation Responsibility Consolidation

ADR-0014 — Documentation Governance Validator

Related Components

src/platform/Documentary/Governance

src/platform/Documentary

src/platform/Knowledge

src/platform/Registry

src/platform/Quality

Related Skills

.ai/skills/ESS-0012-Documentation-Governance.md

.ai/skills/Documentation-Governance-Validator.md

Chapter 1

Rule Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindliche Struktur einer Governance-Regel.

Rule Contract

Jede Regel besitzt verbindlich

ruleId

name

area

description

severity

rationale

essReference

evidenceType

version

Fehlt ein Feld, ist die Regel ungültig und wird nicht ausgeführt.

Rule Identity

Die Regel-ID ist plattformweit eindeutig und stabil.

Format

```
GOV - <AREA> - <NNN>
```

Zulässige Bereichskürzel

```
ESS   ADR   REPO   CONTRACT   DOC   TRACE   KG   TWIN   VER
```

Beispiele

```
GOV - ESS - 001  
GOV - ADR - 004  
GOV - REPO - 002  
GOV - TWIN - 003
```

Die ID bleibt über sämtliche Versionen unverändert.

Eine zurückgezogene Regel wird als **Revoked** gekennzeichnet, ihre ID wird niemals wiederverwendet.

Severity

Die Severity-Stufen aus ESS-0001-CONTRACTS Chapter 12 gelten unverändert:

```
Critical   High   Medium   Low   Information
```

Dieses Dokument definiert keine eigenen Stufen, sondern ordnet ausschließlich zu.

Evidence Contract

Jeder Befund besitzt verbindlich einen Nachweis.

Zulässige Nachweistypen

Typ	Bedeutung
FileReference	Pfad und Zeile
RegistryEntry	Eintrag aus einer Registry
KnowledgeNode	Knoten oder Beziehung
TwinDelta	Abweichung zwischen Repository und Twin
MatrixLink	fehlende oder widersprüchliche Verknüpfung
VersionValue	konkrete Versionsangabe

Ein Befund ohne Nachweis wird verworfen.

Eine Vermutung ist niemals ein Befund.

End of Chapter 1

Chapter 2

Regelwerk

Enterprise Purpose

Dieses Kapitel definiert das verbindliche Mindestregelwerk.

Erweiterungen erfordern keine ADR, solange sie keine bestehende Regel ändern.

2.1 ESS-Regeln

ID	Prüfung	Severity
GOV-ESS-001	ESS-Nummer nicht in der ESS Registry geführt	Critical
GOV-ESS-002	ESS-Dokument ohne Dokumentklasse	High
GOV-ESS-003	ESS-Dokument ohne vollständige Cross-References	High

ID	Prüfung	Severity
GOV-ESS-004	identische Regel in mehreren ESS-Dokumenten	High
GOV-ESS-005	widersprüchliche Regeln in zwei ESS-Dokumenten ohne Vorrangregel	Critical
GOV-ESS-006	Komponente ohne ESS-Referenz	Medium
GOV-ESS-007	ESS-Referenz auf nicht existierendes Dokument	Critical
GOV-ESS-008	Lücke im ESS-Nummernraum	Medium

2.2 ADR-Regeln

ID	Prüfung	Severity
GOV-ADR-001	Strukturänderung ohne ADR	Critical
GOV-ADR-002	Breaking Change ohne ADR	Critical
GOV-ADR-003	ADR-Nummer ohne Dokument	High
GOV-ADR-004	zwei ADRs mit widersprüchlicher Entscheidung ohne SUPERSEDED	Critical
GOV-ADR-005	ADR ohne Implementation-Status	Medium
GOV-ADR-006	ADR mit Status COMPLETE außerhalb von resolved/	Low
GOV-ADR-007	ADR ohne betroffene Komponente	Low
GOV-ADR-008	ADR-Registrierung unvollständig gegenüber Dateibestand	High

2.3 Repository-Regeln

ID	Prüfung	Severity
GOV-REPO-001	Komponente ohne README	High
GOV-REPO-002	Komponente ohne CHANGELOG	High
GOV-REPO-003	Komponente ohne manifest.json	Critical
GOV-REPO-004	Komponente ohne component.yaml	High
GOV-REPO-005	Komponente ohne Ownership oder mit pauschalem Owner	High
GOV-REPO-006	README enthält ausschließlich Platzhaltertext	Medium

ID	Prüfung	Severity
GOV-REPO-007	manifest.json ohne Pflichtfelder aus Chapter 7	High
GOV-REPO-008	leere Beschreibung im Manifest	Medium
GOV-REPO-009	nicht registrierte Strukturabweichung	Critical

2.4 Contract-Regeln

ID	Prüfung	Severity
GOV-CONTRACT-001	identischer Contract in mehreren Dokumenten	High
GOV-CONTRACT-002	widersprüchliche Contracts ohne Vorrangregel	Critical
GOV-CONTRACT-003	Contract ohne definierten Geltungsbereich	High
GOV-CONTRACT-004	Contract von keiner Komponente referenziert	Medium
GOV-CONTRACT-005	Contract mit globaler Wirkung außerhalb ESS-0001-CONTRACTS	Critical
GOV-CONTRACT-006	Contract ohne Validator	Medium

2.5 Dokumentationsregeln

ID	Prüfung	Severity
GOV-DOC-001	Dokument ohne Version	High
GOV-DOC-002	Dokument ohne ESS- oder ADR-Referenz	Medium
GOV-DOC-003	Dokument älter als die zuletzt geänderte referenzierte Komponente	Medium
GOV-DOC-004	Dokument entspricht nicht der Struktur seiner Dokumentklasse	Medium
GOV-DOC-005	Dokumentation außerhalb docs/ ohne registrierte Ausnahme	High
GOV-DOC-006	generiertes Dokument ohne Generator-Kennzeichnung	Medium
GOV-DOC-007	Dokument mit unaufgelöstem Verweis	Low

2.6 Traceability-Regeln

ID	Prüfung	Severity
GOV-TRACE-001	Traceability Matrix nicht vorhanden	High
GOV-TRACE-002	Kettenglied ohne Verknüpfung	High
GOV-TRACE-003	Komponente ohne Test in der Matrix	High
GOV-TRACE-004	Release ohne Verknüpfung zur Version	Medium

Coverage-Schwellwerte und Orphan-Klassen stehen ausschließlich in ESS-0011-CONTRACTS und werden hier nicht wiederholt.

2.7 Knowledge-Graph-Regeln

ID	Prüfung	Severity
GOV-KG-001	Knowledge Graph nicht vorhanden	Critical
GOV-KG-002	verwaister Knoten ohne Beziehung	Medium
GOV-KG-003	doppelte Beziehung zwischen identischem Knotenpaar	Low
GOV-KG-004	Beziehung auf nicht existierenden Knoten	Critical
GOV-KG-005	inkonsistente Gegenrichtung	High

2.8 Digital-Twin-Regeln

ID	Prüfung	Severity
GOV-TWIN-001	Digital Twin nicht vorhanden	Critical
GOV-TWIN-002	Komponente im Repository, nicht im Twin	High
GOV-TWIN-003	Komponente im Twin, nicht im Repository	High
GOV-TWIN-004	Twin-Zustand ungleich Synchronized	Critical
GOV-TWIN-005	Twin älter als die letzte Repository-Änderung	High

Twin-Zustände und Drift-Toleranz stehen in ESS-0001-CONTRACTS Chapter 18.

2.9 Versionsregeln

ID	Prüfung	Severity
GOV-VER-001	widersprüchliche Repository-Versionsangaben	Critical
GOV-VER-002	Komponentenversion ohne Change-log-Eintrag	High
GOV-VER-003	Dokumentversion weicht von Komponentenversion ab	Medium
GOV-VER-004	ESS-Version im Dokument weicht von der Registry ab	High
GOV-VER-005	Version ohne Rollback-Artefakt	High

End of Chapter 2

Chapter 3

Scoring Contracts

Enterprise Purpose

Dieses Kapitel definiert die verbindliche Berechnung der Reifekennzahlen.

Berechnungsgrundsatz

Kennzahlen werden ausschließlich aus Befunden berechnet.

Sie werden niemals manuell gesetzt.

Sie werden niemals aus Dokumentation abgeleitet.

Governance Score

$$\text{Governance Score} = 100 - (\text{Critical} \times 10 + \text{High} \times 4 + \text{Medium} \times 1,5 + \text{Low} \times 0,5)$$

Der Wert wird bei 0 abgeschnitten.

Der Wertebereich beträgt 0 bis 100.

Repository Health Score

Mittelwert aus

Metadatenvollständigkeit

Ownership-Abdeckung

Dokumentationsabdeckung

Strukturkonformität

Jede Teilgröße ist der Anteil konformer Komponenten in Prozent.

Documentation Quality Score

Mittelwert aus

Vollständigkeit

Aktualität

Versionierung

Strukturkonformität

Schwellwerte

Kennzahl	Mindestwert für Produktionsfreigabe
Governance Score	80
Repository Health Score	80
Documentation Quality Score	75

Zusätzlich gilt unabhängig von den Kennzahlen: **kein Critical-Befund**.

Ein hoher Score ersetzt niemals die Behebung eines Critical-Befundes.

End of Chapter 3

Chapter 4

Event Contracts

Enterprise Purpose

Dieses Kapitel definiert die Events des Governance Validator.

Sämtliche Namen folgen dem Naming Contract aus ESS-0001-CONTRACTS Chapter 8.

Erzeugte Events

Event	Auslöser
DocumentationValidatedEvent	Durchlauf abgeschlossen
GovernanceViolationDetectedEvent	Befund ab Severity High
DuplicateContractDetectedEvent	GOV-CONTRACT-001 oder GOV-ESS-004
TraceabilityViolationEvent	Befund im Bereich TRACE
DigitalTwinOutOfSyncEvent	Befund im Bereich TWIN
RepositoryHealthUpdatedEvent	Repository Health Score neu berechnet

Konsumierte Events

RepositoryScannedEvent

KnowledgeUpdatedEvent

TwinSynchronizedEvent

ComponentRegisteredEvent

DocumentationGeneratedEvent

VersionChangedEvent

Event Payload

Jedes erzeugte Event führt zusätzlich zu den Pflichtfeldern aus Chapter 8

runId

ruleId (sofern befundbezogen)

severity

evidence

scoreSnapshot

Der Payload enthält niemals Dateiinhalte, sondern ausschließlich Referenzen.

Correlation

Der Validator übernimmt die Correlation ID des auslösenden Events unverändert.

Eine eigene Correlation ID entsteht ausschließlich bei zeitgesteuerten Vollprüfungen.

End of Chapter 4

Chapter 5

Validation

Enterprise Purpose

Dieses Kapitel definiert die Prüfung des Validators selbst.

Selbstprüfung

Vor jeder Veröffentlichung von Befunden wird geprüft

- ✓ sämtliche geladenen Regeln besitzen vollständige Pflichtfelder
 - ✓ sämtliche Regel-IDs sind eindeutig
 - ✓ sämtliche Befunde besitzen einen Nachweis
 - ✓ keine Befunde ohne zugeordnete Regel
 - ✓ sämtliche Kennzahlen berechnet
 - ✓ Determinismus nachgewiesen
 - ✓ kein Schreibzugriff außerhalb des Befundspeichers
-

Determinism Contract

Bei identischem Repository-Zustand erzeugt der Validator eine byteidentische Befundmenge.

Zeitstempel und Laufzeitangaben sind ausgenommen.

Weicht das Ergebnis ab, gilt der Durchlauf als fehlerhaft und die Befunde werden verworfen.

Enterprise Rules

Keine Regel ohne ID und Severity.

Kein Befund ohne Nachweis.

Keine Kennzahl ohne Berechnung.

Keine Vermutung als Befund.

Kein Schreibzugriff auf den Prüfgegenstand.

Keine Produktionsfreigabe bei Critical-Befunden.

Kein Prüfergebnis ohne Ausführung.

Dieses Dokument definiert niemals globale Enterprise-Regeln.

Success Criteria

Diese Contracts gelten als erfüllt wenn

- ✓ sämtliche Regeln dem Rule Contract entsprechen
 - ✓ sämtliche neun Bereiche durch Regeln abgedeckt sind
 - ✓ sämtliche Kennzahlen deterministisch berechnet werden
 - ✓ sämtliche Events dem Event Contract entsprechen
 - ✓ der Validator sich selbst prüft
 - ✓ keine globale Regel in diesem Dokument definiert wurde
-

Integration

Dieses Dokument ergänzt

ESS-0012 — Documentation Governance



ESS-0001-CONTRACTS Chapter 12 — Validation & Quality Contracts



ESS-0011-CONTRACTS — Enterprise Traceability Matrix Contracts

Governance Statement

ESS-0012-CONTRACTS ist die verbindliche Vertragsgrundlage des Documentation Governance Validator.

Es besitzt keine globale Wirkung.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Fassung der Documentation Governance Contracts, 57 Regeln in neun Bereichen

Approval

Document Status

APPROVED

Enterprise Specification

CAPITAL-AI Core Architecture

Version 1.0.0

End of Document

ESS-0012-CONTRACTS

CAPITAL-AI Documentation Governance Contracts

Version 1.0.0

ESS 0012 Documentation Governance

Quelle: `.ai/skills/ESS-0012-Documentation-Governance.md`

skill: id: ESS-0012 name: Documentation Governance version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority: High

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

classification: type: Technical Specification role: Spezifikation des Documentation Governance Validator contractAuthority: ESS-0001-CONTRACTS ownContracts: ESS-0012-CONTRACTS note: > Dieses Dokument spezifiziert ausschliesslich die Governance-Pruefung der Dokumentation. Validator-Grundregeln verbleiben in ESS-0001-CONTRACTS Chapter 12, Traceability in ESS-0011, Twin-Regeln in Chapter 18.

authority:

controls: - Documentation Governance Validation - Governance Findings - Governance Reports - Repository Health Score

collaborates: - Documentary Engine - Enterprise Traceability - Supervisor - Platform Director - Version Manager - Quality Center

cannot_modify: - Enterprise Specifications - Architecture Decision Records - Enterprise Contracts - Knowledge Graph - Digital Twin - Registry - Dokumentation

crossReference: dependsOn: - ESS-0001-CONTRACTS - ESS-0010 relatedEss: - ESS-0001 - ESS-0002 - ESS-0003 - ESS-0011 - ESS-0011-CONTRACTS - ESS-0012-CONTRACTS relatedAdr: - ADR-0010 - ADR-0013 - ADR-0014 relatedComponents: - src/platform/Documentary/Governance - src/platform/Documentary - src/platform/Knowledge - src/platform/Registry - src/platform/Quality relatedSkills: - `.ai/skills/ESS-0010-Documentary-Engine.md` - `.ai/skills/ESS-0011-Enterprise-Traceability.md` - `.ai/skills/Documentation-Governance-Validator.md`

created: 2026-07-31

Documentation Governance

Enterprise Purpose

Dieses Dokument spezifiziert den **Documentation Governance Validator** — die Prüfinstanz der Documentary Engine.

Die Plattform beschreibt sich seit ESS-0010 selbst und verknüpft sich seit ESS-0011 nachvollziehbar. Sie prüft sich bislang jedoch nicht selbst.

Der Governance Validator schließt diese Lücke: Er überprüft sämtliche Dokumentation, Regeln, Metadaten und Beziehungen automatisch auf Konsistenz, Vollständigkeit und Governance-Konformität.

Er erzeugt ausschließlich Befunde und Berichte.

Er trifft keine Entscheidungen.

Er verändert keine Dokumentation.

Er behebt keine Verstöße.

Notwendigkeitsnachweis

Vor Anlage dieses Dokumentes wurde gemäß der Prüfliste aus ARCH-RESP-0001 geprüft, ob der Regelbereich bereits abgedeckt ist.

Bereich	Bestehende Regelung	Deckt Governance-Prüfung ab?
Validator-Grundregeln, Severity, Quality Gates	ESS-0001-CONTRACTS Chapter 12	teilweise — definiert 16 strukturelle Validatoren, keine dokumentübergreifende Governance-Prüfung
Traceability, Coverage, Orphans	ESS-0011-CONTRACTS	nein — prüft Verknüpfungen, nicht Regelwidersprüche
Twin-Zustände, Drift	ESS-0001-CONTRACTS Chapter 18	nein — prüft Synchronität, nicht Dokumentationsqualität
Knowledge-Node-Regeln	ESS-0001-CONTRACTS Chapter 15	nein — prüft Graphkonsistenz, nicht ESS-/ADR-Semantik
Metadaten-Pflichtfelder	ESS-0001-CONTRACTS Chapter 7	teilweise — prüft Existenz, nicht Widerspruchsfreiheit
Dokumentverantwortung	ARCH-RESP-0001	nein — Zuordnung ohne ausführbare Prüfung

Ergebnis

Nicht abgedeckt sind ausschließlich die **semantischen, dokumentübergreifenden** Governance-Prüfungen:

- doppelte und widersprüchliche ESS-Regeln
- widersprüchliche und veraltete ADRs
- doppelte, widersprüchliche und nicht referenzierte Contracts
- Breaking Changes ohne ADR
- aggregierte Governance- und Repository-Reife

Ein eigenes Dokument ist damit gerechtfertigt und kein Duplikat.

Abgrenzung

Dieses Dokument enthält **nicht**:

Inhalt	Zuständiges Dokument
Validator-Basisvertrag, Severity-Stufen, Quality Gates	ESS-0001-CONTRACTS Chapter 12
Coverage-Schwellwerte, Orphan-Klassen	ESS-0011-CONTRACTS
Twin-Zustände, Drift-Toleranz, Reconciliation	ESS-0001-CONTRACTS Chapter 18
Knoten- und Beziehungsregeln	ESS-0001-CONTRACTS Chapter 15
Metadaten-Pflichtfelder	ESS-0001-CONTRACTS Chapter 7
Event-Namenskonvention, Event-Vertrag	ESS-0001-CONTRACTS Chapter 8
globale Repository-, Naming-, Layer-, Governance-Regeln	ESS-0001-CONTRACTS

Diese Regeln werden referenziert, niemals wiederholt.

Enterprise Principle

Eine Regel, die nicht geprüft wird, ist eine Empfehlung.

Der Validator stellt fest.

Der Supervisor bewertet.

Der Platform Director entscheidet.

Der Validator korrigiert niemals selbst — eine Prüfinstanz, die den Prüfgegenstand verändert, kann ihn nicht mehr unabhängig beurteilen.

Position in der Architektur

Der Governance Validator ist Bestandteil der Documentary Engine und damit des Documentary Layer.

```
src/platform/Documentary/Governance/
```

Zulässige Abhängigkeiten: Core, Shared, Registry, Discovery, Knowledge.

Unzulässig: Version Manager, Supervisor, Platform Director.

Die Kommunikation nach oben erfolgt ausschließlich über Enterprise Events.

End of Chapter 1

Chapter 2

Komponentenarchitektur

Enterprise Purpose

Dieses Kapitel definiert den inneren Aufbau des Governance Validator.

Verzeichnisstruktur

```
src/platform/Documentary/Governance/  
  
Contracts/      komponentenspezifische Contracts  
Validators/     Einzelvalidatoren je Prüfbereich  
Rules/          deklaratives Regelwerk  
Services/       Ausführung und Orchestrierung  
Reports/        Berichtserzeugung  
Events/         Event-Erzeugung und -Konsum  
Models/         Datenmodelle  
Interfaces/     öffentliche Schnittstellen
```

Jedes Unterverzeichnis besitzt genau eine Verantwortung gemäß Chapter 3.

Kernkomponenten

GovernanceEngine

Ausführungssteuerung.

Verantwortung

Reihenfolge der Prüfbereiche

Regelauswahl

Ergebniszusammenführung

Event-Erzeugung

RuleRegistry

Verwaltung des Regelwerks.

Verantwortung

Registrierung der Regeln

Versionierung der Regeln

Auflösung von Regelabhängigkeiten

GovernanceValidators

Neun Einzelvalidatoren je Prüfbereich, siehe Chapter 3.

FindingCollector

Sammlung und Klassifizierung der Befunde.

Verantwortung

Befunderzeugung

Schweregradzuordnung

Deduplizierung

Nachweisführung

ScoreCalculator

Berechnung der Reifekennzahlen.

Verantwortung

Governance Score

Repository Health Score

Dokumentationsqualität

Die Berechnung ist deterministisch und wird niemals manuell gesetzt.

GovernanceReporter

Erzeugung der zehn Berichte gemäß Chapter 5.

Ablaufreihenfolge

→ RuleRegistry	Regelwerk laden
→ GovernanceValidators	neun Prüfbereiche
→ FindingCollector	Befunde sammeln
→ ScoreCalculator	Kennzahlen berechnen
→ GovernanceReporter	Berichte erzeugen

Die Reihenfolge ist verbindlich und deterministisch.

End of Chapter 2

Chapter 3

Prüfbereiche

Enterprise Purpose

Dieses Kapitel definiert die neun verbindlichen Prüfbereiche.

Die konkreten Regeln, Schweregrade und Bezeichner stehen in ESS-0012-CONTRACTS.

3.1 EssValidator

Prüft

doppelte ESS-Regeln über Dokumentgrenzen hinweg

fehlende ESS-Referenzen in Komponenten und Dokumenten

ungültige oder nicht registrierte ESS-Nummern

fehlende Cross-References

widersprüchliche ESS-Regeln

Grundlage: `.ai/registry/ess-registry.json` , `ARCH-RESP-0001` .

3.2 AdrValidator

Prüft

fehlende ADR-Referenzen bei Architekturentscheidungen

widersprüchliche ADRs

veraltete ADRs ohne `SUPERSEDED` -Kennzeichnung

Breaking Changes ohne zugehörigen ADR

ADR-Nummern ohne Dokument

Grundlage: `docs/adr/` , `adr_history.json` .

3.3 RepositoryValidator

Prüft je Komponente

README vorhanden und inhaltlich belegt

CHANGELOG vorhanden

manifest.json vorhanden und vollständig

component.yaml vorhanden

Ownership definiert und nicht pauschal

Ein Platzhaltertext gilt als nicht belegt.

3.4 ContractValidator

Prüft

doppelte Contracts über Dokumentgrenzen hinweg

widersprüchliche Contracts

ungültige Contracts ohne Geltungsbereich

nicht referenzierte Contracts

Contracts mit globaler Wirkung außerhalb von ESS-0001-CONTRACTS

3.5 DocumentationValidator

Prüft

Vollständigkeit gegenüber der Dokumentklasse

Aktualität gegenüber der letzten Codeänderung

Versionierung

Konsistenz zwischen Dokument und Metadaten

Struktur gemäß Dokumentklasse

Formatierung

Enterprise-Konformität

3.6 TraceabilityValidator

Prüft die Kette

ESS → ADR → Code → Tests → Dokumentation → Versionierung → Releases → Production

auf fehlende Beziehungen.

Der Validator **verwendet** die Matrix aus ESS-0011 und erzeugt sie nicht.

Coverage-Schwellwerte und Orphan-Klassen stehen ausschließlich in ESS-0011-CONTRACTS.

3.7 KnowledgeGraphValidator

Prüft

verwaiste Knoten

doppelte Beziehungen

fehlende Beziehungen

inkonsistente Beziehungen

Die Knoten- und Beziehungsregeln selbst stehen in ESS-0001-CONTRACTS Chapter 15.

3.8 DigitalTwinValidator

Vergleicht Repository gegen Digital Twin und erkennt

fehlende Komponenten

neue Komponenten

gelöschte Komponenten

nicht synchronisierte Komponenten

Twin-Zustände und Drift-Toleranz stehen in ESS-0001-CONTRACTS Chapter 18.

3.9 VersionValidator

Prüft auf Konflikte zwischen

Dokumentationsversion

Komponentenversion

Repository-Version

ESS-Version

ADR-Version

Die Versionsstrategie selbst steht in ESS-0001-CONTRACTS Chapter 9.

End of Chapter 3

Chapter 4

Trigger und Workflow

Enterprise Purpose

Dieses Kapitel definiert Auslösung und Ablauf.

Hauptworkflow

```
Trigger
↓
Regelwerk laden
↓
Prüfbereiche ausführen (3.1 bis 3.9)
↓
Befunde sammeln und klassifizieren
↓
Kennzahlen berechnen
↓
Berichte erzeugen
↓
Events veröffentlichen
```

Der Governance Validator läuft ausschließlich **nach** der Documentary Engine und **nach** der Traceability Matrix, da er deren Ergebnisse verwendet.

Trigger Contract

Auslöser	Umfang
DocumentationGeneratedEvent	inkrementell
KnowledgeUpdatedEvent	inkrementell
TwinSynchronizedEvent	inkrementell
ComponentRegisteredEvent	inkrementell
VersionChangedEvent	vollständig
ESS-Änderung	vollständig
ADR-Änderung	vollständig
ReleasePreparedEvent	vollständig
zeitgesteuerte Vollprüfung	vollständig

Vor jeder Produktionsfreigabe erfolgt ein vollständiger Durchlauf.

Determinismus

Bei identischem Repository-Zustand erzeugt der Validator

identische Befunde

identische Schweregrade

identische Kennzahlen

Zeitstempel sind ausgenommen.

Weicht das Ergebnis bei unverändertem Eingangszustand ab, gilt der Durchlauf als fehlerhaft.

Read Only Contract

Der Governance Validator besitzt **keinen** Schreibzugriff auf

Quellcode

Dokumentation

Metadaten

Registry

Knowledge Graph

Digital Twin

ESS-Dokumente

ADR-Dokumente

Er schreibt ausschließlich in seinen eigenen Befund- und Berichtsspeicher.

End of Chapter 4

Chapter 5

Reports

Enterprise Purpose

Dieses Kapitel definiert die zehn verbindlichen Berichte.

Bericht	Inhalt
Documentation Health Report	Vollständigkeit, Aktualität, Struktur je Dokument
Governance Report	sämtliche Governance-Verstöße mit Schweregrad
ESS Coverage Report	Abdeckung der ESS-Regeln durch Komponenten
ADR Coverage Report	Abdeckung der Architekturentscheidungen
Traceability Report	fehlende Beziehungen entlang der Kette
Repository Health Report	Metadaten, Ownership, Struktur je Komponente
Digital Twin Report	Abweichungen Repository gegen Twin
Knowledge Graph Report	verwaiste, doppelte und inkonsistente Beziehungen
Contract Report	Duplikate, Widersprüche, nicht referenzierte Contracts
Architecture Compliance Report	Layer-, Abhängigkeits- und Strukturkonformität

Berichtsablage

docs/quality/

Die physische Erzeugung erfolgt durch die Documentary Engine gemäß ESS-0010.

Der Governance Validator liefert die Daten, nicht das Dokument.

Sämtliche Berichte tragen die Kennzeichnung als generiert gemäß ESS-0001-CONTRACTS Chapter 19, Generated Artifact Contract.

End of Chapter 5

Chapter 6

Integration

Enterprise Purpose

Dieses Kapitel definiert die Einbettung in die Plattform.

Documentary Engine

Der Governance Validator ist Bestandteil der Documentary Engine.

Er verwendet deren Ergebnisse: Knowledge Graph, Registry, Digital Twin, erzeugte Dokumentation.

Er schreibt niemals in diese Quellen zurück.

Enterprise Traceability

Der Governance Validator verwendet die Matrix aus ESS-0011.

Er erzeugt sie nicht und verändert sie nicht.

Fehlt die Matrix, meldet er dies als Befund und setzt den Traceability-Prüfbereich aus.

Supervisor

Der Supervisor erhält gemäß ESS-0002 automatisch Meldung bei

kritischen Dokumentationsfehlern

fehlenden ESS-Referenzen

fehlenden ADR-Referenzen

Governance-Verstößen

Traceability-Verlust

Versionskonflikten

Der Supervisor bewertet die Befunde und entscheidet über Blockade.

Der Validator blockiert selbst niemals.

Platform Director

Der Platform Director erhält gemäß ESS-0003

Governance-Reife

Dokumentationsqualität

Architekturqualität

Repository-Reife

und leitet daraus Handlungsempfehlungen ab.

Die Empfehlungen erzeugt der Platform Director, nicht der Validator.

Version Manager

Der Version Manager verwendet die Ergebnisse zur Versionsbewertung gemäß ESS-0001-CONTRACTS Chapter 9.

Befundklasse	Versionsauswirkung
Breaking Change ohne ADR	blockiert Release
Contract-Widerspruch	Major
neue Regel oder Komponente	Minor
Dokumentationslücke	Patch

Die Versionsentscheidung trifft der Version Manager, nicht der Validator.

AI Integration

Sämtliche KI-Systeme verwenden verbindlich denselben Governance Validator.

System	Pflicht
Claude Code	prüft Änderungen vor Übergabe an die Documentary Engine

System	Pflicht
Google AI Studio	prüft Entwürfe vor Übergabe an Claude Code
ChatGPT	prüft Architektur- und Governance-Vorschläge
Future Enterprise AI	identische Pflicht

Kein KI-System verwendet eigene Prüfregeln.

Kein KI-System deaktiviert Regeln.

Kein KI-System erzeugt Prüfergebnisse ohne Ausführung.

Dies konkretisiert ESS-0001-CONTRACTS Chapter 10 und Chapter 17 für die Dokumentations-Governance.

Enterprise Rules

Der Validator stellt fest, er entscheidet nicht.

Der Validator verändert niemals den Prüfgegenstand.

Keine Regel ohne Bezeichner und Schweregrad.

Kein Befund ohne Nachweis.

Keine Kennzahl ohne Berechnung.

Keine Produktionsfreigabe bei Critical-Befunden.

Kein Prüfergebnis ohne Ausführung.

Success Criteria

Der Governance Validator gilt als erfolgreich implementiert wenn

- ✓ sämtliche neun Prüfbereiche ausführbar sind
- ✓ jeder Befund einen Nachweis besitzt
- ✓ jede Kennzahl deterministisch berechnet wird
- ✓ sämtliche zehn Berichte automatisch erzeugt werden
- ✓ der Supervisor sämtliche kritischen Befunde erhält
- ✓ der Platform Director Reifekennzahlen erhält
- ✓ der Version Manager Versionsauswirkungen ableiten kann
- ✓ sämtliche KI-Systeme denselben Validator verwenden

✓ der Validator keinen Prüfgegenstand verändert

Integration

Dieses Dokument bildet die Grundlage für
ESS-0012-CONTRACTS — Documentation Governance Contracts

Enterprise Final Summary

ESS-0012 Status

Document ID ESS-0012

Titel CAPITAL-AI Documentation Governance

Status Enterprise Specification

Version 1.0.0

Lifecycle Status Approved Enterprise Specification

Governance Statement

ESS-0012 ist die verbindliche Spezifikation des Documentation Governance Validator innerhalb des CAPITAL-AI Core.

Sie baut vollständig auf ESS-0001-CONTRACTS, ESS-0010 und ESS-0011 auf.

Abweichungen erfordern eine neue Architecture Decision Record.

Version History

Version	Status	Beschreibung
1.0.0	Initial Release	Erste Spezifikation des Documenta- tion Governance Validator

Approval

Document Status

APPROVED

Enterprise Specification

CAPITAL-AI Core Architecture

Version 1.0.0

End of Document

ESS-0012

CAPITAL-AI Documentation Governance

Enterprise Specification

Version 1.0.0

ESS 0012 Enterprise Vocabulary Terminology Governance

Quelle: `.ai/skills/ESS-0012-Enterprise-Vocabulary-Terminology-Governance.md`

```
skill:
  id: ESS-0012
  name: Enterprise Vocabulary & Terminology Governance
  version: 1.0.0
  status: Proposed
  owner: Platform Director
  category: Enterprise Governance
  priority: High

capital_ai:
  platform: CAPITAL-AI Core
  repository: Finance
  lifecycle: AI Native Development Lifecycle

classification:
  type: Component Specification
  contractAuthority: ESS-0001-CONTRACTS

references:
  - ESS-0001-CONTRACTS
  - ESS-0003
  - ESS-0009
  - ESS-0010
  - ESS-0011
  - ADR-0044
  - ADR-0045
```

ESS-0012 — Enterprise Vocabulary & Terminology Governance

Deutsch

Zweck

ESS-0012 definiert die komponentenspezifische Governance für kanonische Begriffe, Naming Conventions, zweisprachige Dokumentation und die ereignisbasierte Weitergabe von Terminologieänderungen innerhalb der CAPITAL-AI Wertschöpfungskette. ESS-0001-CONTRACTS bleibt die übergeordnete Contract Authority.

Verbindliche Regeln

- Produktiver Code, technische Identifier, Komponenten, Services, Typen, Interfaces, API-Felder, Events, Branch-Namen und Dateinamen werden in Englisch geführt.
- Enterprise-Dokumentation wird in Deutsch und Englisch bereitgestellt. Technische IDs, ADR-/ESS-Nummern und maschinenlesbare Schlüssel bleiben sprachneutral.

- Benutzerseitige Pull-Request-Informationen werden in deutscher Sprache geführt. Maschinenlesbare Marker, Platzhalter und stabile Schlüssel dürfen Englisch bleiben.
- Jeder kanonische Begriff besitzt eine eindeutige ID, englische Code-Bezeichnung, deutsche und englische Anzeigeform, Definition, zulässige Aliase, verbotene Varianten, Status und Traceability-Referenzen.
- Neue oder geänderte Begriffe erzeugen ein Domain Event und aktualisieren Knowledge, Documentary und Traceability.
- Renames im aktiven Code sind ohne vorangehende Impact-Analyse unzulässig.

Safe-Rename Contract

Vor einem aktiven Rename sind mindestens zu prüfen: 1. statische Referenzen und Import-/Export-Graph, 2. dynamische Imports und Lazy Loading, 3. Route-, API- und Schema-Verwendungen, 4. Konfigurationen und Environment-Referenzen, 5. Regex- und Naming-Validatoren, 6. Dateisystem-Casing und Linux-Deploy-Kompatibilität, 7. TypeScript-/Lint-Prüfung, 8. Tests, 9. Production Build, 10. Deployment-Readiness.

Das Ergebnis wird als SAFE, CONDITIONAL oder BLOCKED klassifiziert. Nur SAFE darf autonom zur Implementierung vorbereitet werden. CONDITIONAL und BLOCKED erzeugen Evidence und benötigen eine explizite Entscheidung.

Event Contract

Mindestens folgende Ereignisse sind vorgesehen: - terminology.change.proposed - terminology.change.approved - terminology.updated - component.rename.proposed - component.rename.validated - documentation.translation.required - documentation.updated - traceability.updated - governance.validation.completed

Integration

ESS-0012 arbeitet mit Platform Director, Supervisor, Enterprise Knowledge Platform, Documentary Engine, Traceability, Quality, Security, Compliance, Version Manager und Release Center zusammen. Die Verarbeitung darf autonom erfolgen; bestehende Human-Approval-Gates für Architektur, Security, geschützte Änderungen, PR-Erstellung und Release werden nicht umgangen.

English

Purpose

ESS-0012 defines component-level governance for canonical terminology, naming conventions, bilingual documentation, and event-driven propagation of terminology changes across the CAPITAL-AI value chain. ESS-0001-CONTRACTS remains the governing contract authority.

Mandatory rules

- Production code, technical identifiers, components, services, types, interfaces, API fields, events, branch names, and file names use English.

- Enterprise documentation is provided in German and English. Technical IDs, ADR/ESS numbers, and machine-readable keys remain language-neutral.
- Human-facing pull request information is written in German. Machine-readable markers, placeholders, and stable keys may remain English.
- Every canonical concept has a stable ID, English code term, German and English display names, definition, allowed aliases, forbidden variants, lifecycle status, and traceability references.
- New or changed terminology emits a domain event and updates Knowledge, Documentary, and Traceability.
- Active-code renames are prohibited without prior impact analysis.

Safe rename contract

Before an active rename, the system must inspect static references and dependencies, dynamic imports, routes/APIs/schemas, configuration and environment references, regex and naming validators, filesystem casing, type checking, tests, production build, and deployment readiness.

The result is classified as **SAFE**, **CONDITIONAL**, or **BLOCKED**. Only **SAFE** changes may be prepared autonomously for implementation. **CONDITIONAL** and **BLOCKED** changes produce evidence and require an explicit decision.

Event integration

Vocabulary governance participates in the platform event lifecycle and integrates with Platform Director, Supervisor, Knowledge, Documentary, Traceability, Quality, Security, Compliance, Version Manager, and Release Center. Automation must never bypass human approval boundaries defined by higher-level contracts or ADRs.

ESS-0013-CONTRACTS

Quelle: `.ai/skills/ESS-0013-Contracts.md`

ESS-0013-CONTRACTS

Enterprise Event Mesh Contracts

Version

1.0.0

Status

Enterprise Specification

Dokumentklassifizierung

Dieses Dokument definiert **ausschließlich** Enterprise-Event-Mesh-Komponenten-Contracts: Event-Katalog-Pflege, Kompatibilitätsprüfung, Registry-Konsistenz, Routing-Garantien, Discovery-Regeln und Report-Inhalte.

Es ist kein globaler Enterprise Standard.

Nicht zulässig sind in diesem Dokument

globale Event-Namensregeln (bleiben in ESS-0001-CONTRACTS Chapter 8)

globale Repository Contracts

globale Layer Contracts

allgemeine Governance-Regeln (bleiben in ESS-0012-CONTRACTS)

Diese verbleiben ausnahmslos in ihrem jeweils zuständigen Dokument.

Bei Konflikt zwischen diesem Dokument und ESS-0001-CONTRACTS gilt ausnahmslos ESS-0001-CONTRACTS.

Cross Reference

Dieses Dokument besitzt kein YAML-Frontmatter. Die Referenzen werden als Markdown-Abschnitt geführt, analog zu ESS-0001-CONTRACTS und ESS-0011-CONTRACTS.

Depends On

ESS-0001-CONTRACTS

ESS-0013

Related ESS

ESS-0001 — Documentary & Code Intelligence Architect

ESS-0010 — Documentary Engine

ESS-0011 — Enterprise Traceability

ESS-0011-CONTRACTS — Link Contract, Coverage, Orphans

ESS-0012 — Documentation Governance

ESS-0012-CONTRACTS — 57 Governance-Regeln, u. a. die sechs Governance-Validator-Events

Related ADR

ADR-0010 — Enterprise Standard Extension

ADR-0013 — ESS Documentation Responsibility Consolidation

ADR-0015 — Enterprise Traceability Component

ADR-0018 — Enterprise Event Mesh

Related Components

`src/platform/EventMesh`

`src/platform/Traceability`

`src/platform/Documentary`

`src/platform/Knowledge`

1. Event Catalog Contract

Der `EventCatalog` führt jedes Event genau einmal.

Ein Event-Name wird niemals zweifach mit unterschiedlicher Bedeutung vergeben.

Bevor ein neues Event registriert wird, prüft `EventContractValidator` gegen den bestehenden Katalog, ob ein bedeutungsgleiches Event bereits existiert (siehe ESS-0013, Abschnitt Standard Event Catalog, für drei bereits aufgelöste Kollisionen).

Ein Event, das ausschließlich einen bereits registrierten Sachverhalt unter neuem Namen beschreibt, wird abgelehnt. Der Producer muss das bestehende Event verwenden.

2. Naming Contract

Jedes Event trägt das Suffix `Event` (Chapter 8, unverändert).

Jedes Event trägt zusätzlich eine der zwölf in ESS-0013 definierten Kategorien als Katalog-Metadatum — nicht als Namensbestandteil.

3. Registry Contract

`EventRegistry`, `ProducerRegistry` und `ConsumerRegistry` werden ausschließlich generiert. Manuelle Einträge sind nicht zulässig.

Ein Event ohne mindestens einen registrierten Producer erhält den Status `proposed` und nicht `registered`.

Ein Event ohne mindestens einen registrierten Consumer ist zulässig (nicht jedes Event muss konsumiert werden), wird jedoch im `EventCoverageReport` gesondert ausgewiesen.

4. Compatibility Contract

Vor jeder neuen Event-Version prüft `EventCompatibilityValidator`:

Eine neue Pflichtfeld-Anforderung im Payload ist ein Breaking Change → Major Version (Chapter 8, Event Versioning, unverändert referenziert).

Eine neue optionale Payload-Eigenschaft ist eine Minor Version.

Eine Beschreibungs- oder Metadatenkorrektur ohne Strukturänderung ist eine Patch Version.

Ein Breaking Change ohne begleitenden ADR wird von `EventCompatibilityValidator` zurückgewiesen — konsistent mit dem repositoryweiten Verbot „Breaking Changes ohne ADR“.

5. Routing Contract

`EventRouter` ist der einzige Ort, an dem eine Zustellentscheidung getroffen wird.

Kein Producer bestimmt selbst seine Consumer. Die Zuordnung erfolgt ausschließlich über `ConsumerRegistry`-Einträge.

Ein Routing-Fehlschlag erzeugt `EventRoutingFailedEvent` (System Events) und wird niemals stillschweigend verworfen.

6. Discovery Contract

`Discovery/` liest ausschließlich deklarierte `manifest.json`-Felder (`events.produces`, `events.consumes`). Es leitet Producer/Consumer niemals aus Quellcode-Analyse ab, solange keine Komponente über ausführbaren Code verfügt.

Eine Komponente mit leerem `events`-Feld gilt als „kein deklarierter Producer/Consumer“, nicht als Fehler. Der `EventCoverageReport` weist dies als offenen Punkt aus, keine Verletzung.

7. Policy Contract

`Policies/` definiert je Event-Kategorie mindestens eine Routing-Policy (wer darf konsumieren) und optional eine Retry-Policy (Zustellversuche bei Fehlschlag).

Eine Policy darf ein Event niemals inhaltlich verändern — ausschließlich Zustellverhalten.

8. Report Contract

Report	Pflichtangaben
<code>EventFlowReport</code>	Zeitraum, Producer, Consumer, Event-Anzahl je Paar
<code>EventCoverageReport</code>	Komponenten ohne Producer-Eintrag, Komponenten ohne Consumer-Eintrag, Events ohne Consumer
<code>EventHealthReport</code>	Fehlerrate, mittlere Zustellzeit, unzustellbare Events der letzten Periode
<code>EventDependencyReport</code>	aus Flüssen abgeleitete Komponentenpaare, absteigend nach Häufigkeit

Reports werden ausschließlich berechnet, niemals manuell gepflegt (ESS-0001-CONTRACTS Chapter 12, analog übernommen).

9. ETM-Referenz-Contract

Jedes registrierte Event führt eine ETM-Referenz (mindestens eine Interface- und eine Komponentenreferenz gemäß ESS-0011 Traceability-Achsen). Fehlt sie, bleibt das Event im Status `proposed`.

10. Abgrenzung gegenüber Chapter 8

Dieses Dokument wiederholt keine der folgenden, bereits in Chapter 8 verbindlichen Regeln: Event-Prinzip, Namensregel (Suffix), Pflichtfelder des Event Contract, Producer-Grundregel („jede Komponente darf veröffentlichen“), Consumer-Grundregel („ausschließlich dokumentierte Events“), zehnteilige Validation-Checkliste.

Bei Erweiterung dieses Dokuments ist vor jeder neuen Regel zu prüfen, ob sie nicht bereits durch Chapter 8 abgedeckt ist (`G0V-ESS-004` , Zero Duplication).

End of ESS-0013-CONTRACTS

ESS 0013 Enterprise Event Mesh

Quelle: [.ai/skills/ESS-0013-Enterprise-Event-Mesh.md](#)

skill: id: ESS-0013 name: Enterprise Event Mesh version: 1.0.0 status: Enterprise Approved maturity: Gold Standard owner: Platform Director category: Enterprise Architecture priority: High

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

classification: type: Technical Specification role: Spezifikation der Enterprise Event Mesh — Komponente, nicht Regelwerk contractAuthority: ESS-0001-CONTRACTS ownContracts: ESS-0013-CONTRACTS note: > Dieses Dokument beschreibt ausschließlich die Enterprise-Event-Mesh-Komponente (Core-Klassen, Registry-Mechanik, Validator-Kette, Report-Typen, Standard-Event- Katalog). Das Event- und Messaging-Regelwerk selbst — Namensregel, Contract- Pflichtfelder, Producer-/Consumer-Grundregeln, Routing-Prinzip, Validation-Kriterien — ist bereits vollständig in ESS-0001-CONTRACTS Chapter 8 normativ definiert und wird hier nicht wiederholt.

authority:

controls: - Event Registry - Event Routing - Event Validation - Event Reports

collaborates: - Documentary Engine - Enterprise Traceability Matrix - Knowledge Graph - Compliance Engine - Version Manager - Supervisor - Platform Director - Security

cannot_modify: - Enterprise Specifications - Architecture Decision Records - fachliche Event-Inhalte anderer Komponenten - ESS-0001-CONTRACTS Chapter 8

crossReference: dependsOn: - ESS-0001-CONTRACTS relatedEss: - ESS-0001 - ESS-0010 - ESS-0011 - ESS-0011-CONTRACTS - ESS-0012 - ESS-0012-CONTRACTS - ESS-0013-CONTRACTS relatedAdr: - ADR-0010 - ADR-0013 - ADR-0014 - ADR-0015 - ADR-0018 relatedComponents: - src/platform/EventMesh - src/platform/Traceability - src/platform/Documentary - src/platform/Knowledge - src/platform/Registry relatedSkills: - [.ai/skills/ESS-0001-Contracts.md](#) - [.ai/skills/ESS-0010-Documentary-Engine.md](#) - [.ai/skills/ESS-0011-Enterprise-Traceability.md](#) - [.ai/skills/Enterprise-Event-Mesh.md](#)

created: 2026-07-31

Enterprise Event Mesh

Enterprise Purpose

Dieses Dokument spezifiziert die Enterprise Event Mesh (EEM) des CAPITAL-AI Core als konkrete Softwarekomponente.

Die Enterprise Event Mesh beantwortet eine einzige Frage lückenlos und maschinell:

Welche Komponente hat wann welches Event veröffentlicht, wer hat es konsumiert, und ist dieser Fluss vertragskonform?

Die Enterprise Event Mesh erzeugt keine fachlichen Events.

Sie erzeugt keine Geschäftslogik.

Sie registriert, validiert, routet und protokolliert ausschließlich Events, die von Fachkomponenten veröffentlicht werden.

Verhältnis zu ESS-0001-CONTRACTS Chapter 8

Chapter 8 ist bereits vollständig und bleibt unverändert. Es definiert normativ:

- das Enterprise-Prinzip („Jede relevante Änderung erzeugt ein Event“)
- die Namensregel (Suffix `Event`)
- sieben Basiskategorien mit Beispielergebnissen
- die Pflichtfelder des Event Contract
- die Grundregeln für Producer, Consumer, Routing
- die Integrationsabschnitte für Documentary, Supervisor, Platform Director, Version Manager
- die zehnteilige Validation-Checkliste

Dieses Dokument (ESS-0013) beschreibt **ausschließlich**, woraus die Komponente besteht, die Chapter 8 umsetzt — analog zum bereits etablierten Verhältnis zwischen Chapter 9 (Versionierungsregeln) und ESS-0004 (Version-Manager-Komponente), sowie zwischen ESS-0011 (Traceability-Spezifikation) und `src/platform/Traceability` (Implementierungsort).

Bei Konflikt zwischen diesem Dokument und ESS-0001-CONTRACTS Chapter 8 gilt ausnahmslos Chapter 8.

Architektur

Die Enterprise Event Mesh besteht mindestens aus folgenden Klassen bzw. Modulen, je Unterverzeichnis von `src/platform/EventMesh/` :

Core

Klasse	Verantwortung
<code>EventBus</code>	zentrale Zustellinstanz; nimmt veröffentlichte Events entgegen
<code>EventDispatcher</code>	verteilt validierte Events an registrierte Consumer

Klasse	Verantwortung
EventPublisher	öffentliche Schnittstelle für Producer zur Veröffentlichung
EventSubscriber	öffentliche Schnittstelle für Consumer zur Registrierung eines Abonnements
EventRouter	einzige Instanz, die über Zustellwege entscheidet (Chapter 8, Event Routing)

Contracts

Typ	Inhalt
EventContract	Gesamtvertrag eines Events — fasst die folgenden vier Typen zusammen
EventPayload	ausschließlich fachliche Information, keine Implementierungsdetails (Chapter 8)
EventMetadata	Name, Event-ID, Timestamp, Source/Target Component, Correlation ID
EventVersion	Major/Minor/Patch gemäß Chapter 8, Event Versioning
EventSchema	strukturelle Definition des Payloads zur Validierung

Registry

Klasse	Verantwortung
EventRegistry	zentrale Registrierung aller bekannten Event-Typen
ProducerRegistry	welche Komponente welches Event veröffentlichen darf
ConsumerRegistry	welche Komponente welches Event abonniert hat
EventCatalog	konsolidierte, lesbare Sicht aus allen drei Registries (siehe Abschnitt Event Registry)

Validators

Klasse	Prüft
EventContractValidator	Vollständigkeit der Pflichtfelder aus Chapter 8
EventSchemaValidator	Payload gegen EventSchema
EventVersionValidator	Versionsformat und Versionsprung-Regeln
EventCompatibilityValidator	Breaking-Change-Erkennung zwischen Event-Versionen

Reports

Report	Inhalt
EventFlowReport	tatsächliche Event-Flüsse Producer → Consumer über einen Zeitraum
EventCoverageReport	

Report	Inhalt
	Anteil registrierter Komponenten mit mindestens einem Producer- oder Consumer-Eintrag
EventHealthReport	Fehlerraten, Zustellverzögerungen, unzustellbare Events
EventDependencyReport	aus Event-Flüssen abgeleitete effektive Abhängigkeiten zwischen Komponenten

Weitere Verzeichnisse

Discovery/ erkennt automatisch mögliche Producer und Consumer anhand deklarierter `manifest.json`-Felder (`events.produces` , `events.consumes`) — siehe Abschnitt Producer- und Consumer-Erkennung.

Policies/ definiert Routing-, Retry- und Kompatibilitätsolicies je Event-Kategorie.

Interfaces/ stellt `IEventBus` , `IEventPublisher` , `IEventSubscriber` , `IEventRouter` , `IEventRegistry` bereit — die einzigen zulässigen Zugriffspunkte für Fachkomponenten.

Models/ enthält die Datenmodelle der Registry- und Katalogeinträge.

Services/ orchestriert Registrierung, Validierung und Routing als ausführbare Abfolge.

Tests/ enthält Vertrags- und Kompatibilitätstests je Event-Kategorie.

Event-Kategorien

Gemäß Anforderung werden mindestens folgende zwölf Kategorien geführt. Sie ordnen sich den sieben Basiskategorien aus Chapter 8 unter bzw. ergänzen sie um die seither in Chapter 11-19 hinzugekommenen Bereiche:

Kategorie	Verhältnis zu Chapter 8
Repository Events	= Chapter 8 Repository Events
Documentation Events	= Chapter 8 Documentary Events
Traceability Events	ergänzt Chapter 8 um ESS-0011 (Traceability besaß bereits einen eigenen Katalog, siehe <code>src/platform/Traceability/manifest.json</code>)
Knowledge Events	= Chapter 8 Knowledge Events
Compliance Events	Teilmenge von Chapter 8 Security Events + Chapter 11
Security Events	Teilmenge von Chapter 8 Security Events + Chapter 11
Versioning Events	= Chapter 8 Version Events
Release Events	Teilmenge von Chapter 8 Version Events
Supervisor Events	ergänzt Chapter 8 Platform Events
Platform Director Events	ergänzt Chapter 8 Platform Events

Kategorie	Verhältnis zu Chapter 8
AI Events	neu — Chapter 10/17 (KI-Entwicklungssysteme, Orchestrierung) besaßen bislang keine eigene Event-Kategorie
System Events	neu — technische Meta-Events der Mesh selbst (siehe <code>manifest.json</code> → <code>events.produces</code>)

Keine dieser Kategorien ersetzt oder verändert die sieben Basiskategorien aus Chapter 8. Sie sind eine feingranularere Untergliederung für den Registry-Katalog.

Standard Event Catalog

Die auslösende Anforderung nennt 15 Standardereignisse. Der Abgleich gegen ESS-0001-CONTRACTS Chapter 8/15/18, ESS-0011 und ESS-0012-CONTRACTS ergab: **12 dieser Namen sind bereits kanonisch, 3 kollidierten mit bereits anders benannten, äquivalenten Events** und wurden auf die bestehende kanonische Bezeichnung abgebildet, um keine Duplikate zu erzeugen (verbindlich aus Zero Duplication, ESS-0001).

#	In der Anforderung genannt	Kanonischer Name im Katalog	Quelle	Kategorie	Producer
1	RepositoryScannedEvent	unverändert	Chapter 8	Repository	Discovery
2	DocumentationGeneratedEvent	unverändert	Chapter 8	Documentation	Documentary
3	DocumentationValidatedEvent	unverändert	Chapter 8, ESS-0012-CONTRACTS	Documentation	Documentary / Governance Validator
4	TraceabilityUpdatedEvent	Traceability-BuildCompletedEvent	ESS-0011 (Traceability/manifest.json)	Traceability	Traceability
5	KnowledgeGraphUpdatedEvent	KnowledgeUpdatedEvent	Chapter 8, Chapter 15	Knowledge	Knowledge Graph
6	ComplianceValidatedEvent	unverändert	Chapter 8	Compliance	Compliance Engine
7	ArchitectureValidatedEvent	unverändert	Chapter 8	Compliance	Compliance Engine
8	VersionCalculatedEvent	unverändert	Chapter 8	Versioning	Version Manager
9		unverändert	Chapter 8	Release	

#	In der Anforderung genannt	Kanonischer Name im Katalog	Quelle	Kategorie	Producer
	ReleasePreparedEvent				Version Manager
10	ReleasePublishedEvent	unverändert	Chapter 8	Release	Version Manager
11	SupervisorAlertEvent	unverändert	Chapter 8	Supervisor	Supervisor
12	PlatformDecisionEvent	unverändert	Chapter 8	Platform Director	Platform Director
13	SecurityViolationDetectedEvent	unverändert	Chapter 11	Security	Security
14	GovernanceViolationDetectedEvent	unverändert	ESS-0012-CONTRACTS	Documentation	Governance Validator
15	DigitalTwinUpdatedEvent	TwinSynchronizedEvent	Chapter 18, Twin Events	System	Digital Twin

Weitere in der Anforderung an anderer Stelle genannte Events, ebenfalls abgeglichen:

Genannt	Befund
ContractViolationDetectedEvent	bereits kanonisch als ContractViolationEvent (Chapter 8, Architecture Events) — keine neue Variante eingeführt
LayerViolationDetectedEvent	bereits kanonisch als LayerViolationEvent (Chapter 8, Architecture Events)
TraceabilityViolationEvent	bereits kanonisch (ESS-0012-CONTRACTS, Governance-Validator-Events)
VersionApprovedEvent	neu — kein kanonisches Äquivalent gefunden; als neue Registrierung aufgenommen
RoadmapUpdatedEvent	neu — kein kanonisches Äquivalent gefunden; als neue Registrierung aufgenommen
ArchitectureDecisionApprovedEvent	neu — kein kanonisches Äquivalent gefunden; als neue Registrierung aufgenommen
CriticalArchitectureViolationEvent	neu — kein kanonisches Äquivalent gefunden; als neue Registrierung aufgenommen
DependencyMappedEvent	neu — kein kanonisches Äquivalent gefunden; als neue Registrierung aufgenommen
KnowledgeRelationCreatedEvent	neu — kein kanonisches Äquivalent gefunden; als neue Registrierung aufgenommen

Genannt	Befund
<code>KnowledgeValidationCompletedEvent</code>	neu — kein kanonisches Äquivalent gefunden; als neue Registrierung aufgenommen

Die vier bereits mit Producer/Consumer benannten Traceability-Events (`TraceabilityBuildStartedEvent` , `TraceabilityBuildFailedEvent` , `CoverageCalculatedEvent` , `CoverageThresholdViolatedEvent` , `OrphanDetectedEvent` , `OrphanResolvedEvent` , `TraceabilityReportGeneratedEvent`) sind unverändert Teil des Katalogs und werden nicht erneut definiert (Quelle: `Traceability/manifest.json`).

Der vollständige, laufend erweiterte Katalog wird durch `EventRegistry` / `EventCatalog` geführt, nicht durch dieses Dokument — dieses Dokument definiert den **Ausgangsbestand**.

Event Contracts

Jedes Event besitzt — wie in Chapter 8 verbindlich vorgegeben — mindestens:

Event Name · Event ID · Version · Timestamp · Source Component · Target Component · Correlation ID · Payload · ESS-Referenzen · ADR-Referenzen

Ergänzend führt jedes über die Enterprise Event Mesh registrierte Event zusätzlich:

ETM-Referenzen — die Traceability-Achse „Event“ (siehe ESS-0011, Traceability- Achsen) verlangt, dass jedes Event auf mindestens ein Interface und mindestens eine Komponente rückverfolgbar ist. Ohne diese Referenz wird ein Event von `EventContractValidator` als unvollständig zurückgewiesen.

Alle Events sind versioniert nach Chapter 8, Event Versioning.

Event Routing

Routing erfolgt ausschließlich über `EventRouter` . Es gibt keinen zweiten Zustellweg.

Direkte Engine-zu-Engine-Kommunikation soll langfristig vollständig durch registrierte Events ersetzt werden — bereits bestehender Code (`server/systemEvents.ts` , Frontend- `CustomEvent` -Nutzung) ist davon **nicht betroffen** (siehe `docs/architecture/ENTERPRISE_EVENT_READINESS_REPORT.md` Abschnitt 3.2); die Regel gilt für künftige Implementierungen unter `src/platform/` .

Producer- und Consumer-Erkennung

`Discovery/` identifiziert mögliche Producer und Consumer **automatisch** anhand der `events.produces` - bzw. `events.consumes` -Felder in `manifest.json` jeder Komponente unter `src/platform/` . Aktueller Bestand (siehe `docs/architecture/ENTERPRISE_EVENT_READINESS_REPORT.md` Abschnitt 3–4):

Mögliche Producer

Documentary Engine · Traceability Engine · Knowledge Graph · Compliance Engine · Version Manager · Supervisor · Platform Director · Security · Repository Discovery

Mögliche Consumer

Documentary · Knowledge Graph · Traceability · Version Manager · Supervisor · Platform Director · Compliance · Security · Release

Befund: Von diesen neun potenziellen Producern führt aktuell nur `Traceability` tatsächlich befüllte `events`-Felder. Die übrigen acht führen `events: []`. Dies ist kein Mangel der Enterprise Event Mesh, sondern ein offener Punkt der jeweils eigenständigen Fachkomponenten — siehe Folgeentscheidungen in ADR-0018.

Komponenten-Integration (Sollzustand gemäß Chapter 8, hier konkretisiert)

Documentary Engine

Veröffentlicht mindestens: `DocumentationGeneratedEvent`, `DocumentationValidatedEvent`, `Twin-SynchronizedEvent`.

Präzisierung gegenüber der Erstfassung (Folgeentscheidung 2, ADR-0018): Die Manifest-Nachpflege ordnet `RepositoryScannedEvent` stattdessen `src/platform/Discovery` als tatsächlichem Producer zu — Discovery führt den technischen Scan aus, Documentary **konsumiert** `RepositoryScannedEvent` und löst darauf die Dokumentationserzeugung aus (konsistent mit Chapter 8, Documentary Integration: „Die Documentary Engine reagiert automatisch auf `RepositoryScannedEvent`“). Kein Widerspruch zur auslösenden Anforderung, die Documentary lediglich als eine der veröffentlichenden Instanzen nannte — die Detailzuordnung erfolgt hier erstmals granular.

Enterprise Traceability Matrix

Veröffentlicht: die bereits in `Traceability/manifest.json` benannten acht Events, insbesondere `TraceabilityBuildCompletedEvent` (deckt den in der Anforderung genannten Zweck von „`TraceabilityUpdatedEvent`“ ab) sowie `TraceabilityViolationEvent` (ESS-0012-CONTRACTS, produziert vom Governance Validator, nicht von der ETM selbst).

Knowledge Graph

Veröffentlicht: `KnowledgeUpdatedEvent`, `KnowledgeRelationCreatedEvent` (neu), `KnowledgeValidationCompletedEvent` (neu).

Enterprise Architecture Compliance Engine

Veröffentlicht: `ArchitectureValidatedEvent`, `ComplianceValidatedEvent`, `LayerViolationEvent`, `ContractViolationEvent` (beide bereits kanonisch, siehe Standard Event Catalog).

Version Manager

Veröffentlicht: `VersionCalculatedEvent`, `VersionApprovedEvent` (neu), `ReleasePreparedEvent`, `ReleasePublishedEvent`.

Supervisor

Veröffentlicht: `SupervisorAlertEvent`, `GovernanceViolationDetectedEvent` (produziert vom Governance Validator, vom Supervisor konsumiert und bei Bedarf eskaliert), `CriticalArchitectureViolationEvent` (neu).

Platform Director

Veröffentlicht: `PlatformDecisionEvent`, `RoadmapUpdatedEvent` (neu), `ArchitectureDecisionApprovedEvent` (neu).

Event Registry

`EventRegistry` führt je Event mindestens: Name, Kategorie, Version, Producer, Consumer, ESS-Referenz, ADR-Referenz, ETM-Referenz, Status (`proposed` / `registered` / `deprecated`).

Jedes Event wird bei Veröffentlichung automatisch registriert — eine manuelle Nachpflege der Registry ist nicht vorgesehen und widerspräche dem Prinzip, dass die Registry ausschließlich generiert wird (analog zur ETM-Regel aus ESS-0011).

Enterprise Traceability Matrix — Integration

Die Enterprise Event Mesh liefert die **Event-Achse** der bereits in ESS-0011 definierten sieben Traceability-Achsen (`ESS` · `ADR` · `Exception` · `Component` · `Interface` · `Event` · `Test`). Sie erzeugt selbst keine neue Achse und keine neue Verknüpfungsart — sie befüllt die bestehende Achse „Event“ mit Daten.

Die in der Anforderung beschriebene Kette

```
ESS → ADR → Event → Producer → Consumer → Komponente → Release → Version
```

ist eine Spezialisierung der bereits in ESS-0011 definierten Vorwärtskette (`ESS-Kapitel` → `ADR` → `Component` → `Interface` → `Event` → `Test`) für den Blickwinkel „ausgehend vom Event“. Beide Ketten sind konsistent: Producer und Consumer sind Komponenten im Sinne von ESS-0011, Release und Version sind die dort bereits vorgesehenen Endpunkte der Versionierungs-Traceability (ESS-0004).

Knowledge Graph — Integration

Jedes registrierte Event wird gemäß ESS-0009 automatisch Teil des Knowledge Graph als Knoten mit Kanten zu Producer- und Consumer-Komponenten. Diese Regel ist keine neue

Regel dieses Dokuments, sondern eine Anwendung der bereits in ESS-0009 definierten Graph-Aufbauregeln auf die neue Datenquelle „Event Registry“.

Digital Twin — Integration

Jedes Event aktualisiert den Digital Twin gemäß der bereits in ESS-0001-CONTRACTS Chapter 18 definierten `TwinSynchronizedEvent`-Kette. Dieses Dokument führt hierfür **kein** neues Event ein — `DigitalTwinUpdatedEvent` aus der auslösenden Anforderung ist durch `TwinSynchronizedEvent` bereits abgedeckt (siehe Standard Event Catalog).

AI Integration

Google AI Studio, Claude Code und ChatGPT müssen, sobald die Enterprise Event Mesh implementiert ist, dieselbe Mesh verwenden. Neue Komponenten dürfen ausschließlich über registrierte Enterprise Events kommunizieren — konkretisiert die bereits in Chapter 10 und Chapter 17 definierte KI-Governance für den Kommunikationskanal.

Validation

Vor jeder Integration eines Events wird geprüft (Erweiterung der zehnteiligen Chapter-8-Checkliste um ETM-Bezug):

- ✓ ESS kompatibel — Chapter 8 sowie ggf. domänenspezifisches Kapitel
 - ✓ ADR kompatibel — ADR-0018 sowie ggf. domänenspezifischer ADR
 - ✓ ETM integriert — Event-Achse befüllt
 - ✓ Documentary integriert — Event wird dokumentiert
 - ✓ Knowledge Graph integriert — Event wird zum Graph-Knoten
 - ✓ Version Manager integriert — Breaking-Change-Bewertung möglich
 - ✓ Supervisor integriert — Event-Fluss überwachbar
 - ✓ Platform Director integriert — Event-Governance anwendbar
 - ✓ Compliance Engine integriert — Event unterliegt Sicherheitsregeln aus Chapter 11
 - ✓ Enterprise Contracts erfüllt — alle Pflichtfelder aus Chapter 8 vorhanden
-

Implementierungsstand

Diese Komponente ist **implementiert (Stufe 1-4)** — als erste Komponente im gesamten `src/platform/`-Baum mit ausführbarem Code (ADR-0018, Folgeentscheidung 1, 2026-07-31).

Stufe	Voraussetzung	Zustand
1	JSON-Schemata für Event Contract	erledigt
2	Core, Interfaces, Models, Registry	erledigt
3	Enterprise Event Bus (Routing)	erledigt
4	Validator-Kette	erledigt

`Discovery/ManifestDiscovery.ts` fand zur Laufzeit 23 Komponenten unter `src/platform/` (mehr als die neun in der Vorab-Analyse identifizierten). Sieben Vertrags- und Kompatibilitätstests (`Tests/eventBus.test.ts`) bestehen.

Unverändert gültig bleibt: Ohne Knowledge Graph und Digital Twin (Stufe 5) kann kein Event vollständig gemäß der Validation-Checkliste (ETM-Referenz, Knowledge-Graph- Verknüpfung) durchlaufen — die Implementierung deckt den Zustellmechanismus ab, nicht die vorgelagerten Datenquellen. Dieselbe Abhängigkeitskette ist bereits für die ETM in ESS-0011 dokumentiert.

Notes

Vollständige Analyse: `docs/architecture/ENTERPRISE_EVENT_READINESS_REPORT.md` (vor dieser Spezifikation erstellt) und `docs/architecture/ENTERPRISE_EVENT_MESH_READINESS_REPORT.md` (danach erstellt).

Operativer KI-Skill: `.ai/skills/Enterprise-Event-Mesh.md`.

ESS-0014-CONTRACTS

Quelle: `.ai/skills/ESS-0014-Contracts.md`

ESS-0014-CONTRACTS

Google Marketing Integration & Protected Change Contracts

Version

1.0.0

Status

Enterprise Specification

Dokumentklassifizierung

Dieses Dokument definiert ausschließlich die Komponenten- und Sicherheitsverträge von ESS-0014. Repositoryweite IAM-, Event-, Release- und Dokumentationsregeln werden nicht dupliziert.

Bei Konflikt gelten in dieser Reihenfolge:

1. geltendes Recht und zwingende Provider-Anforderungen;
 2. ESS-0001-CONTRACTS für globale Enterprise Contracts;
 3. bestehende IAM-/Security-ADRs und ESS-0006;
 4. ESS-0014 für den Google-Marketing-Scope;
 5. dieses Dokument für die technische Contract-Ausprägung.
-

Cross Reference

Depends On

- ESS-0001-CONTRACTS
- ESS-0006 — Security & Compliance
- ESS-0007 — Enterprise Release Center
- ESS-0011 — Enterprise Traceability
- ESS-0012 — Documentation Governance
- ESS-0013 — Enterprise Event Mesh
- ESS-0014 — Google Marketing MCP Governance

Related ADR

- ADR-0003.5 — Owner IAM / Step-up

- ADR-0008 — Central IAM Authorization
- ADR-0009 — CORS/Security Hardening
- ADR-0035 — Protected Google Marketing Integration & Strict CSP

1. Principal Contract

```
type MarketingPrincipal =
  | { type: 'human'; userId: string; iamRole: 'owner' | 'admin' | 'supervisor' | 'user' }
  | { type: 'service_account'; serviceAccountId: string };
```

Service Accounts sind keine `Role` und dürfen `profiles.iam_role` nicht erweitern.

2. Capability Contract

```
type GoogleMarketingCapability =
  | 'google.marketing.read'
  | 'google.marketing.plan'
  | 'google.marketing.apply'
  | 'google.marketing.publish'
  | 'google.marketing.rollback'
  | 'google.marketing.credentials.rotate';
```

Ein Grant enthält mindestens:

```
interface MarketingCapabilityGrant {
  id: string;
  serviceAccountId: string;
  capability: GoogleMarketingCapability;
  resourceScope: string[];
  purpose: string;
  grantedByOwnerUserId: string;
  issuedAt: string;
  expiresAt: string;
  revokedAt?: string;
}
```

Contract

- Grant Create/Update/Revoke: Human OWNER + frischer Step-up.
- Service Account darf keinen Grant erzeugen, bearbeiten, verlängern oder widerrufen.
- `rollback` und `publish` werden niemals aus einem allgemeineren Recht abgeleitet.
- Capability Matching ist exact/fail-closed.

3. Protected Resource Contract

Initial geschützte Ressourcen:

```
index.html
public/cookiehub-init.js
public/google-analytics-consent.js
server.ts::Content-Security-Policy
```

```
CookieHub production configuration
GA4 production property/data stream
GTM production container/version
AdSense publisher/site configuration
Google Ads production customer/campaign resources
Google Marketing MCP/API credential configuration
```

Spätere Implementierungen dürfen den Scope über eine Registry erweitern, aber niemals implizit verkleinern.

4. Protected Change Classification Contract

Eine Änderung ist `protected`, wenn sie:

- einen geschützten Pfad entfernt;
- Consent früher/weiter öffnet;
- eine CSP-Direktive schwächt oder entfernt;
- Provider-/Property-/Publisher-/Customer-Identitäten ändert;
- ein IAM-/Approval-/Audit-Gate entfernt;
- eine zuvor behobene Sicherheits-/Consent-Regression wieder einführt;
- einen Rollback/Revert auf eine frühere Integrationsversion durchführt.

Der Classifier muss konservativ sein: bei Unsicherheit `protected=true`.

5. Impact Disclosure Contract

Vor interaktivem protected change MUSS der Agent mindestens dieses Schema präsentieren:

```
interface ProtectedChangeImpact {
    privacyConsent: string;
    analyticsMeasurement: string;
    adsMonetization: string;
    cspSecurity: string;
    complianceAudit: string;
    runtimeSeoAmp: string;
    affectedResources: Array<{
        resource: string;
        beforeFingerprint: string;
        afterFingerprint: string;
    }>;
}
```

Erst nach expliziter Bestätigung darf ein Approval-Prozess beginnen.

6. Explicit Confirmation Contract

Zulässig:

- eindeutige Bestätigung des konkret beschriebenen Diffs und der Auswirkungen;

- OWNER Approval Artifact für einen Headless-Service-Account-Job.

Nicht zulässig:

- aus einer früheren Unterhaltung abgeleitete Zustimmung;
- Schweigen;
- allgemeine Freigaben wie `mach weiter` ohne Bezug auf den Impact-Diff;
- ein Service Account, der sein eigenes Approval erzeugt;
- ein AI-Agent, der OWNER-Approval simuliert.

7. Human OWNER Authorization Contract

Ein Human protected change verlangt gleichzeitig:

```
verified Supabase identity
AND iam_role == owner
AND fresh one-time step-up
AND explicitConfirmation == true
AND non-expired approval context
AND expectedFingerprint matches live state
```

Fehlt ein Element: `DENY`.

8. Service Account Authorization Contract

Ein Service Account protected change verlangt:

```
verified service principal
AND active exact capability grant
AND target within resourceScope
AND grant not expired/revoked
AND expectedFingerprint matches live state
```

Für `google.marketing.rollback` zusätzlich:

```
one-time job-specific OWNER approval artifact
AND approval target == requested target
AND approval diff fingerprint == requested diff
AND approval unused
AND approval not expired
```

9. Approval Artifact Contract

```
interface ProtectedChangeApproval {
  id: string;
  requestId: string;
  actorType: 'human' | 'service_account';
  actorId: string;
  ownerUserId: string;
  ownerGrantId?: string;
  capability: GoogleMarketingCapability;
  targets: string[];
```

```

beforeFingerprints: string[];
afterFingerprints: string[];
impactSummaryHash: string;
reason: string;
explicitConfirmation: true;
issuedAt: string;
expiresAt: string;
usedAt?: string;
}

```

Approval wird atomar als benutzt markiert. Replay ist unzulässig.

10. Dry-Run Contract

Alle mutierenden Tools:

```

interface MutationRequest {
  dryRun: boolean; // default true
  expectedFingerprint: string;
  reason: string;
  approvalId?: string;
}

```

`dryRun=false` ohne vollständige AuthZ/Approval-Kette -> deny.

11. MCP Separation Contract

Official Google MCP

Google Analytics MCP und Google Ads MCP werden entsprechend dem jeweils dokumentierten Funktionsumfang verwendet. Ist der MCP read-only, bleibt er read-only.

CAPITAL-AI API adapters

Mutierende Provideroperationen laufen in getrennten, explizit benannten Adaptern. Ein MCP Tool darf einen solchen Adapter nur nach Policy-/IAM-/Approval-Gate aufrufen.

Claude

Claude ist Client/Host/Implementer, nicht Trust Root.

12. CSP Nonce Contract

Für production HTML:

```

nonce = cryptographicallyRandomPerResponse();

```

Pflichten:

- mindestens 128 Bit Entropie;
- Base64/Base64URL-safe Darstellung;

- neuer Nonce pro HTML Response;
- derselbe Request-Nonce im CSP Header und allen autorisierten Script Tags dieser Response;
- keine Speicherung als globaler Prozesswert;
- kein Logging des Nonce als Sicherheitsnachweis notwendig;
- statische Assets benötigen keinen eigenen dynamischen Nonce-Header.

AdSense-kompatibler Script-Core:

```
script-src 'nonce-{nonce}' 'unsafe-inline' 'unsafe-eval' 'strict-dynamic' https: http:
```

Zusätzlich:

```
object-src 'none'  
base-uri 'none'
```

13. CookieHub CSP Contract

Mindestens:

```
script/style: cookiehub.net cdn.cookiehub.eu  
connect: ds.cookiehub.net consent.cookiehub.net region-eu.cookiehub.net consent-eu.cookiehub.net  
cookiehub.net cdn.cookiehub.eu
```

Die tatsächlich benötigten Endpoints sind gegen die aktuelle CookieHub-Dokumentation zu validieren.

14. Consent Runtime Contract

Vor zulässigem Consent dürfen keine Analytics-/Marketing-Daten entgegen dem aktiven CAPITAL-AI-Consent-Modell übertragen werden.

Required Google Consent dimensions:

```
analytics_storage  
ad_storage  
ad_user_data  
ad_personalization
```

CookieHub ist die Source of Truth für die Nutzerentscheidung.

15. AMP Contract

`amp-auto-ads` darf nur als aktiv/funktional bewertet werden, wenn das Dokument die AMP-Validierungsanforderungen erfüllt.

Ein normales React/Vite-Dokument mit eingefügtem `amp-auto-ads` Element ist keine bestätigte AMP-Implementierung.

Tests und Dokumentation müssen zwischen `present in markup` und `valid AMP runtime` unterscheiden.

16. Audit Contract

Jeder protected change erzeugt mindestens:

```
ProtectedChangeRequestedEvent  
ProtectedChangeApprovedEvent | ProtectedChangeDeniedEvent  
ProtectedChangeAppliedEvent  
ProtectedChangeVerifiedEvent | ProtectedChangeVerificationFailedEvent
```

Eventnamen müssen vor Registrierung gegen ESS-0013 Event Catalog geprüft werden; existiert bereits ein semantisch gleiches Event, ist dieses wiederzuverwenden.

17. Rollback Contract

Rollback ist selbst eine destructive mutation und **kein Sonderweg um Approval zu umgehen**.

Rollback verlangt:

1. Impact Disclosure;
2. explicit confirmation;
3. OWNER+Step-up oder Service Account + exact rollback grant + one-time OWNER approval;
4. Dry-run diff;
5. Pre-change backup/fingerprint;
6. Apply;
7. Post-change verification;
8. Audit Evidence.

Emergency/Break-Glass darf die OWNER-/Audit-Anforderung nicht abschaffen; ein separater Break-Glass-ADR/Mechanismus kann nur den operativen Ablauf beschleunigen.

18. Test Contract

Pflichttests:

- role denial matrix;
- service-account capability matrix;
- approval expiry/replay;

- fingerprint race/mismatch;
 - per-response nonce uniqueness;
 - CSP header/script nonce parity;
 - CookieHub banner initialisation;
 - no analytics before consent;
 - AdSense CSP compatibility smoke test;
 - AMP markup presence plus separate AMP-validity test;
 - audit evidence completeness.
-

19. Traceability Contract

ESS-0014 muss mindestens zu folgenden Artefakten bidirektional verlinkt werden:

- ADR-0035;
 - CSP implementation;
 - consent runtime;
 - IAM/step-up;
 - service account grant implementation;
 - Google Marketing MCP/API architecture;
 - relevant tests;
 - production evidence.
-

End of ESS-0014-CONTRACTS

ESS-0014 — Google Marketing MCP Governance

Quelle: `.ai/skills/ESS-0014-Google-Marketing-MCP-Governance.md`

ESS-0014 — Google Marketing MCP Governance

Enterprise Specification

Version: 1.0.0

Status: Enterprise Specification

Owner: Platform Director

Security Authority: CAPITAL-AI IAM / Security & Compliance

Related ADR: ADR-0035

Related ESS: ESS-0003, ESS-0006, ESS-0007, ESS-0011, ESS-0012, ESS-0013

Implementation profile: `docs/architecture/CAPITAL_AI_GOOGLE_MARKETING_MCP_ARCHITECTURE.md`

1. Zweck

ESS-0014 definiert die verbindliche Enterprise-Architektur für die Integration von Google-Marketing-Systemen in CAPITAL-AI. Die Spezifikation ist bewusst **modell-, host- und transportneutral**. Claude Code, Claude Desktop, Gemini, ein CAPITAL-AI-Agent oder ein anderer freigegebener MCP-Host sind Implementierungs- oder Ausführungsinstanzen, aber niemals die normative Autorität.

Die Spezifikation umfasst insbesondere:

- Google Analytics 4;
- Google Analytics Data API und Google Analytics Admin API;
- Google Analytics MCP;
- Google Ads API und Google Ads MCP;
- Google Tag Manager API;
- Google AdSense / AdSense Management API;
- Google Consent Mode v2;
- AMP-/AdSense-Auslieferung, sofern eine echte AMP-Dokumentvariante betrieben wird;
- CookieHub als Consent Source of Truth;
- Credential-, IAM-, Delegations-, Approval-, Audit- und Rollback-Governance;
- die Trennung zwischen Google-MCP-Read-Plane und mutierenden Google-API-Control-Plane-Adaptern.

Nicht Bestandteil dieses ESS sind allgemeine repositoryweite IAM-Regeln, globale Event-Namensregeln oder allgemeine Release-Regeln. Diese bleiben in den jeweils zuständigen ESS/Contracts.

2. Normative Grundsätze

1. **CookieHub bleibt Consent Source of Truth.** Google-interne Default-Consent-Konfiguration darf diese Entscheidung nicht stillschweigend überschreiben.
2. **Zero Google data before consent** bleibt für browserseitige Analytics-/Marketing-Daten die Privacy-Invariante, solange kein späteres akzeptiertes ADR ausdrücklich einen anderen Rechts-/Architekturzustand beschließt.
3. **Basic Consent Mode v2** ist der Standard für CAPITAL-AI. Advanced Consent Mode benötigt ein separates ADR mit Datenschutz-/Rechtsbewertung.
4. **Read und Write werden getrennt.** Offizielle Google-MCP-Server werden nur in dem Funktionsumfang genutzt, den Google tatsächlich bereitstellt. Mutationen erfolgen über kontrollierte CAPITAL-AI-Adapter gegen die zuständigen Google APIs.
5. **No hidden rollback.** Kein Agent, MCP-Tool, Refactoring, Drift-Reconciler, CI-Fix oder Dependency-Upgrade darf geschützte Marketing-/Consent-Sicherheitsmechanismen stillschweigend zurückbauen.
6. **Fail closed.** Ist Identität, Rolle, Service-Account-Grant, Step-up, Ressourcenzuordnung, Consent-Zustand, Approval-Artefakt oder Ziel-Fingerprint unklar, wird die Mutation abgelehnt.
7. **Least privilege.** Google OAuth Scopes, Service-Account-Capabilities und API-Operationen werden auf das kleinste erforderliche Recht begrenzt.
8. **Dry-run first.** Jede externe Mutation erstellt zuerst einen Plan/Diff. Produktionswrites sind niemals die Default-Aktion.
9. **Auditability.** Jede privilegierte Änderung erhält Actor, Principal-Typ, Zielressource, vorherigen/nachherigen Fingerprint, Approval-Referenz und Verifikationsergebnis.
10. **No secret in client.** OAuth Refresh Tokens, Client Secrets, Developer Tokens, Service-Account-Secrets und MCP-Admin-Credentials bleiben serverseitig.

3. Systemgrenzen

3.1 Browser Data Plane

Der Browser darf ausschließlich öffentliche IDs und consent-gesteuerte Runtime-Tags erhalten.

```
Browser
|
|--> CookieHub CMP
|   |
|   |--> necessary
|   |--> analytics
|   |--> marketing
```

```

|
+--> GA4 / Google tag      [nur entsprechend Consent]
+--> AdSense               [nur entsprechend Marketing-/CMP-Policy]
+--> AMP Auto Ads          [nur auf echter AMP-Dokumentvariante]

```

3.2 Google MCP Read Plane

```

Approved MCP Host
|
+--> Google Analytics MCP --> read-only reporting
+--> Google Ads MCP       --> read-only discovery/reporting

```

Die offiziellen Google Analytics- und Google Ads-MCP-Server werden als **read-only Evidence Plane** behandelt, solange Google deren dokumentierten Umfang nicht ausdrücklich erweitert.

3.3 CAPITAL-AI Mutation Control Plane

```

Claude / approved MCP host
|
v
CAPITAL-AI Marketing Control Plane
+-- Policy Gate
+-- IAM Gate
+-- Approval Gate
+-- Dry-Run Planner
+-- Idempotency / Drift Guard
+-- Audit Evidence
|
+--> GA Admin API
+--> GTM API
+--> Google Ads API (nur freigegebene Write-Adapter)
+--> AdSense Management API (nur freigegebene Operationen)

```

Der Control Plane darf niemals zu einem generischen, uneingeschränkten Google-API-Proxy werden.

4. Geschützte Integrationsinvarianten

Folgende Elemente sind `PROTECTED_MARKETING_INVARIANTS` :

- CookieHub Loader in `index.html` ;
- `public/cookiehub-init.js` ;
- CookieHub Consent-Kategorien und deren Google-Mapping;
- `public/google-analytics-consent.js` bzw. ein späterer gleichwertiger consent-gated Loader;
- CSP-Nonce-/Strict-CSP-Mechanismus für Google-Marketing-Skripte;
- AdSense Publisher ID und deren kontrollierte Auslieferung;
- Google Consent Mode v2 Defaults/Updates;
- TCF-/CMP-Konfiguration, sofern für EWR/UK/CH und personalisierte Anzeigen relevant;
- Google-Marketing Credential Boundary;
- OWNER-/Service-Account-Approval Gate;

- Audit-/Traceability-Nachweise;
- zukünftige MCP Policy Gate- und Approval Gate-Komponenten.

Ein technisch äquivalenter Ersatz ist zulässig, gilt aber als Architekturänderung und benötigt ADR + Evidence.

5. Protected Change / Rollback Protocol

5.1 Definition

Als Rollback oder protected change gilt jede Änderung, die mindestens eine der folgenden Wirkungen hat:

- Entfernen oder Deaktivieren von CookieHub;
- Rückkehr zu CSP-blockiertem Inline-Consent-Code;
- Entfernen/Abschwächen der CSP-Nonce-/Strict-CSP-Struktur;
- Laden von Analytics/Ads vor der zulässigen Consent-Stufe;
- Entfernen von Consent Mode Signalen;
- Entfernen von Audit-, Approval- oder IAM-Gates;
- Wechsel der Google Property, AdSense Publisher ID, Google Ads Customer ID oder GTM Container Identity;
- Senken des Schutzlevels einer geschützten Komponente;
- Revert eines Commits, der eine geschützte Invariante eingeführt oder repariert hat.

5.2 Pflicht-Nachfrage

Ein interaktiver Agent MUSS vor einem Rollback stoppen und eine explizite Bestätigung anfordern. Die Nachfrage muss die konkreten Auswirkungen nennen, mindestens:

1. Datenschutz/Consent-Auswirkung;
2. Analytics-/Messauswirkung;
3. AdSense-/Monetarisierungsauswirkung;
4. CSP-/XSS-Sicherheitsauswirkung;
5. Compliance-/Audit-/Traceability-Auswirkung;
6. mögliche Produktions-/SEO-/AMP-Auswirkung;
7. betroffene Dateien/Ressourcen und aktueller vs. geplanter Fingerprint.

Eine generische Anweisung wie `fix`, `cleanup`, `restore old implementation`, `make it work`, `revert last changes` oder `simplify CSP` ist **keine** ausreichende Rollback-Freigabe.

5.3 Human OWNER

Ein Human-Principal darf einen protected change nur ausführen, wenn:

- `profiles.iam_role === 'owner'` verifiziert ist;

- ein frischer TOTP Step-up vorliegt;
- eine explizite Auswirkungsbestätigung vorliegt;
- ein Dry-Run-/Diff-Artefakt erzeugt wurde;
- ein Reason/Change-Ticket angegeben ist.

`admin`, `supervisor`, `user` oder Subscription-Tiers erhalten dadurch niemals implizite Berechtigung.

5.4 Service Accounts

Service Accounts sind **keine Human-IAM-Rolle** und dürfen nicht als `owner` in `profiles.iam_role` modelliert werden.

Sie bilden einen getrennten Principal-Typ mit OWNER-delegierten Capabilities. Zulässige Capabilities:

- `google.marketing.read`
- `google.marketing.plan`
- `google.marketing.apply`
- `google.marketing.publish`
- `google.marketing.rollback`
- `google.marketing.credentials.rotate`

Regeln:

1. Nur ein verifizierter Human OWNER mit frischem TOTP Step-up darf Grants anlegen, ändern, verlängern oder widerrufen.
2. Ein Service Account darf seine eigenen Rechte niemals ändern.
3. Ein Service Account darf Rechte niemals an andere Principals delegieren.
4. Grants sind resource-scoped, zweckgebunden und zeitlich begrenzt.
5. `google.marketing.rollback` benötigt zusätzlich ein **job-spezifisches, einmaliges OWNER Approval Artifact**; ein stehender Grant allein reicht nicht.
6. Abgelaufene, widerrufene oder uneindeutige Grants werden fail-closed behandelt.
7. Credential-Rotation darf keine Capability-Eskalation bewirken.

5.5 Approval Artifact

Ein protected change führt mindestens:

```
approvalId
requestId
actorType = human | service_account
actorId
ownerGrantId?           # für Service Account
capability
targetResources[]
currentFingerprints[]
proposedFingerprints[]
impactSummary
explicitConfirmation = true
reason
```

```
issuedAt
expiresAt
usedAt?
```

Das Approval Artifact ist single-use für destructive/rollback actions.

6. IAM-Integration

Bestehende CAPITAL-AI-Bausteine werden wiederverwendet:

- `checkAdminAccess(..., OWNER_ONLY_ROLES)` für Human OWNER;
- `requireStepUp()` für kritische Owner-Aktionen;
- `logIamEvent()` / IAM Audit Logging;
- fail-closed bei nicht verfügbarem IAM-Schema.

Service-Account-Grants werden als getrennte Capability Registry implementiert und dürfen die Human-Rollentabelle nicht umgehen.

7. CSP Contract für Google Marketing

7.1 AdSense

AdSense wird mit **Strict CSP und pro Response zufälligem Nonce** integriert. Der Nonce wird serverseitig erzeugt, in den CSP-Header geschrieben und auf alle `<script>`-Tags der HTML-Antwort übertragen.

Kein statischer Nonce. Kein wiederverwendeter Nonce. Kein `unsafe-inline` als alleinige Lösung.

Google-kompatibler Kern:

```
object-src 'none';
base-uri 'none';
script-src 'nonce-{REQUEST_NONCE}' 'unsafe-inline' 'unsafe-eval' 'strict-dynamic' https: http;;
```

`unsafe-inline` ist in CSP3-Browsern bei vorhandenem Nonce/strict-dynamic nicht die Vertrauenswurzel; `unsafe-eval` bleibt ein dokumentierter AdSense-Kompatibilitäts-Trade-off und muss im ADR nachvollziehbar bleiben.

7.2 CookieHub

Mindestens:

```
script-src: cookiehub.net cdn.cookiehub.eu
style-src: cookiehub.net cdn.cookiehub.eu
connect-src: ds.cookiehub.net consent.cookiehub.net region-eu.cookiehub.net consent-
eu.cookiehub.net cookiehub.net cdn.cookiehub.eu
```

7.3 Analytics

GA4-/Google-Tag-Netzwerkziele werden explizit im `connect-src` / `img-src` -Kontext berücksichtigt. Die Browser-Runtime bleibt consent-gated.

7.4 CSP Drift

Eine CSP-Änderung, die einen geschützten Integrationspfad blockiert oder den Schutzlevel reduziert, ist ein protected change und fällt unter Abschnitt 5.

8. Consent Contract

Consent-Signale:

- `analytics_storage`
- `ad_storage`
- `ad_user_data`
- `ad_personalization`

CookieHub übersetzt die Nutzerentscheidung in die zulässigen Google-Signale. Vor Zustimmung werden Analytics-/Marketing-Tags nach dem CAPITAL-AI Basic-Mode-Prinzip nicht aktiv zur Datenerhebung genutzt.

Bei AdSense in EWR/UK/CH müssen die jeweils aktuellen Google-Publisher- und CMP-Anforderungen einschließlich eines ggf. erforderlichen IAB-TCF-Profiles im Production Handoff geprüft werden.

9. AMP Contract

`amp-auto-ads` ist nur auf Dokumenten zulässig, die als AMP-Dokumente valide sind.

Die Präsenz von:

```
<script async custom-element="amp-auto-ads" src="https://cdn.ampproject.org/v0/amp-auto-ads-0.1.js"></script>
```

und:

```
<amp-auto-ads type="adsense" data-ad-client="ca-pub-1353017943074018"></amp-auto-ads>
```

macht eine React/Vite-SPA **nicht** zu einer AMP-Seite. Vor produktiver Nutzung von AMP Auto Ads ist eine eigenständige AMP-Route/-Dokumentvariante zu validieren. Normale AdSense Auto Ads bleiben davon getrennt.

10. Google MCP Federation

10.1 Official Read Plane

- Google Analytics MCP: Reporting/Data-API Read Plane.
- Google Ads MCP: aktuell Read-only für Discovery, GAQL/Reporting und Ressourcenmetadaten.

10.2 Write Plane

Writes erfolgen nur über explizite CAPITAL-AI Adapter gegen die zuständigen APIs. Ein read-only Google MCP darf nicht durch inoffizielle Seiteneffekte in einen Write-Pfad verwandelt werden.

10.3 Claude

Claude kann:

- MCP Host;
- Implementierungsagent;
- Planner;
- Reviewer;
- Evidence Generator

sein.

Claude kann **nicht**:

- OWNER-Autorisierung ersetzen;
- Step-up ersetzen;
- ein Approval Artifact selbst genehmigen;
- Service-Account-Rechte selbst erweitern;
- geschützte Invarianten ohne Nachfrage zurückbauen.

11. Tool Contract

Jedes mutierende MCP/API-Tool besitzt mindestens:

```
dryRun: boolean = true
approvalId?: string
expectedFingerprint?: string
reason?: string
```

Destructive Tools zusätzlich:

```
requireExplicitConfirmation: true
requiredCapability: google.marketing.rollback | google.marketing.publish
```

Fehlt ein erwarteter Fingerprint oder weicht der Live-Zustand ab, muss neu geplant werden.

12. Audit- und Evidence Contract

Pflicht-Evidence je Mutation:

- actor/principal;
- authz decision;
- Step-up/Grant/Approval Reference;
- dry-run diff;
- API request class (keine Secrets);
- before/after fingerprint;
- result;
- post-change verification;
- rollback readiness;
- Traceability Links zu ESS/ADR/Test/Change.

Secrets und vollständige OAuth Tokens sind von Evidence ausgeschlossen.

13. Tests

Mindestens:

1. OWNER ohne Step-up -> deny;
 2. ADMIN/SUPERVISOR/USER -> deny protected change;
 3. OWNER + Step-up, aber ohne explicit confirmation -> deny;
 4. Service Account ohne Capability -> deny;
 5. Service Account mit allgemeinem apply Grant, aber ohne rollback Capability -> deny rollback;
 6. Service Account mit rollback Capability, aber ohne one-time OWNER approval -> deny;
 7. abgelaufenes Approval -> deny;
 8. Fingerprint mismatch -> deny;
 9. CookieHub init bleibt unter CSP funktionsfähig;
 10. AdSense Script trägt Request Nonce;
 11. Nonce ändert sich pro HTML-Response;
 12. CSP enthält `object-src 'none'`, `base-uri 'none'`, `strict-dynamic`;
 13. Analytics bleibt vor Consent ohne Datenerhebung;
 14. Rollback-Versuch erzeugt Audit Evidence.
-

14. Stop Conditions

Jeder Agent/Host stoppt vor Mutation, wenn:

- die Anweisung einen protected change impliziert, aber keine explizite Bestätigung enthält;
 - Auswirkungen nicht benannt wurden;
 - Human Actor nicht OWNER ist;
 - Step-up fehlt;
 - Service-Account-Grant fehlt/abgelaufen ist;
 - one-time Approval für Rollback fehlt;
 - Live Resource Identity nicht eindeutig ist;
 - Google API-/MCP-Funktionsumfang unklar ist;
 - Consent-Auswirkung nicht verifiziert werden kann.
-

15. Offizielle Referenzen (Stand 2026-08-03)

- Google AdSense Strict CSP: <https://support.google.com/adsense/answer/16283098>
- Google Analytics MCP: <https://developers.google.com/analytics/devguides/reporting/data/v1/mcp>
- Google Ads MCP: <https://developers.google.com/google-ads/api/docs/developer-toolkit/mcp-server>
- CookieHub CSP: <https://docs.cookiehub.com/advanced/csp>

Herstellerdokumentation ist bei jeder Implementierung erneut auf Aktualität zu prüfen.

16. Definition of Done

ESS-0014 gilt als vollständig umgesetzt, wenn:

- Strict CSP mit pro-Response Nonce produktiv verifiziert ist;
- CookieHub/Analytics/AdSense ohne CSP-Verletzung gemäß Zielzustand funktionieren;
- protected change/rollback Gate als ausführbarer Control-Plane-Mechanismus existiert;
- OWNER + Step-up durchgesetzt wird;
- Service-Account Capability Registry mit OWNER-only Grant Management implementiert ist;
- one-time Approval Artifacts für Rollbacks erzwungen werden;
- Google MCP Read Plane und mutierende API Adapter getrennt sind;
- Audit/Traceability/Tests vorhanden sind;
- Produktions-Evidence vorliegt.

Bis dahin bleibt der Implementation-Status `IN PROGRESS` , auch wenn das ESS selbst `published` ist.

ESS-0015 — API Interface Inventory & QA Governance

Quelle: `.ai/skills/ESS-0015-API-Interface-Inventory-QA.md`

ESS-0015 — API Interface Inventory & QA Governance

Status: PUBLISHED

Version: 1.0.0

Owner: Platform Director

Scope: Alle internen und externen API-/Webhook-/SDK-/MCP-/SMTP-/Data-API-Schnittstellen von CAPITAL-AI

Decision Gate: Neue oder geänderte FinTech-API-Anbindungen dürfen erst nach vollständigem Inventory- und QA-Evidence-Review architektonisch freigegeben werden.

1. Zweck

Diese Enterprise Specification führt eine verbindliche, evidenzbasierte Inventarisierung sämtlicher Schnittstellen ein. Ziel ist nicht nur Dokumentation, sondern eine reproduzierbare QA-Prüfung von Sicherheit, Datenqualität, Verfügbarkeit, Kosten, Rate Limits, Authentifizierung, Datenschutz, Observability, Failover und fachlicher Eignung. Das Ergebnis bildet die Entscheidungsgrundlage für eine nachgelagerte ADR zur zukünftigen FinTech-Provider- und Routing-Architektur.

2. Inventar-Pflichtfelder

Jede Schnittstelle MUSS mindestens enthalten:

- `interface_id` — stabile ID, z. B. `API-MARKET-BINANCE-001`
- `provider` und Produkt/Service
- `category` — market-data, ai, payments, database, auth, email, marketing, observability, internal-api, webhook, mcp
- `direction` — inbound, outbound, bidirectional
- `transport` — REST, WebSocket, SDK, SMTP, PostgREST, Edge Function, webhook, MCP
- `runtime_owner` — Render backend, browser, Supabase, external platform
- `code_entrypoints` — konkrete Dateien/Module/Routen
- `public_endpoints` und interne Route-Mappings
- `authentication` — API key, OAuth, JWT, signature, publishable key, service-role, none
- `secret_location` — nur Variablennamen/Secret Store, niemals Secret-Werte
- `data_classes` — market, PII, billing, auth, audit, AI prompt/output usw.
- `asset_classes` bzw. fachlicher Use Case
- `source_of_truth` und Provenance-Status
- `rate_limit` / Quota / bekannte Provider-Limits

- `timeout` , retry, circuit-breaker und fallback
- `cache` / freshness / staleness policy
- `observability` — Logs, Metrics, Correlation ID, Alerts
- `security_controls` — allowlist, schema validation, RLS, signature verification usw.
- `compliance` — DSGVO, MiFID-II-Relevanz, Datenresidenz, Aufbewahrung
- `cost_model` — kostenlos, subscription, per-request, token-basiert
- `sla_slo` — bekannte Verfügbarkeit und internes Ziel
- `qa_status` — NOT_TESTED / FAIL / CONDITIONAL / PASS
- `decision_status` — KEEP / REPLACE / CONSOLIDATE / DEPRECATE / EVALUATE
- `evidence_refs` — Tests, Logs, ADRs, Supabase-Migrationen, CI-Runs

Fehlt ein Pflichtfeld, ist die Schnittstelle nicht decision-ready.

3. Aktuell nachgewiesene Provider-/Schnittstellenfamilien

Der initiale Inventory-Seed muss mindestens folgende im Repository bzw. in der produktiven Supabase-Instanz nachgewiesene Familien erfassen:

3.1 FinTech-/Marktdaten

- Binance Spot Public Market Data — Landing-Kurzanalyse, serverseitig
- CoinGecko — Crypto Snapshot/History/Consensus
- Financial Modeling Prep (FMP) — Index-/Traditional-Quote-Evidence
- Alpha Vantage — Stock Fundamentals / Provider Registry
- weitere im `marketDataProviderRegistry` registrierte oder dokumentierte Provider
- Alpaca — als Kandidat erst dann `ACTIVE` , wenn Code-/Runtime-Evidence vorliegt; reine Render-Secret-Existenz genügt nicht

3.2 AI Provider

- Anthropic Claude API
- OpenAI API
- Google Gemini API
- Multi-Provider Routing `Anthropic -> OpenAI -> Gemini`

3.3 Billing / Payment

- Stripe API
- Stripe Webhooks in Render
- produktive Supabase Edge Functions `stripe-setup` , `stripe-webhook` , `stripe-worker`

3.4 Database / Auth / Platform APIs

- Supabase Auth
- Supabase PostgREST/Data API

- Supabase Edge Functions
- Supabase server-side service-role access

3.5 Messaging / sonstige externe Schnittstellen

- IONOS SMTP / Mailer
- Google OAuth
- Google Marketing-/Analytics-Integration gemäß ESS-0014
- Webscan Radar Badge nur als externe UI-Ressource, nicht als FinTech-Datenquelle

4. QA-Matrix je Schnittstelle

Für jede Interface-ID MUSS QA mindestens folgende Testklassen durchführen und Evidence speichern:

1. **Contract QA** — Request-/Response-Schema, Pflichtfelder, Typen, Versionierung.
2. **Authentication QA** — positive/negative Auth-Fälle, Secret Exposure, Token Scope, Signature-Verifikation.
3. **Authorization QA** — BOLA/IDOR, Rollen, RLS, Service-Role-Grenzen.
4. **Data Quality QA** — Nulls, Ausreißer, Units, Currency, Timestamp, Duplicate, Staleness, Provenance.
5. **No-Demo-Data QA** — bei Provider-Ausfall keine erfundenen Finanzdaten.
6. **Resilience QA** — Timeout, 429, 4xx, 5xx, malformed payload, DNS/network failure.
7. **Rate-Limit QA** — internes Throttling, Retry-After, Backoff, Provider-Quota.
8. **Performance QA** — p50/p95/p99, cold/warm cache, Payload-Größe.
9. **Observability QA** — Correlation ID, strukturierte Logs, Metrics, Alertbarkeit.
10. **Security QA** — SSRF, injection, CORS, CSP-Relevanz, secret scanning, replay bei Webhooks.
11. **Privacy/Compliance QA** — PII-Minimierung, Datenfluss, Retention, DPA/Region soweit relevant.
12. **Cost QA** — Kosten pro Use Case, Worst-Case-Spend, Budget-/Quota-Schutz.
13. **Failover QA** — Priorität, semantische Gleichwertigkeit des Fallbacks, Degradation-Label.
14. **Business Correctness QA** — Eignung für Asset-Klasse, Lizenz-/Nutzungsrestriktionen, Screening-Auswirkungen.

5. Mindest-Evidence für PASS

`qa_status=PASS` ist nur zulässig, wenn:

- automatisierte Contract-/Unit-Tests existieren,
- mindestens ein negativer Auth-/Failure-Test existiert,

- reale Provider-Evidence oder ein deterministischer Mock ausschließlich für Transporttests verwendet wird,
- No-Demo-Data-Verhalten nachgewiesen ist,
- Timeout und Rate-Limit-Verhalten geprüft sind,
- keine High/Critical Security Finding offen ist,
- Source/Provenance in der API-Antwort oder internen Evidence nachvollziehbar bleibt,
- CI den Test auf dem zu entscheidenden Commit erfolgreich ausgeführt hat.

Mocks dürfen niemals als Evidence für reale Marktqualität oder Provider-Verfügbarkeit gelten.

6. Supabase-spezifische QA

Für jede Supabase-exponierte Ressource gilt zusätzlich:

- RLS-Status und Policies inventarisieren.
- `anon`, `authenticated`, `service_role` und Edge-Function-Zugriffe getrennt dokumentieren.
- Edge Functions mit `verify_jwt=false` benötigen eine explizite Begründung und einen alternativen Authentizitätsnachweis im Funktionscode (z. B. Stripe-Signatur). Ohne Evidence: `qa_status=FAIL`.
- Security- und Performance-Advisors werden als QA-Evidence ausgeführt.
- Änderungen an DB/Policies/Functions benötigen Migration-/Deployment-Evidence.

Produktiver Snapshot 2026-08-03: `stripe-setup`, `stripe-webhook`, `stripe-worker` sind aktiv und melden `verify_jwt=false`; diese drei Funktionen sind daher priorisierte QA-Objekte, nicht automatisch Findings.

7. API Inventory Artefakte

Verbindliche Ablage:

- `docs/architecture/api/API_INTERFACE_INVENTORY.md` — menschenlesbare Übersicht
- `docs/architecture/api/API_INTERFACE_INVENTORY.json` — maschinenlesbare Registry
- `docs/architecture/api/API_QA_REPORT.md` — QA-Ergebnisse und Findings
- `docs/architecture/api/API_PROVIDER_DECISION_MATRIX.md` — Entscheidungsmatrix
- `tests/api/` — Contract-/Integration-/Failure-/Security-Tests

Die JSON-Registry ist die kanonische Maschinenquelle. Markdown wird daraus oder synchron dazu gepflegt.

8. Decision Matrix für FinTech Provider

Jeder FinTech-/Market-Data-Provider wird nach einem gewichteten Modell bewertet:

- Datenqualität/Abdeckung: 25 %
- Verfügbarkeit/Resilience: 15 %

- Provenance/Freshness: 15 %
- Security/Compliance: 15 %
- Kosten/Quota: 10 %
- Integrationskomplexität: 10 %
- Vendor Lock-in/Portabilität: 5 %
- Observability/Support: 5 %

Score allein entscheidet nicht. Ein Critical Security Finding, unklare Datenlizenz oder fehlende Provenance ist ein Hard Gate.

9. Architecture Decision Gate

Nach Abschluss des Inventars und QA-Reports MUSS eine neue ADR die zukünftige Anbindung aller FinTech-Schnittstellen entscheiden. Sie muss mindestens zwischen folgenden Mustern abwägen:

- direkte Provider-Adapter je Domain
- zentraler Market Data Gateway
- bestehendes `marketDataProviderRegistry` als verbindliche Provider Control Plane
- MCP nur als Tool-/Agent-Integrationsschicht versus Runtime-Market-Data-Plane
- Primary/Secondary/Consensus Provider je Asset-Klasse
- Cache-/Evidence-Layer und Event-Mesh-Anbindung
- einheitliche Rate-Limit-, Cost- und Circuit-Breaker-Policies

Bis zur ADR-Entscheidung dürfen neue FinTech-Provider nur als `EVALUATE` aufgenommen werden; keine parallele Schattenarchitektur.

10. QA-Abnahmekriterien für die nachgelagerte ADR

Die neue Provider-ADR darf erst `ACCEPTED` werden, wenn:

- 100 % der produktiv verwendeten externen APIs inventarisiert sind,
- 100 % der FinTech-/Market-Data-APIs einen QA-Status besitzen,
- alle Critical/High Findings geschlossen oder formal akzeptiert sind,
- Provider-Duplikate und Funktionsüberschneidungen markiert sind,
- reale Asset-Abdeckung pro Provider dokumentiert ist,
- Failover- und Kostenpfade gemessen wurden,
- Supabase-/Stripe-Schnittstellen als abhängige Plattform-APIs mitbewertet wurden,
- CI-Evidence auf dem finalen Decision Commit vorliegt.

11. Verantwortlichkeiten

- **Platform Director:** finale Architekturentscheidung und Ausnahmefreigaben.
- **Quality Center / QA:** Testmatrix, Evidence, Regression Gates.

- **Security & Compliance:** Auth, RLS, Secret, Privacy, Webhook- und Supply-Chain-Prüfung.
- **Documentary:** Inventory/Reports/Traceability synchron halten.
- **Supervisor:** verhindert Aktivierung nicht freigegebener Provider-Pfade, sobald technisch implementiert.
- **Claude Production Integration:** Produktionskonfiguration und Provider-Secrets gemäß Handoff, keine Secrets im Repository.

12. Definition of Done

ESS-0015 ist operational erfüllt, wenn Inventory JSON + Markdown vollständig, QA automatisiert, Findings triagiert und die nachgelagerte FinTech API Architecture ADR mit gemessener Evidence entschieden ist.

ESS-0016 — Enterprise Market Data Provider & MCP Governance

Quelle: `.ai/skills/ESS-0016-Enterprise-Market-Data-Provider-MCP-Governance.md`

ESS-0016 — Enterprise Market Data Provider & MCP Governance

Enterprise Specification

Version: 1.0.0

Status: Enterprise Specification

Implementation Status: PHASE 3C CONTRACT AND ADAPTER IMPLEMENTED / SCORING WIRING NOT AUTHORIZED / ALPACA PROMOTION NOT AUTHORIZED

Owner: Platform Director

Security Authority: CAPITAL-AI IAM / Security & Compliance

Related ADR: ADR-0041

Related ESS: ESS-0003, ESS-0006, ESS-0008, ESS-0011, ESS-0013, ESS-0014, ESS-0015

Primary Provider: Alpaca Market Data

Secondary / Enrichment Provider: Alpha Vantage

1. Zweck

ESS-0016 definiert die verbindliche Enterprise-Architektur für externe Finanzmarktdaten in CAPITAL-AI. Ziel ist eine providerneutrale, auditierbare und ausfallsichere FinTech Data Pipeline, in der Markt-, Referenz- und Enrichment-Daten nicht direkt an einzelne Drittanbieter-Schemas gekoppelt werden.

Die Spezifikation trennt ausdrücklich zwei technische Pfade:

1. **deterministische Runtime Data Plane** für Screening, Scoring, Backtesting, Monte-Carlo und weitere produktive Berechnungen;
2. **governed MCP Agent Plane** für kontrollierte agentische Recherche, Erklärung, Evidence-Erhebung und Supervisor-gestützte Tool-Nutzung.

MCP ist damit eine Agent-/Tool-Integration und **kein Ersatz** für die deterministische Market-Data-Pipeline.

2. Architekturentscheidung auf ESS-Ebene

CAPITAL-AI verwendet folgende Provider-Rollen:

2.1 Alpaca — Primary Market Data Provider

Alpaca ist der bevorzugte Provider für unterstützte Echtzeit- und historische Marktdaten, insbesondere:

- US Equities;
- Quotes und Trades;
- OHLCV Bars;
- Snapshots;
- Crypto Market Data;
- Options- und Option-Chain-Daten;
- Corporate Actions, sofern für die jeweilige Pipeline benötigt;
- WebSocket-basierte Streaming-Daten, sofern der konkrete Produktionsplan und die Datenlizenz dies erlauben.

2.2 Alpha Vantage — Secondary / Enrichment Provider

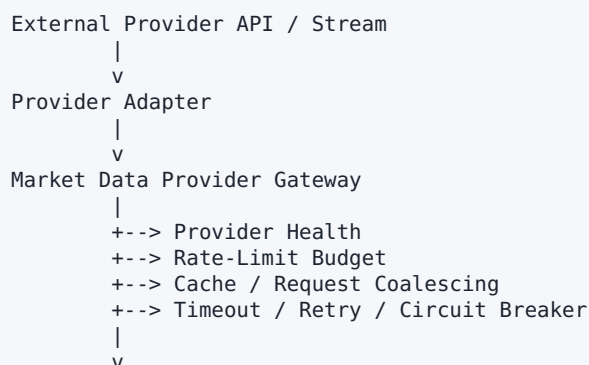
Alpha Vantage wird als ergänzender Provider vorgesehen für:

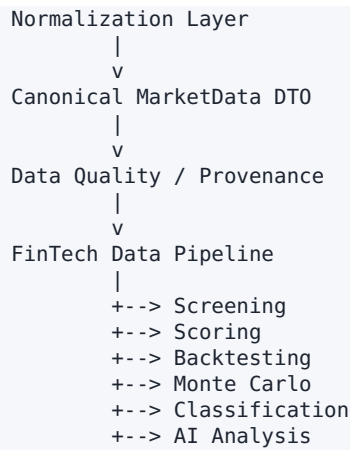
- Fundamentals und Company Data;
- FX;
- Commodities;
- globale Equity-Daten, soweit vom verwendeten Tarif gedeckt;
- technische Indikatoren als unabhängige Referenz bzw. Enrichment;
- agentische Research- und Evidence-Abfragen über den offiziellen Alpha-Vantage-MCP-Server;
- Cross-Provider-Validierung.

Alpha Vantage darf aufgrund tarifabhängiger Request-Limits und Datenentitlements nicht als ungeprüfte alleinige Runtime-Abhängigkeit für die gesamte produktive Screening-Pipeline vorausgesetzt werden.

3. Verbindliche Systemgrenzen

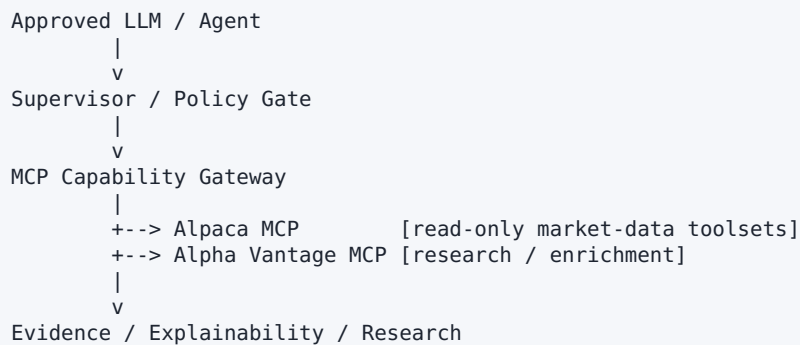
3.1 Runtime Data Plane





Produktive Business-Services dürfen nicht direkt gegen ein Alpaca-, Alpha-Vantage-, FMP- oder anderes Drittanbieter-Response-Schema programmiert werden.

3.2 MCP Agent Plane



MCP darf IAM, Policy Engine, Rate Limits, Audit Logging, Provider Budgets oder den No-Demo-Data-Vertrag nicht umgehen.

4. Provider-Abstraktionsvertrag

Jeder Runtime-Provider MUSS eine providerneutrale Schnittstelle implementieren. Der konkrete Codepfad kann später erweitert werden, muss funktional aber mindestens folgende Capability-Klassen abbilden können:

```

interface MarketDataProvider {
    id: string;
    capabilities(): ProviderCapability[];
    health(): Promise<ProviderHealth>;
    getSnapshot(request: SnapshotRequest): Promise<CanonicalSnapshot>;
    getBars(request: BarsRequest): Promise<CanonicalBars>;
}
  
```

Optionale Capability-Erweiterungen:

- quotes;
- trades;
- order book;

- options chain;
- fundamentals;
- FX;
- commodities;
- corporate actions;
- streaming subscription.

Business-Services referenzieren ausschließlich den Gateway-/Contract-Layer.

5. Canonical Market Data Contract

Jeder normalisierte Datensatz MUSS die Herkunft und zeitliche Qualität nachvollziehbar machen.

Mindestens:

```
provider
providerFeed
symbol
assetClass
currency
sourceTimestamp
ingestedAt
receivedAt
isRealtime
isDelayed
freshnessMs
marketStatus
qualityState
correlationId
```

Für Preis-/Bar-Daten zusätzlich, soweit fachlich anwendbar:

```
price
bid
ask
open
high
low
close
volume
vwap
tradeCount
```

Fehlende Felder dürfen nicht durch erfundene Werte ersetzt werden.

6. Data-Quality-Zustände

CAPITAL-AI unterscheidet mindestens:

```
LIVE
DELAYED
HISTORICAL
STALE
DEGRADED
```

UNAVAILABLE
INVALID

6.1 Freshness

Freshness wird aus Provider-Timestamp und Ingestion-Zeit berechnet. Asset-Klasse, Handelszeit und erwartete Feed-Frequenz sind bei der Bewertung zu berücksichtigen.

6.2 Cross-Provider Validation

Wenn mehrere Provider dieselbe fachliche Information liefern, darf ein Confidence-/Consensus-Layer verwendet werden.

Beispiel:

```
Alpaca Snapshot
+
Secondary Reference
|
v
Divergence Check
|
+--> within tolerance -> high confidence
+--> outside tolerance -> degraded / review
```

Cross-Provider-Divergenz darf nicht still durch Mittelwertbildung verschleiert werden.

7. No Demo Data Policy

Produktive Market-Data-Pfade dürfen bei Provider-Ausfall **keine simulierten, zufälligen oder synthetischen Kursdaten als reale Marktdaten ausgeben**.

Verboten:

```
provider failure
-> generated price fluctuation
-> production score
```

Zulässig:

```
provider failure
-> bounded stale cache + explicit STALE marker
```

oder:

```
provider failure
-> approved secondary provider
-> provenance retained
```

oder:

```
provider failure
-> UNAVAILABLE / DEGRADED
```

Testfixtures und synthetische Daten sind ausschließlich in klar isolierten Testkontexten zulässig und dürfen nicht als Produktions-Evidence persistiert werden.

8. Resilience Contract

Jeder externe Provider-Adapter MUSS mindestens unterstützen:

- bounded timeout;
- begrenzte Retries mit Backoff und Jitter;
- Circuit Breaker;
- Request Coalescing für identische parallele Requests;
- provider- und endpoint-spezifische Rate-Limit-Budgets;
- serverseitiges Caching;
- Health State;
- strukturierte Fehlerklassifikation;
- Correlation ID;
- Telemetrie ohne Secret-Werte.

Ein Fallback darf nur auf einen Provider erfolgen, dessen Capability, Entitlement und Datenqualität für den angeforderten Use Case zulässig sind.

9. Provider Routing

Provider Routing berücksichtigt mindestens:

1. Asset-Klasse;
2. benötigte Capability;
3. Echtzeit-/Delayed-/Historical-Anforderung;
4. Provider-Entitlement;
5. Datenlizenz;
6. Health State;
7. verbleibendes Request-Budget;
8. Freshness;
9. erwartete Latenz;
10. Datenqualität;
11. Kostenprofil;
12. Fallback-Priorität.

Der Provider Router darf keine kostenpflichtige oder lizenzpflichtige Capability implizit annehmen.

10. Alpaca MCP Governance

Der offizielle Alpaca MCP Server V2 unterstützt Toolset-Filtering. Für CAPITAL-AI wird initial ausschließlich ein **Read-only Market Data Profile** genehmigt.

Zulässige Toolset-Klassen, abhängig von tatsächlichem Bedarf:

```
assets  
stock-data  
crypto-data  
options-data  
corporate-actions  
news
```

Nicht freigegeben durch dieses ESS/ADR:

```
trading  
order placement  
order replacement  
order cancellation  
account mutation  
portfolio mutation
```

Wird ein MCP-Host direkt konfiguriert, MUSS die Toolauswahl explizit eingeschränkt werden. Eine Default-Konfiguration mit allen verfügbaren Alpaca-Tools ist für den Produktionskontext nicht zulässig.

11. Alpha Vantage MCP Governance

Der offizielle Alpha-Vantage-MCP-Server darf für kontrollierte Research-, Enrichment- und Evidence-Aufgaben eingebunden werden.

Geeignete Anwendungsfälle:

- Fundamentals;
- FX;
- Commodities;
- technische Indikatoren;
- historische Research-Abfragen;
- unabhängige Referenzwerte;
- agentische Erklärungen.

Der Alpha-Vantage-MCP darf keine versteckte Abhängigkeit in der deterministischen Runtime-Pipeline erzeugen. Runtime-Nutzung erfolgt über einen expliziten Provider Adapter mit Rate-Limit- und Entitlement-Kontrolle.

12. Secrets und Render

Provider-Secrets werden ausschließlich serverseitig verwaltet.

Beispielhafte Server-Secret-Namen:

```
ALPACA_API_KEY  
ALPACA_SECRET_KEY  
ALPHA_VANTAGE_API_KEY
```

Regeln:

1. Kein Secret erhält ein `VITE_`-Präfix.
2. Kein Secret wird in Client Bundles, Logs, Metrics, Audit-Dokumente oder Fehlermeldungen geschrieben.
3. Repository-Dokumentation enthält nur Secret-Namen und den Contract, niemals Werte.
4. Secret Rotation darf keinen Code-Change erzwingen.
5. Fehlende Pflicht-Secrets führen für den betroffenen Provider zu `UNAVAILABLE`, nicht zu Demo-/Fake-Daten.
6. Ein in Render vorhandener Key wird nicht als Nachweis gewertet, dass alle für die konkrete Provider-Funktion erforderlichen Credentials und Entitlements vollständig sind.

Render-Mutationen und Secret-Werte selbst gehören zur Production Control Plane und werden nicht durch dieses Dokument verändert.

13. IAM und MCP Capability Policy

MCP-Zugriff wird als Capability behandelt, nicht als allgemeiner Netzwerkzugriff.

Beispielhafte Capabilities:

```
marketdata.alpaca.read  
marketdata.alphavantage.read  
marketdata.provider.health.read  
marketdata.provider.config.read  
marketdata.provider.config.plan
```

Nicht Bestandteil dieses ESS:

```
marketdata.trade.execute  
marketdata.order.cancel  
marketdata.portfolio.modify
```

Eine spätere Trading-Integration benötigt ein eigenes ADR mit separater IAM-, Step-up-, Suitability-, Compliance-, Audit- und Failure-Mode-Bewertung.

14. Observability Contract

Mindestens folgende technische Signale werden vorgesehen:

```
market_data_requests_total  
market_data_errors_total  
market_data_cache_hit_total
```

```
market_data_rate_limit_total
market_data_fallback_total
market_data_stale_total
market_data_unavailable_total
market_data_provider_latency_ms
market_data_provider_divergence
```

Logs enthalten Provider-ID, Capability, Status, Correlation ID und Latenz, aber keine API-Keys oder vollständigen sensiblen Response-Payloads.

15. Data Licensing & Redistribution

Marktdaten dürfen nur im Rahmen des tatsächlich gebuchten Provider-Plans und der geltenden Lizenz-/Redistributionsrechte verwendet werden.

Vor einer Funktion, die Daten an Endnutzer weiterverteilt, exportiert, dauerhaft speichert oder in größerem Umfang weiterverarbeitet, MUSS geprüft werden:

- Echtzeit-/Delayed-Entitlement;
- Exchange-/Feed-Lizenz;
- Redistributable vs. display-only;
- historische Speicherung;
- geografische Beschränkungen;
- Nutzer-/Geräte-/Display-Regeln;
- Tarif- und Nutzungsgrenzen.

Das Vorhandensein eines API-Keys allein ist kein Lizenznachweis.

16. Integration in CAPITAL-AI

Zielstruktur:

```
src/platform/MarketData/
  contracts/
    MarketDataProvider.ts
    MarketDataSnapshot.ts
    ProviderCapability.ts
  providers/
    AlpacaProvider.ts
    AlphaVantageProvider.ts
  ProviderRegistry.ts
  ProviderRouter.ts
  DataNormalizer.ts
  DataQualityService.ts
  MarketDataGateway.ts
```

Bestehende Scoring-/Screening-Services werden schrittweise von direkten Provider-Aufrufen auf `MarketDataGateway` migriert.

Der vorhandene Request-Orchestrator kann für Concurrency-/Queue-Schutz wiederverwendet werden; provider-spezifische Rate-Limit-Budgets bleiben zusätzlich erforderlich.

17. FinTech Use Cases

17.1 Equity Screening

Alpaca Snapshot
-> Canonical DTO
-> Technical Signals
-> Traditional Asset Scoring
-> Screener

17.2 Historical Backtesting

Alpaca Historical Bars
-> Normalization
-> Data Quality
-> Backtest Engine

17.3 Options Intelligence

Options Chains, Quotes, Implied Volatility und Greeks können für zusätzliche Risiko-, Volatilitäts- und Derivate-Scores verwendet werden, soweit Feed und Tarif diese Daten bereitstellen.

17.4 Cross-Provider Confidence

Alpaca kann als Primary Signal mit Alpha Vantage oder einem anderen freigegebenen Secondary Provider auf Divergenz geprüft werden.

17.5 Agentic Explainability

Ein Supervisor-gesteuerter Agent darf aktuelle Read-only-Market-Data-Tools nutzen, um Score-Erklärungen oder Research-Evidence zu erzeugen. Der Agent darf den deterministischen Score nicht durch ungeprüfte MCP-Ausgaben überschreiben.

17.6 Data Quality Auditor

Ein Quality-/Compliance-Agent kann prüfen:

- Timestamp/Freshness;
- Provider/Feed;
- Missing Fields;
- Divergence;
- Delayed-vs-Realtime Status;
- Entitlement-/Capability-Mismatch;
- Fallback-Häufigkeit.

18. Event-Mesh-Integration

Nach Umsetzung sollten providerbezogene Events in den bestehenden Enterprise Event Mesh eingebunden werden, zum Beispiel:

```
marketdata.provider.healthy  
marketdata.provider.degraded  
marketdata.provider.unavailable  
marketdata.rate_limit.hit  
marketdata.fallback.activated  
marketdata.data.stale  
marketdata.divergence.detected
```

Event-Namens- und Registry-Regeln werden nicht in ESS-0016 neu definiert, sondern aus ESS-0013 übernommen.

19. Implementation Phases

Phase 1 — Contracts & Adapter

- `MarketDataProvider` Contract;
- Canonical DTO;
- Alpaca REST Adapter;
- Secret/Env validation;
- unit tests.

Phase-1-Nachweis (2026-08-14): kanonischer Snapshot-Vertrag, Quality Assessment, Provider Registry, fail-closed Gateway-Grundgrenze und Alpaca REST Adapter sind implementiert. Alpaca bleibt technisch `shadow` und wird ohne `includeShadow: true` nicht geroutet. Bestehende Scoring-/Screening-Pfade sind noch nicht migriert.

Phase 2 — Gateway

- Provider Registry;
- Routing;
- caching;
- rate-limit budget;
- circuit breaker;
- observability.

Phase-2-Nachweis (2026-08-14): explizites Registry-Routing, bounded Cache mit erneuter Freshness-Prüfung, Request Coalescing, provider-/capability-spezifisches Rate-Limit-Budget, Circuit Breaker und strukturierte payloadfreie Telemetrie sind implementiert. Die Komponenten sind weiterhin nicht in Scoring-/Screening-Call-Sites verdrahtet; Alpaca bleibt `shadow`.

Phase 3 — Pipeline Migration

- traditionelle Asset-Services auf Gateway migrieren;
- bestehende direkte Provider-Zugriffe inventarisieren;
- No-Demo-Data-Invariante verifizieren.

Phase-3A-Nachweis (2026-08-15): Der bestehende Twelve-Data-Aktien-/Forex-Quote-Pfad wird über einen typisierten Provider-Adapter und den kanonischen MarketDataGateway geführt. Der bisherige Traditional-Quote-Vertrag bleibt als Compatibility Boundary erhalten. Index, Historie, Fundamentals und Scoring-Inputs sind noch nicht migriert; Alpaca bleibt score-neutraler Shadow und ist nicht als Primary autorisiert.

Phase-3B-Nachweis (2026-08-15): FMP-Indexquotes werden über einen injizierten read-only Loader-Adapter und den MarketDataGateway geführt. Bestehende Ticker-Allowlist, Cooldown und Server-Cache bleiben erhalten; der Platform-Layer importiert keine Server-Interns. Indexhistorie, Fundamentals und Scoring-Inputs bleiben offen.

Phase-3C-Nachweis (2026-08-15)

Der kanonische Vertrag `market-data-history/1.0.0`, eine separate History-Provider-Registry, ein fail-closed History-Gateway und ein injizierter FMP-Index-History-Adapter sind implementiert. Der Adapter normalisiert ausschließlich positive, datierte EOD-Schlusswerte, bewahrt Correlation- und Evidence-IDs und erzeugt keine synthetischen Historienwerte. Shadow-Provider sind standardmäßig ausgeschlossen.

Die aktive Composition Boundary in `server/fmpIndices.ts`, bestehende Scoring-Inputs, Fundamentals und produktive Historienkonsumenten sind ausdrücklich noch nicht migriert. Dieser Schritt autorisiert weder neue Provider-Aufrufe noch Alpaca-Promotion.

Phase 4 — Streaming

- Alpaca WebSocket nur nach Entitlement-/Planprüfung;
- reconnect/backpressure;
- bounded fan-out;
- freshness monitoring.

Phase 5 — MCP Read Plane

- Alpaca MCP mit explizitem Read-only Toolset;
- Supervisor/Policy Gate;
- Audit Trail;
- keine Trading Tools.

Phase 6 — Alpha Vantage Enrichment

- Alpha Vantage Provider Adapter;
- offizieller Remote MCP für Agent Research;
- Quota Tracking;

- Cross-Provider Validation.

Phase 7 — Production Evidence

- latency/failure metrics;
 - failover tests;
 - entitlement evidence;
 - licensing review;
 - traceability links;
 - operational runbook.
-

20. Acceptance Criteria

ESS-0016 gilt technisch als umgesetzt, wenn mindestens:

- ein providerneutraler MarketData Contract existiert;
 - Alpaca über einen Server-side Adapter angebunden ist;
 - Scoring-/Screening-Code nicht direkt an Alpaca-Response-Schemas gekoppelt ist;
 - Provider Health, Timeouts, Rate Limits und Cache instrumentiert sind;
 - Provenance und Freshness in normalisierten Daten enthalten sind;
 - bei Provider-Ausfall keine simulierten Produktionsdaten erzeugt werden;
 - ein expliziter Fallback-/Degraded-State existiert;
 - Alpaca MCP ausschließlich read-only/toolset-restricted betrieben wird;
 - Alpha Vantage MCP als Enrichment/Research Plane getrennt bleibt;
 - Trading-/Order-Funktionen nicht freigegeben sind;
 - Secrets ausschließlich serverseitig vorliegen;
 - Lizenz-/Entitlement-Annahmen dokumentiert und verifiziert sind;
 - Tests die No-Demo-Data- und Fail-Closed-Invarianten abdecken;
 - ESS/ADR/Implementation in der Traceability Matrix verknüpft sind.
-

21. Externe Referenzen

Stand der Provider-Eigenschaften wurde am **2026-08-03** geprüft.

- Alpaca MCP Server V2: <https://alpaca.markets/blog/alpaca-launches-mcp-server-v2/>
- Alpaca MCP toolset guidance: <https://alpaca.markets/learn/mcp-trading-with-claude-alpaca-google-sheets>
- Alpha Vantage official MCP: <https://mcp.alphavantage.co/>
- Alpha Vantage API documentation: <https://www.alphavantage.co/documentation/>
- Alpha Vantage support / quota information: <https://www.alphavantage.co/support/>

Provider-Funktionen, Preise, Entitlements und Limits sind externe Verträge und müssen vor produktionskritischen Änderungen erneut verifiziert werden.

ESS-0017-CONTRACTS

Quelle: `.ai/skills/ESS-0017-Contracts.md`

ESS-0017-CONTRACTS

Vocabulary Governance Contracts

Version

1.0.0

Status

Proposed Enterprise Specification

Dokumentklassifizierung

Dieses Dokument definiert ausschließlich Vocabulary-, Terminology- und Naming-Verträge für ESS-0017.

Es definiert keine globalen Repository-, Event-, Layer-, Security-, Release- oder Documentation-Governance-Regeln. Diese verbleiben in den jeweils zuständigen Authorities, insbesondere ESS-0001-CONTRACTS und ESS-0012/ESS-0012-CONTRACTS.

Bei Konflikt gilt folgende Reihenfolge:

1. ESS-0001-CONTRACTS für globale Enterprise Contracts;
 2. bestehende Security-/Compliance-/IAM-Authorities;
 3. ESS-0013/ESS-0013-CONTRACTS für EventMesh-Regeln;
 4. ESS-0012/ESS-0012-CONTRACTS für Documentation Governance;
 5. ESS-0017;
 6. dieses Dokument für Vocabulary-spezifische Contract-Ausprägungen.
-

Chapter 1 — Canonical Concept Contract

Jeder Vocabulary-Eintrag besitzt verbindlich:

- `id`
- `canonicalCodeTerm`
- `displayNameDE`
- `displayNameEN`
- `definitionDE`

- `definitionEN`
- `aliases`
- `forbiddenTerms`
- `category`
- `status`
- `version`
- `essReferences`
- `adrReferences`
- `traceabilityReferences`

Fehlt ein Pflichtfeld, ist der Eintrag nicht `approved`-fähig.

Concept Identity

Format:

```
VOC-<CATEGORY>-<NNNN>
```

Beispiele:

```
VOC-BILLING-0001  
VOC-ASSET-0004  
VOC-IAM-0012
```

Die ID ist unveränderlich und wird nach Retirement nie wiederverwendet.

Chapter 2 — Naming Contracts

Technical Code Naming

Technische Identifier sind Englisch. Dies umfasst insbesondere:

- Dateien und Verzeichnisse des aktiven Codes;
- Komponenten, Klassen, Interfaces und Types;
- Funktionen, Methoden und Variablen;
- API-Felder und Schemas;
- Event-Namen;
- technische Konfigurationsschlüssel.

Ein bestehender Identifier darf nicht allein zur Erfüllung dieser Regel umbenannt werden. Für aktive Renames gilt zwingend der Safe Rename Contract.

Human-Facing Documentation

Enterprise-Dokumentation wird in Deutsch und Englisch geführt. Beide Varianten referenzieren dieselben Concept IDs.

Pull Request Information

Menschlich sichtbare Pull-Request-Informationen werden auf Deutsch geführt. Maschinenlesbare Schlüssel dürfen Englisch bleiben, wenn sie von Automationen oder Validatoren verarbeitet werden.

Chapter 3 — Alias and Forbidden-Term Contracts

- Ein Alias muss genau einem kanonischen Concept zugeordnet sein.
 - Ein Alias darf nicht gleichzeitig kanonischer Term eines anderen aktiven Concepts sein.
 - `forbiddenTerms` dürfen in neuem governed content nicht als kanonische Bezeichnung eingeführt werden.
 - Historische Vorkommen werden als Evidence klassifiziert und nicht blind ersetzt.
 - Collision Detection ist vor Freigabe eines Concepts verpflichtend.
-

Chapter 4 — Bilingual Mapping Contract

`displayNameDE` und `displayNameEN` gehören immer zur identischen Concept ID.

Fehlende Übersetzungen sind ein Governance Finding. Semantisch widersprüchliche Übersetzungen blockieren den Status `approved`.

Dokumentationsgeneratoren dürfen keine neue Concept ID erzeugen, nur weil eine Sprache einen anderen Ausdruck verwendet.

Chapter 5 — Safe Rename Contract

Vor jedem Rename eines aktiven technischen Identifiers müssen mindestens folgende Prüfungen erfolgreich ausgeführt oder als Evidence bewertet werden:

1. Repository reference scan;
2. import/export dependency graph;
3. dynamic imports and lazy loading;
4. routes, APIs and schemas;
5. configuration and environment references;
6. regex and naming policies;

7. case-sensitive filesystem impact;
8. TypeScript/lint validation;
9. relevant tests;
10. production build;
11. deployment-readiness validation.

Ergebnis:

SAFE
CONDITIONAL
BLOCKED

Nur **SAFE** darf automatisiert zur Umsetzung vorbereitet werden. **CONDITIONAL** und **BLOCKED** benötigen explizite Evidence und Entscheidung.

Chapter 6 — Event Contracts

Vocabulary Governance führt keinen eigenen Event Bus ein.

Alle Events müssen über die bestehenden öffentlichen Interfaces der Enterprise Event Mesh veröffentlicht und konsumiert werden.

Vorgesehene Event-Typen, vorbehaltlich EventCatalog-Kompatibilitätsprüfung:

- VocabularyConceptProposedEvent
- VocabularyConceptApprovedEvent
- VocabularyConceptDeprecatedEvent
- VocabularyTranslationMissingEvent
- VocabularyCollisionDetectedEvent
- VocabularyRenameImpactAssessedEvent

Vor Registrierung ist zu prüfen, ob bereits ein semantisch äquivalentes kanonisches Event existiert.

Event Payload Minimum

Vocabulary Events enthalten mindestens:

- conceptId
- conceptVersion
- changeType
- evidenceReferences
- essReferences
- adrReferences

Correlation und causation werden gemäß ESS-0001-CONTRACTS und ESS-0013-CONTRACTS übernommen.

Chapter 7 — Source Authority Contract

Zulässige Quellen für kanonische Vocabulary-Entscheidungen:

1. aktueller aktiver Code;
2. ESS und Contracts;
3. aktive ADRs;
4. Registry-Einträge;
5. Traceability Evidence;
6. aktuelle Architecture Specifications;
7. revalidierte Domain-Dokumentation.

Nicht automatisch autoritativ:

- Legacy-Dokumente;
- historische Audit- oder Review-Snapshots;
- superseded blueprints;
- nicht revalidierte provider-/modellbezogene Dokumentation.

PR #142 dient als Klassifizierungsbasis für diese Unterscheidung.

Chapter 8 — Traceability Contract

Jedes `approved` Concept muss mindestens auf seine fachliche Authority verweisen. Bei Architektur- oder Governance-Begriffen sind ESS-/ADR-Referenzen verpflichtend.

Vocabulary-Einträge werden nicht als paralleler Knowledge Graph modelliert. Beziehungen zu Komponenten, Events und Dokumenten werden über die bestehende Traceability-/Knowledge-Architektur angebunden.

Chapter 9 — Validation

Vor Freigabe eines Concepts müssen mindestens validiert werden:

- eindeutige Concept ID;
- eindeutiger `canonicalCodeTerm` im Geltungsbereich;
- gültige DE/EN-Zuordnung;
- Alias-Kollisionen;

- Forbidden-Term-Kollisionen;
- Authority References;
- Event-Kompatibilität, wenn Events betroffen sind;
- Rename Impact, wenn aktiver Code betroffen ist.

Ein Finding ohne Evidence wird verworfen.

Success Criteria

ESS-0017-CONTRACTS gilt als erfüllt, wenn:

- Concept Schema und IDs deterministisch validiert werden;
- DE/EN-Mapping aus einer gemeinsamen Concept Identity erfolgt;
- Aliase und Forbidden Terms kollisionsfrei geprüft werden;
- Safe Rename Gate vor aktiven Renames erzwungen wird;
- Events ausschließlich über die bestehende EventMesh laufen;
- Vocabulary keine parallele Knowledge-, Traceability- oder Event-Infrastruktur erzeugt;
- Legacy-Inhalte nur nach Revalidierung kanonische Terminologie beeinflussen.

ESS 0017 Vocabulary Governance

Quelle: [.ai/skills/ESS-0017-Vocabulary-Governance.md](#)

skill: id: ESS-0017 name: Vocabulary Governance version: 1.0.0 status: Proposed maturity: Foundation owner: Platform Director category: Enterprise Architecture priority: High

capital_ai: platform: CAPITAL-AI Core repository: Finance architecture: Enterprise lifecycle: AI Native Development Lifecycle

classification: type: Technical Specification role: Spezifikation der Canonical Vocabulary Governance contractAuthority: ESS-0001-CONTRACTS ownContracts: ESS-0017-CONTRACTS note: > Dieses Dokument spezifiziert ausschließlich Vocabulary-, Terminology- und Naming-Governance. Globale Repository-, Event-, Layer- und Release-Regeln verbleiben in ESS-0001-CONTRACTS. Documentation Governance verbleibt unter ESS-0012.

authority: controls: - Canonical Vocabulary Registry - Technical Naming Governance - Product Wording Governance - Terminology Lifecycle - Alias and Forbidden-Term Governance - Bilingual Terminology Mapping collaborates: - Documentary Engine - Documentation Governance - Enterprise Knowledge Platform - Enterprise Traceability - Enterprise Event Mesh - Supervisor - Platform Director - Quality Center - Security & Compliance - Version Manager - Release Center cannot_modify: - Enterprise Specifications without approved governance flow - Architecture Decision Records without approved governance flow - Runtime code names without Safe Rename Gate - API contracts without impact validation - Environment keys without impact validation - Event names without EventMesh compatibility validation

crossReference: dependsOn: - ESS-0001-CONTRACTS - ESS-0012 - ESS-0013 relatedEss: - ESS-0003 - ESS-0005 - ESS-0006 - ESS-0009 - ESS-0010 - ESS-0011 - ESS-0012-CONTRACTS - ESS-0013-CONTRACTS relatedAdr: - ADR-0046 relatedComponents: - src/platform/Vocabulary - src/platform/Documentary - src/platform/Knowledge - src/platform/Traceability - src/platform/EventMesh relatedDocs: - docs/architecture/VOCABULARY_GOVERNANCE_MIGRATION_ROADMAP.md - docs/architecture/VOCABULARY_GOVERNANCE_PHASE_1_5_BASELINE.md - docs/architecture/LEGACY_DOCUMENTATION_CONSOLIDATION_2026-08-09.md

created: 2026-08-09

ESS-0017 — Vocabulary Governance

1. Enterprise Purpose

ESS-0017 defines the canonical governance model for names, terms, wording and bilingual terminology across CAPITAL-AI.

It does not replace Documentation Governance, the Enterprise Knowledge Platform, Enterprise Traceability or the Enterprise Event Mesh. It provides a dedicated terminology authority that integrates with those components through existing contracts and events.

2. Normative Principles

1. Technical code identifiers are English.
2. Enterprise documentation is maintained in German and English.
3. Human-facing pull request information is German.
4. Canonical concepts have stable, language-neutral identities.
5. German and English labels map to the same concept identity.
6. Aliases are explicit; forbidden terms are explicit.
7. Active code renames require a Safe Rename Gate before implementation.
8. Legacy documentation is not a canonical terminology source unless revalidated against current code and current authorities.
9. Vocabulary changes are traceable to ESS, ADR, events and evidence.
10. Vocabulary lifecycle actions integrate into the CAPITAL-AI event-driven value chain without bypassing human approval boundaries.

3. Component Boundary

Planned implementation location:

```
src/platform/Vocabulary/  
  Domain/  
  Registry/  
  Services/  
  Validators/  
  Events/  
  Interfaces/  
  Types/  
  Tests/
```

The component may depend only on approved shared contracts and public interfaces. It must not introduce a second Event Bus, second Knowledge Graph or second Traceability store.

4. Canonical Concept Model

Every canonical concept contains at minimum:

- `id`
- `canonicalCodeTerm`
- `displayNameDE`
- `displayNameEN`
- `definitionDE`
- `definitionEN`

- `aliases`
- `forbiddenTerms`
- `category`
- `status`
- `essReferences`
- `adrReferences`
- `traceabilityReferences`
- `version`

Concept IDs are stable and language-neutral. Labels may evolve under version control without changing identity.

5. Lifecycle

Allowed concept states:

```
draft -> proposed -> approved -> deprecated -> retired
```

A deprecated or retired concept remains resolvable for historical evidence and traceability. Its ID is never reused.

6. Naming Domains

Vocabulary Governance separates at least:

- code and component naming;
- file and directory naming;
- API and schema naming;
- event naming;
- product terminology;
- UI wording;
- documentation terminology;
- compliance and governance terminology.

A UI translation must never change a technical identifier implicitly.

7. Safe Rename Boundary

No active rename is authorized solely by a vocabulary decision. The later Safe Rename Gate must validate at minimum repository references, imports/exports, dynamic imports, routes/APIs/schemas, configuration/environment references, regex/naming policies, filesystem casing, TypeScript/lint, tests, production build and deployment readiness.

Classification is `SAFE`, `CONDITIONAL` or `BLOCKED`.

8. Event-Driven Integration

Vocabulary Governance produces and consumes only registered Enterprise Events through public EventMesh interfaces. Direct implementation coupling to EventMesh internals is prohibited.

Planned vocabulary lifecycle events are defined in ESS-0017-CONTRACTS and must be checked against the canonical Event Catalog before registration.

Downstream integration targets include Documentary, Knowledge, Traceability, Quality, Security/Compliance, Supervisor, Platform Director, Version Manager and Release.

9. Bilingual Documentation

German and English documentation derive terminology from the same concept IDs. Language variants may differ stylistically, but they must remain semantically equivalent for governed terms.

Missing or diverging translations are governance findings and later event-producing conditions.

10. Pull Request Wording

Human-facing PR title/body information is German. Machine-readable identifiers and stable technical keys remain English where required for deterministic automation.

11. Legacy Documentation Rule

Documents classified by PR #142 as historical, superseded or evidence snapshots may seed review candidates only. They cannot automatically establish canonical terminology.

12. Success Criteria

ESS-0017 is implemented when:

- the Canonical Vocabulary Registry exists as a single terminology source;
- concepts are typed, versioned and collision-checked;
- DE/EN mappings are deterministic;
- aliases and forbidden terms are validated;
- EventMesh integration uses canonical registered events;
- Traceability links concepts to ESS/ADR/components/evidence;
- Documentation Governance can validate terminology usage;
- active renames cannot bypass the Safe Rename Gate;
- no duplicate terminology authority exists elsewhere in the platform.

ESS-0018 — Agentic Supabase Tool Governance

Quelle: `.ai/skills/ESS-0018-Agentic-Supabase-Tool-Governance.md`

ESS-0018 — Agentic Supabase Tool Governance

Enterprise Specification

Version: 1.0.0 **Status:** Enterprise Specification **Implementation Status:** PHASE 1 IMPLEMENTED / PHASE 2 VOLLSTAENDIG UMGESETZT (Foundation + beide in §4.2 spezifizierten Schreib-Capabilities) **Owner:** Platform Director **Security Authority:** CAPITAL-AI IAM / Security & Compliance **Related ADR:** ADR-0043 (Supabase Privilege Separation), ADR-0050 (Agent Tool & Capability IAM Foundation, Phase 1), ADR-0051 (Capability/Grant IAM & Approval Workflow, Phase 2) **Related ESS:** ESS-0001-CONTRACTS, ESS-0014, ESS-0016

1. Zweck

ESS-0018 definiert die verbindliche Architektur dafür, wie CAPITAL-AI-KI-Agenten kontrolliert auf die eigene Supabase-Datenbank zugreifen dürfen. Ziel ist eine auditierbare, least-privilege Agent-Fähigkeits-Ebene, die niemals über den privilegierten Datenbankzugriff hinausgeht, den ein menschlicher Administrator ohnehin schon hat, und die niemals Rohzugriff (raw SQL, Schema-Mutation, Projekt-Verwaltung) an ein Sprachmodell delegiert.

Diese Spezifikation deckt zwei konkrete Agenten ab:

1. **Screening/Scoring-Erklärbarkeits-Agent** — lesend, erklärt bereits persistierte Scoring-Ergebnisse anhand zitierter Datenbank-Evidenz. **Phase 1, implementiert.**
2. **Admin/Support/Diagnose-Agent** — für CAPITAL-AI-Systemadministratoren, lesend plus eng umrissene, freigegebene Schreiboperationen, gekoppelt an Compliance (Policy-Oberfläche) und Supervisor (Ausführungs-Wrapper). **Phase 2, Foundation implementiert (ADR-0051); beide in §4.2 spezifizierten Schreib-Capabilities (`alert_subscription.disable` , `alert_subscription.resend_confirmation`) implementiert.**

2. Ausgangslage (Stand 2026-08-09, verifiziert im Code)

- Es existiert **kein** MCP-Server-Code in `src/` oder `server/`. MCP ist bislang ausschließlich Spezifikation (ADR-0041: „SPECIFIED / NOT YET IMPLEMENTED“). Der einzige real laufende MCP-Server (`ga4-analytics` , `.mcp.json`) ist Entwickler-Tooling für Claude-Code-Sitzungen — kein Muster für In-App-Agenten-Fähigkeiten.
- Agenten (`src/agents/*.ts`) erzeugen bislang ausschließlich strukturiertes JSON/Freitext über `src/services/agentModelRouting.ts` (Anthropic → OpenAI → Gemini-Fallback). Es gab vor Phase 1 **kein** Tool-/Function-Calling im Agentensystem.

- IAM (`src/platform/Security/authMiddleware.ts`) kennt ausschließlich grobe Rollen (`'owner' | 'admin' | 'supervisor' | 'user'`), **kein** Capability-/Grant-System.
- Supervisor (`src/platform/Supervisor/supervisor.ts`) ist real, aber Retry-/Observability-Wrapper — kein Approval-Gate. Compliance (`src/platform/Compliance/`) ist real, aber ein statischer Security-Scanner ohne Laufzeit-Autorität (eigener Code-Kommentar: „darf keine Berechtigungen vergeben, keine IAM-Regeln verändern, keine Sicherheitsmechanismen umgehen“).
- Das in CLAUDE.md geforderte Kettenmodell
`Policy -> IAM/Grant -> Approval -> Dry-run -> Fingerprint -> Apply -> Verify -> Audit`
 existiert für Provider-Mutationen bislang nirgends als Laufzeit-Code.
- ADR-0043 trennt `getPrivilegedServerSupabase()` (Secret/Service-Role, umgeht RLS, backend-only) strikt von `getRLsServerSupabase()` (Publishable/Anon, kein privilegierter Fallback). `score_snapshots` und `alert_subscriptions` haben keine `user_id`-Spalte und eine RLS-Policy, die ausschließlich `service_role`-Zugriff erlaubt — das ist gewollt (globale Symbol-/Betriebsdaten, keine personenbezogenen Datensätze).

Konsequenz: Phase 1 baute die kleinstmögliche echte, sichere Scheibe (ein lesendes Tool, ein Agent, ein Endpunkt). Phase 2 (ADR-0051) hat die Capability-IAM, das Approval-Artefakt und die Compliance/Supervisor-Gates additiv nachgezogen und an einer real angebundenen, bewusst risikoarmen Schreib-Capability bewiesen — ohne einen der oben genannten, damals als sicherheitskritisch identifizierten Bestandteile zu verändern.

3. Systemgrenzen

```

Screening/Scoring-Engines (deterministisch)
  |
  v
score_snapshots (Supabase, service-role-only RLS)
  |
  v
SupabaseScoreEvidenceTool [READ-ONLY, allowlisted parametrisierte Query]
  |
  v
Row-Evidence-Bundle (Zitat, Qualitätsverdikt HIGH/MEDIUM/LOW/NO_EVIDENCE, fail-closed)
  |
  v
ScoreExplainabilityAgent -> generateTextWithFallback (unverändert)
  |
  v
GET /api/scoring/explain/:symbol (Admin/Supervisor-Rolle erforderlich, kein Browser-anonymer Zugriff)

```

Kein Netzwerkpfad, kein MCP-Protokoll-Server, kein Schreibpfad in Phase 1. Die Agent-Plane bleibt strikt vom deterministischen Scoring-Runtime-Pfad (`server/scoreValidation.ts`, `recordDailySnapshots`) getrennt — der Agent liest nur, er beeinflusst niemals den persistierten Score.

4. Capability Policy

4.1 Phase 1 — Screening/Scoring-Erklärbarkeits-Agent (implementiert)

Erlaubt:


```
supabase.score_snapshots.read
```

Konkret: eine einzige, allowlisted parametrisierte Lesefunktion (`getScoreSnapshotEvidence(symbol, fromDate?, toDate?)`), ausschließlich über den Supabase-JS-Query-Builder (`.eq/.gte/.lte/.order/.limit`), niemals String-Konkatenation, niemals freie SQL-Eingabe vom Modell. Symbol und Datumsangaben werden vor jeder Abfrage validiert (Regex-Whitelist), nicht erst nach Rückgabe gefiltert.

Nicht erlaubt (Phase 1):

```
supabase.*.write
supabase.sql.execute
supabase.schema.*
supabase.project.*
supabase.alert_subscriptions.* (kein Anwendungsfall in Phase 1)
```

4.2 Phase 2 — Admin/Support/Diagnose-Agent (Foundation + eine Schreib-Capability implementiert, ADR-0051)

Implementierte Capability-Namen (`src/platform/Security/capabilities.ts` , `CAPABILITIES`):

<code>supabase.admin.diagnostics.read</code>	(Lesen, implementiert – inkl. Quota-Auswertung)
<code>wurde</code>	fuer eine E-Mail; eine gesonderte <code>supabase.admin.quota.read</code> -Capability
<code>supabase.admin.subscription_status.read</code>	NICHT angelegt, siehe Anmerkung unten)
<code>supabase.admin.alert_subscription.disable</code>	(Lesen, implementiert)
Schreib-Capability, ADR-0051)	(Schreiben, implementiert – erste reale
<code>supabase.admin.alert_subscription.resend_confirmation</code>	(Schreiben, implementiert – zweite
Schreib-Capability, demselben Muster folgend)	

Anmerkung zu `quota.read` : §4.2 hatte urspruenglich eine eigene `supabase.admin.quota.read`-Capability vorgesehen. In der tatsaechlichen Implementierung (`getAdminDiagnostics()` , `src/services/agentTools/supabaseAdminDiagnosticsTool.ts`) liefert der bereits bestehende `ADMIN_DIAGNOSTICS_READ`-Aufruf Subscription-Tier UND Quota in einem Aufruf zurueck — eine separate Capability haette hier nur denselben, bereits gelesenen Datensatz nochmal gated, ohne einen zusaetzlichen Angriffsflaechen- oder Autorisierungsgewinn. Diese Spezifikation wird daher als durch `ADMIN_DIAGNOSTICS_READ` erfuellt betrachtet, nicht als separat offener Punkt.

Keine weiteren offenen Schreib-Capabilities aus §4.2.

Explizit **nicht** Bestandteil dieser ESS und in keiner Phase vorgesehen:

```
supabase.sql.execute (kein Raw-SQL-Tool für ein Modell, niemals)
supabase.admin.migration.apply
supabase.admin.project.pause
supabase.admin.project.delete
supabase.admin.branch.*
supabase.admin.user.role.grant (IAM-Rollenvergabe bleibt menschliche Owner-Aktion, siehe CLAUDE.md)
```

Jede Phase-2-Schreib-Capability MUSS an die vollständige, in CLAUDE.md verbindlich vorgeschriebene Kette gebunden sein:

```
Policy -> IAM/Grant -> Approval -> Dry-run -> Fingerprint -> Apply -> Verify -> Audit
```

mit `Compliance` (`src/platform/Compliance/PolicyGate.ts` , additiv neben dem unveränderten `Ganz-App-Auditor`) als Policy-Oberfläche und `Supervisor` (`executeApprovedSupervisedAction()` , additiv neben dem unveränderten `executeSupervised()`) als Ausführungs-Wrapper, der Approval-Konsum vor jedem Apply erzwingt. Fingerprint-Prüfung ist Aufgabe des jeweiligen Schreib-Tools (siehe ADR-0051). Für `supabase.admin.alert_subscription.disable` UND `supabase.admin.alert_subscription.resend_confirmation` ist diese Kette jetzt real implementiert, jeweils mit eigenem, auf die konkrete Mutation zugeschnittenem Fingerprint (Zeilen-Zustand bei `disable` , `confirmed` -Flag bei `resend_confirmation` — ein erneuter Versand ist ein realer Seiteneffekt, auch wenn er im Regelfall keine Zeile mutiert).

5. MCP-Ebene

Phase 1 baut **keinen** MCP-Protokoll-Server (kein stdio, kein Streamable HTTP). Begründung: das einzige reale MCP-Muster im Repo (`ga4-analytics`) ist externes Dev-Tooling für Claude-Code, kein In-App-Agenten-Muster; In-App-Fähigkeiten sind bislang immer einfache interne TypeScript-Services. Mit genau einem internen Aufrufer (`ScoreExplainabilityAgent`) würde MCP-Protokoll-Maschinerie (Transport, Tool-Schema, eigene Prozessgrenze) Komplexität ohne Nutzen hinzufügen.

Auslöser für eine spätere MCP-Protokoll-Ebene: sobald ein zweiter echter, unabhängiger Konsument entsteht — z. B. eine externe Ops-Oberfläche, oder der Phase-2-Admin-Agent, der aus Isolationsgründen (Schreibrechte!) eine eigene Prozess-/Vertrauensgrenze braucht. Erst dann wird geprüft, ob ein echter MCP-Server (analog zur in ADR-0041 spezifizierten, aber nie gebauten Provisioner-Architektur) den Aufwand rechtfertigt.

6. Secrets und Zugriff

- Kein neues Secret in Phase 1. `getPrivilegedServerSupabase()` (`server/db.ts`) wird unverändert wiederverwendet — `score_snapshots` ist bewusst service-role-only, ein Zugriff über den RLS-gebundenen Client würde grundsätzlich null Zeilen liefern.
- `SupabaseScoreEvidenceTool` und `ScoreExplainabilityAgent` sind **backend-only**. Kein Import aus Vite-/Browser-erreichbarem `src/` -Code außerhalb von Server-Routern erlaubt.
- Der HTTP-Endpunkt (`server/scoreExplainability.ts`) erfordert `checkAdminAccess` mit `SUPERVISOR_ZONE_ROLES` (owner/admin/supervisor) — kein anonymer, kein reiner `user` -Zugriff.
- Jeder Tool-Aufruf wird über `logSystemEvent('ORCHESTRATOR', ...)` protokolliert: Symbol, Datumsbereich, Zeilenanzahl, Qualitätsverdikt — niemals der Datensatzinhalt selbst.

7. Acceptance Criteria

Phase 1 (diese ESS gilt für Phase 1 als umgesetzt, wenn:)

- `getScoreSnapshotEvidence` existiert, exportiert keine Write/Upsert/Delete-Funktion, nutzt ausschließlich den Supabase-JS-Query-Builder mit validierten Eingaben.
- Der Ergebnis-Bundle-Typ liefert ein fail-closed `NO_EVIDENCE`-Verdikt bei leeren Ergebnissen statt eine erfundene Erklärung.
- `ScoreExplainabilityAgent` erfindet keine Erklärung ohne Evidenz und zitiert die tatsächlich gelesenen Snapshots.
- `GET /api/scoring/explain/:symbol` verlangt `checkAdminAccess` und ist ohne gültigen Admin/Supervisor-Token nicht erreichbar.
- Unit-Tests decken Qualitätsverdikt-Schwellenwerte, Eingabvalidierung und die exakte Query-Builder-Aufrufform ab.
- `npm run lint`, `npm test`, `npm run build`, `npm run predeploy:check` sind grün.

Phase 2 — Foundation + erste Schreib-Capability (ADR-0051, umgesetzt, wenn:)

- `checkCapability()` fail-closed bei jedem DB-/Verbindungsfehler, keine Wildcard-Capability erreichbar.
- `consumeApproval()` verweigert fehlende, abgelaufene, bereits verbrauchte und plan-/aktions-fremde Genehmigungen; Konsum ist atomar/race-sicher (Single-Use).
- `evaluateWritePolicy()` / `evaluateReadPolicy()` lassen ausschließlich die dokumentierte Allowlist zu, jede Raw-SQL-förmige Capability erhält `DENY`.
- `executeApprovedSupervisedAction()` ruft die eigentliche Aktion niemals ohne konsumiertes, passendes Approval auf; ein `DENY` an der Policy-Stufe erreicht die Approval-Stufe gar nicht erst.
- `disableAlertSubscription()` verweigert Apply bei Fingerprint-Mismatch (fail-closed statt stillem Überschreiben) und ist bei bereits inaktiver Zeile idempotent.
- `resendAlertSubscriptionConfirmation()` verweigert Apply bei Fingerprint-Mismatch, ist bei bereits bestätigtem Abo idempotent (kein Mailversand) und wirft statt still zu "erfolgreich" zu werden, wenn der Mailversand selbst fehlschlägt.
- Grant-/Approval-Ausstellung ist ausschließlich hinter Owner-Rolle + frischem TOTP-Step-up erreichbar (428 ohne Step-up, analog `VersionManager`).
- `npm run lint`, `npm test`, `npm run build`, `npm run predeploy:check` sind grün.

Weiterhin offen: ein UI für Grant-/Approval-Verwaltung, und die produktive Anwendung der beiden Phase-2-Migrationen gegen die echte Supabase-Instanz (bleibt kontrolliertes Handoff, CLAUDE.md). Keine offenen Schreib-Capabilities aus §4.2 mehr.

8. Externe Referenzen

Keine externen Provider-Abhängigkeiten in Phase 1 (rein interne Supabase-Datenbank). Phase-2-Design orientiert sich an der in CLAUDE.md verbindlich vorgeschriebenen Gover-

nance-Kette sowie am Struktur-Vorbild ESS-0016 (Allowed/Not-Allowed-Listen, Phasenmodell, Acceptance Criteria).

ESS-0019 — Universal AI Agent Control Plane

Quelle: `.ai/skills/ESS-0019-Universal-AI-Agent-Control-Plane.md`

ESS-0019 — Universal AI Agent Control Plane

Status: ACCEPTED (provider-set clarified 2026-08-16) Version: 1.1.0 Date: 2026-08-11

Owner: Platform Director Scope: CAPITAL-AI DevelopmentChain M2-M9

1. Purpose

ESS-0019 defines the provider-neutral control plane for every AI application, coding agent, research assistant or model that can observe, propose or execute work against CAPITAL-AI.

The controlling invariant is:

```
AI product != trust root.
```

Canonical active providers (Owner 2026-08-16): ChatGPT, Claude and Grok are execution or research profiles. **Google AI Studio, NotebookLM and Gemini are not part of the active DEVELOPMENT Chain or AI value chain** (retired aliases). Authorization, policy, audit, evidence and production mutation authority remain inside CAPITAL-AI governance.

2. Plane separation

CAPITAL-AI separates four planes:

1. **Research & Evidence Plane** — read-only source-grounded analysis, repository/document research and evidence production.
2. **Agent Execution Plane** — planning, file/tool operations and bounded changes inside an isolated workspace/branch.
3. **Control Plane** — identity, capability grants, risk classification, policy decisions, approvals, step-up, kill switch and audit correlation.
4. **Production Plane** — protected deployment/runtime/data/billing mutations requiring explicit production authority.

No provider may bypass the Control Plane to reach the Production Plane.

3. Provider profiles

- **ChatGPT** (`chatgpt-github-connector`): execution/research client through GitHub/connectors/MCP/Apps; never receives implicit Owner authority.
- **Claude / Claude Code** (`claude-code-cli`): execution client with permission-scoped tools/MCP; plan/read modes preferred before write; bypass-permissions modes prohibited for CAPITAL-AI production work.

- **Grok** (`grok-xai-connector`): research + controlled execution client via GitHub MCP / Grok Chat connector; same control-plane policy as ChatGPT/Claude; model name never elevates authority.
- **Google AI Studio / Gemini / NotebookLM: RETIRED** — not registered as active value-chain profiles; control plane returns DENY/RETIRED.

4. Capability model

Canonical capabilities:

```
READ -> ANALYZE -> PLAN -> BRANCH -> COMMIT -> PR -> CI_REQUEST -> DEPLOY_REQUEST -> PRODUCTION_MUTATION
```

Capabilities are granted to attributable principals, not model brands.

Default policy is deny-by-default. A higher capability never follows automatically from possession of a lower capability.

5. Risk classes

- LOW — documentation, formatting, non-executable metadata.
- MEDIUM — isolated frontend/business logic with bounded blast radius.
- HIGH — IAM, auth, financial scoring, CI/CD, dependencies, security controls.
- CRITICAL — secrets, production DB/RLS, billing, deployment, Owner IAM, break-glass.

HIGH and CRITICAL actions require independent policy evaluation and human/step-up approval where defined by the related ADRs.

6. Execution contract

Every command/action MUST be attributable through:

```
human_actor -> app/client -> agent/session -> request -> capability -> policy decision -> tool execution -> repository/resource result -> evidence.
```

The minimum correlation set is defined by ADR-0059 and the audit schema under `docs/architecture/ai-agent/`.

7. Security invariants

- No raw long-lived provider or infrastructure secret is exposed to a model when a connector/tool can hold it.
- Read-only evidence gathering is separated from mutation authority.
- Tool inputs and retrieved content are untrusted data and cannot elevate capability.
- Agent self-approval is forbidden for HIGH/CRITICAL changes.
- Production mutation is impossible from retired profiles and from research-only ceilings.
- Production changes require rollback and postcondition evidence.

- Security audit evidence is not sampled away with normal telemetry.

8. Observability and audit

OpenTelemetry/W3C Trace Context is the preferred cross-provider correlation mechanism. Operational telemetry and security audit evidence are separate retention classes. Redaction occurs before export.

Supervisor observes the agent provider chain (ChatGPT/Claude/Grok) via `observeAgentProviderChain` (ESS-0002).

9. Supply chain

Agent-generated code is not trusted by origin. It passes the same branch, PR, CI, SBOM, provenance, review and deployment gates as human-authored changes.

10. Related authorities

- ESS-0002 Supervisor, ESS-0006 Security & Compliance, ESS-0008 AI Agent Framework
- ADR-0050, ADR-0051, ADR-0056, ADR-0057..ADR-0063
- `docs/evidence/m8/M8_PROVIDER_SET_CORRECTION_2026-08-16.md`
- `docs/architecture/ai-agent/AI_AGENT_PROVIDER_PROFILE_CONTRACT.md`

ESS-0020 — Supabase Native MFA / AAL2 Hardening

Quelle: `.ai/skills/ESS-0020-Supabase-Native-MFA-AAL2-Hardening.md`

ESS-0020 — Supabase Native MFA / AAL2 Hardening

Enterprise Specification

Version: 1.0.0 **Status:** PROPOSED — HUMAN/OWNER REVIEW REQUIRED **Implementation Status:** NOT STARTED — M5A BASELINE COMPLETE **Owner:** Platform Director **Security Authority:** CAPITAL-AI IAM / Security & Compliance **Roadmap Phase:** M5A — Supabase MFA/TOTP/AAL2 Hardening **Related ADR:** ADR-0003.5, ADR-0058, ADR-0064 **Related ESS:** ESS-0018, ESS-0019

1. Zweck

ESS-0020 definiert den verbindlichen Authentifizierungsvertrag für privilegierte CAPITAL-AI-Identitäten und ersetzt für den M5A-Scope die bisherige Annahme, dass ein in `public.profiles` gespeichertes `totp_enabled` oder ein CAPITAL-AI-eigenes TOTP-Secret einen Supabase-MFA-Nachweis darstellt.

Für M5A ist **Supabase Auth die MFA-Authority**. Ein privilegierter Request gilt nur dann als MFA-verifiziert, wenn die vertrauenswürdig verifizierte Supabase-Session einen aktuellen, nicht-stalen `aal2`-Zustand nachweist.

2. Verifizierte Ausgangslage 2026-08-12

Read-only Evidence gegen `AIFINANCIAL (ryzywoktptyhwzxmstyu)` und `Finance@main`:

- `auth.mfa_factors` : **0** Faktoren;
- `auth.sessions` : **2 × aal1, 0 × aal2**;
- `public.profiles` : **2 Owner-Profile mit legacy `totp_enabled=true`**, 2 User-Profile ohne Legacy-TOTP;
- `public.break_glass_codes` : **20 unbenutzte Legacy-Recovery-Codes**;
- `public.step_up_tokens` : **0 aktive Tokens**;
- Supabase Security Advisor: `auth_insufficient_mfa_options` aktiv;
- Organization Plan: **Free**; Leaked Password Protection bleibt planbedingt separat deferred;
- Repository-Code implementiert TOTP aktuell selbst in `src/platform/Security/totp.ts` und `server/stepUp.ts` und verwendet nicht `supabase.auth.mfa.enroll/challenge/verify`;
- `checkAdminAccess()` verifiziert Identität und IAM-Rolle, aber aktuell **kein aal**;
- `loginStepUpRequirement()` enthält für Status-/Netzwerkfehler einen **fail-open**-Pfad;

- Passkey-Bestätigung kann den aktuellen Login-Step-Up alleine erfüllen, obwohl ADR-0003.5 TOTP für Owner unabhängig vom Passkey fordert.

Keine Secret-Werte, E-Mail-Adressen, Factor-IDs, Recovery-Codes oder Token-Werte werden in dieser Evidence gespeichert.

3. Authority Contract

3.1 Verbindliche MFA-Quelle

Für M5A zählen ausschließlich Supabase-Auth-Nachweise:

- verifizierter Supabase-Bearer/JWT;
- `aal === 'aal2'` ;
- Faktorstatus über Supabase Auth (`mfa.listFactors` / serverseitig validierte AAL-Evidence);
- TOTP-Verifikation über den unterstützten Supabase-Flow `enroll → challenge → verify` .

Folgende Signale sind **keine MFA-Authority**:

- `profiles.totp_enabled` ;
- `profiles.totp_secret_encrypted` ;
- Browser- `sessionStorage` -Marker;
- UI-Zustand;
- bloßes Vorhandensein eines Faktors ohne erfolgreiche Challenge/Verify;
- Passkey allein, solange dessen Supabase-Session keinen für M5A akzeptierten `aal2` - Nachweis liefert;
- CAPITAL-AI-eigener `x-step-up-token` ohne gleichzeitig gültige AAL2-Session.

3.2 Privilegierte Rollen

Für `owner` und `admin` gilt bei privilegierten Aktionen:

```
VALID SUPABASE SESSION
+ AUTHORIZED IAM ROLE
+ CURRENT AAL2
+ NON-STALE FACTOR STATE
+ PURPOSE-BOUND STEP-UP WHERE THE ACTION CONTRACT REQUIRES IT
= PRIVILEGED ACTION ELIGIBLE
```

`aal1` muss fail-closed abgewiesen werden. Ein Zustand `currentLevel=aal2` und `nextLevel=aal1` ist als stale/downgraded zu behandeln und darf keine privilegierte Aktion autorisieren.

3.3 Enrollment

Native TOTP-Einschreibung:

1. `supabase.auth.mfa.enroll({ factorType: 'totp' })` ;
2. QR/Secret nur dem aktuell authentifizierten Benutzer anzeigen;

3. `supabase.auth.mfa.challenge({ factorId })` ;
4. `supabase.auth.mfa.verify({ factorId, challengeId, code })` ;
5. Session/AAL nach erfolgreicher Verifikation neu bewerten;
6. `aal2` muss technisch nachgewiesen sein, bevor die Einschreibung als abgeschlossen gilt.

Enrollment ohne erfolgreiche Verification ist kein MFA-Erfolg.

3.4 Login und Session Restore

Nach Primärlogin und bei wiederhergestellten Sessions muss vor privilegiertem Zugriff `getAuthenticatorAssuranceLevel()` bzw. eine äquivalente serverseitig validierte AAL-Prüfung stattfinden.

Die Logik ist fail-closed für privilegierte Rollen:

- `aal1/aal1` : kein verifizierter MFA-Faktor → privilegierter Zugriff gesperrt / Enrollment erforderlich;
- `aal1/aal2` : Faktor vorhanden, Challenge fehlt → Challenge erzwingen;
- `aal2/aal2` : MFA erfolgreich → Zugriff nach IAM-/Action-Gates möglich;
- `aal2/aal1` : stale/downgraded Session → Zugriff sperren, Session aktualisieren/re-authentifizieren.

Netzwerk-/Auth-Fehler dürfen bei Owner/Admin **nicht** zu `none` bzw. Dashboard-Freigabe degradieren.

3.5 Server/API Enforcement

Eine zentrale serverseitige AAL2-Prüfung muss privilegierten Endpunkten vorgeschaltet werden. Die Browser-UI ist ausschließlich UX, nicht Security Boundary.

Die Prüfung muss mindestens validieren:

- Bearer-Token gehört zu einem realen Supabase-Benutzer;
- IAM-Rolle erlaubt die Zielzone;
- AAL2 wird aus vertrauenswürdig validierter Session/JWT-Evidence abgeleitet;
- stale/downlevel Zustand fail-closed;
- Fehlerpfade liefern keine autorisierte Entscheidung.

Für direkt durch `authenticated` aufgerufene sensible Datenbankpfade ist nach separater Tabellen-/Policy-Inventur ein restriktiver AAL2-RLS-Contract zu verwenden. Service-Role-Servverpfade werden nicht durch RLS geschützt und müssen AAL2 deshalb vor dem privilegierten Server-Apply selbst erzwingen.

3.6 Purpose-bound Step-Up

Der bestehende `x-step-up-token` kann als zusätzliche, kurzlebige, einmalige und zweckgebundene Kontrolle erhalten bleiben, aber nur als **Defense-in-Depth über AAL2**.

Er darf AAL2 niemals ersetzen. Ausstellung und Konsum müssen an dieselbe verifizierte Identität gebunden sein. Für besonders kritische Owner-Aktionen ist frische MFA-Evidence zu testen und zu dokumentieren; Annahmen über AMR-Freshness dürfen nicht ohne Integrationstest als erfüllt gelten.

4. Recovery / Break-Glass

Supabase Native MFA besitzt für App-Benutzer keinen eingebauten Recovery-Code-Vertrag, der den vorhandenen CAPITAL-AI-Codes entspricht. M5A definiert daher:

- primär mindestens einen getesteten Backup-Faktor-/Recovery-Prozess für Owner;
- vorhandene Legacy-Recovery-Codes gelten während der Migration nicht automatisch als Native-MFA-Recovery;
- ein Owner-kontrollierter Break-Glass-Pfad darf native Faktoren nur serverseitig über die unterstützte Admin-MFA-Factor-Operation entfernen;
- das Entfernen eines verifizierten Faktors muss als Security-Critical Event auditiert werden und aktive Sessions entsprechend Supabase-Verhalten invalidieren;
- Passkeys dürfen nicht mehr implizit zusammen mit TOTP gelöscht werden, sofern eine getrennte Recovery-Entscheidung möglich ist;
- keine Recovery-Operation darf Rollen erhöhen oder `aal2` simulieren.

5. Legacy-TOTP-Migration

Die bestehenden eigenen Secrets werden nicht in-place als Supabase-Faktoren behandelt.

Verbindlicher Cutover:

1. Native MFA-Codepfad implementieren und testen;
2. Owner re-enrollen ihren TOTP-Faktor über Supabase Auth;
3. AAL2 für beide Owner-Identitäten verifizieren;
4. Recovery/Backup-Faktor testen;
5. privilegierte Serverpfade auf AAL2 schalten;
6. Legacy-TOTP-Pfade auf read-only/deprecated setzen;
7. erst nach separatem Owner-Approval Legacy-Secrets/-Spalten/-Tabellen entfernen.

DDL-/Datenbereinigung ist **nicht** Bestandteil der M5A-Baseline.

6. Mutation Classification

Bereich	Status nach Baseline
Repository/Application Code	REQUIRED
Supabase Auth Factor Enrollment	REQUIRED für die beiden Owner-Identitäten
Supabase Project Auth Configuration	CONDITIONAL / NOT YET VERIFIED — Connector kann die Dashboard-Verification-Einstellung nicht auslesen

Bereich	Status nach Baseline
Postgres DDL für Native MFA	NOT REQUIRED
Legacy TOTP Schema Cleanup	DEFERRED / SEPARATE OWNER-APPROVED MUTATION
Render	NOT REQUIRED für Baseline/Design; Deployment erst nach Code-Merge über bestehende Chain
Stripe	NOT REQUIRED

7. Required Negative Tests

M5A darf nicht `VERIFIED PASS` werden, bevor mindestens diese Fälle fail-closed bewiesen sind:

1. Owner/Admin mit `aa11` → DENY;
2. eingeschriebener aber nicht verifizierter Faktor → DENY;
3. falscher TOTP-Code → DENY;
4. abgelaufene/ungültige Challenge → DENY;
5. fehlender Faktor → DENY;
6. stale `aa12/aa11` → DENY;
7. Auth-/AAL-Check Netzwerkfehler → DENY für privilegierte Rollen;
8. `x-step-up-token` ohne AAL2 → DENY;
9. AAL2 ohne erforderlichen purpose-bound Step-Up bei Critical Action → DENY;
10. Step-Up-Token für falschen Purpose/User → DENY;
11. verbrauchter/replayed Step-Up → DENY;
12. Recovery ohne Owner-/Recovery-Gate → DENY.

Positive Tests:

- verifizierter Owner + `aa12/aa12` + gültiger Action-Step-Up → ALLOW für explizit erlaubte Aktion;
- Standardnutzer bleiben gemäß Produktpolicy ohne globale MFA-Pflicht nutzbar, solange kein privilegierter Pfad betroffen ist.

8. Evidence / Audit

M5A Evidence muss ohne Secrets dokumentieren:

- aggregierte Factor-/Session-Zustände;
- getestete AAL-Transitions;
- negative Testresultate;
- Advisor vor/nach Mutation;
- PR/CI/Merge SHA;
- Mutation State;

- Recovery-Testresultat;
- Rollback-Entscheidung.

9. Exit Gate

M5A ist erst `COMPLETE / VERIFIED PASS` , wenn:

1. ESS-0020 + ADR-0064 Human/Owner-approved sind;
2. Native MFA-Code und zentrale serverseitige AAL2-Enforcement implementiert sind;
3. Required CI PASS ist;
4. zwei Owner-Identitäten native TOTP-Faktoren erfolgreich verifiziert haben;
5. privilegierte AAL1-/stale-/error-Pfade fail-closed getestet sind;
6. Recovery/Backup-Verfahren verifiziert ist;
7. Security Advisor nach Mutation erneut ausgeführt wurde;
8. Evidence und Roadmap final synchronisiert sind.

M6 bleibt bis dahin blockiert.

ESS-0021 — Systemadmin Roadmap Executor

Quelle: `.ai/skills/ESS-0021-Systemadmin-Roadmap-Executor.md`

ESS-0021 — Systemadmin Roadmap Executor

Status: PROPOSED Version: 1.0.0 Date: 2026-08-12 Owner: Platform Director / Repository
Owner Scope: CAPITAL-AI DevelopmentChain cross-cutting execution profile

1. Purpose

ESS-0021 defines a privileged, provider-neutral **Systemadmin Roadmap Executor** that can autonomously execute larger Owner-approved Roadmap work packages from analysis through a review-ready Pull Request.

The goal is higher execution throughput without converting the model into a trust root.

Core invariant:

Owner approves bounded Roadmap authority once → agent executes many scoped technical steps →
Human reviews/merges final PR

2. Agent identity

Canonical logical agent id:

`capital-ai-systemadmin-roadmap-executor`

Provider/model metadata is non-authoritative. The profile may execute through ChatGPT, Claude or a future client only when the provider-neutral Control Plane can attribute the action to the same logical agent and Owner mandate.

3. Standing authorization contract

The Systemadmin profile operates only under an active **Roadmap Execution Mandate (REM)**.

A REM is a bounded, expiring Owner authorization tied to:

- Owner actor;
- subject agent;
- Finance repository;
- base branch;
- Roadmap/ESS/ADR work-package references;
- allowed paths/targets;
- explicit capabilities;

- maximum risk;
- permitted mutation classes;
- validity window;
- kill switch;
- tests/rollback/evidence.

The REM is the authority for repeated BRANCH/COMMIT/PR operations inside the same Roadmap segment. It is not a wildcard administrator grant.

4. Capability profile

Default delegated capabilities under a valid REM:

`READ, ANALYZE, PLAN, BRANCH, COMMIT, PR, CI_REQUEST`

`DEPLOY_REQUEST` and `PRODUCTION_MUTATION` are not granted by default.

`MERGE` is permanently outside this profile and outside the Agent IAM capability vocabulary.

Capabilities do not inherit. Each capability must be explicitly present in the REM and pass policy evaluation for the current target.

5. Autonomous repository workflow

The Systemadmin profile may execute the following loop without a new Human prompt per technical action:

1. read current Roadmap/main/evidence;
2. select the next mandate-covered unblocked work package;
3. perform security and overlap preflight;
4. create a fresh branch from current main;
5. implement code/documentation/tests;
6. run targeted checks;
7. perform security self-check / negative-test review;
8. commit scoped changes;
9. iterate until exit criteria are met;
10. open/update the PR;
11. repair pre-review CI/governance defects within scope;
12. stop at Human/Owner final review and merge boundary.

6. External mutation model

Repository mutation is the initial autonomous mutation domain.

External production mutation requires a later technically enforceable REM-bound Control-Plane capability. Until that implementation is VERIFIED PASS, this profile MUST NOT treat policy text alone as production write authority.

A future REM-bound production mutation can only be valid when:

- exact target and mutation type are predeclared;
- Roadmap/ADR/ESS require it;
- preconditions and rollback are deterministic;
- the mutation is bounded and reversible;
- postconditions are verifiable;
- audit correlation is complete;
- the action is not in the non-delegable Owner list.

7. Non-delegable actions

The Systemadmin profile MUST NOT autonomously:

- merge Pull Requests;
- weaken repository protections or Human review controls;
- elevate Owner/admin IAM;
- reset/remove Owner MFA or invoke break-glass;
- expose secrets or reusable credentials;
- execute destructive production data operations;
- mutate live billing in ways that can change customer money/entitlements;
- delete production resources;
- change DNS/TLS/domain ownership;
- disable security/audit/RLS/consent controls;
- expand or extend its own mandate.

8. Security preflight

Each Roadmap work package begins read-only and must verify:

- current main SHA;
- current Roadmap phase/gate;
- mandate validity;
- open PR/file overlap;
- relevant architecture/ESS/ADR/evidence;
- external current-state evidence where needed;
- PR check class D/C/R/M;
- threat model requirement;

- negative tests;
- rollback path;
- CI cost/scope;
- no reserved Owner-only action is required.

Security-critical ambiguity fails closed.

9. Audit contract

Every mutating action is correlated to:

```
mandateId + roadmapItem + humanActor + agent + client + session + request + capability +  
risk + target + decision + result
```

Where repository state exists, also record branch, commit and PR identifiers.

The M5 append-only audit path is the preferred durable evidence sink. Redaction/omission requirements from ADR-0059 remain mandatory.

10. Kill switch

The Owner or Control Plane may revoke the active REM at any time. Kill-switch activation blocks all mutating capabilities immediately.

Automatic STOP conditions include expiry, scope drift, requested self-elevation, reserved action, repeated failed verification, unresolvable concurrent change, missing audit identity or security-control regression.

11. PR governance interaction

ADR-0039 remains the default for normal agents. ADR-0065 introduces the only Systemadmin exception: a valid Owner-approved REM counts as standing PR-creation authorization for PRs fully inside the mandate.

The existing final Human/Owner PR gate remains unchanged:

```
Files changed / Viewed → current-head review → Owner checkboxes → build-and-test → separate  
Human merge instruction
```

12. Branch lifecycle

Every work package uses a fresh branch from current main. After successful merge into `Finance`, the branch MUST be deleted. Closed/superseded branches are also deleted after Evidence retention. Merged branches are never reused.

13. Verification requirements before enablement

Technical enablement requires at minimum negative tests proving:

- expired mandate → DENY;

- wrong agent → DENY;
- wrong repository/target/path → DENY;
- missing capability → DENY;
- maximum-risk violation → DENY;
- reserved Owner action → DENY;
- self-extension/self-approval → DENY;
- kill switch active → DENY;
- concurrent/scope conflict without approved resolution → STOP;
- valid mandate + valid scoped repository action → ALLOW.

14. Related authorities

- ESS-0019 Universal AI Agent Control Plane
- ESS-0018 Agentic Supabase Tool Governance
- ADR-0058 Provider-neutral Agent IAM
- ADR-0059 Agent Audit / OTel Correlation
- ADR-0065 Systemadmin Roadmap Execution Mandate
- `docs/governance/SYSTEMADMIN_AGENT_ROADMAP_EXECUTION_POLICY.md`
- `docs/governance/AUTONOMOUS_AGENT_CONCEPT_GATE.md`
- `docs/governance/HUMAN_OWNER_PR_APPROVAL_POLICY.md`

ESS-0022 — Marketing Roadmap Executor

Quelle: `.ai/skills/ESS-0022-Marketing-Roadmap-Executor.md`

ESS-0022 — Marketing Roadmap Executor

Status: PROPOSED / NOT ENABLED Version: 0.1.0 Date: 2026-08-12 Owner: Platform Director / Repository Owner Scope: CAPITAL-AI Marketing, SEO, Social Content and Content-Distribution DevelopmentChain

1. Purpose

ESS-0022 defines the provider-neutral logical agent `capital-ai-marketing-roadmap-executor`.

The Marketing Roadmap Executor is a domain-limited development and operations agent for Owner-approved Marketing Roadmap work packages. It is deliberately separate from the broader `capital-ai-systemadmin-roadmap-executor` defined by ESS-0021.

Core invariant:

`Marketing automation != infrastructure authority != publishing authority`.

The Marketing Agent may eventually plan and implement SEO, content-generation, media-rendering, social-distribution and analytics-feedback work, but it MUST NOT inherit Systemadmin privileges merely because it reuses the same execution-host and audit foundations.

2. Best-practice basis

The profile applies the following reference principles:

- NIST AI RMF / Generative AI Profile: Govern, Map, Measure and Manage AI risk throughout the lifecycle;
- NIST SSDF: integrate secure software practices into the existing SDLC instead of creating a parallel insecure automation path;
- ISO/IEC 42001: accountable AI governance, traceability, transparency, continual improvement and risk treatment;
- OWASP Agentic Security Initiative: treat tools, retrieved content, memory and inter-agent messages as untrusted; prevent excessive agency, tool misuse and privilege expansion;
- CAPITAL-AI ESS-0019: AI provider/client is never the trust root;
- CAPITAL-AI ESS-0021 / SA Roadmap: bounded mandate, exact capabilities, audit-before-side-effect, Human-only merge and branch deletion.

External standards are crosswalk references. CAPITAL-AI ESS/ADR/Policy remains the executable repository authority.

3. Logical agent identity

Canonical logical agent id:

`capital-ai-marketing-roadmap-executor`

Provider/model metadata is non-authoritative. ChatGPT, Claude, Gemini or a later approved host may execute the logical profile only when the CAPITAL-AI Control Plane can attribute the action to the same logical agent, mandate, human owner and request.

4. Initial state

This ESS creates the specification only.

Current enablement state:

`READ / ANALYZE / PLAN only`.

The profile has no standing repository mutation authority, no social-platform publishing authority and no production mutation authority until the later Marketing Agent Roadmap gates are individually VERIFIED PASS.

5. Future capability ceiling

After the required Control-Plane and execution-host gates are proven, a Marketing Roadmap Execution Mandate may delegate only explicitly listed capabilities:

`READ, ANALYZE, PLAN, BRANCH, COMMIT, PR, CI_REQUEST`.

Capabilities never inherit.

`MERGE` is permanently prohibited.

`DEPLOY_REQUEST` and `PRODUCTION_MUTATION` are not part of the default Marketing profile and require separate future ADRs, exact-target technical enforcement and Human/Owner approval.

Public social publishing is treated as an external business mutation even if the underlying API is technically simple.

6. Domain authority

The Marketing Agent is intended for the following domains:

- SEO management and measurement;
- SocialMediaEngine content-generation contracts;
- content templates and platform variants;
- AI content provenance and disclosure;
- content-compliance gates;
- media-rendering adapters;
- social publishing bridge code;

- marketing analytics feedback;
- marketing documentation, tests and evidence.

The Marketing Agent is NOT a general backend, infrastructure, IAM, billing, scoring or release administrator.

7. Initial repository path scope

Candidate owned paths for later mandate-controlled mutation:

- `src/platform/SocialMediaEngine/**`;
- `server/socialMedia/**`;
- `src/routes/socialMediaRoutes.ts` only when explicitly in the work package;
- `src/platform/SeoEngine/**` when implemented;
- `docs/seo/**`;
- `docs/architecture/*CONTENT*` / Marketing-specific architecture documents;
- Marketing-specific tests and evidence;
- Marketing-specific `.ai/contracts/**` and `.ai/skills/**` entries after Human-approved governance changes.

Shared integration paths are NOT implicitly owned:

- `AGENTS.md`;
- `package.json` / lock file;
- `.github/workflows/**`;
- global security/IAM files;
- global Event Mesh registries;
- global Traceability registries;
- `server.ts` / application bootstrap;
- deployment configuration;
- production database schemas shared with unrelated domains.

A shared integration change requires explicit scope, conflict inspection and Human/Systemadmin coordination.

8. Marketing content trust boundary

AI-generated marketing content MUST be separated into these planes:

1. source/evidence;
2. content planning/generation;
3. financial-claim and policy validation;
4. provenance/disclosure;
5. Human review/approval;

6. media rendering;
7. asset validation;
8. existing CAPITAL-AI publishing;
9. analytics/feedback.

No generative model or media renderer receives Social OAuth tokens, Supabase service credentials, Stripe credentials, GitHub credentials, Owner session material, TOTP secrets or passkey material.

9. Financial-content invariants

Public quantitative financial claims require attributable source evidence.

Synthetic, heuristic, placeholder or unverified values **MUST NOT** be presented as live market facts.

The Marketing Agent **MUST** fail closed or require Human review when content contains:

- investment recommendations or recommendation-like language;
- market forecasts;
- rankings or scores used as persuasive claims;
- performance promises;
- unsupported financial statistics;
- regulatory/compliance-sensitive claims;
- conflicting source evidence;
- incomplete AI provenance.

10. Human approval boundary

Until a later separately approved architecture proves a lower-risk autonomous publication class, Human/Owner approval remains mandatory before generated financial marketing content is published externally.

Approval **SHOULD** be bound to the final content hash, final asset hashes and target platforms. Material content or asset changes invalidate approval.

11. Reuse of Systemadmin foundations

The Marketing Agent **SHOULD** reuse verified foundations from the SA Roadmap rather than introduce a second security stack:

- provider-neutral Agent IAM;
- execution-host identity;
- OIDC where applicable;
- Policy Gate;
- durable append-only M5 audit;

- permit-before-side-effect enforcement;
- kill switch;
- branch/PR evidence;
- Human review and Human-only merge;
- fresh branch and post-merge branch deletion.

Reuse of infrastructure does not imply reuse of authority. The Marketing profile has its own subject identity, mandate, allowed paths, capabilities and risk ceiling.

12. Non-delegable actions

The Marketing Agent MUST NOT autonomously:

- merge Pull Requests;
- weaken Human review or repository protections;
- modify its own authorization policy or expand its own mandate;
- change Owner/admin IAM or MFA;
- disclose or rotate unrestricted credentials;
- mutate billing/customer entitlements;
- change financial scoring formulas or weights unless a separate scoring-domain mandate explicitly authorizes it;
- mutate production market data;
- delete production resources;
- change DNS/TLS/domain ownership;
- disable security, audit, RLS or consent controls;
- publish generated financial content without the required content approval;
- bypass platform review requirements.

13. Threat model requirements

At minimum, implementation must test:

- prompt/retrieval injection cannot elevate capability;
- external content cannot issue tool instructions;
- renderer/provider compromise cannot access publishing credentials;
- wrong mandate/agent/path/capability -> DENY;
- stale source evidence -> REVIEW_REQUIRED or DENY;
- unsupported financial claim -> DENY;
- approval hash mismatch -> DENY;
- cross-tenant content/asset access -> DENY;
- invalid/private media URL -> DENY;
- kill switch active -> DENY;

- audit persistence failure -> zero repository/external side effect;
- publication cannot precede required approval.

14. Enablement sequence

MA0 documentation package -> Human acceptance -> MA1 read-only profile/validator -> MA2 content contracts -> MA3 provenance/compliance/approval -> MA4 rendering -> MA5 bounded repository pilot -> MA6 separately governed external publishing control -> MA7 feedback loop.

Mutation authority MUST NOT be activated before the relevant SA execution-host foundation is VERIFIED PASS and the Marketing-specific negative tests succeed.

15. Branch lifecycle

Every future mutating Marketing work package uses a fresh scoped branch from current `main`.

After successful Human merge into `Finance`, the remote work branch MUST be deleted. Closed or superseded branches are deleted after required evidence retention. Merged branches are never reused.

16. Related authorities

- ESS-0006 Security & Compliance
- ESS-0008 AI Agent Framework
- ESS-0011 Enterprise Traceability
- ESS-0013 Enterprise Event Mesh
- ESS-0014 Google Marketing MCP Governance
- ESS-0019 Universal AI Agent Control Plane
- ESS-0021 Systemadmin Roadmap Executor
- ADR-0039 Human-authorized PR creation
- ADR-0058 Provider-neutral Agent IAM
- ADR-0059 Agent Audit / OTel Correlation
- ADR-0065 Systemadmin Roadmap Execution Mandate
- ADR-0067 Systemadmin execution-host binding
- ADR-0068 Marketing Roadmap Executor and Content Automation Boundary
- `docs/governance/AUTONOMOUS_AGENT_CONCEPT_GATE.md`
- `docs/roadmaps/MARKETING_AGENT_ROADMAP.md`
- `docs/governance/MARKETING_AGENT_ROADMAP_EXECUTION_POLICY.md`

17. Registry note

ESS-0022 was checked as unoccupied in the current repository before this draft was created. The canonical ESS Registry **MUST** be updated as part of the final Human-approved documentation PR before this specification is treated as published authority.

ESS-0022 — Passkey-only Human/Owner PR Authorization

Quelle: `.ai/skills/ESS-0022-Passkey-Only-Owner-PR-Authorization.md`

ESS-0022 — Passkey-only Human/Owner PR Authorization

Status: PROPOSED — IMPLEMENTATION BLOCKED BY M9 Version: `1.0.0` Date: 2026-08-12

Scope: CAPITAL-AI DEVELOPMENT Chain M10 Owner: `SvenKulesa` Repository: `SvenKulesa/Finance` Authority: ADR-0066, DEVELOPMENT Chain Execution Policy, Human Owner PR Approval Policy

1. Purpose

Define the normative Enterprise System Specification for replacing the transitional emoji/text + PR-body checkbox authorization before expensive CI with a CAPITAL-AI-owned, PR-state-bound WebAuthn/passkey approval transaction.

M10 does **not** automate merge. Human file review and Human/Owner merge remain separate controls.

2. Target Sequence

```
PR OPEN / UPDATE
→ Human reviews complete changed-file set
→ all files Viewed
→ trusted service resolves exact current PR state
→ server generates single-use WebAuthn challenge
→ Owner completes passkey assertion with required user verification
→ server verifies assertion + exact PR context
→ immutable approval evidence persists
→ exactly one CI request consumes approval
→ build-and-test
→ Human merge
```

3. Authorization Context

Every `AUTHORIZE_PR_CI` transaction MUST bind at least:

- Owner: `SvenKulesa` ;
- Repository: `SvenKulesa/Finance` ;
- Pull Request number;
- base branch;
- exact base SHA;
- exact current head SHA;
- canonical sorted changed-file-set hash;
- canonical PR diff/review digest;
- action: `AUTHORIZE_PR_CI` ;

- challenge ID;
- challenge issue time;
- challenge expiry;
- approval consumption/replay state.

A new commit, force-push, base change, changed-file-set change or material diff change MUST invalidate an earlier approval.

4. WebAuthn Verification Requirements

Server-side verification MUST enforce:

- cryptographically random server-generated challenge;
- short expiry;
- single-use challenge;
- expected HTTPS origin;
- expected RP ID;
- registered Owner credential binding;
- assertion signature verification;
- authenticator `UP` (User Presence);
- authenticator `UV` (User Verification) required;
- expected challenge match;
- credential revocation state;
- replay/consumption check;
- exact PR authorization-context match.

Client-side device identity, browser state or provider assertions are not sufficient authority.

5. Human File Review

Human file review remains mandatory.

The system MUST derive the canonical changed-file set from trusted GitHub state rather than accepting an agent-supplied file list as authoritative.

The approval transaction MUST bind the reviewed file set and current diff/review digest. `Viewed` status remains Human workflow evidence; the cryptographic approval binds the resolved state that the Owner is authorizing.

6. Audit

Before CI can be requested, immutable M5-compatible approval evidence MUST be persisted.

Evidence SHOULD include:

- human actor ID;
- repository / PR;
- base/head SHA;
- file-set hash;
- diff/review digest;
- action;
- challenge/approval IDs;
- credential public identifier/reference suitable for audit;
- UP/UV verification result;
- RP/origin verification result;
- issued/verified/consumed timestamps;
- CI request/result reference.

Evidence MUST NOT include:

- private keys;
- biometric material;
- reusable authenticator secrets;
- raw passwords;
- TOTP secrets/codes;
- recovery codes;
- reusable bearer credentials.

7. Single CI Consumption

A valid approval may authorize exactly one normal expensive CI request for its exact PR state.

Duplicate requests for an already consumed approval/head MUST be rejected or deterministically deduplicated.

An unconsumed approval cannot be retargeted to another PR/base/head/file set/diff/action.

8. Legacy Cutover

Until M10 `VERIFIED PASS`, the current transitional Owner gate remains authoritative.

After controlled cutover, the following MUST no longer authorize expensive CI:

- `;`;
- `okay` ;
- generic GitHub review status;
- PR-body Owner checkboxes;

- reactions;
- comments;
- labels;
- device ID;
- provider/model identity;
- GitHub login method alone.

Legacy signals may remain as non-authoritative workflow/display evidence only where useful.

9. Recovery / Break-Glass

Passkey recovery is a separate strong Human/Owner flow.

Recovery MUST:

- verify the Owner through an approved strong recovery path;
- be reason-bound and audited;
- avoid silent fallback to emoji/checkbox/TOTP-only PR authorization;
- not grant agent merge authority;
- support credential revocation/re-enrollment;
- invalidate affected outstanding challenges/approvals when necessary.

Break-glass is disabled by default and governed by M9/ADR-0063.

10. Agent Boundary

No agent may:

- self-enroll an Owner credential;
- answer the Owner WebAuthn challenge as the Owner;
- modify the approval context after assertion;
- accept agent-provided file/diff hashes as authoritative without trusted recomputation;
- consume approval for a different action;
- grant itself `MERGE` ;
- weaken RP/origin/UP/UV/replay/audit checks.

Audit/verifier unavailable → CI authorization `DENY` .

11. Threat Model

Normative threat model:

`docs/architecture/ai-agent/M10_PASSKEY_OWNER_PR_AUTHORIZATION_THREAT_MODEL.md`

It covers RP/origin confusion, replay, TOCTOU/state drift, forged review/diff context, stolen session, compromised agent, recovery bypass, credential revocation, audit outage and duplicate CI triggering.

12. Implementation Runbook

Normative runbook:

`docs/runbooks/M10_PASSKEY_OWNER_PR_AUTHORIZATION.md`

Required rollout:

1. architecture/ESS/threat model approval;
2. implementation;
3. unit/integration/security tests;
4. Owner passkey enrollment + recovery proof;
5. shadow mode while legacy gate remains authoritative;
6. decision comparison;
7. negative/replay/freshness/revocation tests;
8. controlled passkey-only cutover;
9. legacy authorization cleanup;
10. final Evidence and Roadmap sync.

13. Mandatory Positive Tests

- correct Owner credential + exact current PR state → verified approval;
- immutable audit record persisted;
- one CI request consumes approval;
- approved current head runs required build-and-test;
- Human merge remains independent after green CI.

14. Mandatory Negative Tests

- wrong credential → DENY;
- wrong RP ID/origin → DENY;
- `UP=false` → DENY;
- `UV=false` → DENY;
- expired challenge → DENY;
- replayed challenge/assertion → DENY;
- revoked credential → DENY;
- wrong repository/PR/base/head → DENY;
- changed file set/diff after challenge issue → invalidate/DENY;
- unavailable verifier → DENY;

- unavailable durable audit → DENY;
- duplicate CI consumption → DENY/DEDUPE;
- forged emoji/comment/reaction/checkbox → no authorization;
- agent self-approval → DENY.

15. Rollback

Before legacy removal, rollback disables M10 enforcement and returns to the still-authoritative transitional gate.

After legacy cleanup, rollback MUST use a separately pre-approved recovery design and MUST NOT silently re-enable weaker legacy authorization without Human/Owner incident approval.

16. Exit Gate

M10 is COMPLETE / VERIFIED PASS only when:

- M9 prerequisite is VERIFIED PASS ;
- exact-state WebAuthn approval works end-to-end;
- required positive/negative/replay/recovery tests PASS;
- shadow comparison is accepted;
- immutable approval/audit evidence works;
- exactly one CI request per approval/head is proven;
- legacy emoji/checkbox authorization is removed from the CI authorization path;
- Human file review remains;
- Human-only merge remains;
- rollback/recovery is proven;
- Evidence/Roadmap/Traceability are synchronized;
- implementation branch is deleted after Human merge.

ESS-0023 — Konsolidierter Systemadministrator-Prototyp- und Mutationsanfragevertrag

Quelle: `.ai/skills/ESS-0023-Consolidated-Systemadmin-Prototype-and-Mutation-Request.md`

ESS-0023 — Konsolidierter Systemadministrator-Prototyp- und Mutationsanfragevertrag

Status: ACTIVE — DOCUMENTATION CONTRACT

Version: 1.1.0

Datum: 2026-08-14

Authority: ADR-0071

Owner: SvenKulesa

1. Zweck

Dieser Vertrag steuert die Ableitung codebasierter Systemadministrator-Prototypen aus der konsolidierten CAPITAL-AI-Roadmap und erlaubt dem Agenten, notwendige Mutationen beim Owner anzufragen, ohne Selbstautorisierung zu erzeugen.

2. Rollen

- **Owner:** entscheidet, genehmigt oder lehnt Mutationen ab; führt Human Review und Merge aus.
- **Systemadministrator-Agent:** diagnostiziert, erstellt Optionen, Proposal, Prototyp, Tests und Evidence; darf nach Freigabe nur über einen autorisierten Execution Host handeln.
- **Execution Host:** erzwingt IAM/REM/Target/Approval/Precheck und audit-before-side-effect.
- **Reviewer/CI:** prüft Code, Contracts und negative Tests; erzeugt keine Owner-Authority.

3. Eingaben je Prototyp

Pflichtfelder:

- `prototypeId` ;
- Roadmap-Item und Priorität;
- current-main SHA;
- Scope und erlaubte Pfade;
- verbotene Pfade/Operationen;
- Risiko- und Datenklasse;
- ADR-/ESS-/Runbook-Verweise;

- Tests;
- Evidence-Ziel;
- Rollback;
- Branch- und PR-Plan.

Fehlt ein Pflichtfeld, lautet die Entscheidung `DENY_INCOMPLETE_WORK_PACKAGE`.

4. Mutationsanfrage

Zulässige Initiatoren:

- `OWNER` ;
- `SYSTEMADMIN_AGENT` .

Der Systemadministrator-Agent darf eine Anfrage stellen, wenn read-only Evidence zeigt, dass eine Mutation erforderlich oder sinnvoll sein könnte. Die Anfrage muss enthalten:

1. Ist-Zustand;
2. exaktes Ziel;
3. sichere Standardlösung;
4. Alternativen mit Trade-offs, wenn relevant;
5. Security-/Integritäts-/Verfügbarkeitsauswirkung;
6. Precheck;
7. atomare Operation;
8. Postcheck;
9. Rollback;
10. Evidence-Pfad;
11. Ablauf-/Invalidierungsbedingungen.

Die Anfrage darf keine Mutation auslösen.

5. Approval

Für externe Plattformen, Produktion, Supabase, Render, Stripe, IONOS, GitHub-Governance, Secrets, IAM, Billing, DDL/DML sowie HIGH/CRITICAL gilt:

- Approval nur durch Owner;
- exact target, operation, version/SHA und Zeitfenster;
- keine implizite Freigabe;
- keine Wiederverwendung nach Änderung;
- keine Agenten-, Provider- oder Modellselbstfreigabe;
- `OWNER_APPROVED` muss vor Side Effect persistent und referenzierbar sein.

5.1 Einmalige Abschnittsfreigabe für jeden Administrator

Diese Regeln gelten ausnahmslos für jeden menschlichen oder agentischen Administrator-Principal:

- genau ein benannter Roadmap-Abschnitt bzw. ein Work Package;
- genau ein Administrator-Principal;
- Bindung an `approvalId`, `main`-SHA, Branch, Ziel, Operations-Allowlist, Risikoklasse und Ablaufzeit;
- atomarer Single-Use-Verbrauch vor dem ersten Side Effect;
- nach Erfolg, Fehler, Abbruch oder Rollback keine Wiederverwendung;
- Replay oder paralleler Verbrauch → `DENY_APPROVAL_REPLAY` ;
- Scope-/Principal-/SHA-/Target-/Operations-/Risiko-/Zeitfensterdrift → `DENY_SCOPE_DRIFT` ;
- Precheck, Postcheck, Rollback, Kill-Switch und append-only Audit sind verpflichtend;
- Erweiterungen oder Folgemutationen benötigen eine neue Owner-Freigabe.

Eine Abschnittsfreigabe kann mehrere vorab vollständig enumerierte atomare Operationen enthalten. Sie ist verbraucht, sobald der Execution Host den Abschnitt zur Ausführung claimt. Sie darf niemals Merge, Owner-IAM/MFA, Secret-Offenlegung, Live-Billing/Entitlement, destructive production data, DNS/TLS, Security-Control-Abschaltung oder Selbst-Erweiterung delegieren.

6. Prototypmodi

Modus	Erlaubt	Verboten
READ_ONLY	Inventar, Vergleich, Evidence-Plan	Writes
MOCK	Contract-/Unit-Tests	externe Calls mit Side Effects
SANDBOX	isolierte Testressourcen nach Authority	Produktion
REPOSITORY	scoped Code/Test/Docs in Branch	Merge, Secrets, Plattformmutation
MUTATION_HANDOFF	Proposal und maschinenlesbarer Auftrag	eigenständige Ausführung
AUTHORIZED_EXECUTION	exakte genehmigte Operation	Scope-Erweiterung

Standard ist `READ_ONLY` oder `MOCK`.

7. Pflichtentscheidungen

- `ALLOW_PROTOTYPE` ;
- `REVIEW_REQUIRED` ;
- `DENY_INCOMPLETE_WORK_PACKAGE` ;
- `DENY_SCOPE_DRIFT` ;
- `DENY_MISSING_OWNER_APPROVAL` ;

- `DENY_TARGET_MISMATCH ;`
- `DENY_PRECHECK_FAILED ;`
- `DENY_AUDIT_UNAVAILABLE ;`
- `DENY_APPROVAL_REPLAY ;`
- `STOP_AND_ROLLBACK .`

8. Native MFA

Owner-Attestation: native MFA aktiviert.

Der Agent darf:

- read-only AAL-/Faktorstatus prüfen, soweit sicher und autorisiert;
- redigierte Positive-/Negative-/Recovery-Evidence vorbereiten;
- fehlende Evidence oder notwendige Korrektur als Proposal anfragen.

Der Agent darf nicht:

- Faktoren enrollen, entfernen oder resettet;
- MFA-Policy verändern;
- Recovery oder Owner-Zugriff mutieren;
- Secrets oder TOTP-Seeds erfassen;
- Aktivierung als VERIFIED PASS behaupten, solange Exit-Evidence fehlt.

9. Tests

Mindestens:

- gültiges Owner-Proposal → bis Approval nur `REVIEW_REQUIRED`;
- Agent-Proposal ohne Approval → `DENY`;
- Ziel-/SHA-Änderung nach Approval → `DENY`;
- fehlender Auditstore → `DENY`;
- Precheck fail → zero side effect;
- Postcheck fail → `STOP_AND_ROLLBACK`;
- Scope drift → `DENY`;
- Secret im Output → Redaction/`DENY`;
- abgelaufene Approval → `DENY`;
- zweiter oder paralleler Verbrauch derselben Abschnittsfreigabe → `DENY_APPROVAL_REPLAY`;
- anderer Administrator-Principal mit derselben approvalId → `DENY`;
- Fehler/Abbruch/Rollback und anschließender Wiederholungsversuch ohne neue Freigabe → `DENY`;
- Branch nach Merge noch vorhanden → Arbeitspaket nicht geschlossen.

10. Evidence

Evidence ist redigiert, append-only und korreliert:

- request/proposal ID;
- requestor;
- Owner-Approval-Referenz und Single-Use-Verbrauchsstatus;
- target hash/version;
- Precheck;
- authorization event vor Side Effect;
- Operationsergebnis;
- Postcheck;
- Rollback, falls ausgelöst;
- PR/CI/Merge;
- Branch-Cleanup.

11. Branch Lifecycle

Jeder Repository-Prototyp:

```
current main → fresh agent/<scope> → scoped diff → deutscher Draft-PR  
→ Human Review → CI → Human Merge → Remote-Branch löschen
```

Kein Branch wird nach Merge wiederverwendet.

12. Stopbedingungen

Sofort stoppen bei:

- fehlender oder mehrdeutiger Authority;
- ungeklärtem Ziel/Tenant/Umgebung;
- fehlendem Rollback;
- fehlender Auditpersistenz;
- Scope Drift;
- Secret-/PII-Risiko;
- widersprüchlicher Evidence;
- Versuch der Selbstfreigabe;
- neuem HIGH/CRITICAL-Risiko ohne erneutes Owner Review.